

Agent-Based Modeling and Simulation

I

**Practical guide to the analysis
of complex systems**

Juan Martín García



Agent-Based Modeling and Simulation I

Practical guide to the analysis
of complex systems

Juan Martín García



Agent-Based Modeling and Simulation I
Practical guide to the analysis of complex systems
Juan Martín García 2021 Spain
jmg@atc-innova.com
ISBN: 9798599111306

Juan Martin Garcia is a renowned expert in System Dynamics and System Thinking, PhD Industrial Engineer (Spain) and Postgraduate Diploma in Business Dynamics at the Sloan School of Management of the Massachusetts Institute of Technology (USA). He has extensive industrial experience in engineering and manufacturing, and 20 years' experience teaching building simulation models in large companies and universities, now teaches the online courses of Vensim in <http://vensim.com/vensim-online-courses/>.

Reviewers: Lening Mora, M.S Environmental & Occupational Health (San Diego, California) and Postgraduate Diploma in Healthcare Modeling and Simulation at Naval Postgraduate School (Monterey, California USA). Dr. Francisco Campuzano Bolarín, Professor of Business Organization at the Universidad Politécnica de Cartagena (UPCT). Author of 27 publications indexed in JCR, and several books published in Springer and Pearson. (Cartagena, Spain). Professor Gavin Melles, PhD, MSc Swinburne University (Victoria, Australia). Steven Warwick BSc (Hons) BEng (Hons) MRSC FGS FRSS. PhD Researcher. Geoenvironmental Research Centre. School of Engineering. Cardiff University. UK

© All Rights Reserved. No part of this book may be reproduced, stored in a retrieval system or transmitted without written permission from the author.

Ventity® is a trademark of Ventana Systems Inc. USA.

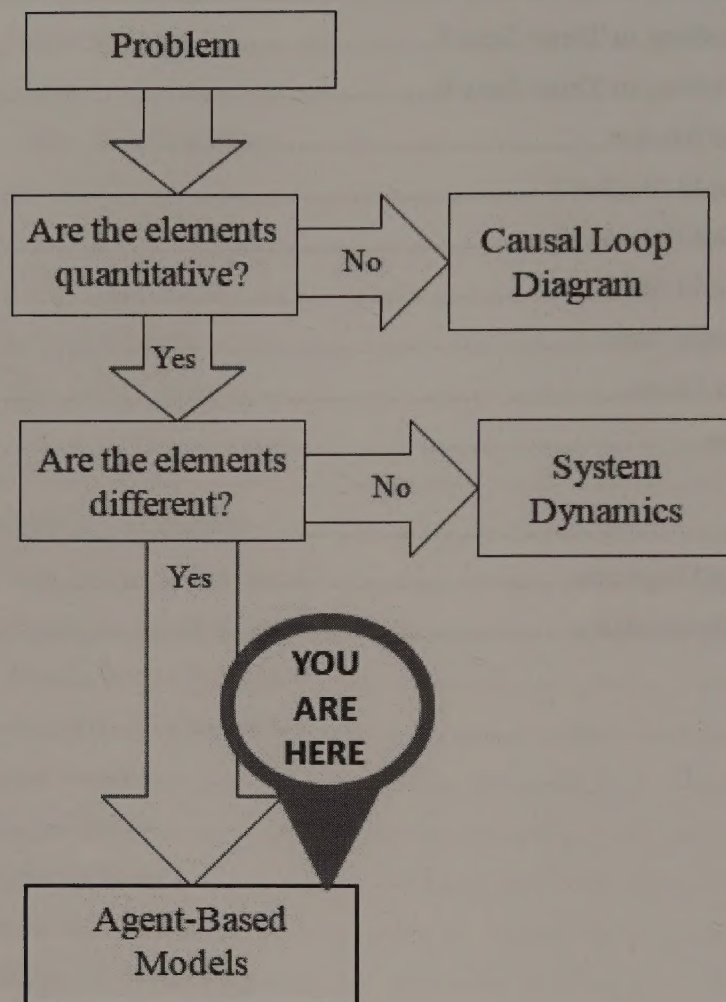
Cover and images: pixabay.com and coquephotography.com ©

INDEX

Preface	7
Key points	9
Glossary	13
Software	17
Installing the software.....	19
Panels and windows.....	23
A model in 1 minute	30
Concepts	33
Index of models	35
Used terms	37
Functions and Lookups	41
Units.....	47
Variables	49
Model: Traffic Light	50
Model: Paris Rome	53
Attributes	56
Model: Basic River 2	59
Model: Truck Fleet	60
Collections and aggregates	64
Model: Dragons and Castles	65
Model: Parents and Children	72
Model: The Four Pirates	75
References.....	79
Model: Black and White	81
Model: Black and White 2	85
Model: Black and White 3	87
Comments	91

Tools	93
Entities initial parameters	95
Model: Horse Racing	98
Time Parameters	101
Model: Launching Satellites	102
External Data Files.....	105
Formats	105
Import initial data.....	106
Import time series data.....	108
Model: My Three Dogs.....	111
Exercises	115
Model: Population of Rabbits	117
Model: Population of Rabbits 2	121
Model: Population of Rabbits 3	123
Model: Population of Rabbits 4	125
Model: Population of Rabbits 5	128
Model: Sweet Candies	136
Model: Cheese Shop	145
Model Chesse Shop 2	151
Model: Pilots of Formula 1	155
Model: Patients and Hospitals	161
Model: Horse Breeding.....	169
Model: Horse Breeding 2.....	172
Model: Horse Breeding 3.....	174
Model: Horse Breeding 4.....	176
Model: Horse Breeding 5.....	180
Model: Horse Breeding 6.....	186
Model: Horse Breeding 7.....	192
Model: Aircraft Squadron.....	195
Model: Aircraft Squadron 2	202
Model: Fishing in Three Seas	207
Model: Fishing in Three Seas 2	211

Model: Fishing in Three Seas 3	217
Model: Fishing in Three Seas 4	220
Model: Fishing in Three Seas 5	222
Model: Fishing in Three Seas 6	223
Model: Gold Market	225
Model: Gold Market 2	227
Model: Gold Market 3	228
Model: Gold Market 4	231
Model: Jewelry Store	235
Model: Beer Game	239
Continuation	249
Annex	251
Causal Loop Diagrams	253
Download the models	259



Preface

Simulation models founded on the concept of system dynamics assume all the elements defined in the system are homogeneous. For example, by including the concept of *young people* within a particular modeling scenario it is assumed the same characteristics and behavior apply throughout. Such a simplification may not be possible and, in the real world as is often the case, homogeneity does not always exist. Expanding the *young people* example where the buying preferences of 10 young consumers are quite different and these preferences require analysis, then agent-based models [ABMs] are better suited than system dynamics models [SDMs].

ABMs can be applied to many disciplines including business, the social sciences and the pure sciences. They offer the opportunity to simulate the actions and interactions of these complex systems, holistically. Economists assume rational behavior when applying aggregate mathematical models to problems leading to an equilibrium or optimal position. By using ABMs we are able: (i) build from the individual or single entity as the basic unit and (ii) provide a natural modeling alternative in which they allow the use of disaggregated data as a means to overcome limitations associated with assumptions such as rational behavior.

Agent-based modeling can be achieved using generic software including Java, Anylogic, NetLogo, Matlab, C++ or Python. However, to construct an ABM with any of these systems requires a working knowledge of that system and/or programming. This book introduces Ventity as a specific ABM software that requires no programming knowledge and is free of charge for personal or educational use. By following the examples contained in this book you will develop your knowledge and skill in the application of Ventity. These can then be applied to your own area(s) of expertise and facilitate the construction of more complex agent-based models.

Learning ABM requires an understanding of its concepts and the ability to apply them. This book is a learning resource. An understanding of the concepts is developed by learning to use the software. Their application is achieved by following and completing the many practical exercises which are written as a step-by-step guide to maximize your learning experience. The book includes a complete set of instructions on downloading Ventity along with a link to the exercises included in this book.

My objective in writing this book is to provide you with an introduction to ABMs and how to create them. Modeling can be a frustrating experience, not least the initial learning but also experiencing the disappointment when a model fails. It is my hope, the approach taken in this book is both friendly and sufficiently well explained to overcome problems associated with learning. With regard to disappointment let me say this. I have designed this book so that you are able to get the correct results – from the start. This will encourage you to move forward, in a progressive manner, and acquire the understanding and confidence to take-on the more complex problems. Good modeling!

The author

What is an Agent-Based Model?

An ABM is a methodology that enables the creation of simulation models based on identifying individual behavior of the entities (people, animals, things, etc.) that make up a system, and is used to analyze what changes can be introduced to the real system to achieve the desired behavior.

When using an ABM

Prior to creating a model with the ABM methodology, we have to brainstorm the following options:

1. Often times the problem can be analyzed well with the help of an Excel. If so, without hesitation use Excel.
2. If there's a lot of available historical data and you want to make a forecast, consider using statistical software. For example, we want to make a forecast for the next month of traffic on a highway, of which there are daily historical traffic data for the last five years. No construction or any other specific event is scheduled for next month. In this case, software such as SPSS or Matlab is best suited for this option.
3. Under the notion the main elements of the system are homogeneous, in this case the ideal tool is to create a System Dynamics model. For example, you want to establish the limit of tuna catches in a certain area. It can be assumed that all tunas are the same, and the fisheries are classified into 3 types. A software with the capability to create System Dynamics models such as Vensim is the best option.

Discarding the previous options, we already have the certainty that an ABM may be just what we need. An ABM model is used when we have a system composed of elements that, although similar, have characteristics that make them unique and critical to understand the system as a whole to decide policies that must be applied to manage it and achieve the proposed objective.

For example, let's say you want to plan the activities of a taxi fleet, and you cannot assume that they are the same, due to different vehicle brands, different purchase dates and daily accumulated kilometers, etc. Therefore, although they are all similar, the characteristics of the taxis cannot be based on an average value, the same for all, because that average taxi does not exist.

What is ABM useful for?

ABM simulation models are used across multiple industries and fields. The following are a few examples on the usage of ABMs: In the academic field ABMs are useful for research projects, final projects and doctoral theses e.g., since it allows the concepts to be clearly ordered, rigorous calculation made to finally offer convincing results. ABMs are useful as a planning tool for production and managing warehouses. In economics useful to manage multiple public resources, especially investments. The environment to analyze the environmental impact of human activity on natural ecosystems. The multiple disciplines of the social sciences can integrate ABM models as an analysis tool allowing analysis of small groups to groups that encompass millions of people.

What does the book contribute

Starting the usage of a new methodology always requires effort. This book shows in an orderly fashion the concepts as well as the steps to follow to build an ABM. The step-by-step creation of various models of increasing complexity are shown, in order for the reader to progressively gain mastery of this tool.

The themes of the models are diverse reflecting the multiple applications of this tool. The reason for this diversity of themes is not so much to show possible applications, but instead to make reading the book an enjoyable experience.

Focus of the book

It is assumed that the reader is completely unaware of the ABM models, beyond having an idea of their possible utility. Therefore, the book begins by explaining the most basic concepts and the terms used and then progressively explaining in each of the practical exercises the benefits the software offers. Each exercise brings some news and also helps to consolidate what has been seen up to that moment.

Obviously, a certain effort of concentration is required from the reader, but the content of the book is designed so that the reader can advance at a good pace. To achieve this objective, the book has been refined with the help of many people who have volunteered to do so, to achieve a text that is easy to understand and free of any misprints.

The approach is therefore to start from the most basic level and progress until the reader can make their own model applying the ABM methodology. This is the goal of this book, but it is not the end of the road thus, a follow-up book introduces the reader to additional features such as creating entities, defining cohorts, and doing sensitivity analysis, optimization, and calibration.

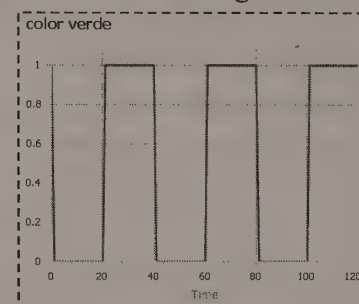
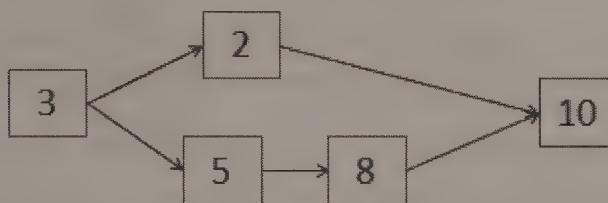
Previous knowledge

Reading the book and doing the exercises does not require any programming knowledge, which is a great advantage. Nor are special mathematical knowledge necessary, an average user of Excel can approach this book with complete confidence and peace of mind.

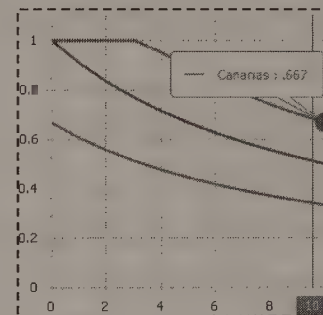
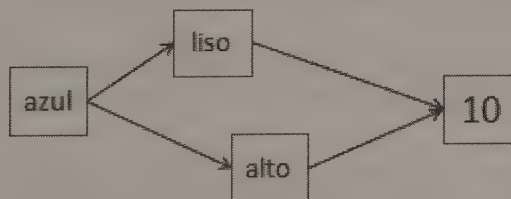
The person who has prior programming knowledge will be able to directly manipulate the instructions of the model, since it is ultimately a txt file, and the individual will also be able to find some equivalence with the existing knowledge from programming.

What am I going to learn

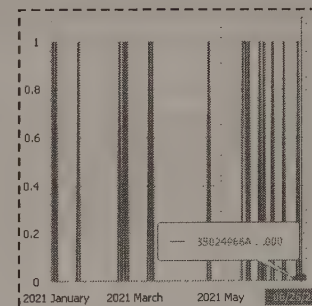
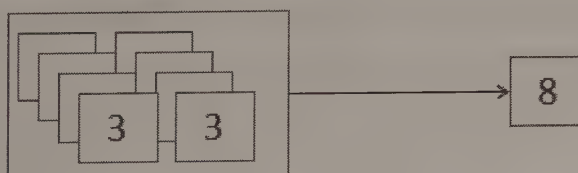
1. Create models with quantitative variables. In many models we can assign values to all variables, and the result is also numerical.



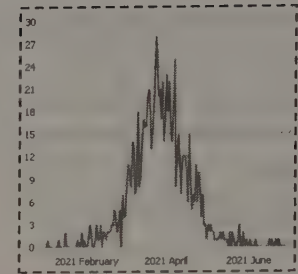
2. Create models with qualitative variables. Sometimes we need to use alphanumeric attributes to some variables, with one result being numeric.



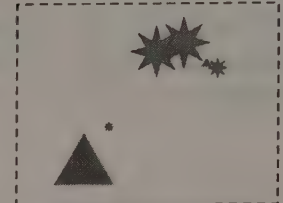
3. Create models with grouped variables. Sometimes we must group some entities by one of their attributes to obtain clearer results.



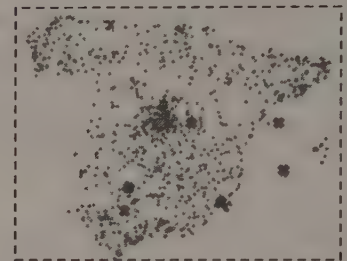
4. Create and delete entities during simulation. This option is used when some entities are incorporated or deleted during simulation



5. Get results in 2D. Feature that is used when the dynamic representation of the behavior of the system entities on a two-dimensional plane is of interest.



6. Get results on GIS images. Feature that allows you to view the dynamics of the system entities when they have a geographic references.



These last 3 points are developed in the book that is a continuation of this one.

In addition, you will learn the use of chronological dates in the simulation, the capture of data from an Excel sheet, the transfer of results to external files, the use of entity cohorts and many other additional features.

Software

The software used is exclusive to create simulation models applying the ABM methodology, it does not require programming knowledge of any kind. You are going to use Ventity software which is a free software for personal and educational use, consequently it is not necessary to buy it in order to complete the exercises from the book.

Ventity is a very intuitive software to use, since the diagrams are created with the help of the icons and the variables used allow a colloquial language such as "change to green" or "open the tap".

Glossary

Action - Instructions for making changes to stocks and attributes and also for creating or deleting entities. They indicate what model should do.

Aggregate - It is a variable that adds together the value of other variables whose entities have been grouped by any of their attributes.

Attribute - It is a characteristic of an entity. The value of an attribute is a text string: letters, numbers or other characters but not a number.

Arrows - They indicate that there is a causal relationship between two variables. The software validates that there is coherence between the arrows that affect a variable and the variables used in the equation. So, a warning is generated if the diagram shows an influence that does not exist in the equation.

Auxiliary - It is a type of variable whose value depends on other variables and is calculated with an equation; it is not a Stock or a Flow.

Calendar - The simulation can be done using calendar dates.

Calibration - The process of calculating the difference between the results of the model and the actual data.

Causal Tracing – A causal tree that shows which variables a variable depends on and which variables are influenced by that same variable.

Cloud - It is a stock of infinite value and origin of the units that accumulate in a stock. Although it can be named, a cloud never influences calculations.

Collection - Group of some entities based on some of their attributes. The objective is to have aggregate variables of these entities.

Constant - This is an element whose value does not change during the simulation, although it may have a different value in each simulation.

Count - It is a counter of the entities that meet certain conditions.

Data Entity Type - Entity that is built to collect some external data, for example in an Excel sheet, that will be used in the simulation. It is built to link model variables with Excel fields.

Data Set - Set of data sources, for example several Excel files, used in the simulation. In the Run Control you can select which one will be used in the simulation.

Data Source - Individual, non-grouped data sources that can be used in a simulation.

Delay - Set of functions that allow one to simulate that a variable responds with a certain delay to changes in another or several variables.

Edit - Modifies the equation of a variable.

Entity - An entity, agent or element of the system is the smallest unit of analysis, which can be both an individual and a group.

Equation - It is the mathematical expression that allows us to calculate the value of a variable based on the value of other variables.

Singular Entity - It is a unique entity type in a model that has variables used in the calculations of other entities, so that those entities can access them without having to define any reference. These entities do not have collections.

Entity Initialization Data - Folder where the files are placed with the values that the model variables will have when an entity begins to exist.

Entity Picker - Folder that allows us to select which entities, or a set of them, we want to see in the results charts and tables.

Exp - Function that allows exponential calculations.

Export – Process by which we pass the results of the model to an external file.

Flow - It is a variable that indicates the rate of change of a Stock.

GIS - Geographic Information Systems files contain information on a geographic basis that can be used if the entities of the model vary their geographic location and this is relevant for the analysis of the system.

Grids - On the diagram there is a grid to show the references that an entity type has, and another grid to show or hide the drawing icons.

If - Conditional function that allows defining what value a variable takes if a condition is met, and what value it takes if it is not met.

Inspector - Area of the screen that allows us to see the variables, attributes, collections, references and actions of an entity type. It is opened from the Model Overview of each type of entity.

Invoker - It is a reference that indicates which is the entity that owns the trigger that executes an Action. These references are used to access the target attribute or the stock to be modified.

Key - It is a characteristic of an attribute that indicates that this attribute and not another uniquely identifies an entity of a certain type.

Lookup - It is a way, alternative to an equation, to represent the relationship that exists between two variables. This relationship is defined by points on an XY graph, each of these points indicates the value that the dependent variable takes as a function of the other variable.

Mapping - Screen used to link the fields of an input file with the model variables.

Model Overview - Work screen area that allows us access to the diagrams of all the entity types, the collections, the actions, and the folders that contain the input files.

Node - Intermediate variable that is automatically generated to identify which attributes or stocks an Action can modify.

Optimization - Calculation process that allows us identify what values some variables must take so that another variable takes a desired value.

Parent - It is a reference that indicates which is the entity that owns the Action. These references are used to access the target attribute or the stock to be modified.

RAMP - Function that allows us to simulate the gradual change of a variable.

Random - A set of functions to simulate random values in a variable.

References - Definition of which attributes allow us to establish the links that exist between various entities.

Refers to - Indicates that an attribute of an entity corresponds to another entity.

Run Control - Screen to define the temporary parameters, as well as to identify the input files and the units, to be used in a simulation.

Sensitivity - The sensitivity analysis allows us to identify the degree of influence that some variables have on other variables.

Simulate - The model calculates the evolution of each of the system entities between the initial period of time that has been defined and the final one.

STEP - Function that allows us to simulate a sudden and continuous change of a variable.

Stock - Variable that shows accumulation in the system.

Time - This is a variable that acts as a time counter, which takes the same time values that have been defined for the simulation, be it a sequence that begins with 0 or for example 2020. If the sequence is based on calendar dates, this variable starts with the value 0.

Time Series Data - Folder where the files with series of time values used in the model are placed.

Trigger - Indicates what conditions must be met for an Action that has already been defined to be executed.

Units - Indicates the units of measure for each variable in the model. Although they do not intervene in the calculations, and therefore in the simulation results, they can be used to check the consistency of the equations.

Variable - A variable is an element of the system whose value changes over time. It can be Stock, Flow or Auxiliary.

Visualization - Folder that allows us to define the format in which the results will be displayed when the entities of the model have changes in a two spatial dimensions.

Software

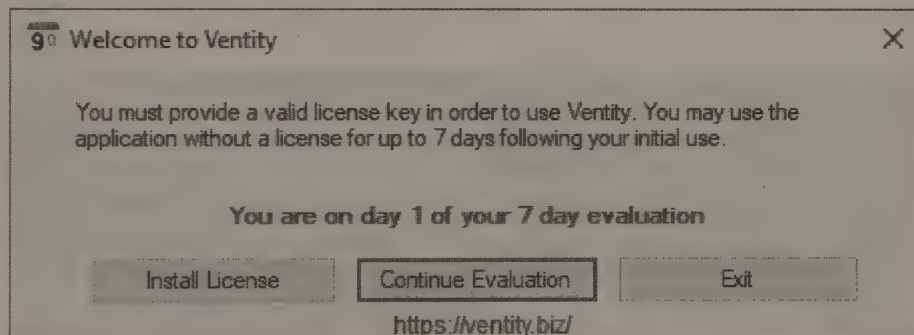
Installing the software

Ventity is the software used here to build the simulation models with the ABM methodology. It is a specific software for ABM, therefore the user does not need to have any prior knowledge of programming languages. There is general information about the software and its applications at <https://ventity.biz/>

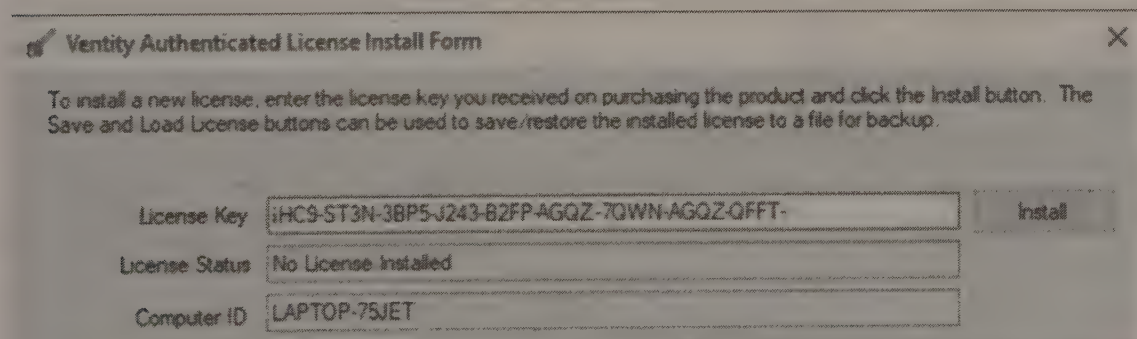
To download the Ventity software license, enter your personal data at <https://ventity.biz/download>.

When all the data has been entered, an email will be received with instructions, the steps to follow to download the software are:

1. Copy the password you have received.
2. Click on the web shown in the email.
3. Choose the version you want to download (for example Ventity 4.0 beta 64 bit)
4. A zip file is downloaded at the bottom of your screen. Open it.
5. Run the VentitySetupx64 or similar application that is in the zip.
6. Follow the instructions that appear (Accept the Agreement, ...) until Install.
7. Click Finish.
8. A welcome message appears.



Click on "Install License", in the new window paste the license code that you received in the email, then click Install.

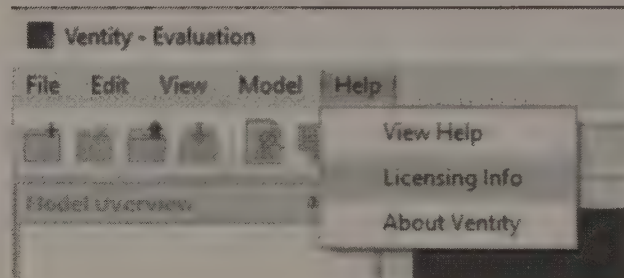


9. The key is verified, and the expiration date appears in the License Status. Press Close.

The screenshot shows a dialog box titled "Ventity Authenticated License Install Form". It contains instructions: "To install a new license, enter the license key you received on purchasing the product and click the Install button. The Save and Load License buttons can be used to save/restore the installed license to a file for backup." Below the instructions are three input fields: "License Key" (empty), "License Status" (containing "License Installed - Expires: 12/25/2020 11:34"), and "Computer ID" (containing "LAPTOP-75JETETF"). At the bottom left are three buttons: "Load License", "Save License", and "Uninstall License". At the bottom right is a "Close" button. At the very bottom, it says "To obtain a license or edit/update your existing license, please visit: <https://ventity.biz/>".

and step 10. Ventity opens, and also on the Desktop there should be the Ventity icon.

You already have the Ventity software installed and can use it without restrictions until the expiration date indicated in the License Status. The next step is to request a free license for personal or academic use. Open Ventity, go to Help - Licensing Info.



Click on the button to edit the license. Your license details appear and a link to request a free license for academic use appears at the bottom of the page. Click on that link.

The screenshot shows the Ventity application window with the "VENTITY" logo at the top. Below the logo is the "License Information" section, which displays the following details: "Date Issued : 10/26/2020 10:34:15 AM", "Key : A3Q3-8AAL-UGJC-78A2-GHC9-ST3N-3BP5-J243-B2FP-CDBG-7QWN", "SerialNo : 101414", "License type : Evaluation", and "ExpiryDate : 12/25/2020 10:34:15 AM". Below this is the "Apply for an academic use license" section, which contains the text: "If you would like to apply for an academic use license, please click the following link. You will be asked to complete a form and agree to the restrictions in place on an academic use license. If you meet the criteria for an academic use license, we will extend the expiry on your license by a year. After that time, you will need to re-apply for a license extension." At the bottom of this section is a blue hyperlink: "Click here to apply for an academic use license." A black arrow points to this link.

Fill in the information requested, and at the bottom of the page press the button to send this information.

Educational licence policy.

Academic use licences are available only to students and faculty of accredited educational institutions. The software may only be used for educational purposes.

If you believe you qualify for an academic use licence, please complete the "Educational Discount Policy" form below.

If you have any questions please feel free to email us at info@ventity.biz.

Please read this carefully and ensure that you understand the licence restrictions. Agree to the terms, and submit the form.

IMPORTANT : Please use your official email address.

Name

Institution

Position

Full address

Telephone number

Email

Confirm Email

Intended use of Ventity (please complete in as much detail as possible)

You will receive a message stating that you will receive the answer shortly.



Thank you for your application. We hope to process this within the next 5 working days.

Ventity customer center v1.0.

The response email usually arrives in a few minutes, indicating that the license expires in a year, at which point it will be necessary to reapply.

Ventity academic license request



noreply@ventity.biz <noreply@ventity.biz>

12:02

Your request for an academic use license has been approved. The license will expire one year from now; at that time you will need to re-apply.

You don't need to reinstall the license key; its status will update automatically, usually within 24 hours, when you start Ventity.

Kind regards,

The Ventity team

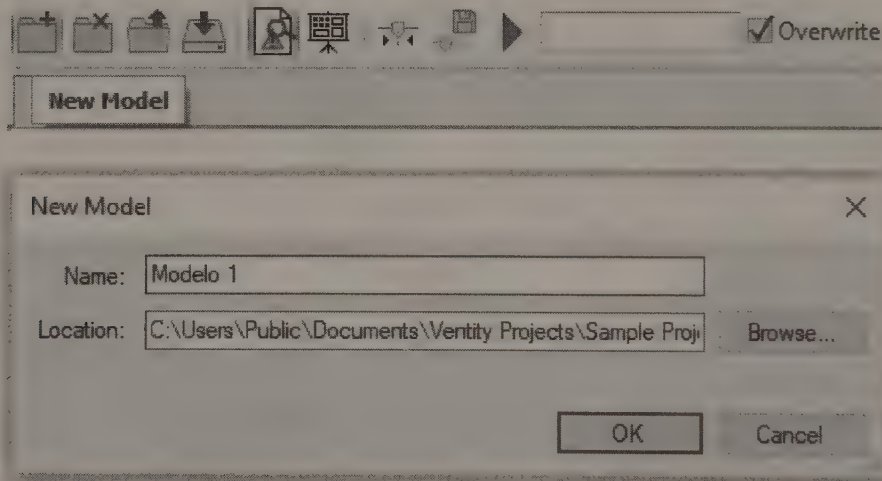
All licenses for personal, academic or user use with Vensim Professional and DSS licenses are free. In the latter cases, enter the license data on the website <https://ventity.biz/update>, download the zip file, then open it and run the exe. In any case, the license can be purchased at <https://ventity.biz/purchase>. If you need help, consult info@ventity.biz.

Ventity software is powerful and has many advantages, but it is sometimes difficult to correct any errors. Remember to save the model frequently and make successive versions of the model that allow you to go back to a previous version. Sometimes it is not enough to close the model, it is necessary to also close Ventity and reopen it.

Once the software has been downloaded and the software installation is complete, click on the "New Model" option to start. The window that displays the model creation area opens.



Write the name (Model 1) and OK.

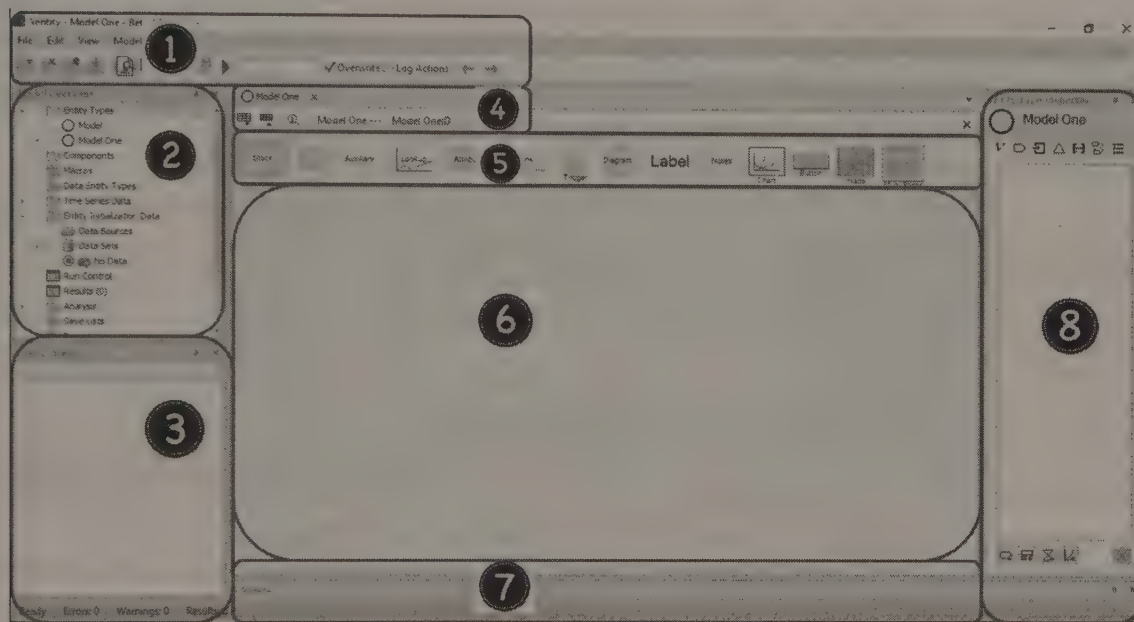


Installation is complete, you can start designing your model. Remember that your license details are available in Help - Licensing Info.

Panels and windows

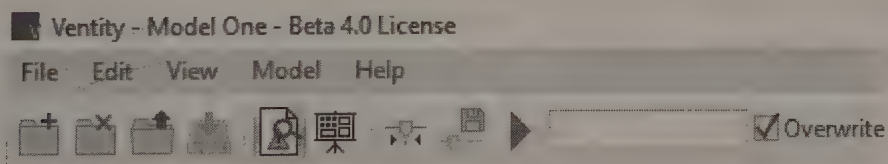
The screen is divided into 8 panels. Panels can be resized and rearranged by moving the top title bar to place it on the desired part of the screen. Any panel can be closed with the x button at the top right, and can then be opened from the View menu. The most common layout is to close windows 3 and 7, to leave more space at windows 2 and 6.

When a model is created, the work screen opens:



1. Top bar

The top bar contains icons to create, open or save a model, as well as simulate it (using the green arrow), including writing the name of the simulation. By default, if you do a simulation with the same name as the previous one, only the last one is saved (Overwrite mark).



To create versions of a model press File - Save As. By default a model is created with the same name as the current one followed by a consecutive numbering.

2. Model Overview

- Entity types. It shows the Types of entities, their Collections and Actions. Double clicking on any of these opens the diagram screen and on the right the inspector for that Type of entity, Collection or Action.

By default, the entity "Model" appears with the temporary parameters of the model, and an entity with the name of the model, for example "Aircrafts".

- Data Entity Types are variables with external data that are used in the model calculations.

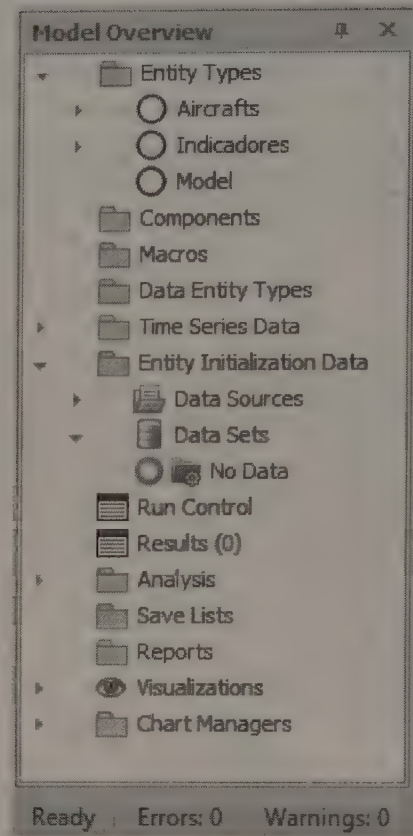
- Time Series Data hosts the external files with time series data that are used in the model

- Entity Initialization Data includes initial data sets. Right-click on a data set to select it as the source for the start of the next simulation. Double-click on a file to browse or edit it.

- Run Control configures temporary parameters and active initial data files. Press to edit the time horizon and time interval of the simulation.

- Results shows the simulation results. Right-click on a run to browse, save or export the results.

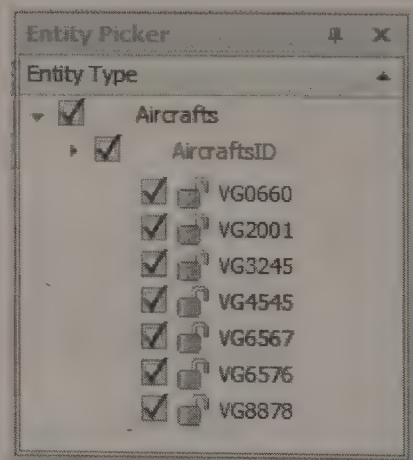
- Reports shows additional tools for viewing the output. Right-click on a visualization to view or configure its settings.



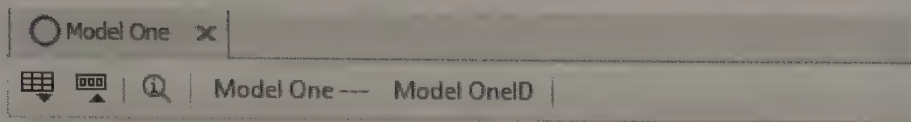
3. Entity Picker

In an ABM model hundreds of entities can be managed; the Entity Picker allows selecting one or more entities in order to analyze their individual behavior and verify if it is correct.

Check or uncheck the boxes and locks to determine which entity or combination of entities will be displayed on the charts.



4. References



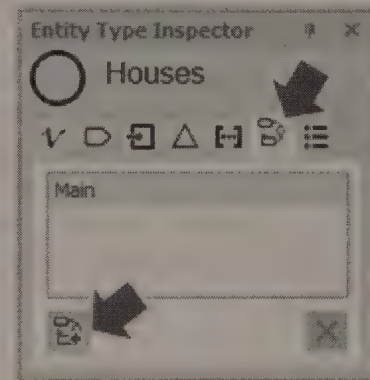
Click the grid on the left to see the references, the second grid shows or hides the drawing palette icons. To show this area if it is hidden, right click on the central diagram and the option "Show Diagram Toolbar"

5. Diagram

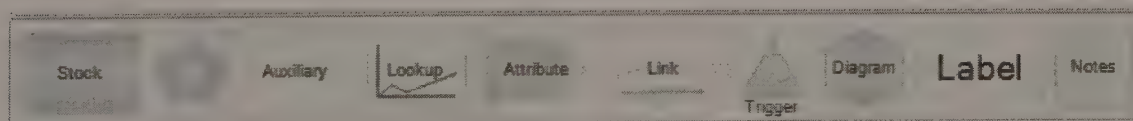
A diagram show the properties of an entity and the causal relationships between its components. Each entity has a Diagram with its name. If one diagram is not enough, the entity definition can be divided into separate sections, each with its own diagram. Thus, the entity type "production" can have a diagram dedicated to the raw materials stock, another dedicated to the process of production, and another to the finished products stock. You can create a fourth diagram that includes entities from the other three, without changing the calculation structure of the model.

Entities, collections, and actions appear in the Model Overview, and have their own inspectors and diagrams.

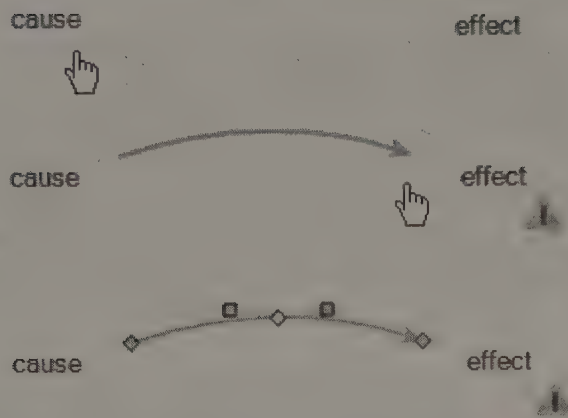
To create an additional diagram of an entity, use the + button at the bottom of the entity inspector. Enter a name for the diagram when prompted.



To define an entity type, visual symbols are used. Variables are of four types: Stocks, Flows, Auxiliary and Aggregates (in collections). When you click on one of the icons in the palette, it is selected, and you can move it to the drawing area to incorporate it into the diagram.



A Stock is a variable that accumulates. A Flow is an entry or exit to a Stock. Flows are the rates of change of Stocks. They are drawn as an arrow between two Stocks or between a Stocks and a cloud. Hover the cursor over the edge of the source variable until it becomes a hand, then click and move the arrow to the target variable.



The green and yellow dots are used to change the shape of the arrow. A warning next to the variable "effect" points out that its equation or units are missing.

Auxiliary

An Auxiliary is any variable that is not Stock or Flow. An Aggregate (Aggregate icon) is an auxiliary variable, which is calculated (total amount, average, etc ...) based on a collection of entities.

Sea

A cloud -that can have a name- is a Stock of infinite value, which we use as the origin or destination of the Flows, to specify that there is no restriction on the origin or destination of the units entering or leaving the Stock.

Link

The Link allows one to draw perpendicular links between variables or attributes.

Label

Notes

When a Label or Notes icon is moved to the diagram, a large font title or text field is added for comment.



Chart

The Chart icon adds a chart to the diagram. Integrated Chart graphics update when simulations are done.



Button

Button is used to make it easier for the user to move from one diagram to another. Move it to the diagram, right-click on it, add its name and define the target diagram.



Image

The Image icon allows one to add bmp, png, jpg, ... format images to the diagram. Move it to the diagram, press Select Image and choose it.



Multi-Slider creates cursors in the diagram that allow one to change the value of a constant variable to another value and make the model run again with this new number. Move the icon within the diagram, right-click on it and press Edit to define the parameters.



If an entity has a very complex diagram, it is useful to divide it into smaller parts. Each part, called a group, has its own diagram. Each group can be represented in other diagrams by the Diagram icon. To create a new group, move the icon to the diagram. A second new diagram will be created, and the group node in the first diagram will become a hyperlink to the second.



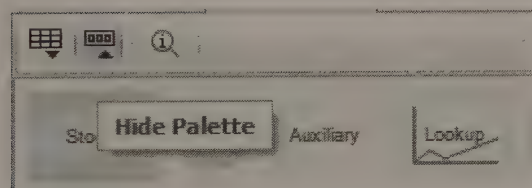
An attribute is a property of an entity. Each attribute of an entity has a name (for example, "hair color") and a value (for example, "brown"). Unlike variables, whose values are numbers, the value of an attribute can be alphanumeric.



Each action must have a Trigger that specifies if and when the action occurs. A trigger can be created on the same entity where the action is defined, or on a different entity. The entity where the action is defined is called the Parent of the action, the entity where the trigger is defined is called the Invoker.



A shadow variable is the duplicate of an original variable, it has no causal source arrows. Their appearance is a green font, is used to avoid arrow overload, such as when a variable appears more than once in a diagram or variables of one entity type are used in another type



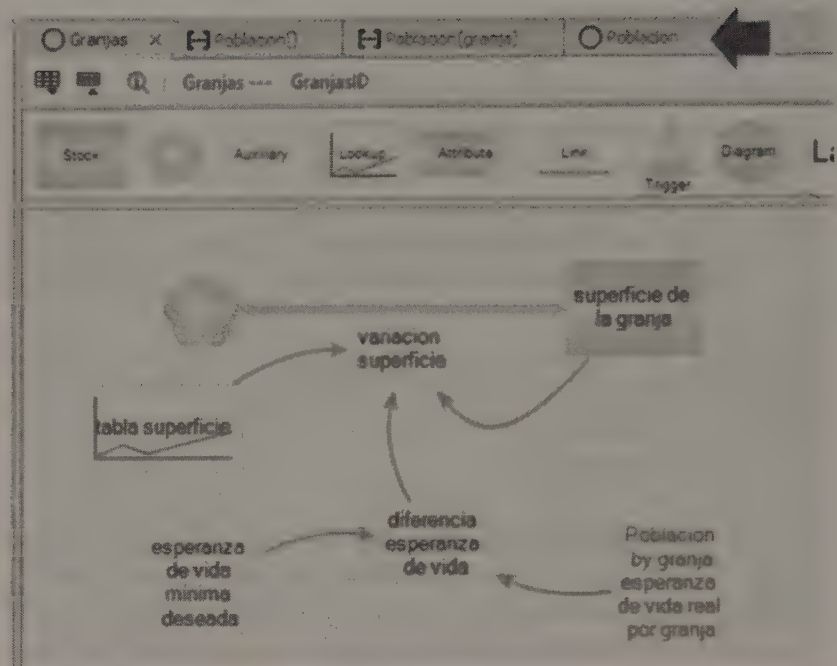
The drawing palette can be shown or hidden by right-clicking on the icon.

The calculation process is:

- The initial values of the Stocks are taken.
- The values of all the Flows and Auxiliaries are calculated.
- The Flows and Auxiliary values are used to update the Stocks after a period of time (for example, one day).
- Flows are recalculated for the next time interval.
- Stockss are updated based on the recalculated Flows
- ... and so on until the end of the simulation.

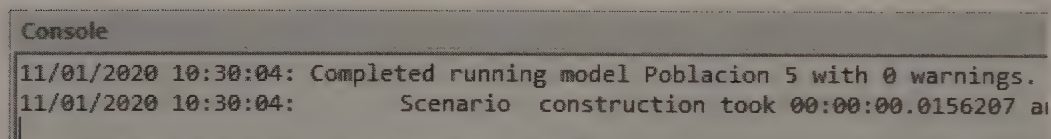
6. Diagram drawing area

The drawing area displays the diagram of an entity, collection, or action. The diagram can be zoomed in or out by selecting the diagram screen and, while holding down the Ctrl key, scrolling the mouse wheel in or out. The diagram can be changed from one type of entity to another with the upper tabs.



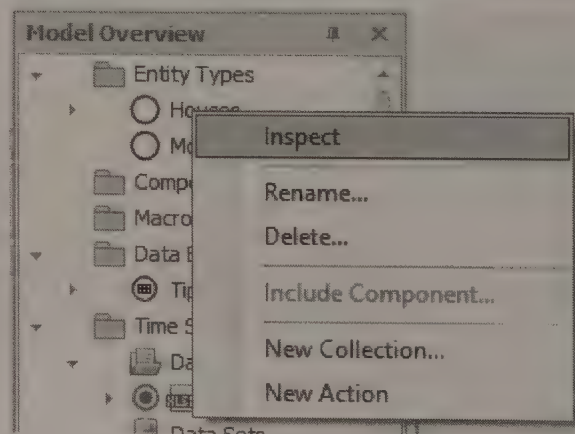
7. Console

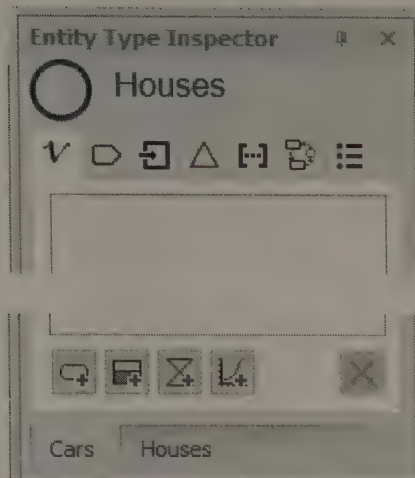
The Console panel is the window in the lower area that shows the results of the simulation, as well as warning messages and errors. It is displayed by clicking View and "Console". The View - Error List option shows warnings and errors in the same Console area



8. Inspector

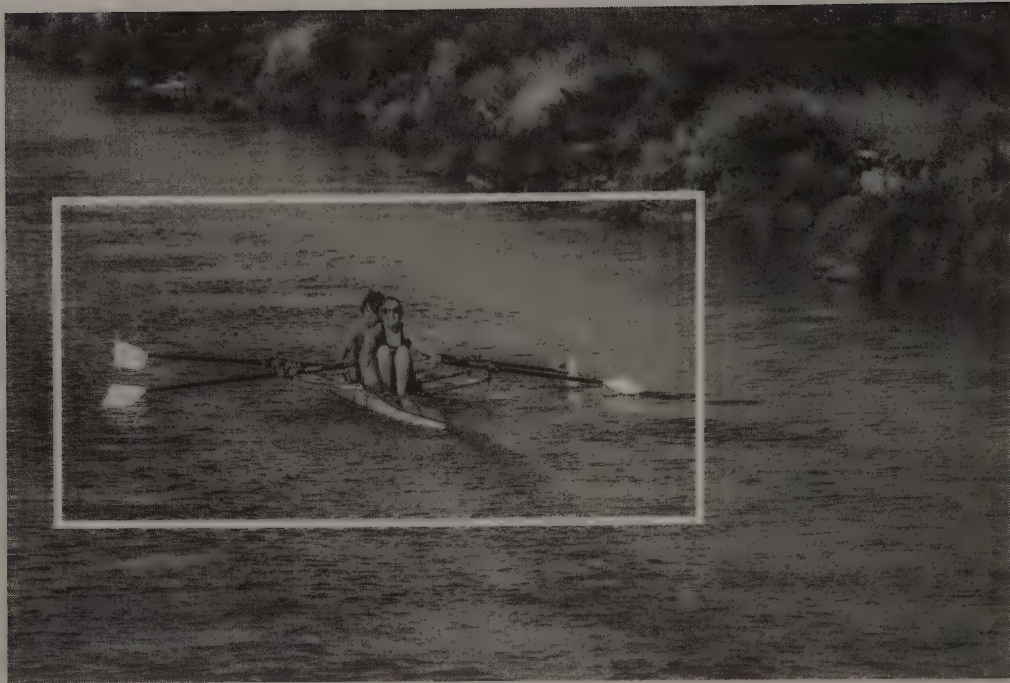
This window allows one to view the details of any given entity, action or collection. To open the inspector, right-click on the entity, action or collection that you want to see in the Model Overview and select "Inspect" or double-click on it.





The Inspector window for an entity displays the various variables, collections, attributes, references, actions, diagrams, and properties of an entity. They can be selected by clicking on any of the tabs.

The lower part of the inspector allows one to open each of them by clicking on the tab with its name. To delete a variable; it must be deleted from the inspector, not from the diagram. If you delete it from the diagram with "Remove from Diagram" the variable is still active in the model.



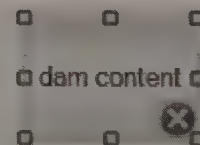
A model in 1 minute

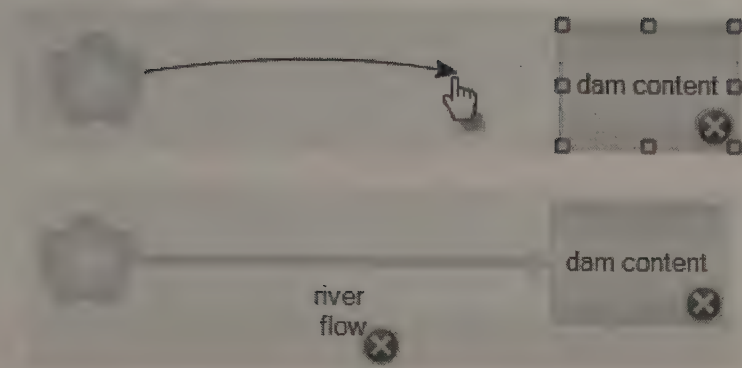
Once the software is installed, we can obtain a very quick overview of how to create a simple model and obtain the first results in a one minute practical exercise. We are going to model the causal path of a river that goes to a dam.

See a video here: <https://youtu.be/EVOjzILbTic>



Go to File - New Model and write the name River, which is the name of the new model. In Model Overview - Entity Types an entity type called River is created. Move the Stock icon to the center of the diagram, in the window that opens type "dam content" (without quotes), press enter. Move the cloud icon to the left of the Stock icon. Move the cursor over the cloud icon, when a hand appears, click on the cloud icon and while pressing the mouse, move the pointer to the Stock you have created, release it in the center, in the window that opens, type "river flow" (without quotes or accent), which is the name of the Flow, press enter.





Flow - River.river flow

Name: river flow

Owner: River

Units:

From Stock:

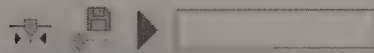
To Stock: dam content

☐ Constant

5

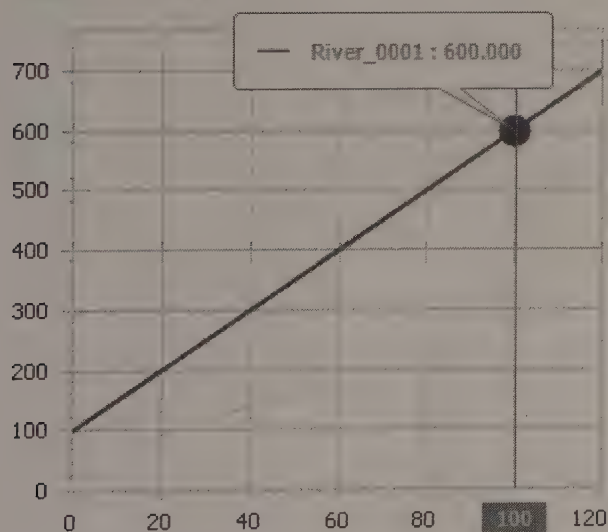
Right click on the Stock, choose Edit Stock, in Default Initial Value type 100, close that window. Right click on the Flow, choose Edit Flow, type 5 to the right of the = sign.

The model is now complete, the yellow signals in the diagram means that the units of each variable are still to be defined, but this does not affect the simulation.



Press the green arrow (Run Model) at the top of the screen to simulate the model.

dam content



Right-click on the variable "dam content", choose the option Plot Variable, the graph appears showing the evolution over time.

All right, so you've already drawn a diagram, simulated the model, and seen the results. This is the basic outline of any model.

Concepts

Practical exercises usually show these points:

- Create a model and the types of entities.
- Create the diagram of an entity: stocks, flows, auxiliaries, arrows, attributes.
- Define the start and end time parameters (Run Control).
- Use the inspector.
- Define the initial data of the constants (Builtin Data).
- Create collections and aggregate variables.
- Create references between entities.
- Define units of each variable.
- Write the equations. Use shadow variables.
- Simulate the model.
- Define the graphs and display graphs of results.

Furthermore the models show additional aspects:

Run Control

Define Schedule: Pirates - Horses 7 - Gold 2-3-4 - Satellites

Use Active Dataset: Restaurant - Horses 2-3

Units - Synonyms

Horses 1

Entity Picker

Cheese 2 - Satellites

Features

IF: Pirates - Mother - Hospital - Candy - Town 4 - Fishing 2,3 - Beer

DELAY: Cheese - Satellites

TABLE: Population 4, 5

RANDOM: Cheeses - Airplanes

STEP: Cheese

RAMP: Cheese

MIN: Beer

MAX: Beer

AND: Planes - Fishing 6

PULSE: Airplanes

Export results

Candy - Horses 7 - Fishing 2

Causal Tree

Beer

Graphics - Edit Chart:

Cheese 2 - Formula 1 - Airplanes - Fishing 4

Import initial data

Aircraft 2 - Fishing 6

Import historical data series

Rabbits - Gold 1, 2, 4

Collections and aggregates

Dragons - Parents - Pirates - Black and White - Hospital - Restaurant

Attribute references

Black and white - Hospital - Restaurant - Town 5 - Horses 4 - Airplane 1 - Trucks - Candy

Reference to a complete collection of entities

Black and White 3

Reference to a collection

Black and White 2 - Population 5 - Candy - Cheese 2 - Horses 5, 6 - Airplane 2 - Fishing 2 - Formula 1

Reference to historical data entities

Rabbits - Gold 1, 4

Entity

An entity, agent or object is the smallest unit that we wish to study. An entity can be an individual entity or a collective entity. We can imagine an entity as a box, within it there may be a person, a liter of water, a rabbit, etc. Each box is an entity. If we have 20 boxes each with a rabbit inside; we have 20 entities, and so for example we can analyze the movements of each one of them within the farm. On the other hand, we can have 20 rabbits in 5 boxes, with 4 rabbits each. In this case we have 5 entities, and although there may be different breeds of rabbits, we are assuming that the 4 rabbits in the same box are identical (and in this case 4 will be the initial value of each of the 5 entities).

In a model of prey-predator differential equations, it is assumed that all prey are equal and that all predators are equal. With an ABM we can define individual properties and simulate the evolution of each of the prey and each predator, even where there are thousands of them, and see individual and aggregate behavior.

Entity types

Entities that have common characteristics are said to be of the same "Type". Thus, there are entities (a certain rabbit), their possible groupings (white rabbits) and the "types of entities" (rabbits), which is a set of different but homogeneous entities. Thus, we can define as types of entities: rabbits, wolves, grass, etc. The first step is to define what types of entities will exist in the model:

- A construction project may require workers and tasks as entities.
- The analysis of an ecosystem may require rabbits and wolves as entities.
- The analysis of a bank may require balances, customers and technology as entities.

Variable

A variable is a property of the entity whose value changes over time. A variable has a name, an equation that is used to calculate its value, and a value at each moment during the model simulation. A constant is a variable whose value does not vary. A variable can be a stock, flow, auxiliary or aggregate.

Stock

Stocks are variables that show accumulations that exist in an entity. When designing a model, it is advisable to start by identifying the stocks. Examples of stocks are the contents of a water tank, the balance of a bank account, the number of trees in a forest, the number of cars in a parking lot, the number of passengers on a train, etc.

Remember, a Stock with value 80 (people) does not mean that there are 80 entities, but ONE ENTITY with 80 homogeneous people.

Flows

Flows are variables that show the inputs and outputs of the Stocks. The units of a flow are the same as the units of the Stock it regulates divided by a unit of time. Thus, examples of Flows are: liters of water input per minute, amount of monthly income in the bank account, trees felled each year, etc.

Cloud

Every flow must have an origin and a destination. When the origin or destination is not relevant to the problem analyzed, a cloud is used instead. Thus, if the water in a small reservoir is the object of study and comes from a million m³ dam, it makes no sense to include the volume of the dam in the model, and instead a cloud is used.

Auxiliaries

Any variable that is not a stock or a flow is of the auxiliary type. They are used to divide calculations into parts of understandable size. For example, it is useful to break down income and expenses as into separate auxiliary variables to calculate profit.

Aggregates

It is a variable calculated from the properties of a collection of entities, for example the average age of a company's workforce.

Initial value of an entity

The stocks require a value to be entered at the start of the simulation. It is required to specify this value when writing the equation. It can be a number or a equation. In the Entity Initialization Data, specific values can be included for each entity, if none is specified, the one defined in the equation is used for all. Thus, the stock of "clients" can have an initial value of 100, and in the start file it can be defined: 180 clients in Madrid, 60 clients in Seville 60, 20 clients in Malaga. If these values are not specified, the calculations are made with the value 100 for each client.

Constant

The flow and auxiliary equation has a "Constant" box. If this box is checked it is possible to assign different values for each entity in the Entity Initialization Data. If "Constant" is not checked, the value of the variable will be determined from the equation at each time interval.

Variable names

The software is not case-sensitive for variable names, and omits any whitespace or underscores. Therefore, "Waiting patients", "waiting patients", and "waiting_patients" are the same variable.

Causal relationship

All variables depend on other variables, except independent, external or exogenous variables. For example, profit depends on income and expenses, thus, income and expenses are the causes of profit.

Singular entity type

A singular entity has variables that are used by many other entities in the model, and it is not necessary to create a reference for each entity, such as the weather entity, which would affect both wolf and rabbit entities. You can designate an entity type as singular by checking the "Singular" box on the Properties tab of the inspector. There can only be one entity type defined as singular in a simulation (other than Model). Singular entities have no collections and no more than one of them can be initialized in a simulation. At the same time, no reference is needed to access the information in a singular entity. An example of a singular entity is the Model entity, you can't have collections, but Model variables can be referenced by writing Model.time.

Attribute

An attribute is a property or characteristic of an entity. Each attribute of an entity has a name (for example, "hair color") and a value (for example, "brown"). Unlike variables, whose values are numbers, the value of an attribute is any combination of letters, numbers, or other characters. They cannot be modified by other variables, only by actions.

Collection

You can create entities that are the aggregation of entities of a type A (for example children) based on entities of another type B (for example parents). You can do this if you have placed an attribute of entity B (parents) in entity A (children), and thus, for example, calculate the number of children that each entity of type parents has.

Reference

A reference is the name of an entity, or part (collection) of an entity, whose information will be used for a calculation. It must be specified which entity is the origin of the information and what information it has.

Calculations

The software simulates each of the entities (which has its own ID) in each of the time periods defined in the model. Models built based on the ABM methodology allow to simulate interactions between individual objects called entities. Four stages must be followed.

1. Define what types of entities exist, creating the diagram and writing the equations that determine their behavior.
2. Define which entities make up each entity type and their particular characteristics, e.g. definition of the initial parameters.
3. Simulate the behavior of the model.
4. Analyze the results.

In a simulation, entities are started using the Entity Initialization Data, and:

1. The triggers are evaluated and the actions of period 0 are performed.
2. The flows are calculated and applied to update the stocks.
3. The stock and flow values for the period are calculated.
4. Triggers are evaluated and do any action of the period.
5. Step 1 is repeated for each period until reaching the final period.

Results

Available options:

- Plot variable - displays a plot of the variable
- Bar Chart Variable - displays a bar chart
- Plot Causes - graphs of the variables that influence the selected one
- Plot Uses - graphs of the dependent variables of the selected one
- View Data - Numerical results in table form
- Causes Data Table - table of variables that influence the selected one
- Causes Tree Diagram - causal tree with the variables that influence the selected one
- Uses Tree Diagram - causal tree of the dependent variables of the selected one
- Causes-Uses Tree Diagram - causal tree of influences and dependencies

Save the model

A model consists of a .vmdl file, and several folders in its same directory: Data, DataSourceMapping, EntityTypes, GridSettings, StyleSheets, and Visualizers. To simulate a model all these elements must be present. All files are in text or XML format.

Logical functions

`==, !=, <, <=, >, >=, !, not, &&`

Comparison operators. Comparisons between two quantities return True or False. "`!=`" and "`<`" both mean different; "`!`" and "not" mean no, "`&&`" and "and" mean and. "`=`" and "`==`" are equivalent.

True, false

Special values that can be used in place of a logical test.

If (C) {X} Else {Y}

Returns X if condition C is true and Y if it is false.

If (C1) {X1} Else If (C2) {X2} ... Else {Y}

Returns X1 if condition C1 is true, X2 if condition C2 is true, and so on. If none of the conditions is true, it returns Y. If more than one condition is true, it returns the first one in the sequence. You can add as many "Else If (condition) {results}" as you like between the initial "If" and the final "Else {result}".

Random numbers

The "seed" value of the random numbers is defined in the Run Control.

RandomUniform (min, max), Random ()

RandomUniform returns a random number evenly distributed between minimum and maximum. Random () is short for RandomUniform (0,1).

RandomNormal (mean, stdev [, min, max])

Returns a random number normally distributed around the mean with standard deviation stdev. With four arguments, the output is limited between minimum and maximum.

RandomBinomial (p, n)

RandomBinomial returns a number from a binomial distribution with probability of success p and n trials.

RandomPoisson (mean [, min, max])

Returns a random number with a Poisson distribution. With three arguments, the output is limited between the minimum and maximum.

Math functions

Min (a, b [, c, d, e]) and Max (a, b [, c, d, e])

Min and Max return the minimum or maximum of up to five values.

Bound (x, a, b)

Bound returns a if $x < a$, x if $a \leq x \leq b$, and b if $x > b$ and is equivalent to $\min(\max(a, x), b)$.

Module (a, b)

The remainder when a is divided by b . Same as the Residual formula in Excel,

Residual (n, d) = $n - d * \text{integer}(n / d)$. Example: Module (179,7) = 4

Sin (a), Cos (a), Tan (a)

Sine, cosine, or tangent of the angle in radians a , which must be dimensionless.

Abs (x)

Abs calculates the absolute value of x , e.g. Abs (-5.7) = 5.7

Exp (x), Pow (b, x)

Exp raises e to the argument, Pow raises the base b to the power x . b^x is the equivalent of pow (b, x).

Xidz (a, b, x) and Zidz (a, b)

Xidz ("x if divided by zero") returns x if $b = 0$, or a / b otherwise. Zidz ("zero if divided by zero") is equivalent to Xidz (a, b, 0).

ValidOrX (value, x) and ValidOrZero (value)

Returns the input value or a specified value if the input is invalid.

Test Inputs

Step (height, time)

It has the value 0 until the time period, then it takes the value defined in height, and keeps it until the end of the simulation.

Ramp (slope, start, stop)

It has the value 0 until the start period, increases with the slope until the stop period, and is a constant value thereafter.

Pulse (start, duration)

It has the value 0 until the start period, then the value 1 in the start period, and 0 thereafter.

PulseTrain (start, duration, interval, stop)

It has the value 0 until the start period, then the value 1 during the periods indicated by duration, the value 0 during the interval, and 0 from the stop period.

String functions for starting attributes

RoundToString (value)

Rounds a value to a whole number and converts it to a string (alphabetical); Useful for initializing an attribute with a sequence number.

Concatenate (value1, value2)

Concatenate two strings without blanks. The string can be "quoted text", another attribute, or RoundToString. The list of values can contain up to four arguments.

Combine (value1, value2)

Combine two strings with a gap.

CombineWithDelimiter ("delimiter", value1, value2)

Combine two strings with a delimiter character or a string.

Reference functions

These functions can be used to construct attribute string values.

EntityExists (collection, entitykey)

Checks if an entity with the given key exists in a collection. Useful to determine if a relationship that joins two entities already exists.

IsActive (entity reference)

Checks if a reference points to an entity that is "alive" (not deleted).

IsNull (entity reference)

Checks if an entity reference is defined (points to a valid entity).

Rounding

Ceiling (x,p), Floor (x,p), Round (x,p), Truncate (x,p)

Floor rounds x down to the next integer, Ceiling rounds up, Truncate rounds to zero, Round rounds to the nearest integer (choosing even if x is in the middle). All of them with the number of decimals (0,1,2,3,...) defined by p.

Floor (5.8) = 5 Floor (5.2) = 5

Ceiling (5.8) = 6 Ceiling (5.2) = 6

Truncate (5.8) = 5 Truncate (5.2) = 5

Round (5.8) = 6 Round (5.2) = 5

Delays

DelayInformation (variable, timedelay, initial value)

Returns the value of the input variable with a delay based on the timedelay variable, such as Vensim's FIXED DELAY. DelayInformation can only be used to calculate based on passed data, and only positive integers are accepted in the timedelay variable. HAND WRITE ALL THIS FUNCTION.

SMOOTH (input, delay time)

SMOOTHI (input, delay time, initial value)

DELAY1 (input, delay time)

DELAY1I (input, delay time, initial value)

DELAY3 (input, delay time)

DELAY3I (input, delay time, initial value)

Returns a first or third order exponential smoothing of the input variable during the delay time with / without initial value.

TREND (input, average time, initial value)

Returns the average rate of growth or decline of the input. The initial value indicates the initial trend of growth or decline.

FORECAST (input, average time, horizon)

Returns the forecast of the input value at a given time and horizon.

NPV (input stream, discount rate, initial value, factor)

NPVE (input stream, discount rate, initial value, factor)

Returns the net present value of the inflow using the discount rate and the initial value. Calculates the current value of the inflow at the end of the period and the discount rate in a discrete period, it is the same calculation sequence as in Excel.

Active Initial

ActiveInitial (main expression, initial expression)

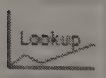
Starts the main expression after the initial expression has been started during model execution.

Classification

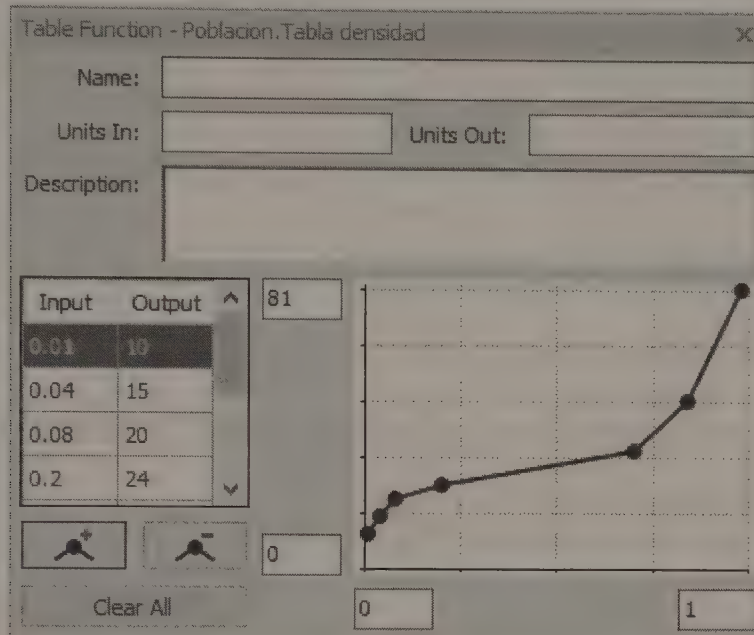
GetCategory (collection reference, variable)

Sort a collection by a variable. Generates a list of the collection, then searches for those that match the parameters set by the variable. This function can only be used in an Action, and to use it it is necessary to have the collection already created, which is referenced through the name of the collection entity and the variable that orders the collection.

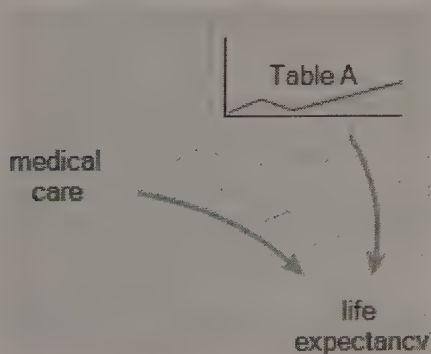
Lookup



Move the Lookup icon from the palette to a diagram and give it a name. Right-click on the icon to edit the function and a table and an X-Y graph will open. The X axis represents the input values and the Y axis shows the corresponding output. Enter the values in the table or indicate them on the graph. The window can be enlarged with the minimum and maximum fields near each axis of the graph.



If the input values (input) are below or above the values indicated in the table, the output value (output) is extrapolated using the slopes defined by the first two points for inputs less than the lowest input and the last two points for values greater than the largest input.



In the diagram draw a table from the variable output (Y) that will also depend on one or more variables input (X). Write the equation of the output variable with the format $Y = \text{table}(X)$, for example:

$$\text{life expectancy} = \text{Table A}(\text{medical care})$$

Once created, a table can also be edited in Data Sources - Builtin, and thus an entry pasted from an Excel in format (x1, y1), (x2, y2) ... etc. is accepted.

Table functions

All Table Functions require a linked table. To link a table, create a helper to hold the function and draw a line from the table to the helper, or in the arguments for the table's function type the name of the table, right-click the underlined name and select "Draw arrow from (table name)".

LookupArea (table)

Find the area under the line of the table function.

LookupAreaByRange (table, x, time)

Starting at the x-value, finds the area under the line of the table function.

LookupExtrapolate (table, x)

Finds the slope of the line, then returns the expected y value for the given value of x using that slope.

LookupForward (table, time)

The next value of y is used between the values of x instead of being interpolated.

LookupBackward (table, time)

The value of y above is kept between the values of x instead of being interpolated.

LookupInvert (table, time)

Reverses the search for x and y values, returning y for x.

LookupSlope (table, time)

Finds the change in y divided by the change in x from one point to the next.

LookupMaxValue (table)

Finds the highest value of and the table.

LookupMaxValueByRange (table, x1, x2)

Finds the highest value of y for the table between the two values of x given (x1, x2).

LookupMinValue (table)

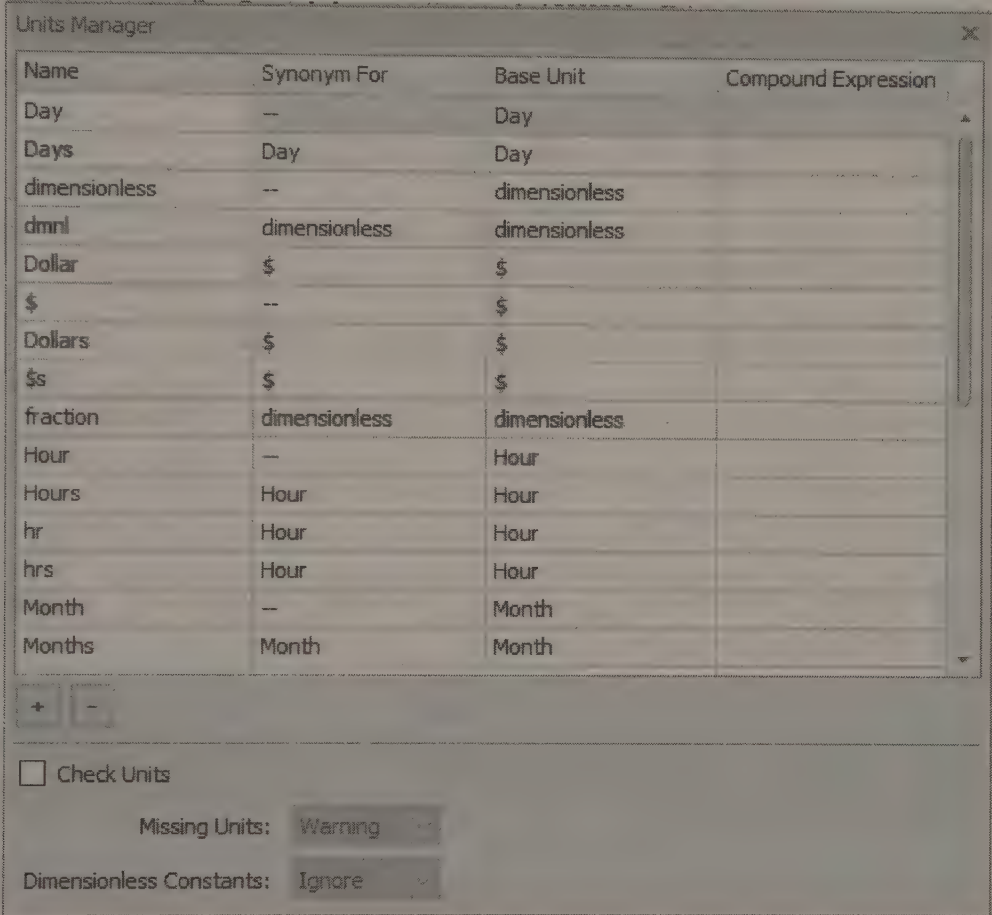
Finds the lowest value of y for the table.

LookupMinValueByRange (table, x1, x2)

Finds the lowest value for y in the table between the two given x values (x1, x2)

Units

Each system variable has its units (liters, kilos, people, ...) and the model can validate that they are consistent with those variables on which it depends. In View - Units Manager you can define synonyms for units and uncheck this validation. If a variable has no units (such as a proportion), you should use dmnl (dimensionless).



Name	Synonym For	Base Unit	Compound Expression
Day	--	Day	
Days	Day	Day	
dimensionless	--	dimensionless	
dmnl	dimensionless	dimensionless	
Dollar	\$	\$	
\$	--	\$	
Dollars	\$	\$	
\$s	\$	\$	
fraction	dimensionless	dimensionless	
Hour	--	Hour	
Hours	Hour	Hour	
hr	Hour	Hour	
hrs	Hour	Hour	
Month	--	Month	
Months	Month	Month	

☐ Check Units

Missing Units: Warning

Dimensionless Constants: Ignore

Synonyms are used to avoid the most common cause of errors in units, which is the use of different names for the same unit: people, employees, workers, etc., except in this case, unit errors indicate inconsistency of equations.

Remember this example

A person is an Entity, and the set of similar entities form an Entity Type. A group of 12 people can also be ONE entity.

The weight of a person is a Stock. The monthly variation of your weight is a Flow. The monthly salary earned by each person is an Auxiliary Variable. The number of working days in the year is a Constant. A Cloud can represent the set of workers in a country when we analyze a single company.

A person's profession ("policeman") is an Attribute.

If the profession influences the salary, we draw an Arrow (causal relationship) from profession to salary.

People can be grouped by profession by creating a Collection.

We can calculate the Aggregate salary of the members of a collection.

The relationship between Professions and Wages is defined with a Relationship.

The attributes defined as Key (the driver's license number) are unique to each entity and can be used to establish relationships between two types of entities.

The values of a variable, such as salary, defined for the start of the model can be modified in subsequent periods either with the salary equation or from the initialization file.

The variables whose values are imported from Excel files cannot be modified during the simulation, for example the geographical coordinates of a house.

- . -

Create successive versions of the model, to be able to resume the previous version as many times as necessary.

Use American notation (points for decimals), MM / DD / YYYY dates, and variable names without accents.

Variables

The simplest models include essential variables to see how an entity works. These models can include Stocks, Flows and some Auxiliary variables which are connected by arrows indicating causal relationships between those elements.

There are no specific rules on how to create a model diagram. The steps below are a framework to get you started to create a model:

1. Take a mental snapshot of the system and the included elements (e.g., people, sq. kms, litres, animals, etc.). These elements will be the stocks of the model.
2. Find or create elements that constitute the 'stock variations' (persons/day, litres/hour, etc.). These are considered the flows of the models.
3. The remaining elements are the auxiliary variables for the model.

The steps above are only a general guide. At a later time, it will be shown how you can make changes to those stocks which will remain constants (e.g., sq m of a room). Now let us take a closer look at how these elements are represented.



Stock

The 'stocks' are those elements showing the state of the model at each instant - stocks present an accumulation and only vary as a function of other elements called 'flows. The 'clouds' in a flow diagram are stocks with inexhaustible content. Stocks in our examples will be represented by a rectangle.

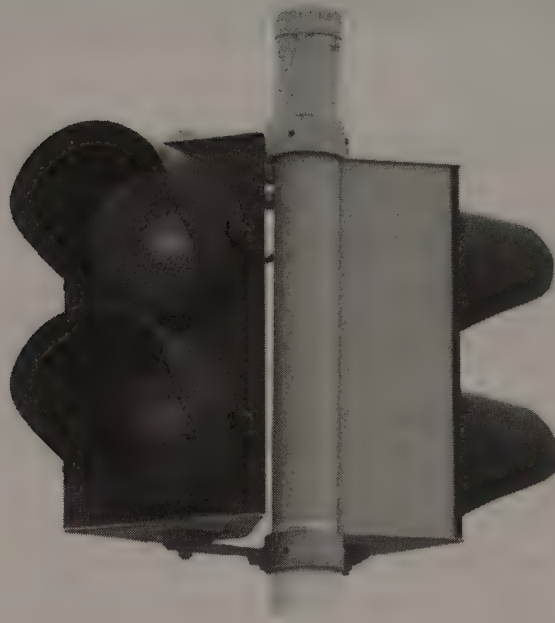
Flows are elements that can be defined as time functions. They can be regarded as representation of decisions taken in the system which determine variations of the stocks.

Auxiliary

'Auxiliary variables' and 'constants' are both parameters which are useful in providing a simpler visualisation of aspects that can affect the behaviour of flows.

The magnitudes between flows and stocks are transmitted through so-called "material channels". On the other hand, "information channels" transmit elements that do not accumulate.

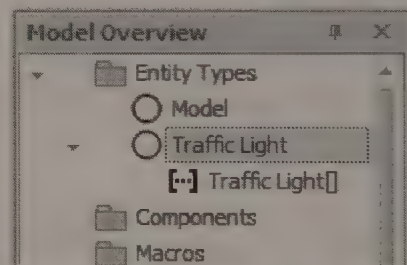
Lastly, "delays" are primarily functions used to simulate the delays occurring in the transmission of materials or information. In social and economic systems, delays are used frequently, and these are of great importance value in the behaviour of the system.



Model: Traffic Light

We are now going to make a model representing the operation of a traffic light with two positions red or green, changing at 20 seconds intervals. The objective of this model is to see the representation structure of a model with the following variables: Stock, Flow and an Auxiliary variable. In this model we have ONE entity, the traffic light.

Open Ventity and Press File - New Model and type Traffic Light to create a model with this name. The type of entity with this name appears in the Model Overview, the empty diagram is shown in the center of the screen.

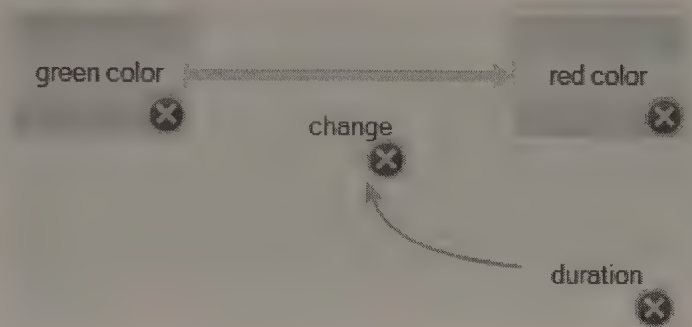


Move the Stock icon to the screen and type "green color" as the name of the Stock, close. Create the "red color" stock to the right of the previous one. Move the cursor over the "green color" rectangle until a hand appears, click on the rectangle and move the arrow that created the "red color" stock.





Upon releasing the mouse button, the flow will be created type the name "change". In all three cases, a red icon is displayed indicating equations have not yet been written. Move the Auxiliary icon to the diagram and name it "duration". Add an arrow from duration to the change flow as shown below.



The stocks will have the value 1 when the traffic light is assigned the same color, green or red, and 0 when that color is off. The duration variable will have the value of the amount of time the traffic light is of that given color, and we will use the change flow to simulate when the traffic light changes color. Keep in mind in this model the entity or agent is ONE traffic light.

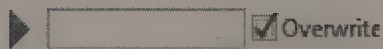
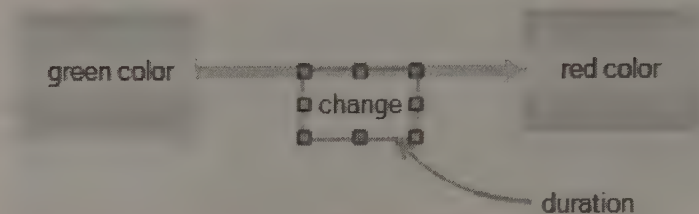
Before writing the equations, go to the top bar View - Units Manager and uncheck the Check Units option, because in this model we will not use the units. In Model Overview, double click on Run Control, the simulation is 120 periods, in Units For Time write "seconds".

Right click on "green color" and type 1 in the initial value and Close the screen. Write the value 0 to the initial value of the red traffic light, to show the color is off. Press on duration and type the value 20.

There are many existing techniques to make a model that reproduces the dynamics of the traffic light, and there are also multiple valid equations. Right-click on "change", choose Edit Flow and type the equation below to activate the traffic light and choose the colors:

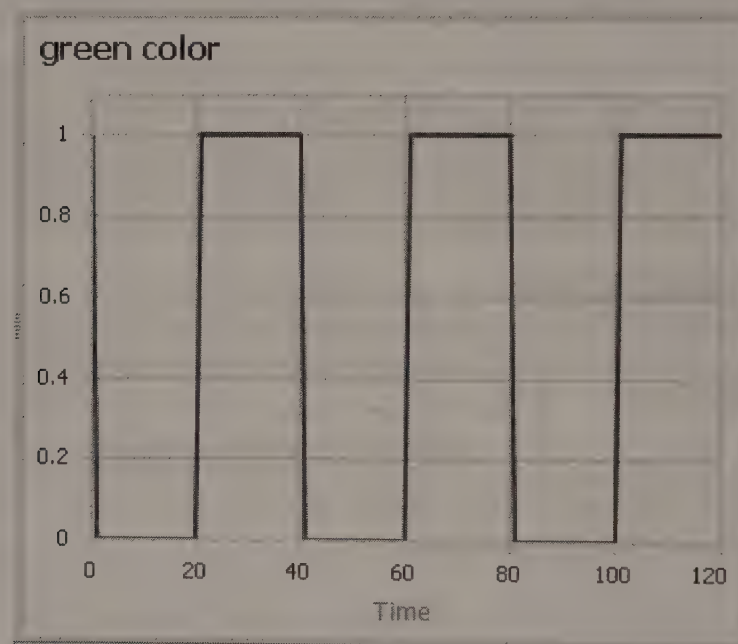
$$\text{shift} = \text{PulseTrain}(0, 1, \text{duration} * 2, 120) - \text{PulseTrain}(\text{duration}, 1, \text{duration} * 2, 120)$$

Type "Pulse" in the window that opens the feasibility to write the equation, various options are displayed, choose the PulseTrain option and fill in the fields. This function will have the value 0 by default and takes the value 1 in the interval indicated. The first Pulse function generates a change and the second Pulse function cancels it.



Simulate the model by clicking the green arrow icon on the top bar.

To see the results of the simulation, right click on the "green color" stock and select the Plot Variable option. The graph shows in which periods the traffic light will be green or red color for example, between intervals of 20 and 40 seconds or between 60 and 80 second intervals etc. which is when the value of this variable equals to 1.



Ensure the "color red" variable takes the value 1 (the traffic light is red) just in the periods that it is not green.



Model: Paris Rome

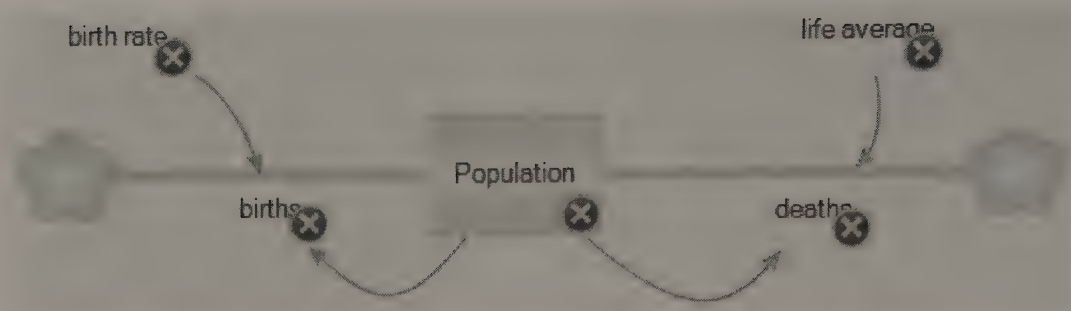
The population of Paris is 2.14 million inhabitants, its annual birth rate is 11.5 per 100,000 inhabitants, and its average life expectancy is 78 years. It is assumed that there are no migrations and the current age distribution of the population is uniform. If the birth rate and life expectancy are kept constant, we will obtain a certain temporal evolution of the number of individuals.

The respective data for Rome is 2.87 million, rate of 12.8 and life expectancy of 80 years. We want to estimate the population 20 years from now.

The objective of the model is to use entities (two) that group many individuals, which are assumed to be homogeneous for the purpose of the study.

Press File - New Model and write the name City, we will not create an entity for each city in this example because the elements to take into account for this calculation will be the same.

Draw the following diagram, with the Population Stock, the Flows of births and deaths, and the Auxiliary variables of birth rate and life average. Add the arrows in order to execute the calculations at a later time.



Edit the variables and transcribe the following equations. This is a generic diagram, valid for any city, and therefore we will place 0 as the initial value for the population and the two constants. Later we will define the values of each city.

Population. initial value: 0 units: people

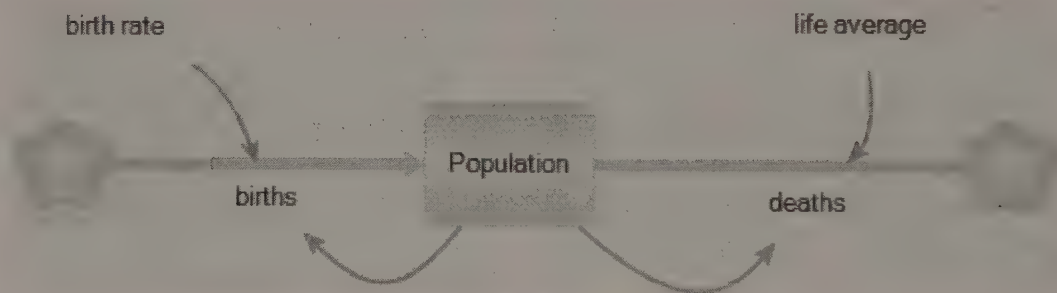
births = Population * birth rate / 1000 units: people / Year (a)

deaths = Population / life expectancy units: people / Year

birth rate = 0 click Constant, units 1 / Year

life expectancy = 0 click Constant, units Year

Press twice on Model Overview - Run Control, on Final Time write 20 and on Units Of Time write Year. If the model shows any warning signs it will be necessary to review the equations or units.



Now we will enter the values of each city. Go to Model Overview - Entity Initialization Data and right click on Data Sources, select Add Builtin Data, in the new window that opens enter the data and close.

City						
	☒	Time	CityID	birth rate	life average	Population
1	☒		Paris	11.5	78	2.14
2	☒		Rome	12.8	80	2.87

Equation (a) stated above is divided by 1000 since the birth rate is per 100,000 inhabitants and the number of inhabitants is in millions.

Simulate the model by clicking the green arrow at the top. To see the result, right-click on Population and in the menu that opens. Select View Data, the annual population figures for each year and city are displayed in table form.

Results: Run - Component: City - Element: Population			
DateTime	Run	CityID	Population
	15 Run	Paris	2,098
	16 Run	Paris	2,095
	17 Run	Paris	2,092
	18 Run	Paris	2,090
	19 Run	Paris	2,087
	20 Run	Paris	2,084
	0 Run	Rome	2,870
	1 Run	Rome	2,871
	2 Run	Rome	2,872

Go to Model Overview and double-click on Results. On the screen shown, double-click the XLSX icon to be able export the results to Excel and be able to use them in other applications. A screen opens to indicate the name and location that is desired for the Excel.

	A	B	C	D	E	F	G
1	DateTime	CityID	birth rate	births	deaths	life average	Population
2		0 Paris	11,50	0,025	0,027	78	2,14
3		1 Paris	11,50	0,025	0,027	78	2,14
4		2 Paris	11,50	0,025	0,027	78	2,13
5		3 Paris	11,50	0,025	0,027	78	2,13
6		4 Paris	11,50	0,024	0,027	78	2,13
7		5 Paris	11,50	0,024	0,027	78	2,13
8		6 Paris	11,50	0,024	0,027	78	2,12

Ensure you save the model.

Attributes

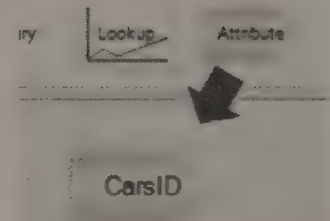
An attribute is a property or characteristic of an entity. Each attribute of an entity has a name (for example, "hair color") and a value (for example, "brown"). Unlike variables, whose values are numbers, the value of an attribute is any combination of letters, numbers, or other characters.

Attributes are used:

- as keys to uniquely identify each entity in the model (Agent 007).
- as discrete values that can be used in equations (28).
- to identify relationships with other entities (city of birth).
- to categorize entities in collections, in order to have added values (born in Oslo).

Create an attribute

To create an attribute on an entity, move the Attribute icon from the palette to the diagram. You must write a descriptive name, such as "hair color", "hometown". If it is going to be a Key attribute (which identifies each entity) it is convenient for the name to end with ID.



Edit an attribute

You can edit an attribute by right-clicking and selecting "Edit", a window opens indicating the Name, which can be modified, in which entity (Owner) is located, which is the same entity of the diagram where it is located. Additional fields can also be found in the attribute window.

The screenshot shows a dialog box titled 'Attribute - Houses.price'. It contains the following fields and options:

Name:	price
Owner:	Houses
Refers to:	<none>
	☐ Key
Initial Value:	"good"

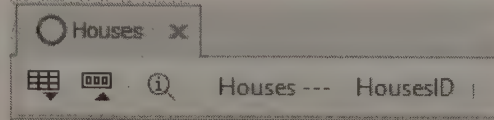
Refers to

If the attribute describes a property with another entity, the "Refers to" menu is used to select that other entity. For example: the entity "car" may have an attribute called "owner", which refers to an entity of type "person" <none> is chosen if the attribute is not linked to any entity.

Placing an attribute of an A entity in the diagram of a B entity, allows defining in the Entity Initialization Data - Data Sources a link between each entity of type B with each entity of type A.

Key

Each entity (except singular entities) must have a unique key to identify it. By default, a key is created for each entity. A key is a single attribute value or a unique combination of attribute values. The "name" attribute can be a suitable key for entities of restaurants types, as long as there are no two restaurants with the same name. The license plate of a car is a good key for cars as well because each car has its own unique license plate.



When creating a new entity, an attribute is automatically created for that entity to use as a key. This attribute has the same name as the entity followed by ID. For example, the Rabbit entity will have an attribute key called RabbitID. Each rabbit will have a unique value for its unique rabbit ID, which will be rabbit-1, rabbit-2, rabbit-3 etc.

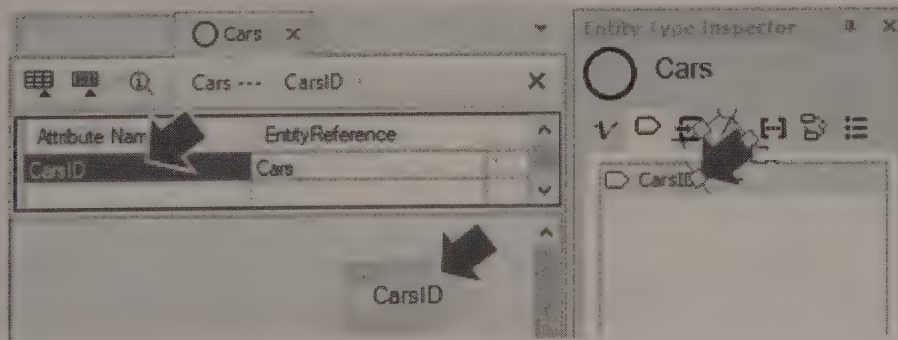
To define your own attribute names, choose a single attribute to use as a key. Check the Key box for that attribute and make sure no other attributes for that entity for that key has been checked. A composite key attribute can be created with up to four attributes. The key attributes for an entity type are listed at the top of each diagram for that specific entity type.

Initial value

The initial value of the attribute is set during initialization. However, if no value is provided during initialization, the initial value is determined from what you enter in the Initial Value field. It can be a string in quotes. For example, each new insect entity begins with the current stage of the "egg" life cycle, or it can be an expression. A different scenario if $(\text{random}() < 0.5) \{ \text{"blue"} \}$ else $\{ \text{"white"} \}$. In this case once the simulation has started; the value of an attribute can only be changed by an "action".

Display

The attributes of an entity are visible in three places: The attribute grid at the top of each diagram, in diagrams, and in the Attributes tab of the entity inspector.



Summary

Each type of entity has a key attribute (Key) which helps to uniquely identify each entity of the same type.

Attributes can contain discrete values of state (as opposed to continuous values, for which variables or actions are used), which can be used in equations whose output depends on conditions. For example, `fun = if ("hair color" == "black") {1} else {0}`.

If an attribute is empty, the attribute will receive the value "" (a string of length 0). This can be used to see if a value has been assigned:

```
hair status=if (hair color=="") {"data missing"} else {"follow"}
```

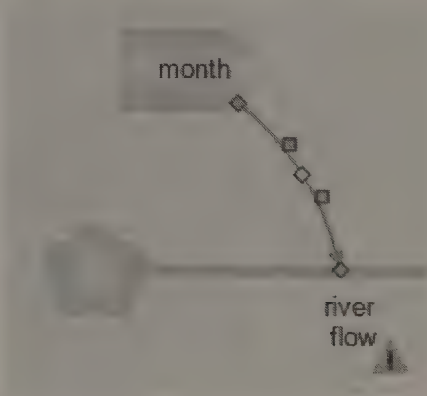
Attributes can be used to group entities into collections in order to calculate aggregate values (such as the attribute example given on the rabbit entity type i.e., rabbit-1, rabbit-2, rabbit-3 etc.).

Attributes can be used to establish relationships with other entities. Information about those related entities is defined through references.

Remember Attributes can ONLY be modified through actions.

Model: Basic River 2

We are going to take the River model created before adding the condition that if it is January the flow is 5 and the rest of the year the flow is 0.



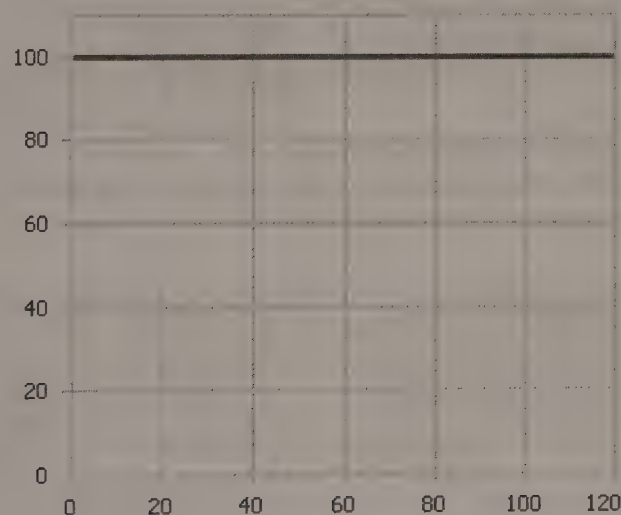
Open the model and move the Attribute icon to the diagram, as name write "month", press enter. Add an arrow from the Attribute to the Flow.

Right click on the "month" attribute, choose Edit Attribute, write "June" (in quotation marks) and close.

Right click on the Flow "river flow" and choose Edit Flow, replace the value 5 with this equation = if (month = "January") {5} else {0} which makes the month condition true = January the value of the flow is 5, if it is not fulfilled the value is 0. In this case it is not fulfilled, because in the attribute we have defined June.

Simulate the model. The flow is always 0 and the content of the dam remains constant at 100.

dam content



In summary, an attribute is an alphabetic variable instead of a numeric one, with the nuance that it cannot depend on other variables, but is modified through actions.



Model: Truck Fleet

This model shows the process of how to include the one-to-one relationships between two entities of the model. In this example we will use the Attributes. The goal of creating such a relationship is for one entity to be able to use the information from another entity.

In this case, the model will capture the one-to-one relationship between a truck with only one driver, and each driver will only drive one truck at a time.

Create a new model (File-New Model) named "Trucks". In Model Overview (left side) - Entity Types press the right button and select New Entity Type name the entity "Drivers" and press OK. Check that the entities Trucks and Drivers appear in the Model Overview at the top of the diagram.

In the Drivers diagram create the "salary" variable by dragging the Auxiliary icon. Then Edit Auxiliary by right click, name variable salary and type 1000 euros, click Constant.

Go to the Trucks diagram, repeat the steps above and create another variable and name it “consumption”. Units will be litre/km and value 0.12. Click Constant.

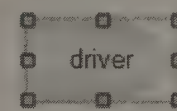
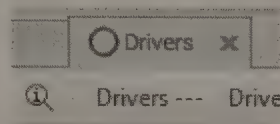
Attributes

We want to be able to use the Truck variables in the Drivers diagram and vice versa, in order for this to be possible we will need to add attributes of each one to the other. Go Trucks diagram and create an attribute named "driver", to do this by dragging the Attribute icon to the screen and the name the attribute driver. Repeat the process and name the attribute "trucks".

Right click on the driver attribute to Edit Attribute, under Refers select Trucks. Link the trucks attribute with the Drivers entity.

Auxiliary - Trucks.consumption

Name:	consumption
Owner:	Trucks
Units:	liter / km
☒ Constant	
	0.12



salary

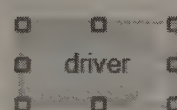
consumption

trucks

salary

Attribute - Drivers.trucks

Name:	trucks
Owner:	Drivers
Refers to:	Trucks
☐ Key	

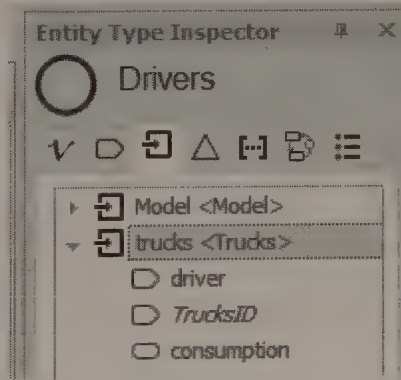
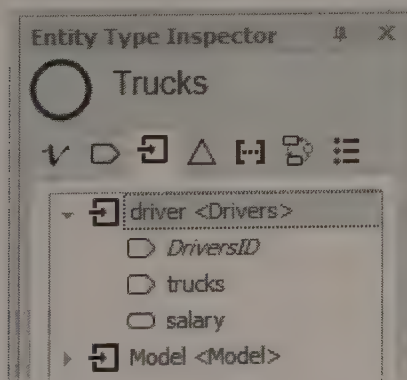


consumption

Attribute - Trucks.driver

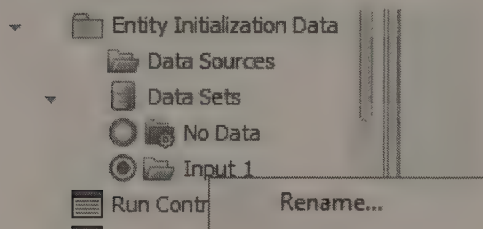
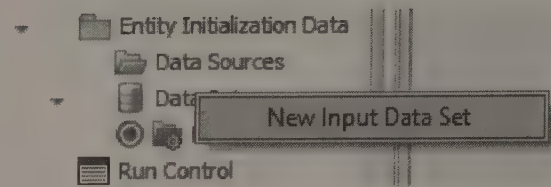
Name:	driver
Owner:	Trucks
Refers to:	Drivers
☐ Key	

This has created references between the two entities, visible in the reference tab of each inspector. If the inspector is not visible, go to Model Overview, select an entity, and click Inspect.



These references allow each truck entity to use driver variables such as salary, and driver entity equations use truck variables such as consumption.

The next step is to establish the pairs during initialization, matching the truck IDs with the driver IDs. In this example, we have initialized two truck entities, Ford and Toyota. Go to Data Sets, right-click, choose New Input Data Set. Input 1 is created in the dataset list.

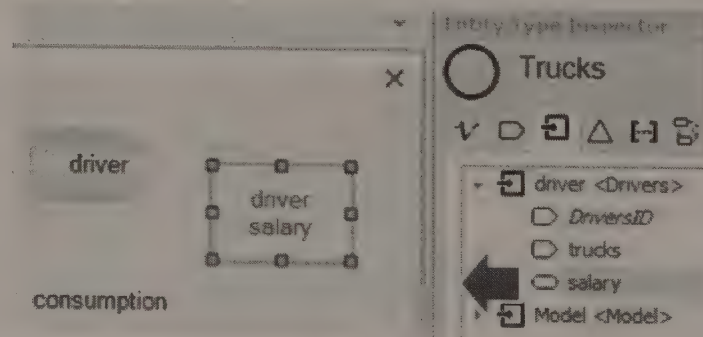


Right click on Input 1 and select Rename, write "one by one". Select, and choose Add Builin data Entities 1 is created, in the Trucks tab indicate the truck IDs the driver data, and conversely in the Drivers tab indicate the driver IDs and the truck data. Close the two tabs. Save the model.

Entities 1						
Drivers		Model		Trucks		
	☒	Time	DriversID	trucks	salary	Cal
1	☒		Antonio	Ford		
2	☒		Luis	Toyota		

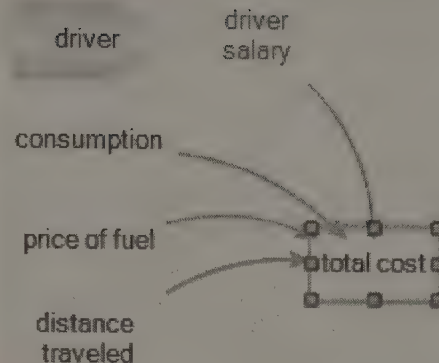
Drivers		Model		Trucks		
	☒	Time	TrucksID	driver	consumption	
1	☒		Ford	Antonio		
2	☒		Toyota	Luis		

From now on, the entities of the types trucks and drivers are linked in a one-to-one way. Go to the Entity type Trucks inspector and from the References tab move the Driver salary attribute to the Trucks screen.



Add the variables "price of fuel" (1.50 euros/liter) and "distance traveled" each period (1500 km) and "total cost" (euros), which is the sum of the driver's salary plus the cost of fuel (price x consumption x distance).

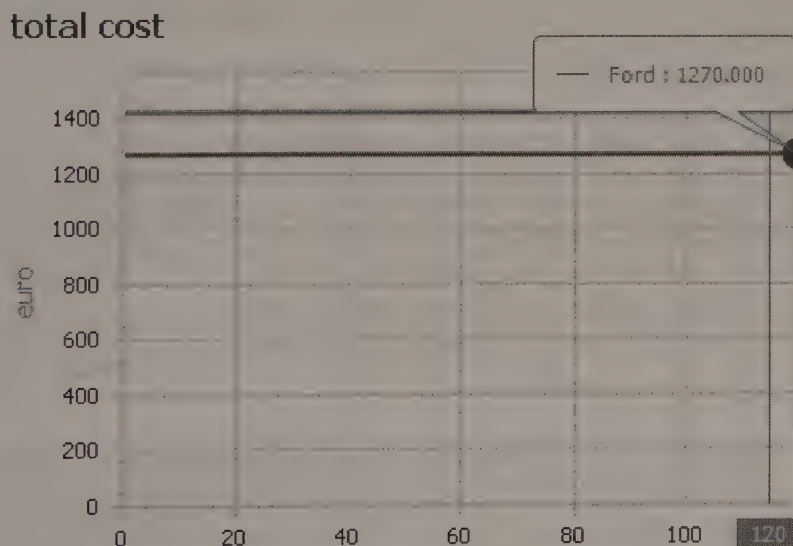
In Data Sets - Entities 1, go to the Trucks tab and enter the consumption of each truck and the price of the fuel it uses. In the driver's tab input the salary amounts.



Entities 1						
Drivers			Trucks			
	☒	Time	TrucksID	driver	consumption	Calc
1	☒		Ford	Antonio	0.12	
2	☒		Toyota	Luis	0.1	

Drivers			Trucks			
	☒	Time	DriversID	trucks	salary	Calc
1	☒		Antonio	Ford	1000	
2	☒		Luis	Toyota	1200	

- Simulate, select "total cost", press the right button and choose Plot Variable, the graph shows the cost of each truck, including the driver's salary.



Collections and aggregates

Sometimes an equation requires information not about a single entity (house), but about a group of entities (expensive houses). These variables are called aggregate variables. An aggregated variable could be the number of "horse" entities at the end of each month. Or the average height of all "young tree" entities. Or the maximum of the "income generated" by the sellers in the North zone. All these aggregates are based on creating collections or groups of entities.

A collection is therefore a set of entities of the same type. A collection can be very inclusive (all entities of type e.g., "horse"), or entities can be divided into sub-collections based on their attributes (black horses). Splitting attributes of "color" and "race" would result in multiple sub-collections.

The collections may be empty: for example, we may not have black dwarf horses at the beginning of the model, but can be added at a later time.

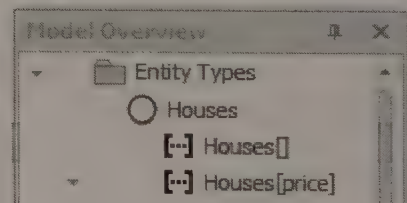
In an Agent Based Models (ABMs) the number of elements can be very large but not all of them are absolutely different, in general ABMs tend to share some properties. The creation of collections of elements facilitates the understanding of the behaviour and the final conclusions from the model.

Complete collections

The most basic collection comprises ALL entities of a given type. Collections are enclosed in brackets [] with the entity name next to it. If there is nothing in the brackets, the collection includes all entities of that type. The collection of all entities of a type is automatically defined when the entity type is created, and can be viewed in the Collections tab in the entity type inspector.

Sub-collections

When you want to make calculations of a subset of all the entities of a type, sub-collections are created, these are defined by the attributes of the entities (not by their values or parameters), for example their color (but not their weight), or the attribute price (good, bad, expensive...).



Later, during the practical exercises chapter the creation of simulation models using ABM is shown in detail, but for now we will create some basic models to help in the integration of concepts.



Model: Dragons and Castles

There are 4 dragons living in 2 castles the green dragons are determined can only eat corn. The red dragons are determined only can eat meat. In this example we want to calculate how many kilos of corn it will be necessary to bring to each castle monthly with the following given data.

Open the software. In the upper part of the screen make File - New Model name the model "Dragons", in the Model Overview - Entity Types an entity type with the same name is created. Then right click on Entity Types, choose New Entity Type and write "Castles". You have defined the two types of entities: Dragons and Castles.

In Model Overview, click twice on Run Control, in Final Time type 12, and in Units for Time = Month. Close.

Save List	Using Calendar Time	Initial Time	Final Time	Initial Calendar Time	Final Calendar Time	Time Step	Unit Of Time
None (Saves Everything)	☐	0	12	10/09/2020	02/09/2021	1 (13 tot...	Month

It was determined that the amount and type of food of each dragon depends on its unique weight and color. Only green dragons eat corn. We will create an attribute to specify the color of each dragon.

Attribute

In the Dragons diagram, move the Attribute icon to the diagram, right-click on it, choose Edit Attribute and write the name "color".

color

Attribute - Dragons.castle

Name:

Owner:

Refers to:

☐ Key

Initial Value:

Once complete create another attribute and name it "castle" thus, we can later specify in which castle each dragon lives. Once this attribute is created, click on it with the right button and in the Refers to select Castles. This indicates it is an attribute that refers to the Castles entity, not Dragons.

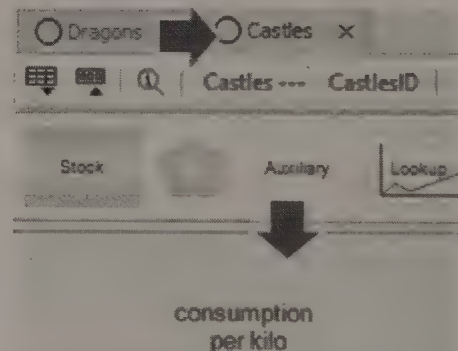
Move the Auxiliary icon to the Dragons diagram and create the variable "weight", press the right button, choose Edit, and set the value to 150 and units kg. Click Constant. This value will be used by default if the weight of each dragon is not specified.

color

weight

castle

At the top of the diagram, click on Castles to open the diagram, move the Auxiliary icon to the centre of the diagram and type the name "consumption per kilo", click on the variable with the right button, choose Edit Auxiliary. Enter quantity 2, units' kg/kg/month, and click Constant. Consequently, we specified that each dragon eats twice its weight each month.



We have placed the variable "consumption per kilo" in the entity Castles and not in Dragons because they tell us that the eating consumption depends on the castle where each dragon lives on.

We already defined some parameters for the two types of entities dragons and castles. Now we are going to identify each of the entities, which are the 4 dragons and the 2 castles.

Display the Model Overview - Entity Initialization Data options and right-click on Data Sources, choose the Add Builtin Data option, Entities 1 is created, which is the place where we will enter the initial values of the variables and attributes.

Once entities 1 has been created, click twice on the name, and in the window that opens check and ensure there are two tabs: one for Dragons and the other for Castles. If not shown check Show All Types. Go to the Dragons tab and enter the following data as shown below. Once complete go to the Castles tab and enter the rest of the data.

Entities 1

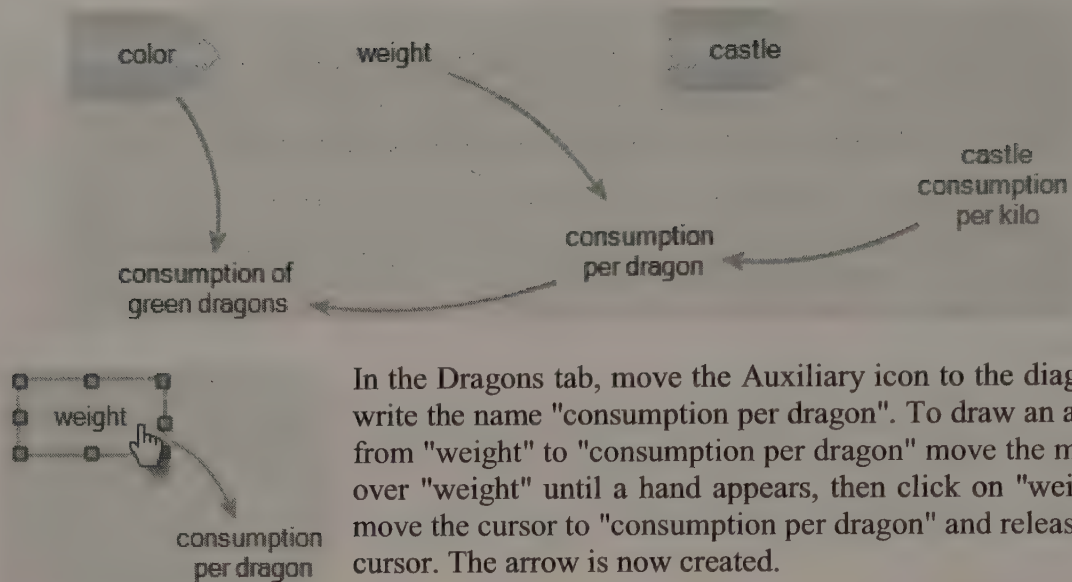
Number of rows to add:       All None Invert ☐ Show

Castles				Dragons			
	☒	Time	CalendarTime	DragonsID	color	weight	castle
▶ 1	☒	0		Neron	green	140	Segovia
2	☒	3		Galba	green	100	Segovia
3	☒	6		Trajano	red	130	Segovia
4	☒	9		Tiberio	green	90	Avila
*	☐						

Castles				Dragons	
	☒	Time	CalendarTime	CastlesID	consumption per kilo
▶ 1	☒			Segovia	2
2	☒			Avila	4

In DragonsID type the name of each dragon in the other columns type the specific color, the weight of each one and the castle where each dragon lives on. In the Time column specify the period in which that entity is created (the month when each dragon begins to live in the castle), since it was established the dragons will arrive to the castle every 3 months.

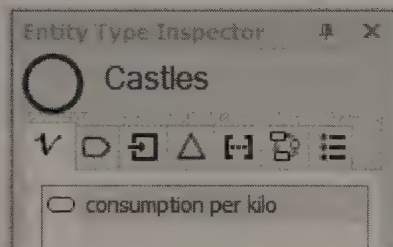
In CastlesID write the name of the castles, and the consumption of corn per kilo of weight of the dragon (keep in mind only green dragons eat corn) Close that window. Now you need to do some calculations. In the Dragons diagram, create this diagram as explained.



In the Dragons tab, move the Auxiliary icon to the diagram, write the name "consumption per dragon". To draw an arrow from "weight" to "consumption per dragon" move the mouse over "weight" until a hand appears, then click on "weight", move the cursor to "consumption per dragon" and release the cursor. The arrow is now created.

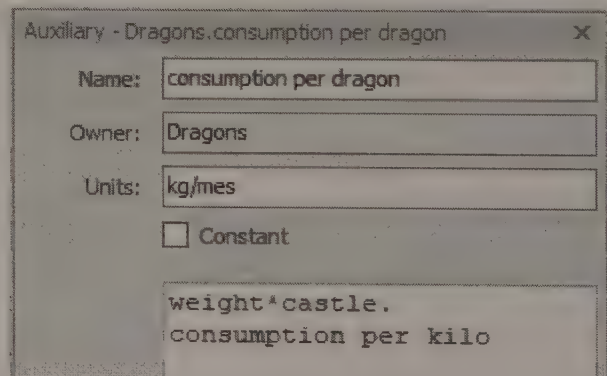
Arrows are placed because in the calculation of the destination variable ("consumption per dragon") the source variable intervenes ("weight"). Subsequently, we will write the equations of each variable.

Repeat the same process to create the variable "consumption of green dragons", and add the arrows from the attribute "color" as well as the variable "consumption per dragon".



Go to the Model Overview and right-click on Castles, choose the Inspect option. This opens the inspector on the right side of the screen. In the Variables tab the "consumption per kilo" is shown move it to the Dragons screen. A shadow variable (in green) is created with the name "castle consumption per kilo".

Right click on the variable "consumption per dragon" and select Edit Auxiliary. In the window that opens type the units. To write the equation, click on the area to the right of the "=", write the first letter of the variable, a menu is displayed with all the variables, attributes and functions that start with that letter, choose the correct variable by clicking twice on it and complete the equation.

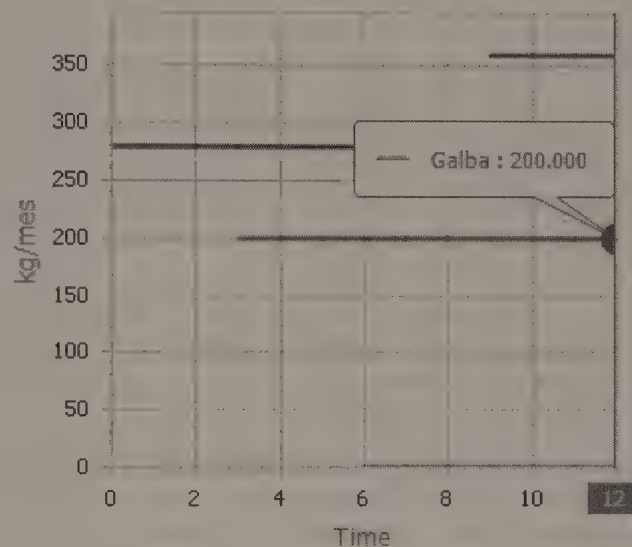


Then click on the variable "green dragons consumption", and write the equation = if (color = "green") {dragon consumption} else {0}. In this equation we specify that if the color of the dragon is green, the corn consumption is calculated in the variable "consumption per dragon", otherwise (if the dragon is red), the value is 0 (it eats another person or thing).

If you have problems writing an equation write it in a text editor, and copy and paste in the area of the equation.

To simulate the model, go to the upper bar and click on the green arrow (Run Model). Then, with the right button, click on "green dragons' consumption" and select Plot Variable. The graph of the consumption of each dragon is shown, and the month in which it begins. Click on the line to see the dragon's name.

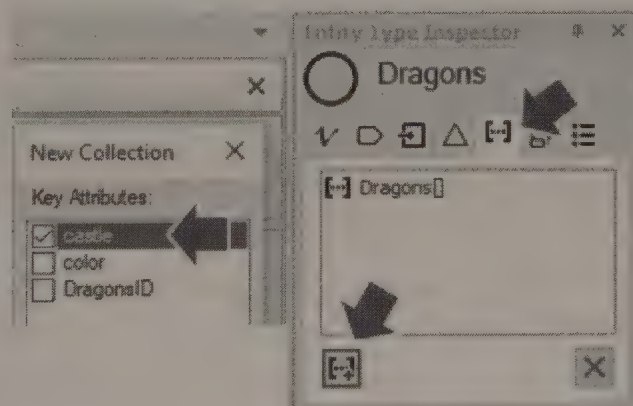
consumption of green dragons

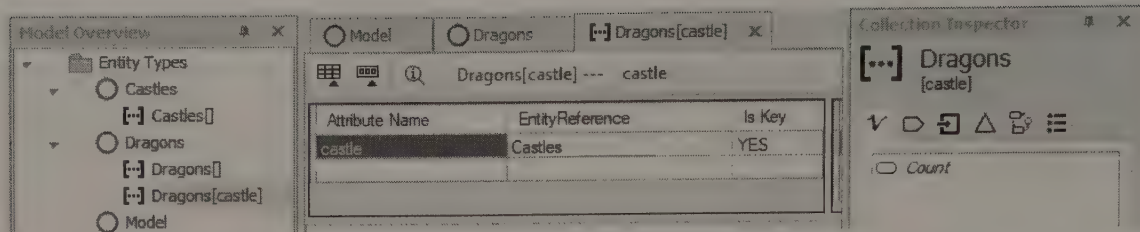


This information is of interest nonetheless, we need to know how much corn we should bring each month to each castle, and for this reason we will create a new entity to collect information grouped from dragons and castles.

Go to the Dragons inspector, open the Collections tab. At the bottom click on the "New Collection" icon. In the new window that opens, click on the castle option, since the intention is to have the dragons grouped by castles. Close.

A new collection will be created that will appear as Dragons [castle] (read "dragons grouped by castle"). This collection represents the entire population of dragons divided into subsets by the castle where the dragons live. You will also see a new inspector for the Dragons [castle] collection, which is used to view and set the properties of the new collection.

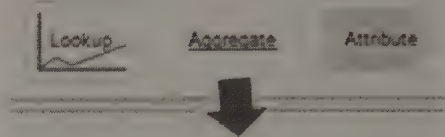




Subcollections can be created using more than one attribute. Selecting "color" and "castle" as key attributes of a sub-collection would create the Dragons [color, castle] collection.

Aggregates

Collections are also entities and can be found in the Model Overview. Collections can be viewed in the Entity inspectors and most importantly, their variables, attributes and references can be added.



Collections can also have diagrams showing the structure of any of their properties. The variable Count is automatically included with each collection and reports the number of existing entities of each attribute at any given time.

The variable "aggregates" are available in collections. If the dragon entity has a variable called "consumption", any collection of dragon entities can provide the average, sum, maximum, minimum, median, product, or standard deviation of the consumption of the dragons in that collection.

Aggregate - Dragons[castle].Sum consumption of green dragons

Name:	Sum consumption of green dragons
Owner:	Dragons[castle]
Units:	kg/mes
Base Type:	Dragons
Aggregation Function:	Sum
Target Variable:	consumption of green dragons

Go to the Dragons [castle] diagram by double clicking on this entity in the Model Overview.

Move the Aggregate icon from the palette to the diagram. Right click on the variable to edit it and select the Aggregation Function (Sum) and the Target Variable to add, in this case the consumption of green dragons.

This result is the total weight of the green dragons that live in the castle. Variables that are aggregations are named automatically, according to their definition. For example, in this case it would be named "Sum consumption of green dragons"

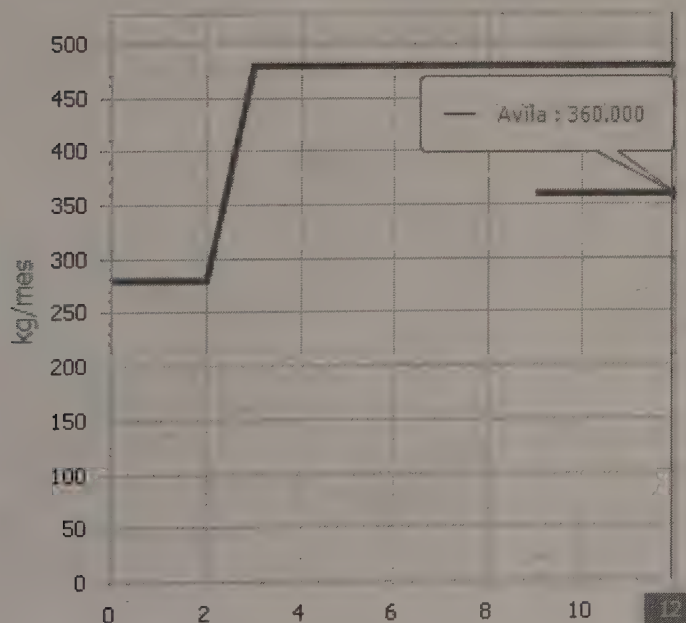
If there are no entities in the collection, the Min, Max, Average, StDev, and Median functions will return NaN, and Count, Sum, and Product will return zero.

Results: Run - Component: Dragons[castle] - Element: Sum

DateTime	Run	castle	Sum consumption of green dragons
0	Run	Segovia	280
1	Run	Segovia	280
2	Run	Segovia	280
3	Run	Segovia	480
4	Run	Segovia	480
5	Run	Segovia	480
6	Run	Segovia	480

Simulate the model and then right-click on the variable "Sum consumption of green dragons", choose the View Data option; it shows us a table with the monthly values of corn consumption for each castle.

Sum consumption of green dragons



Click on Plot Variable. The graph shows those same values in a descriptive way, it is for this reason we are able to determine the castle of Segovia will need 280 kilos of corn in the first month. While the castle of Avila will need 360 kilos at month 9.

Save the model



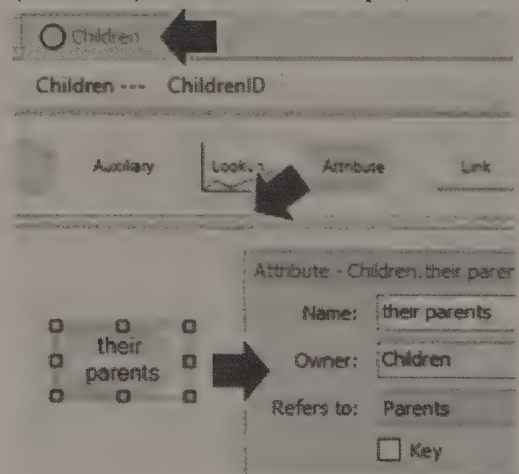
Model: Parents and Children

Adding by an attribute of another entity

You can create entities that are the aggregation of entities of type A, e.g. children, based on entities of another type B, e.g. parents. For this you have to place an attribute of entity B (Parents) in the entity A (Children); thus, for example, we can calculate the number of children that each entity of the type parents has.

Create a model with File - New Model with the name Parents. Then create a new type of entity (Entity Type - New Entity Type) with the name Children.

In the Children diagram move the Attribute icon to the drawing area, press edit, write the name "their parents" and in Refers to choose the Parents option. In this way, we can now go to the initialization menu (Entity



Initialization Data) and specify which parents each child has.

Go to Model Overview, right click on Entity Initialization Data - Data Sources - Add Builtin Data. In the new window click Show All Types, go to the Parents tab and write their names. Then go to the Children tab and enter their respective parents.

We will use the Time column to specify the year of birth. Close the window.

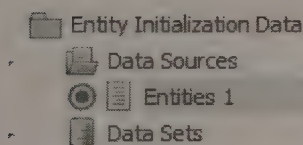
Entities 1

Number of rows to add: All None Invert

Children		Time	CalendarTime	ChildrenID	their parents
▶ 1	☒	2015		Borja	Rosa y Juan
2	☒	2019		Alfonso	Rosa y Juan
3	☒	2019		Carlos	Ana y Pedro
4	☒	2020		Luisa	Ana y Pedro
*	☐				

Children		Time	CalendarTime	ParentsID
▶ 1	☒			Rosa y Juan
2	☒			Ana y Pedro

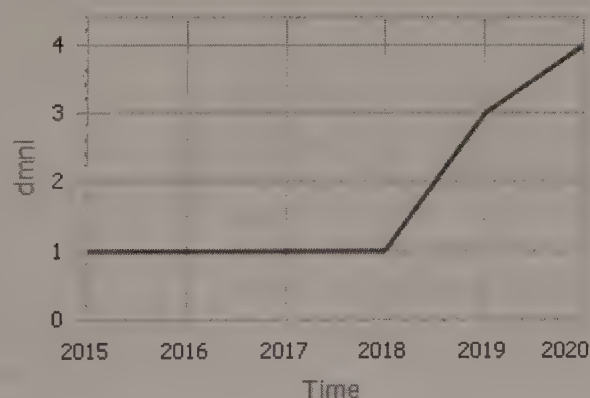
In Model Overview click twice on Run Control, write Units for time = year, Final Time = 2020 and Initial Time = 2015 (in this order), close the window.



Check that the Builtin Data that has been created is active, and if it is not, click it.

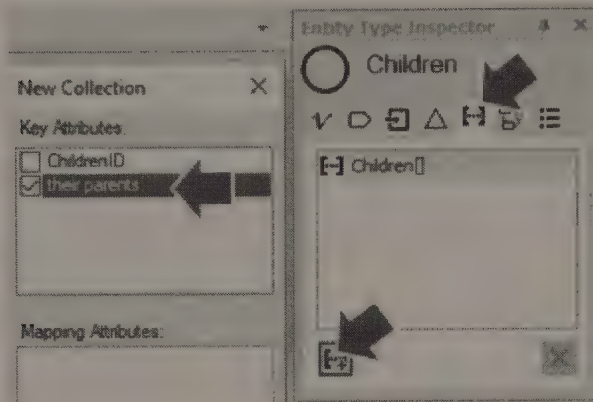
Simulate with the green arrow on the top bar. Then, in the Model Overview - Entity Types, display Children and double click on Children [], it is a collection that only has a Count variable that acts as a counter. Go to the inspector that has been opened on the right of the screen, right-click on Count and choose Plot Variable.

Count



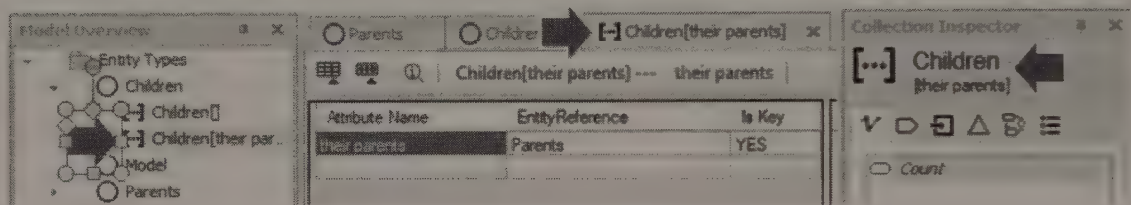
The graph shows the total number of children each year, according to the years of birth that have been defined previously.

Next, we are going to calculate how many children each family has or set of parents have. To do this go to the Children inspector (by clicking twice on Children in the Model Overview), select the Collections tab and click the New Collections icon at the bottom.



Choose the option "their parents".

A collection named Children [their parents] has been created. A new inspector, a new diagram and a link appear in Entity Types displaying the Children entity (since it is Children information).

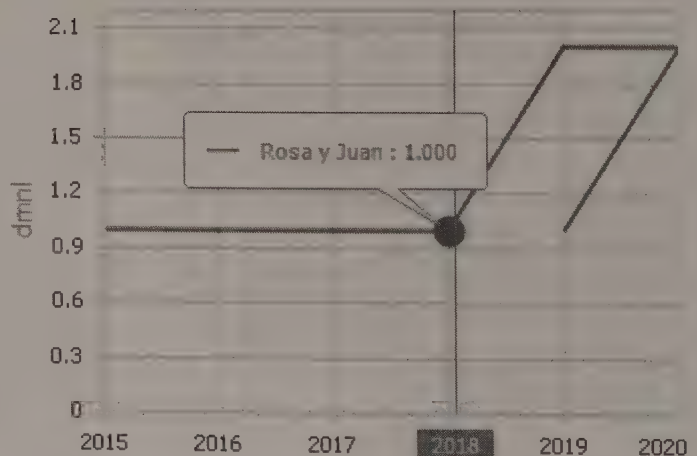


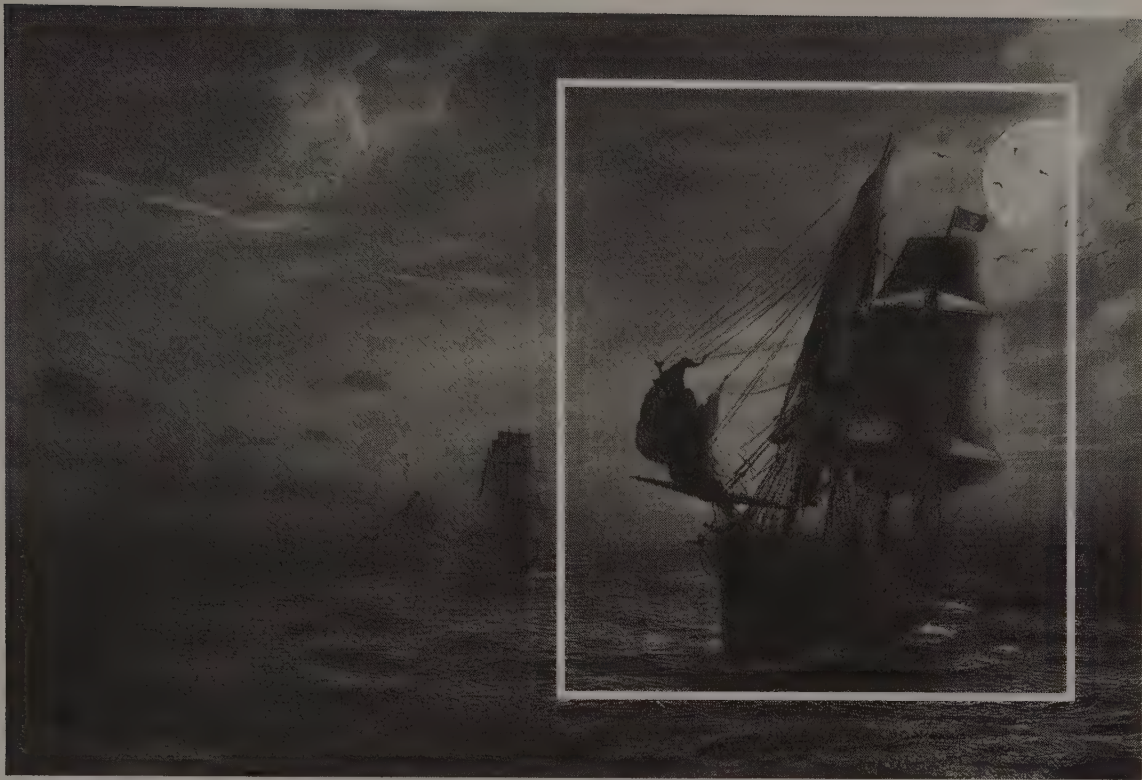
Simulate the model. In the Inspector of Children [their parents] right click on Count and then Plot Variable.

Count

The graph shows how both families reach the year 2020 with two children, starting with one child in 2015 (the family of Rosa and Juan), and the other starting in 2019. In that same menu, the View Data option shows the results in numerical format.

Save the model.





Model: The Four Pirates

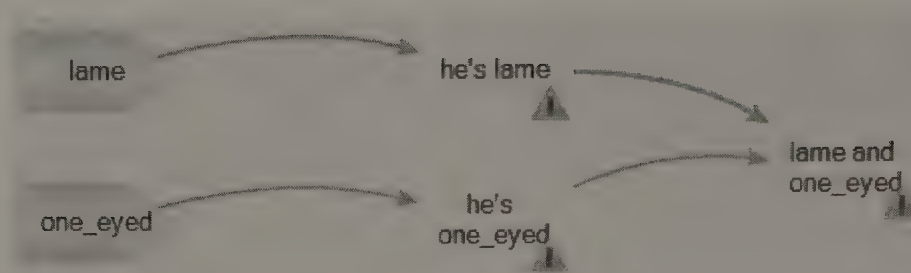
From a group of pirates (Morgan, Drake, Calico and Hawkins) we want to know how many of them and in what years they were lame and one-eyed. We have these annual data:

	lame	one-eyed
Morgan	1570	1590
Drake	1520	1510
Calico	1580	1580
Hawkins	1530	1510

Instructions:

1. Create a model named "Pirates", this creates a type of entity with this name.
2. In Run Control write Final Time 1600, Initial Time = 1500 and Units for Time = year
3. Create two attributes: lame and one-eyed.
4. Press Add Builtin Data, enter the names as PiratesID, in Time enter the years between 1500 and 1600 in which the pirate is lamed or one-eyed, and in the attribute columns enter "yes" on those dates (see next page).

Draw the model diagram



The yellow triangles show that we have not entered units. To uncheck checking, go to View - Units Manager and uncheck Check Units.

In the Builtin Data write the year in which the entity (Pirates) is registered as lame or one-eyed. The name of both attributes appears, write "yes" in the relevant cell (as below).

Entities 1

Number of rows to add:



Pirates

	☒	Time	CalendarTime	PiratesID	lame	one_eyed
▶ 1	☒	1570		Morgan	yes	
2	☒	1590		Morgan		yes
3	☒	1510		Drake		yes
4	☒	1520		Drake	yes	
5	☒	1580		Calico	yes	yes
6	☒	1510		Hawkins		yes
7	☒	1530		Hawkins	yes	

The corresponding equations are:

he's lame = if (lame = "yes") {1} else {0}

he's one_eyed = if (one_eyed = "yes") {1} else {0}

lame and one_eyed = he's lame * he's one_eyed

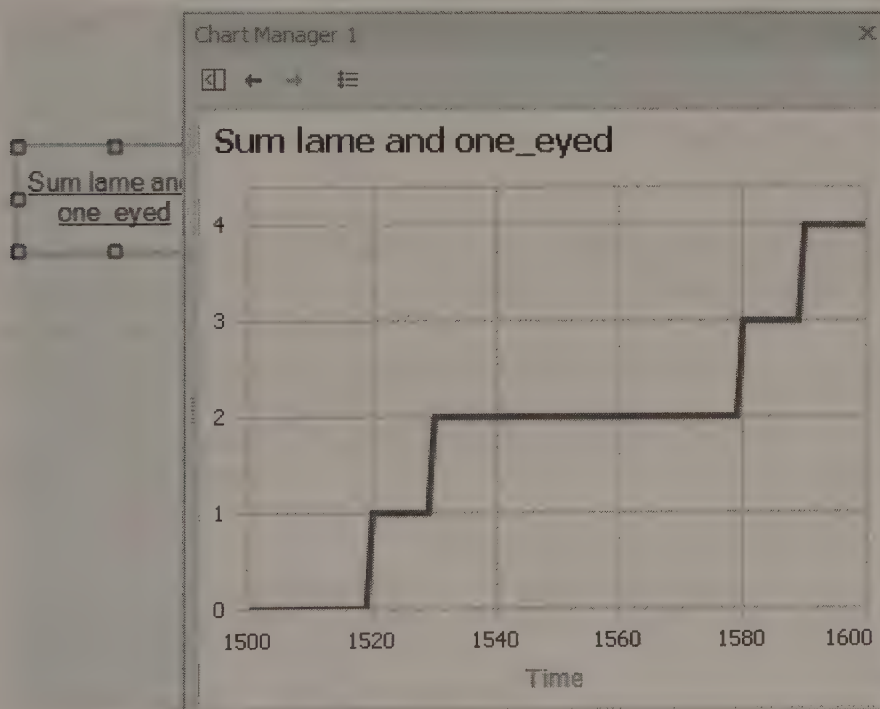
We transform the literal "if" in 1, so that both conditions are fulfilled, and the simplest thing is to multiply both binary states. The result is 1 (1x1=1) in that case.

In Model Overview display the Pirates entity type. It shows Pirates [] is created by default. Click twice on Pirates [] to open the diagram, move the Aggregate cursor to the diagram, choose the Sum option as Aggregation Function and the "lame and

one_eyed" variable in Target Variable. The Name is created automatically from that selection.

Aggregate - Pirates[], Sum lame and one_eyed	
Name:	Sum lame and one_eyed
Owner:	Pirates[]
Units:	
Base Type:	Pirates
Aggregation Function:	Sum
Target Variable:	lame and one_eyed

Simulate the model, right-click on "Sum lame and one-eyed", then in Plot Variable, a graph is displayed that shows the annual value of the pirates that meet both conditions.



Summary of Attributes

Attribute >

Key

Refers to

An attribute points to a characteristic, e.g. hair color = gray, of an entity, e.g. person, which can be common to other entities, e.g. people with gray hair.

We can create a collection of entities that have the same attribute, e.g. people with gray hair color.

A Key attribute points to an attribute that is unique for each of the entities, e.g. passport number.

The Refers to element of an attribute notes that the attribute is characteristic only of that type of entity, e.g. hair color refers to people, or that this attribute, e.g. car brand, refers to another type of entity, e.g. cars. As a result, we can identify, for example, which people with a Toyota brand car have gray hair.

Attributes cannot be modified with equations but only with "actions".

References

A reference is a link between two types of entities – a source (S) and destination (D). Information from the source (S) entity -or a collection- is used as an input in a calculation of another entity (D). The source entity of the information (S) must be specified, and what information will be used.

How to create a reference

1. Place a variable of entity S (source) in the diagram of entity D (destination)
2. Create in D an attribute X with Refers to S
3. Go to the inspector of entity type D, open the References tab edit S <S>
4. The Target Type is the type of entity S, in Value to Match write the name of the attribute X of the entity S (the attribute can be the Key or not if it is the aggregate of a collection).

Steps to follow

References are created between two types of entities; one entity is the source (S) of the data and another is the destination (D). There are several possibilities as listed below.

Add to entity D1 of entity type D, the value of an aggregate of all entities type S

Comment: We are adding in diagram D the value of a variable of entity D1 with the value of an aggregate of all entities S.

1. Enter in Initialization Data the values of the entities of S and D
2. Create an aggregate variable of the entities from S to S []
3. Place the aggregate variable of S in the entity type diagram D
4. Write equations and simulate

Example: Black and White Model 3

Add to entity D1 of entity type D, the value of entity O1 of type O

Comment: We are adding in diagram D the value of a variable from entity D1 with the value of another variable from entity S1.

1. Place a variable of entity S1 in the diagram of D
2. Create in D an attribute X with Refers to S
3. Enter in Initialization Data the values of S1 and D1, and the correspondence of S1 and D1
4. Create a reference in the D inspector with the X attribute of S
5. Write equations and simulate

Example: Black and White Model 1

Add to entity D1 of entity type D, the value of an aggregate of entities of type S

Comment: We are adding in diagram D the value of a variable of entity D1 with the value of an aggregate of entities of type S.

1. Enter in Initialization Data the values of the entities of S and D
 2. Create in S an attribute X and an aggregate variable of some entities by X
 3. Place the aggregate variable of entity S1 in diagram D
 4. Create in the D inspector a reference with the added variable of S, indicating a condition of the attribute X
 5. Write equations and simulate
- Example: Black and White Model 2

Add to entity D1 of entity type D, the value of a variable of a singular entity S

Comment: We are adding in diagram D the value of a variable of entity D1 with the value of an entity of type S.

1. Enter in Initialization Data the values of the entities of S and D
2. Place the variable of S in the diagram of the type of entities D
4. Write equations and simulate

Example: Mothers Model

Add to entity D1 of entity type D, a variable from a historical data entity S.

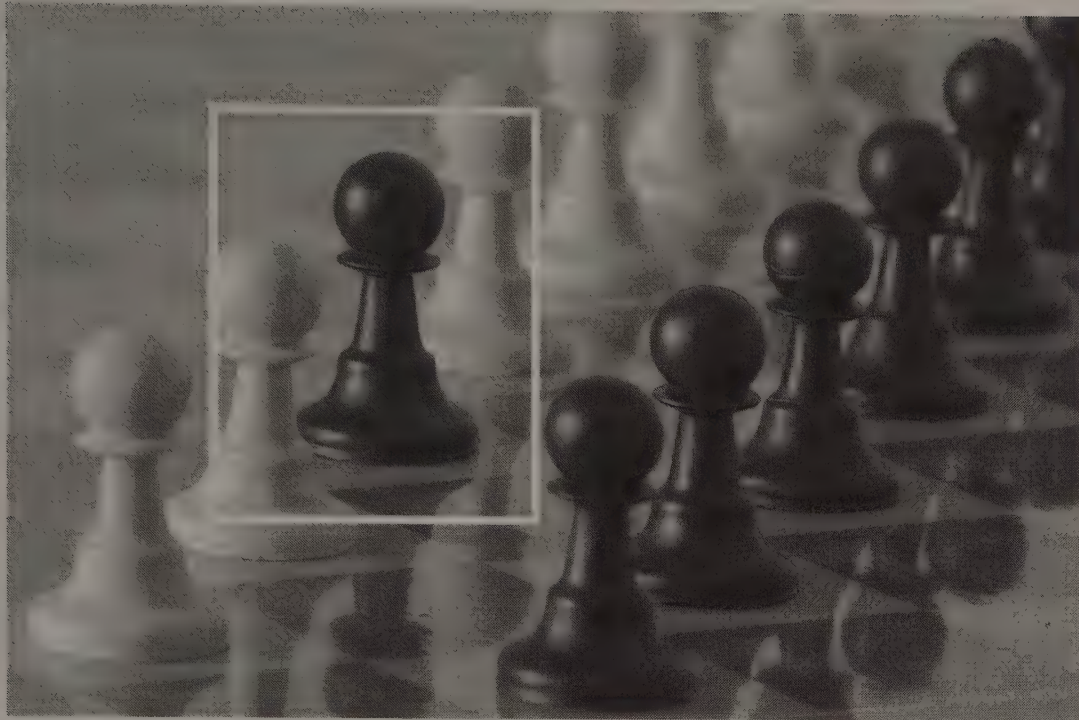
1. Create the Data Entity Type S and the variables that will contain the historical data
2. Add the external data file in Time Series Data, and create the Mapping that links it to the entity S
3. Create a reference in the D inspector with the X attribute of S

Example: Rabbits Model

Add to entity D1 of entity type D, a variable of an initial data entity S.

1. Create the Data Entity Type S and the variables that will contain the initial data
2. Add the external data file in Time Series Data of S
3. Create a reference in the D inspector with the X attribute of S

Example: Oil Model 3, 6 (in the next book)



Model: Black and White

The purpose of this model is to clarify the concept of references between entities. We have 2 types of entities: Black and White, with 2 entities in each type white1, white2, black1 and black2.

We want to calculate the difference between the value of a variable of a White entity and a variable of a Black entity; later we will see what this means.

Make File - New Model and write the name White. The White entity type is created. Click the right button on Model Overview - Entity Types, choose New Entity Type and enter the name Black. The Black entity type is created.

Go to Model Overview - Control and change the Run Time to 12. Then press Final Close. Go to View - Units Manager and uncheck the Check Units option because you will not enter information units.

Create variables Auxiliary

Open the diagram White, move the icon Auxiliary to the diagram, write the name "Auxiliary white". Click the right button on this variable. Choose Edit Variable, mark Constant (to define an initial value of each entity) and value 80.

In the diagram of Black, create the variable "Auxiliary black" in the same way that Auxiliary white was created. Mark Constant and put the value 80.

Link the two Auxiliary variables

We are going to calculate the difference between the values of "Auxiliary white" and "Auxiliary black"; one is on a White entity and the other is on a Black entity. Go to the White diagram (upper tab) and open the Black inspector (at the bottom of the inspector there are two tabs, press Black to open that inspector if it is not visible).

In the Black inspector go to the Variables tab, move "Auxiliary black" to the White diagram. The variable "Black Auxiliary black" is created in green because it is a duplicate variable of the original Black one. "Black" indicates that it is such a variable, and "Auxiliary black" is the name of the variable.



Move the Auxiliary icon to the White diagram, and create the Difference variable, add arrows from "Auxiliary white" and "Black Auxiliary black" to Difference.

Right click on Difference, choose Edit Auxiliary and write the equation:

$$\text{Difference} = \text{Black Auxiliary black} - \text{Auxiliary white}$$

Placing the variable "Auxiliary black" on the White diagram automatically creates a relationship. You can go to the White inspector, hit the References tab and you will see one with the name Black <Black>.

Enter initial data

In Model Overview go to Entity Initialization Data, right-click on Data Sources and choose the New Builtin Data option. The Entities 1 file is created and opened. There should be a White and a Black tab, if not, check Show All Types.

Enter the following data. There are four entities white1, white2, black1 and black2, and we can enter the values of "Auxiliary white" and "Auxiliary black" in their respective tabs because we define them as Constant. Close.

Entities 1						
Number of rows to add:						
Black White						
	☒	Time	CalendarTime	WhiteID	Auxiliary white	its black
1	☒			white1	85	black1
2	☒			white2	90	black2
Black White						
	☒	Time	CalendarTime	BlackID	Auxiliary black	
1	☒			black1		95
2	☒			black2		105

Add an attribute

We must add an attribute to identify which entities should be subtracted: either black1-white1 or black1-white2. To do this, we will create an attribute on the White entity type.

Go to the White diagram, move the Attribute icon to the diagram and name it "its black". Right click on the attribute, choose Edit Attribute. In Refers to choose the Black option. Thus, we indicate that this attribute will have information of the type of entities Black.

In Model Overview double click on Entities 1 to open it and go to the Target tab. A column "its black" appears, enter the data from the following figure, which shows the correspondence between the entities of White and Black. Close the window.

White		
WhiteID	Auxiliary white	its black
white1	85	black1
white2	90	black2

Attribute - White.its black

Name:

Owner:

Refers to:

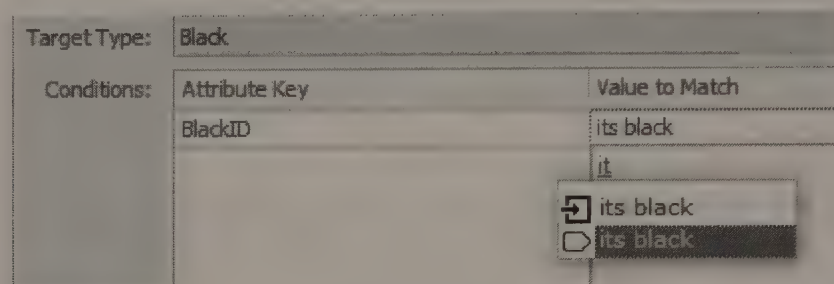
☐ Key

Placing the attribute "its blacks" on the White diagram, defines as Refers to Black and automatically creates a relationship. You can go to the White inspector, hit the References tab and you will see one with the name "your blacks <Black>".

Add the reference

In the upper part of the White diagram the relationships are shown. The Condition column points out with ??? what remains to define a relationship. We will define the correspondence between the entities of Black (BlackID) and the values of the attribute "its black" that we have just entered in Entities 1.

Click on ???, in the window that opens go to the Value to Match box and write "its black" and in the options that open choose the option of the attribute "its black". Close.



We have already defined the reference so that each entity BlackID (black1 and black2) corresponds to the values of the attribute "its black" (black1 and black2), to calculate the Difference.

Simulate

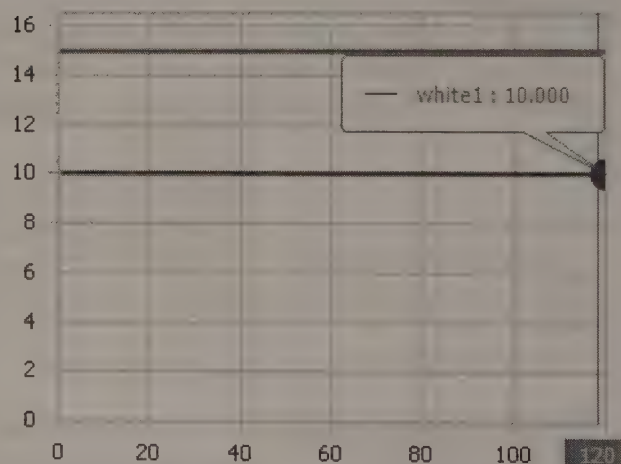
Simulate the model, right-click on Difference and choose the Plot Variable option. A graph of the white 1 and white 2 entities is shown since we are in the White diagram. For example, the value of the Difference of white1 is white1 (85) minus black1 (95). We have defined the Difference as black minus white, so $95 - 85 = 10$.

Summary

To subtract values of entities of different types (Black and White) it is necessary to:

1. Place an attribute on an entity type (White) that refers to entities of the other type (Black).
2. Define in the initialization file (Entities 1) the one-to-one correspondence between entities of one type (White) and another (Black)
3. Define the reference that links the entities of one type (BlackID) with the attribute located in the other entity (its black) that links it.

Difference

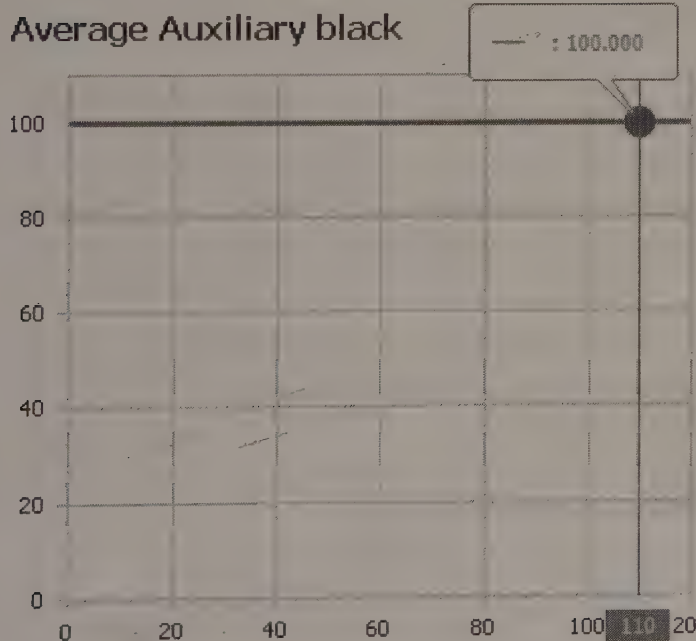
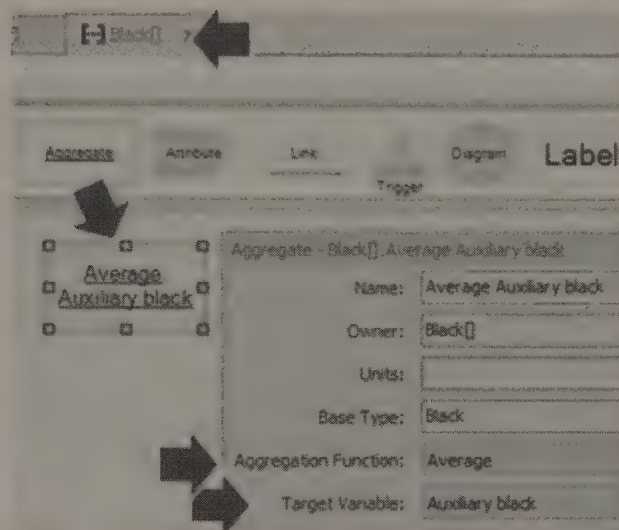


In this model we are going to relate the White entities with an aggregate of ALL to the Black entities. We are going to calculate the difference of the value of White entities (white1 and white2) from the average value of all the Black entities. Use the previous model.

Create an aggregate

Since we want to have information on all the Black entities grouped together without this calculation being linked to the White entities, we will use the Black entity type [], which is created automatically and is a collection of all the Black entities.

In Model Overview display the Black entity type, double click on Black [] and the diagram and the inspector open. Move the Aggregate icon to the diagram, Aggregate1 is created. Right-click on Aggregate 1, and choose Edit Aggregate.



In Aggregation Function choose Average to calculate the mean value, in Target Variable choose "Auxiliary black". The Name of the aggregate changes automatically.

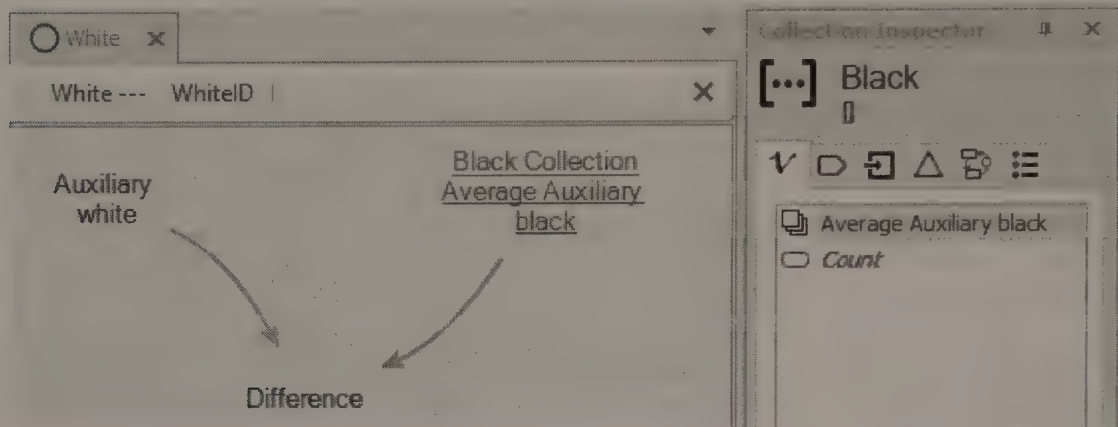
Simulate the model, make Plot Variable of the "Black Auxiliary Average" and check that the result is $(105 + 95) / 2 = 100$.

Calculate the difference

From the Variables tab of the Black [] inspector, move "Auxiliary Average black" to the White diagram. The variable "Black Collection Average Auxiliary black" is created, which is a duplicate variable and is therefore shown in green.

Delete the previous arrow from "Black Auxiliary black" to Difference, and add an arrow to Difference from "Black Collection Average Auxiliary black". Right click on Difference, choose Edit Auxiliary and write the equation:

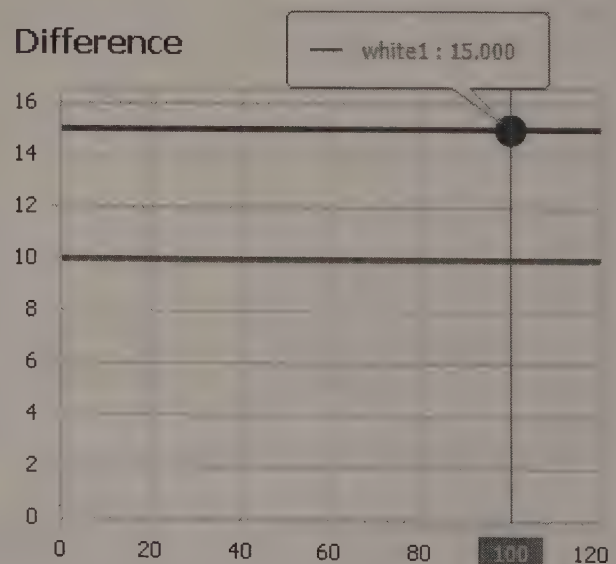
$$\text{Difference} = \text{Black Collection Average Auxiliary black} - \text{Auxiliary white}$$



Results

Simulate the model. The mean value of the Black entities is 100, as we have seen. The entity white1 has the value 85, therefore the Difference of white1 should be $100 - 85 = 15$.

Click the Variable Plot of Difference.



Note: When we use the aggregate of ALL entities of an entity type (Black) it is not necessary to define a reference between the entity types (Black and White).

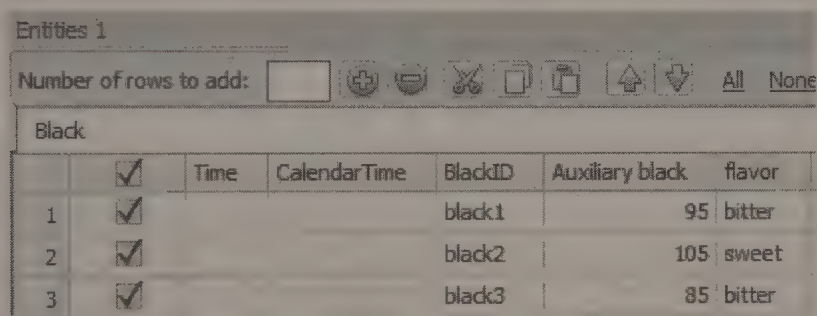
Model: Black and White 3

In this model we are going to relate the White entities with an aggregate of SOME of the Black entities. Let's calculate the difference of the value of the White entities with the average value of the Black entities that have the attribute "flavor". We will use the previous model.

Create a new attribute

Go to the Black diagram, move the Attribute icon to the diagram and give it the name "flavor". Make Edit Attribute by right clicking on it, and in Refers to choose the Black option. Thus, we indicate that this attribute is exclusive to those entities.

In Model Overview go to Entity Initialization Data and open Entities 1 by pressing twice, in the Black tab we add the following information, in which we have added information in the flavor column and a new entity, black3. Close to finish.

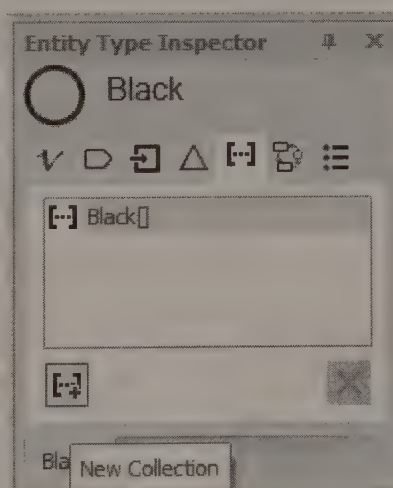


	☒	Time	CalendarTime	BlackID	Auxiliary black	flavor
1	☒			black1	95	bitter
2	☒			black2	105	sweet
3	☒			black3	85	bitter

Create an aggregate

In the Black inspector go to the Collections tab (in the Model Overview, right-click on Black and choose Inspect). At the bottom click New Collection.

In the menu that opens mark "flavor" and OK. Thus, we will have the Black entities grouped by their flavor.



Aggregate - Black[flavor], Average Auxiliary black

Name:	Average Auxiliary black
Owner:	Black[flavor]
Units:	
Base Type:	Black
Aggregation Function:	Average
Target Variable:	Auxiliary black

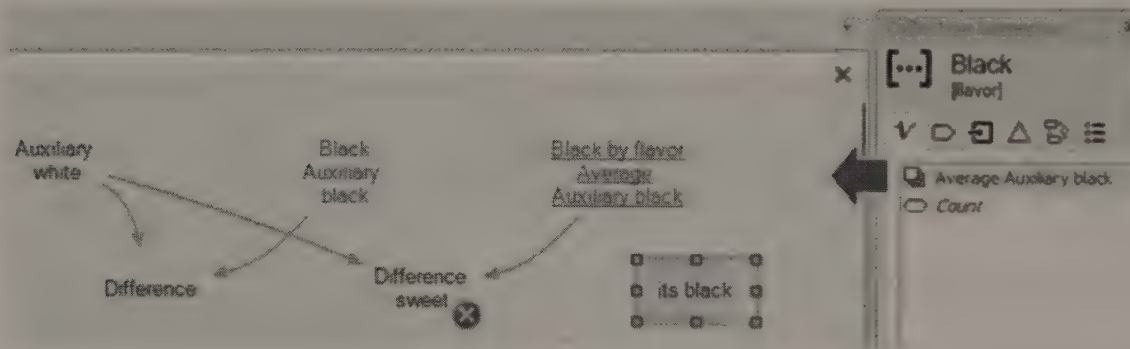
On the Black [flavor] screen move the Aggregate icon to the diagram which is empty. Right-click on it and choose Edit Aggregate.

In the window that opens choose Average for Aggregation Function and Auxiliary black for Target Variable.

Close.

Open the White diagram by clicking on its tab at the top. Open the Black [flavor] inspector. To do this go to the Model Overview, display Black, right-click on Black [flavor] and choose the Inspect option.

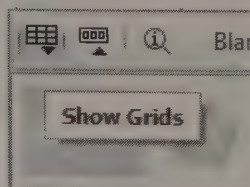
Move the "Average Auxiliary black" variable from the Black [flavor] inspector to the White diagram. The duplicate variable (in green) "Black by flavor Average Auxiliary black" is created. In the same screen create the auxiliary variable "Difference sweet", and add arrows to it from "Black by flavor Average Auxiliary black" and "Auxiliary white".



Right-click on Sweet Difference, choose Auxiliary Edit, and enter the equation:

Difference sweet= Black by flavor Average Auxiliary black-Auxiliary white

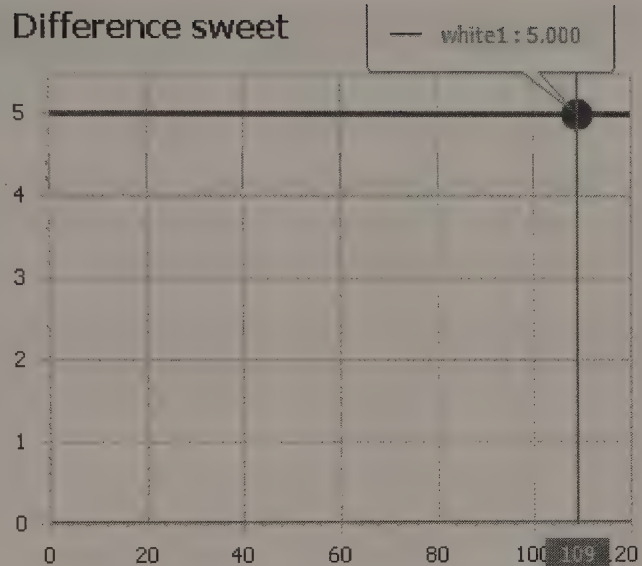
If you simulate the model you will see an error message appear at the bottom.



Click the Show Grids icon at the top to display the references. On the right side you will see that the selected flavor that is marked with ??? is not identified in Condition. Double click on [flavor] = ??? In the menu that opens go to Value to Match and type "sweet" (in quotation marks).

Before simulating the model, let's see what values we should obtain. The mean value for Black entities with a sweet taste is $(95 + 85) / 2 = 90$. The value of white1 is 85 so the Difference sweet should be $90 - 85 = 5$. The value of white2 is 90 so the Difference sweet should be $90 - 90 = 0$.

Simulate the model, select the Difference sweet variable, right-click on it and choose Plot Variable. The result is that white1 has the value 5 and white2 the value 0.



Once you have reached this point in the book, where you have read the concepts used in an ABM simulation model, such as attributes, collections, references, ... and done the previous exercises, it is normal that there is some confusion and even discouragement.

Next you will see purely mechanical aspects of the creation of the model, e.g. how the initial data is defined, how external data is imported, etcetera, which does not require thinking about. Just take note of the steps to follow for each of these actions.

In the next chapter many practical exercises are explained step by step and in great detail, that is where - I hope - you see clearly those concepts that you have just read now. Read again the previous pages when needed, and consolidate these concepts.

The concepts of attributes and references need to be well understood and there is no need to spend effort on this.

Review the previous pages and add the notes that you deem necessary. In any case during the following exercises you will consolidate these concepts.

The content of the book has been refined to facilitate a quick and complete learning, thanks to the kind collaboration of many other people who, like you, started from scratch, arrived at this same point albeit discouraged, but continued on and are now able to build their own simulation models based on the ABM methodology.

You are now at a critical moment, it is normal and common. So please don't be discouraged, have a little patience, and move on.

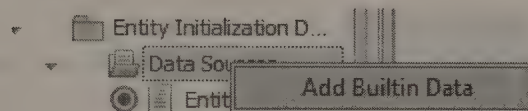
Tools

Entities initial parameters

A Builtin Data File contains the value of the constants and the initial values of each entity. For example, if we have a type of entity that is trucks, in the initial parameters file we can define for each entity (truck): license plate, purchase value, brand, purchase date, etc.

These files have data that is either not tied to a specific period, such as constants (whose values are the same at all times) or are values defined at startup (whose values apply at any time) or are designated as the start period of the entity, e.g., month 8.

To create a data file, go to Model Overview - Entity Initialization Data - Data Source - New Builtin Data. A new file called "Entities 1" appears in the entity's data folder and its name can be changed (Edited) by clicking with the right mouse button on it.



The embedded file opens automatically when created, or double-click on it and choose "Edit". A window appears with a tab for each entity.

A screenshot of a software window titled 'Entities 1'. It contains a table with the following data:

	☒	Time	CalendarTime	BlackID	Auxiliary black	flavor
1	☒			black1	95	bitter
2	☒			black2	105	sweet
3	☒			black3	85	bitter

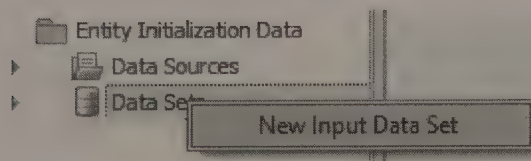
- To activate a row, check the checkbox on the left.
- The "Time" column points out the period in which the variable will be activated in the simulation. If the value is 0 or blank, the variable will be active from the start of the simulation.
- The values that you type in each column establish the initial values for the variable in that row. If no data is placed, the initial value of the action or the definition of the attribute defined in the diagram is taken.
- To add more rows, click on the (+) button. To delete rows, select the rows and press the (-) button. If no row is selected, the last row will be deleted.
- All cells of embedded data can be cut, copied and pasted, e.g., from/to Excel.
- Check the Show All Types option if some type of entity is not shown.

Once a data file has been created, it can be used on multiple input Data Sets: this requires moving it from one to another input in the Model Overview.

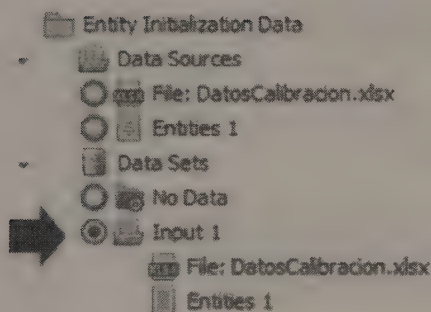
If a variable is renamed in the model, its name in the embedded data files is automatically updated.

Entity Initialization Data Sets

The initial data files (Builtin Data) can be grouped into packages (Data Sets). This is useful when we want to use several files in a simulation. In this way the Data Set is activated instead of each of the Builtin Data.



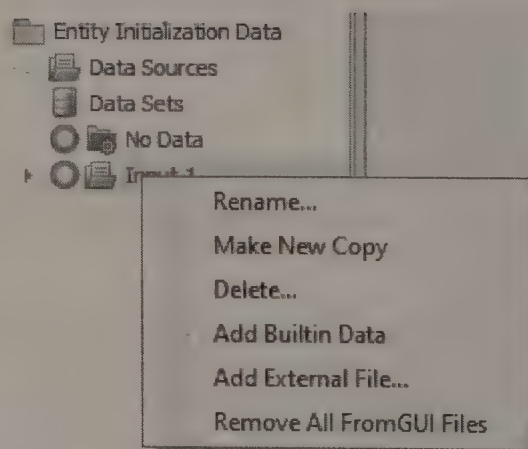
In Model Overview go to Entity Initialization Data - Data Sets and choose New Input Data Set. Input 1 folder is created



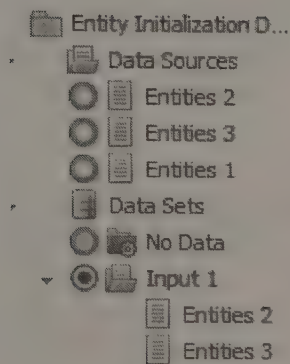
The most common choice is to place in a Data Sets folder all the initial data (Builtin Data) and external files that will be used in the model. Thus, they also appear in Data Sources in case they are needed individually.

The name of the folder can be changed (Rename), and in particular data can be entered either manually (Add Builtin Data) or from an external file (Add External File).

When using the Add External File option an established format is required, which we will introduce later.



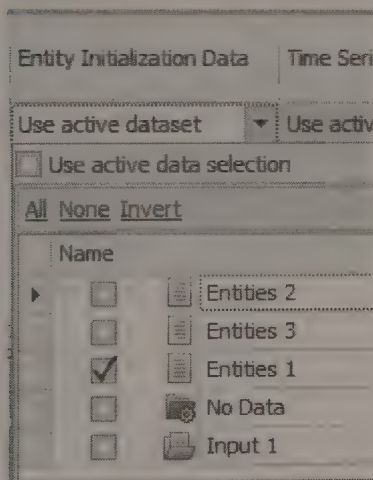
Order of files



During initialization, the values from the first (top) file in the set will be applied, then the second, then the third, and so on. If the same variable is initialized in two or more files, the last value found is the one used during the simulation. This allows you to organize a group of simulations by placing a set of standard values at the top of the list and then, for each simulation, switching to a smaller set with file-specific overrides towards the bottom of the list.

In the figure, where the Input 1 file folder is selected, the data from Entities 3 will be used if that same entity is also defined in Entities 2.

If a key entity (identified by its unique key ID) appears in an input file that did not appear in previous files, that entity will be created. The default values will be used for any initial values not provided.



The Entity Initialization Data menu only allows one option to be left active, but in Run Control you can define whether you want to use the active Dataset (in the previous image it is the Input 1 folder) or whether you want to select a specific file (such as Entities 1). To do this uncheck "Use active dataset" and mark the desired files.

Notes:

- If you define a file in Data Sets, it is automatically added into the Data Sources, which does not happen if you define it in Data Sources.
- Initial data is applied when an entity is incorporated into the model, which can be at the start of the simulation or later.
- In short, the initial data files of each entity can be added in a single file (the most common option in small models) or in the form of packages (Data Sets).

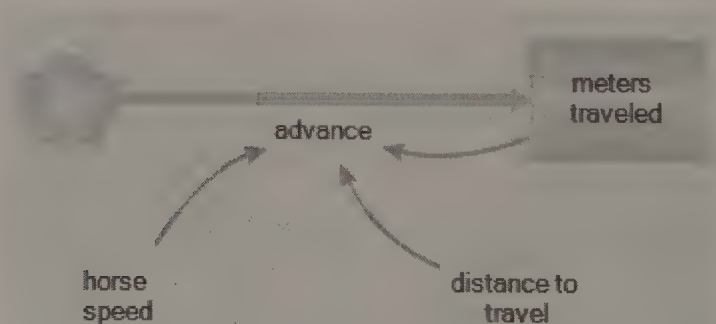


Model: Horse Racing

We are going to simulate the race of 8 horses that must travel 2000 meters, we have information on the speeds of each horse.

Press File - New Model and write the name "Race". In Model Overview go to Run Control and set Final Time to 180 and Units for Time write seconds.

Draw the following diagram on the Race screen, starting with the Stock, then the cloud, the flow, the Auxiliary variables, and the arrows.



Initial parameters

In Model Overview - Entity Initialization Data, right click on Data Sources and choose Add Builtin Data. Entities 1 is created (name that could be changed). The Time column indicates when the entity starts, in this case all the horses start the race in period 0 except the horses number 7 and 8 that leave behind.

RaceID is the identifier of each entity (horse), in this case the number of each horse. It is a literal, not a numeric value. "horse speed" is the meters per second that the horse travels.

Race				
	☒	Time	RaceID	horse speed
1	☒	0	1	13
2	☒	0	2	21
3	☒	0	3	15
4	☒	0	4	14
5	☒	0	5	16
6	☒	0	6	18
7	☒	5	7	20
8	☒	10	8	19

Equations

meters traveled
Initial value = 0
Units = meters

distance to travel = 2000
Units = meters

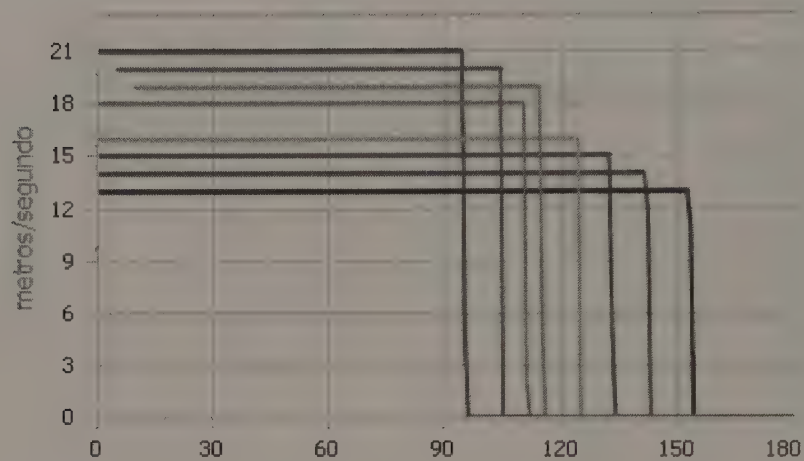
horse speed = 20 (Constant)
Units = meters / second

advance = if (distance to travel-meters traveled < horse speed) {distance to travel-meters traveled} else {horse speed}
Units: meters / second
If the remaining distance to the goal is less than the horse's speed, the value of that distance is taken.

Simulate

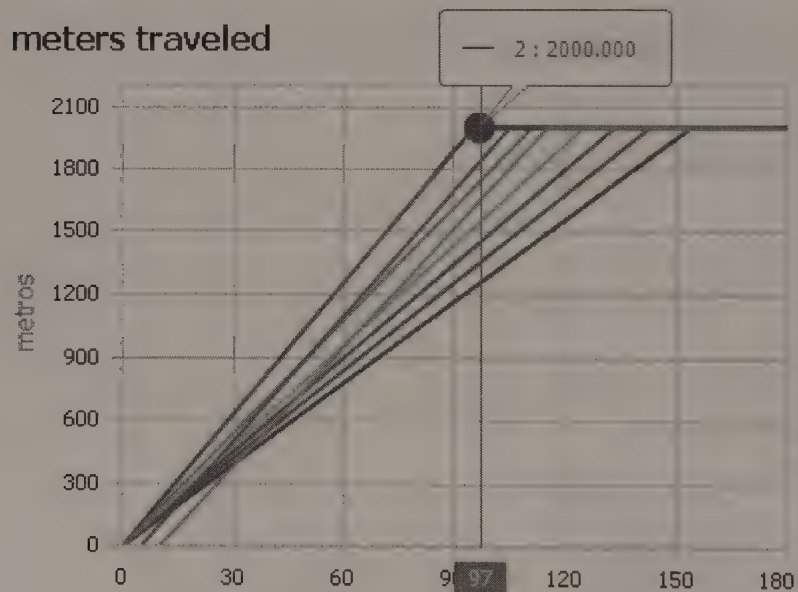
The graph shows the start of each horse and its progress until it reaches the goal.

advance



The graph shows that horse number 2 is the first to reach the finish line.

meters traveled



Time Parameters

Each model includes a single "singular" entity called Model. Four variables are defined in that entity:

- Initial Time = The simulation start time period.
- Final Time = The final time period of the simulation.
- Time = A counter of the time variable, it reads 1 in period 1, etc.
- Time Step = The duration of a time interval, which is the amount of time that is advanced during each period of the simulation.

The values of the four variables are defined in Model Overview - Run Control. Entities can access these variables through the Model entity. To use the Time counter in an equation, it must be written in the equation: "Model.Time".

CurrentTimeSlice () is the number of the current time interval. The first time interval of the simulation is 0 and the number increases in each time interval regardless of the Initial Time, Final Time and Time Step defined.

Run Control

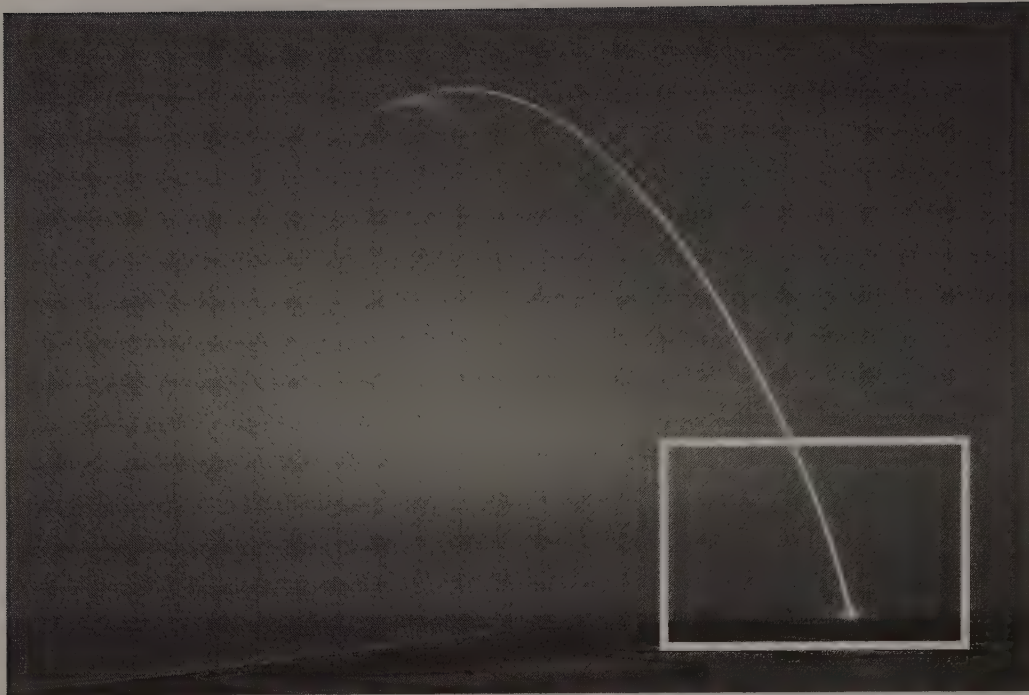
To define the temporary parameters of a model, go to Model Overview - Run Control and click twice to open the window. Write the Initial Time, Final Time, Time Step and Units for Time (the latter is important if you check Using Calendar Time).

Using Calendar Time	Initial Time	Final Time	Initial Calendar Time	Final Calendar Time	Time Step	Unit Of Time
☐	0	180	11/16/2020	03/16/2021	1 (181 t...	seconds

If you check "Using Calendar Time" you will have to define the Initial Calendar Time and the Final Calendar Time. Remember to use the MM / DD / YYYY format and the Units of Time.

Save Time Step		Initial Save Time	Final Save Time	Initial Save Calendar Time	Final Save Calendar Time
1	0	12	09/28/2020	01/28/2021	

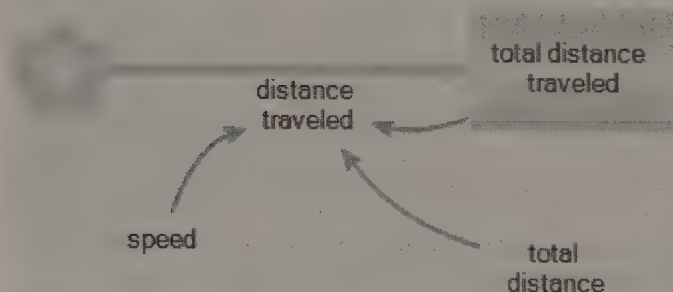
The Entity Initialization Data option allows you to select which file (external) of the Data Sources will be used in the simulation, by default an empty file is used.



Model: Launching Satellites

We want to simulate the launching and putting into orbit of 12 observation satellites to the Moon. It is planned that the launch of the rockets will be done at intervals of 15 days from January 15, 2022, the distance to travel is 50,000 km. and the average speed of the rocket is 1000 kilometers per day, which is regulated according to the distance to the destination point.

Press File - New model and indicate the name Satellite. Go to Run Control and check Using Calendar Time, then Final Time = 01/01/2022, Initial Time = 01/31/2022, Units for time = day.



Draw this diagram in the Satellites tab, starting with the Stock, then the cloud, creating the Flow, the Auxiliary variables and finally adding the arrows.

Equations

total distance traveled

Initial Value = 0

units: km

total distance = 50000

units: km

speed = 1000

units: km / day

distance traveled = speed * (1 + DELAY3 ((total distance-total distance traveled) / total distance, 30))

The distance traveled is a function of the speed and an adjustment as a function of the proportion traveled over the total, acting with a certain delay.

Start data

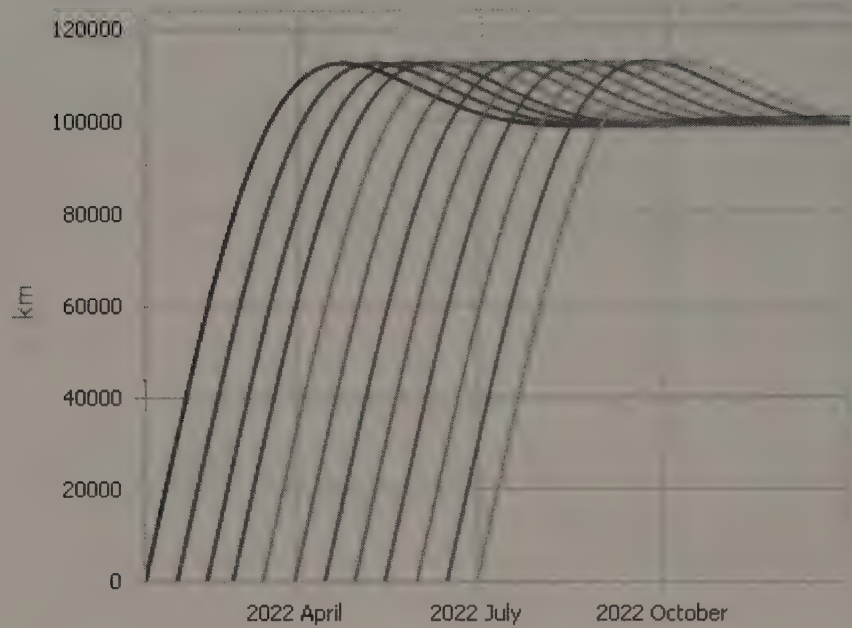
It is necessary to write the release dates. Go to Model Overview - Entity Initialization Data, click with the right button on Data Sources and choose Add Builtin Data. Entities 1 is created. Enter the data in Calendar Time and the satellite code. Close.

Satellites			
	☒	CalendarTime	SatellitesID
1	☒	01/15/2022	ESP 1
2	☒	01/31/2022	ESP 2
3	☒	02/15/2022	ESP 3
4	☒	02/28/2022	ESP 4
5	☒	03/15/2022	ESP 5
6	☒	03/31/2022	ESP 6
7	☒	04/15/2022	ESP 7
8	☒	04/30/2022	ESP 8
9	☒	05/15/2022	ESP 9
10	☒	05/31/2022	ESP 10
11	☒	06/15/2022	ESP 11
12	☒	06/30/2022	ESP 12

Simulate

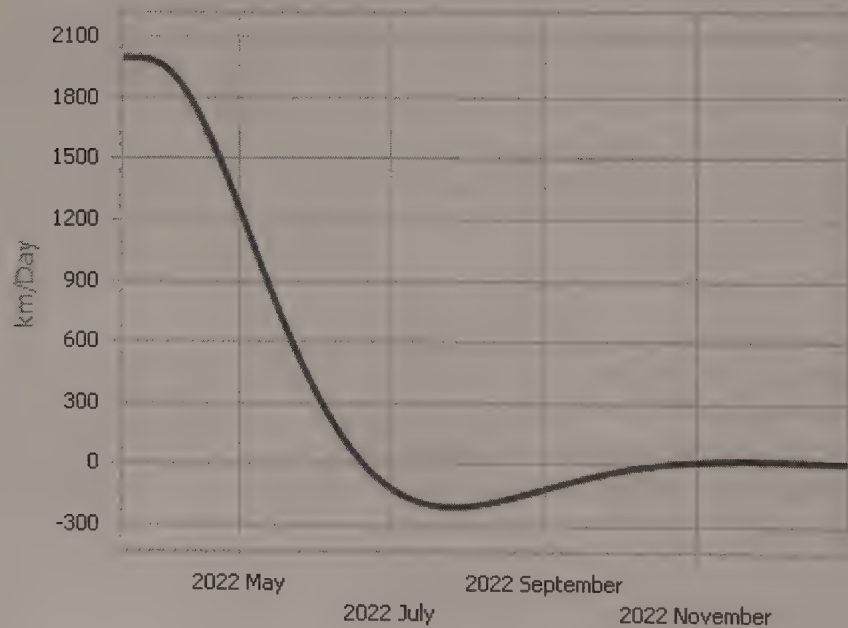
Click on the green Run Model icon to simulate the model. Click on the right button on the variable "total distance traveled" and choose the option Plot Variable. The following image is displayed.

total distance traveled



For a more detailed analysis go to the Entity Picker (bottom left) to display the Satellites menu, then display the SatellitesID menu to show all the satellites. Uncheck Satellites and activate ESP5 to see only that satellite. Select the variable "distance traveled" and then with the right button choose Plot Variable.

distance traveled



External Data Files

The model can incorporate two types of data from external files:

- 1) Initial values of an entity in Entity Initialization Data.
- 2) Time series data of an entity in Time Series Data.

The external files incorporated in the Entity Initialization Data indicate what entities there are, what type they are, when they start, and what values they take in what period. The most common is that they indicate the attribute of each entity and its value in period 0.

On the other hand, the external files incorporated as Time Series Data, have data of some variables of the model during the whole period of the simulation.

Both types of data are imported from a file and must be saved in the Data folder of the model. These data are used as exogenous information, which will not depend on other variables. In this book we will use only Excel files.

In both cases, a specific entity has to be created that links the external data with variables in the model, so that the other entities in the model can access the external data in their equation. Temporal data can be used in calibration to measure the mismatch between the model result and the actual data.

Formats

If you import a file with data in the Entity Initialization Data, remember that:



1. External files must have DateTime and CalendarTime columns (even if they are empty).
2. The Key attribute (ending in ID) must be an Excel column.
3. The names of the variables must be the same in the Excel and the entity.
4. The name of the Excel sheet must be the same as the name of the entity type.

If instead you import a series of temporary data, it is not necessary that the names of variables or the Excel sheet match, because a Mapping of equivalences of the Excel with the entity of Data Entity Types is created.

Excel data can be in four formats, the software detects which one is being used. The following examples are for importing the variables weight and height of the entity type "dogs" with the key attribute dogsID.

Format 1. Tidy

Each row specifies an entity and a time; each column, a variable. The first row contains the key attribute, "DateTime", as well as variable names.

	A	B	C	D
1	dogsID	DateTime	weight	height
2	Buddy	0	1,5	40
3	Buddy	1	1,6	42
4	Lucy	0	6,2	75
5	Lucy	1	6,8	70

Format 2. Relational

Each row specifies a variable, entity, and time. The first row contains the keywords "DateTime", "Varname", and the Key attribute.

	A	B	C	D
1	dogsID	DateTime	Varname	Value
2	Buddy	0	weight	1,5
3	Buddy	1	weight	1,6
4	Buddy	0	height	40
5	Buddy	1	height	42
6	Lucy	0	weight	6,2
7	Lucy	1	weight	6,8
8	Lucy	0	height	75
9	Lucy	1	height	70

Format 3. Tabular (dates beside)

Each row specifies an entity and a variable; each column, a time. The first row contains the Varname, the key, and the time values.

	A	B	C	D
1	Varname	dogsID	0	1
2	weight	Buddy	1,5	1,6
3	height	Buddy	40	42
4	weight	Lucy	6,2	6,8
5	height	Lucy	75	70

Format 4. Tabular (vertical dates)

Each row specifies a time; each column, an entity and a variable. The first column contains the Key attribute, the "Varname", and the time values.

	A	B	C	D	E
1	dogsID	Buddy	Buddy	Lucy	Lucy
2	Varname	weight	height	weight	height
3	0	1,5	40	6,2	75
4	1	1,6	42	6,8	70

Import initial data

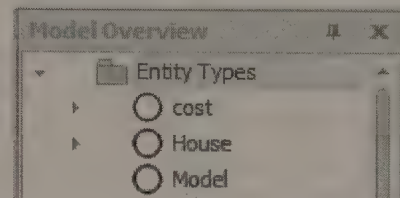
The steps to follow are described below, later some exercises allows to see these steps in more detail.

Remember: 1. The Excel files with the external data must have the Time and CalendarTime columns (even if they are empty), 2. The sheet with the data must have

the same name as the entity where it is used, 3. The Key attribute (finished in ID) must be a column of the Excel, and 4. The names of the variables must be the same in the Excel and the entity.

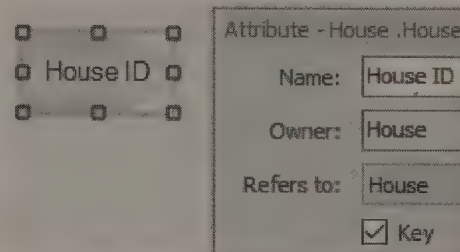
Step 1. Create an entity

Create an entity in Entity Types where the data imported from Excel will be. The name of the new entity must be the same as the Excel sheet with the data.



The name of the Key attribute of the new entity must be the same as the Excel column that identifies them.

Auxiliary variables (marked Constant) are created in the new entity that will take the data of interest from Excel, which may have a lot of other information that is not imported.



The name of the variables to import must be exactly the same as they have in the Excel column with the data.

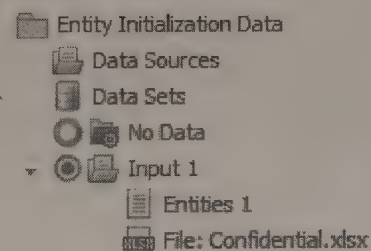
Step 2. Define the entities

The key attributes (Key) of each entity must be defined. Go to Entity Initialization Data, right-click on Datasets and choose New Input Dataset, Input 1 is created. In this directory all the data of the new simulation are placed. Right-click on Input 1 and choose Add Builtin Data. Entities 1 is created. It is necessary to define an entity for the model to work, just check the tab on the left for this.

Aircrafts		
	☒	AircraftsID
1	☒	N751
2	☒	N752
3	☒	N753
4	☒	N754
5	☒	N755
6	☒	N756

Step 3. Identify the external file

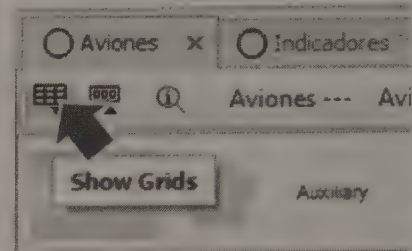
Open Input 1 and choose Add External File. When you do this, the Data folder of the model is automatically opened and the external files are displayed. The corresponding file is chosen, whose name is added in the Entity Initialization Data - Data Sets - Input 1, and also in Data Sources.



Step 4. Link the data entity to another entity in the model

In the model we have created an entity (with the same name as the data sheet) that captures the Excel data. This data is used in the calculations of other entities, it will be necessary to create a link between attributes of both types of entities.

Open the data entity inspector and move a variable to the linked entity diagram. A variable of one entity type has been placed in another entity. You need to create a reference that links them. Press the Show Grids icon.



The reference grid shows that this type of entity has an undefined reference (indicated with ???), click there and in the window that opens choose the attribute of that same entity in the Value to Match. Thus we indicate that both Key match.

A screenshot of a software window titled 'Indicators --- IndicatorsID'. It contains two tables. The first table has columns 'Attribute Name', 'EntityReference', and 'Is Key'. The second table has columns 'Reference Name', 'EntityReference', and 'Condition'.

Attribute Name	EntityReference	Is Key
IndicatorsID	Indicators	YES
its aircrafts	Aircrafts	No

Reference Name	EntityReference	Condition
Aircrafts	Aircrafts	[AircraftsID]= IndicatorsID
its aircrafts	Aircrafts	attribute value
Model	Model	

Import time series data

Using data entities to import data series requires:

1. An external file with data and dates relative to one or more variables of the model. For example, an Excel sheet with data of "daily weight" and its dates.
2. A Data Entity with the model variables linked to those in the file. For example, the variable "daily weight" is linked to "WeightDay" in the model.
3. A data Map, which links specific variables in Excel with variables in the Data Entity and specifies interpolation methods.

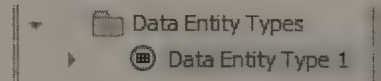
Keep in mind that data entities provide external information to the model during the course of the simulation. This is different from the initialization data, which tells the model at the start of the simulation how many entities there are of each type, when they appear and with what characteristics.

In Excel, each sheet can have information for several entities, all entities on the same sheet must have an identifier, e.g. the passport number, of the same type (alphabetical, date or numeric).

If there is more than one sheet for an entity type, e.g. Factory, the sheets must be named Factory, Factory_A, Factory_B, Factory_C, and so on. The text after the underscore does not have to be A, B, C; it can be another symbol. The name of the Excel sheet must begin with the name of the entity type.

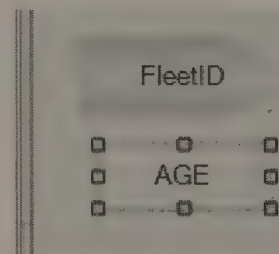
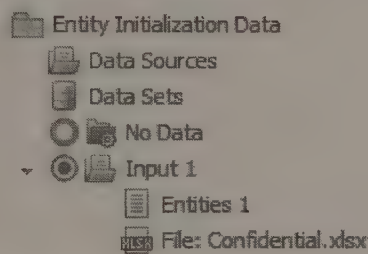
Step 1. Create a data entity

In Data Entity Types create a new entity that will be the one that captures the data from the external file.



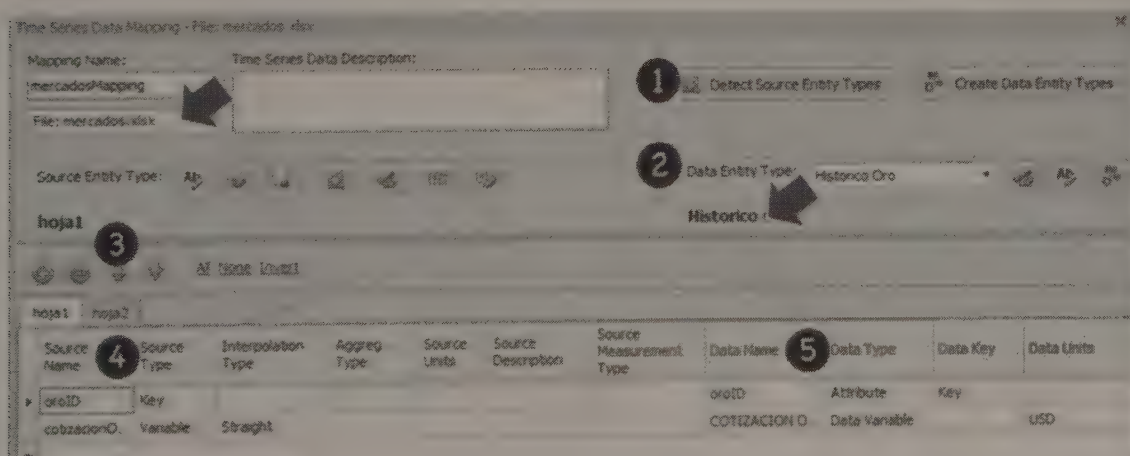
Step 2. Identify the external file

Open Input 1 and choose Add External File. When you do this, the Model Data folder automatically opens, and the external files are displayed. Choose the file that corresponds, whose name is added in the Entity Initialization Data - Data Sets - Input 1, and in Data Sources.



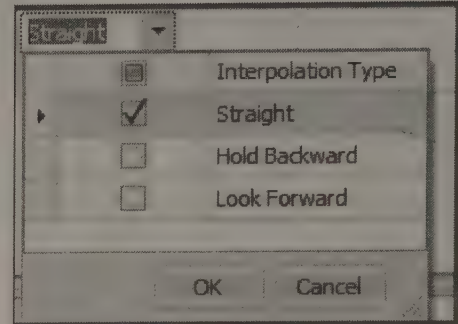
Step 3. Mapping. Link external file and data entity

On the Time Series Data Mapping screen, press (1) for the software to detect the file to import (shown in the red arrow on the left). Next in (2) the "Data Entity Type" of the model that will be linked to that Excel file is chosen. The selected entity is shown in blue. On the left side (3) indicate the Excel sheet where the data of the selected entity to import are located.



In (4) are the Excel fields with information and the identifiers that appear in the Excel. On the right you can choose the interpolation that will be used in the model when some data is missing, the options are:

- Raw. None - The model will not use a value.
- Straight. Linear: use a linear interpolation between the two closest points.
- Hold backward. Previous - Use the most recent value from the data source.
- Look forward. Next: use the next value provided in the data source.

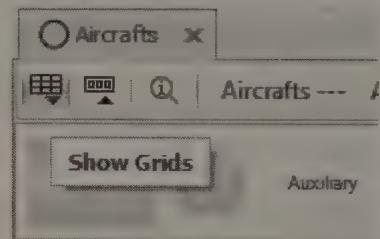


The same Excel variable can be linked to several variables of the model, if several types of interpolation are chosen.

In (5) are the model variables. Open the drop-down menu, select one in the Data Name, display the menu and choose the one that corresponds. The name of the model variable and the corresponding Excel variable may be different. Choosing the model variable automatically completes the Data Type, Data Key and Data Units information, as defined in the entity.

Step 4. Link the data entity to another entity in the model

Open the data entity inspector and move a variable to the linked entity diagram. A variable of one entity type has been placed in another entity. You need to create a reference that links them. Press the Show Grids icon.



The reference grid shows that this type of entity has an undefined reference, indicated with ???, click there and in the window that opens choose the attribute of that same entity in the Value to Match. Thus we indicate that both Key match.

Aircrafts			Aircrafts --- AircraftsID		
Attribute Name	EntityReference	Is Key	Reference Name	EntityReference	Condition
AircraftsID	Aircrafts	YES	BaseMarbella	Base Marbella	[ReetID]= ???
			Model	Model	



Model: My Three Dogs

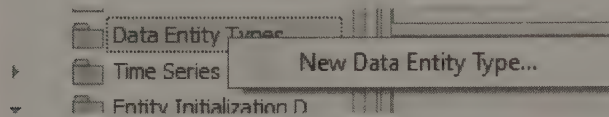
Create a new model (File-New Model) with the name "Dogs" so that all the model folders are created. In Model Overview - Run Control put Final Time = 10 and Units for Time = Months. We will see how to import a time series.

Open Excel (available with the book) with the name "file dogs.xlsx" and save it in the Data folder of the model "Dogs".

	A	B	C	D
1	DogsID	DateTime	weight	height
2	Buddy	0	0,5	40
3	Buddy	1	0,5	40
4	Buddy	2	0,5	40
5	Buddy	3	0,5	45

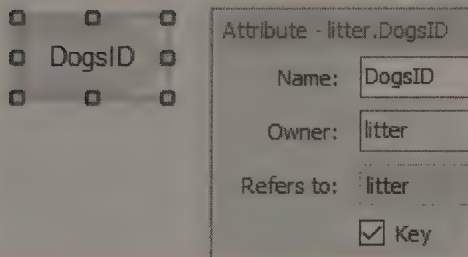
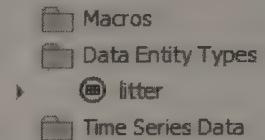
In Excel the data is in ordered format (Tidy). Thus, in the first row is the key of the type of entity that we are going to import the data from (DogsID), the DateTime column has the period of the data, and the variables (weight and height) in separate columns.

The following rows contain the values of the entity (the dog) in each time period. In the Excel "file dogs.xlsx" there is only Page1.



Create a New Data Entity Type, enter the name "litter", which is the entity where the data will be imported.

The Data Entity Type "litter" has been created.

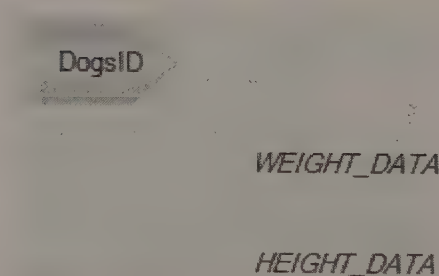
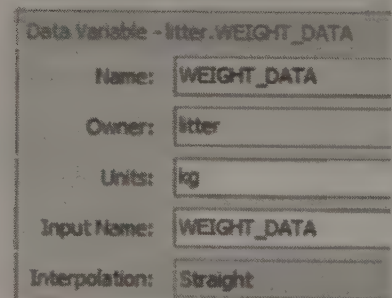
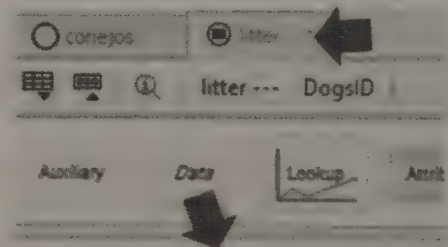


The first step is to create a Key attribute in the layer that identifies each of the entities that we are going to import.

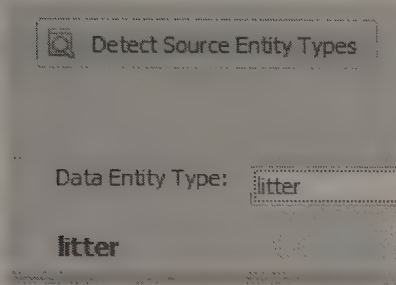
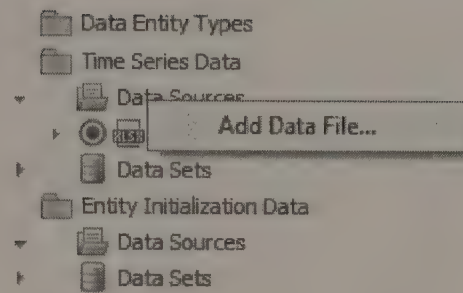
Move the Attribute icon to the litter diagram, right-click on it, choose Edit, write the name DogsID (the same name used in Excel), put Refers to: litter, mark Key and close.

In the layer diagram, move the Data icon to the screen, enter the Input Name DATA_WEIGHT and kg in Units. Repeat and create the variable HEIGHT_DATA with cm in Units and close.

The variables WEIGHT_DATA and HEIGHT_DATA have been created in the diagram.

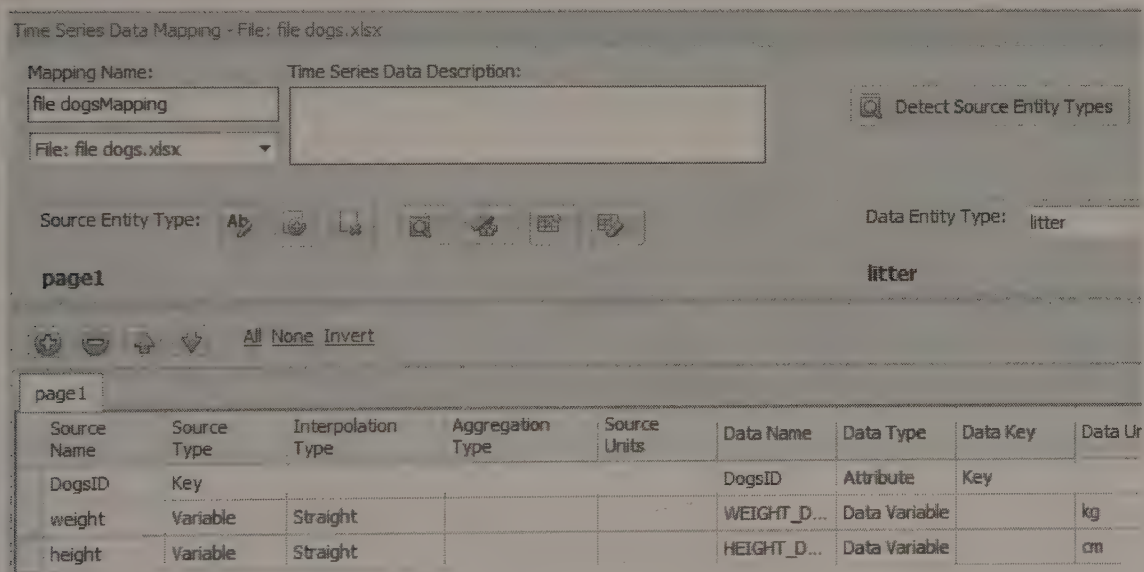


Go to Time Series Data - Data Sources - click right button and choose Add Data File. A window opens with the Data folder for this model. Double click on "file dogs.xlsx"



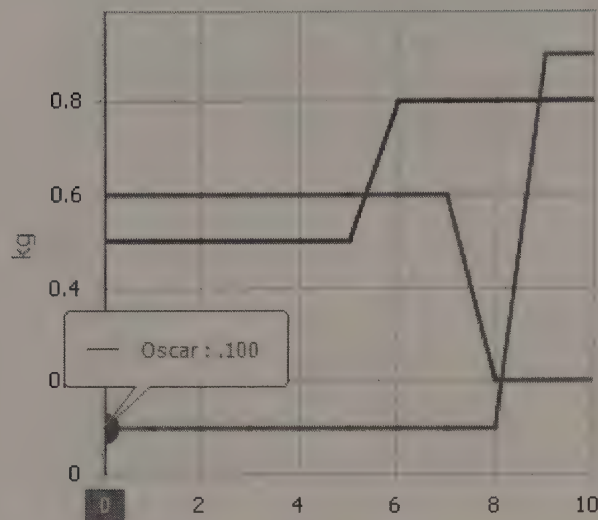
In the new window that opens, click on Detect Source Entity Types, so that it looks for the file - wait a few seconds for the bottom part to be completed. Then, as the only option is litter for Data Entity Type, this links the Excel data with the "litter" entity that has been created.

After a few seconds the screen is partially completed. Click on the boxes in the Data Name column and choose the corresponding variables from the diagram. Close.

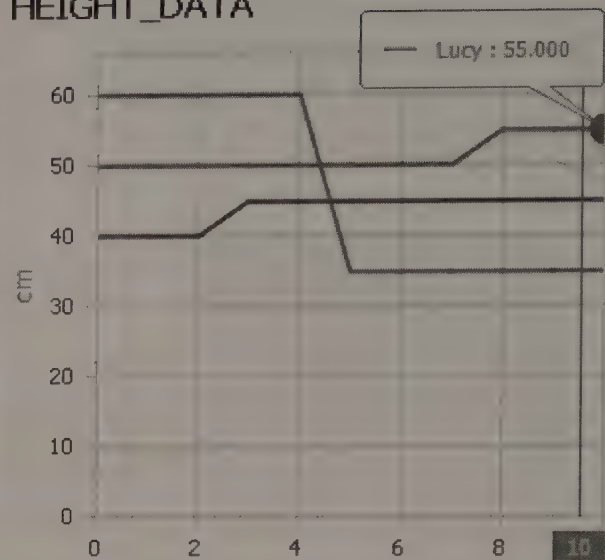


Simulate the model. We can now select the weight and height variables and make Plot Variable to verify that the behavior of the variables matches the content of the Excel. Now it is possible to use these two variables like any other in the model, although of course it cannot be modified during the simulation.

WEIGHT_DATA

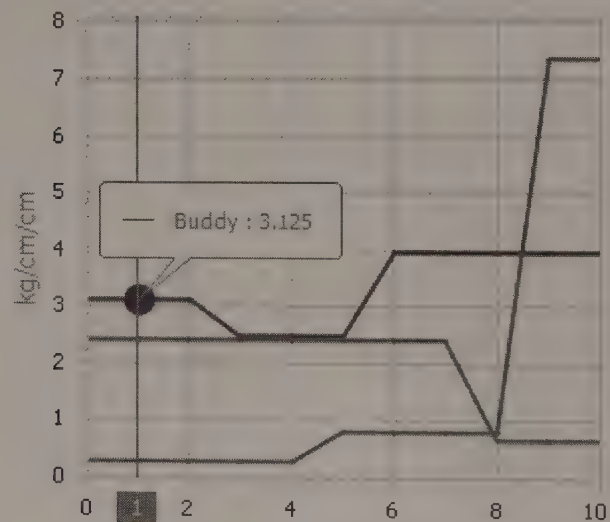


HEIGHT_DATA



With these variables we can calculate the Body Mass Index (BMI). Create this variable with the equation $BMI = DATA_WEIGHT / (DATA_HEIGHT / 100)^2$ and units kg / cm / cm

BMI



Note that it has not been necessary to create anything in Entity Initialization Data, to define which entities exist (Buddy, Oscar, Lucy), it has been enough to define the entities in Excel from which the data is imported.

Exercises

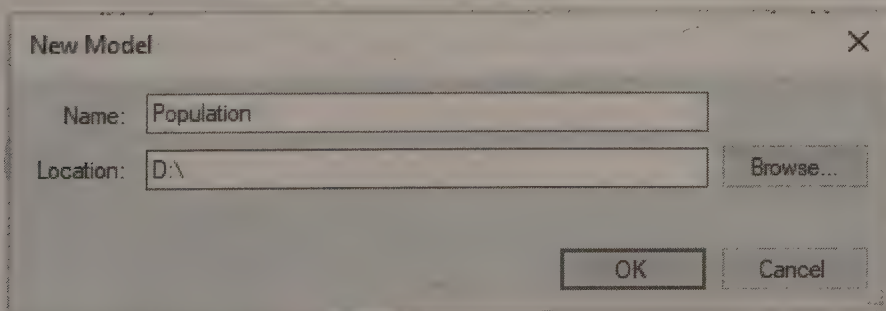


Model: Population of Rabbits

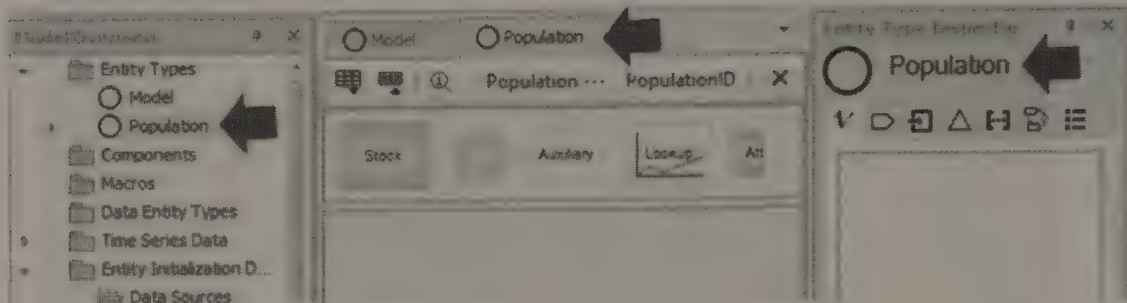
We are going to create a basic model that allows one to see: how an entity is created; how the variables are drawn in the diagram; how their respective equations are written; the time horizon set; the time units entered; how it is simulated and the results obtained.

We will simulate a population of rabbits that has a birth rate of 0.10 per month and a life expectancy of 12 months, assuming that the age distribution of the population is uniform and that the initial population is 80 rabbits.

The first step to create a model is to click the New Model icon and assign a name to the model, for example Population. It is also necessary to choose in which folder it should be saved, then press OK. The diagram area is shown empty.



An entity type with the same model name (Population) is created in the "Model Overview" column. A tab at the top shows that the drawing area of the diagram corresponds to that type of entity (Population -of rabbits-) and on the right the inspector window for this type of entity appears.



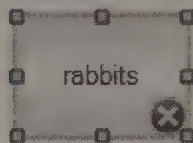
Before starting to draw the diagram, define in the model the temporary variables, which are month for Units Of Time, and a final period of 12. To do this, click twice on Run Control locating in the Model Overview column and enter these two parameters. Close the window.

Initial Time	Final Time	Initial Calendar Time	Final Calendar Time	Time Step	Unit Of Time
0	12	09/28/2020	01/28/2021	1 (13 tot...	Month

Next, draw the diagram formed by a Stock that is the number of rabbits in each period, and the inflows for births and outflows for deaths. Then add two auxiliary variables with the birth rate and life expectancy.



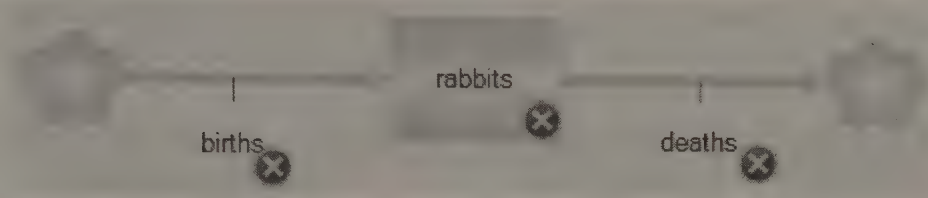
Move the Stock icon to the center of the screen, a window appears to assign a name, type rabbits and Enter. This Stock is shown with a red symbol because this variable has a pending equation.



From the icon grid, move the cloud icon twice, and place it once to the right of the stock and once to the left, leaving space to place the flows.



Click on the left cloud to select it, a green rectangle appears, then move the cursor over the cloud until a hand appears. Click on the cloud and move the cursor to the right (an arrow is displayed) and click in the center of the stock. A window is displayed above the arrow - write births. Repeat the process on the right side, starting at the stock and ending in the cloud. Write the name deaths.

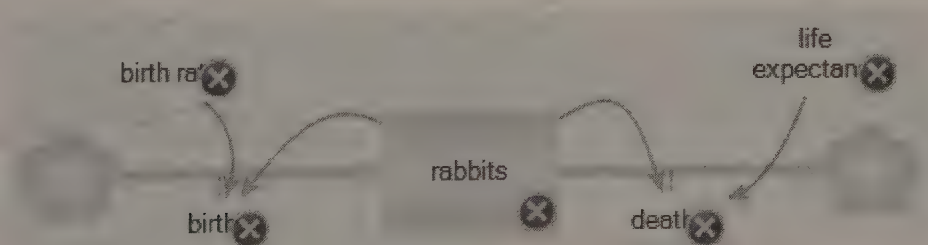


In this model, to calculate the inflow and outflows it is necessary to know the value of the stock, and therefore they must be linked with arrows that come out from the rabbit stock and end in each of the flows.

To draw the arrows, move the cursor over the rabbits stock, press to select it and move it until a hand is shown. Then click on the stock and move the cursor until it is over the name of the flow and click on it. An arrow appears with yellow and green rectangles that can be used to shape the arrow.



The next step is to add two auxiliary variables to the diagram, which are the birth rate and life expectancy. Move the Auxiliary icon to the top left of the diagram, and type birth rate in the window that opens, then press Enter. Repeat the process to create the life expectancy variable. Add arrows from these variables to the corresponding flow.



To write the equations, right click on the rabbits stock and choose the Edit Stock option. Write 80 in the initial value and rabbits in units. Close. Repeat the process in the auxiliary variables of birth rate and life expectancy, write the value of each variable, its units, mark Constant and close.

In the case of flows, clicking on the area of the equation shows the variables that influence the flow. Double clicking on them adds them to the equation without having to write the name of each variable. Write the units and close when finished.

Stock - Population.rabbits

Name: rabbits

Owner: Population

Units: rabbits

Default Initial Value: 80

Flow - Population.births

Name: births

Owner: Population

Units: rabbits/Month

From Stock:

To Stock: rabbits

☐ Constant

rabbits*birth rate

Flow - Population.deaths

Name: deaths

Owner: Population

Units: rabbits/Month

From Stock: rabbits

To Stock:

☐ Constant

rabbits/life expectancy

Auxiliary - Population.birth rate

Name: birth rate

Owner: Population

Units: 1/Month

☐ Constant

0.1

Auxiliary - Population.life expectancy

Name: life expectancy

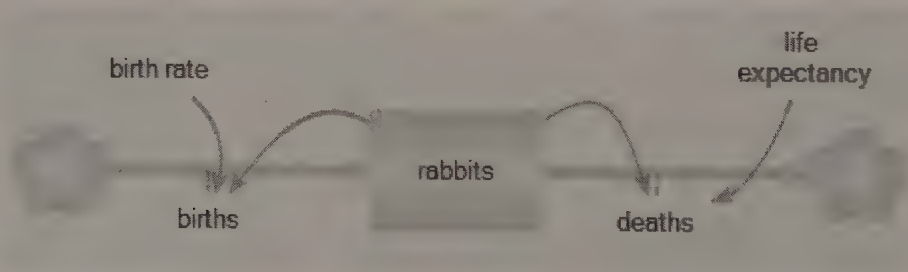
Owner: Population

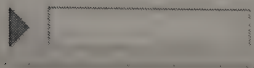
Units: Month

☐ Constant

12

The model is now complete and ready to be simulated.

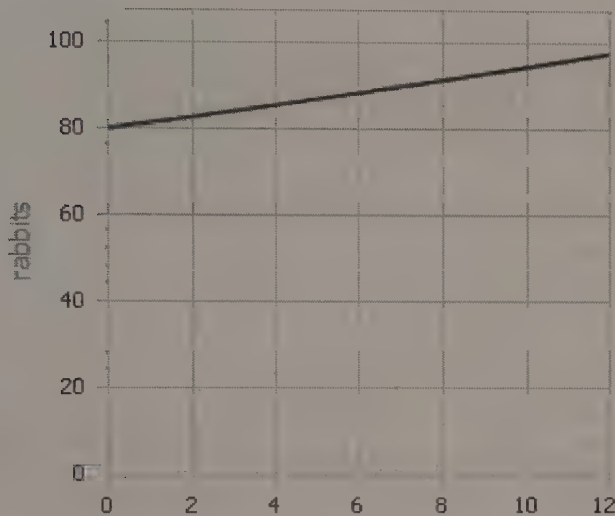




Click on the green arrow located on the top bar to simulate the model. The simulation name can be left blank.

Select the rabbits stock with the right mouse button and in the menu that opens choose the option Plot Variable. The window opens that shows in graphical format the evolution of the number of rabbits during 12 months. Save the Population model.

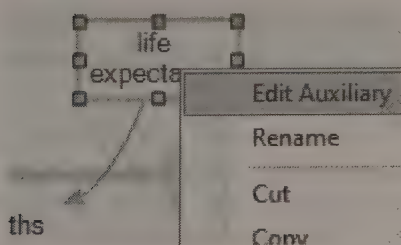
rabbits



The correct interpretation of the graph is that there is ONE entity made up of a set of 80 rabbits that progressively increases in number.

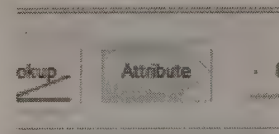
There are not 80 "rabbit" entities in the model or new entities being born or dying. In that case we should have identified each of the initial 80 rabbits and then indicated to the model (with an action) that new entities are to be created.

Model: Population of Rabbits 2

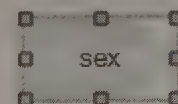


Next, we are going to differentiate the population by their sex since life expectancy is very different in males than in females. We will create 2 entities: males and females with their different PopulationID. Right click on life expectancy and choose Edit Auxiliary. In the window that opens, check the Constant box.

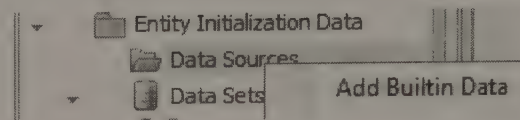
This will allow us to define different values of life expectancy for males and females.



We are going to create an attribute that differentiates the population between males and females. To do this move the



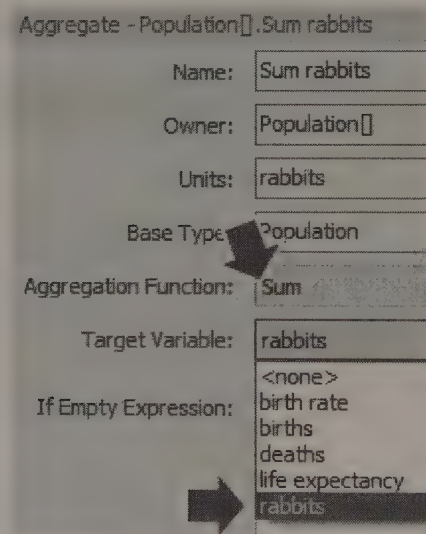
Attribute icon from the icon bar to the drawing area and name it sex.



In the Model Overview go to Entity Initialization Data - Data Sources - Add Builtin Data and write the values for males and females and close the window. Check "Show All Types" if the Population tab is not visible.

Entities 1							
Number of rows to add:							
Population							
	☒	Time	PopulationID	sex	rabbits	life expectancy	birth rate
1	☒		M	male	35		10
2	☒		F	female	45		16

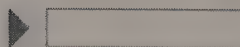
These changes will give us information on the evolution of the population of the two entities (males and females), but we also want to know how the total evolves. For this we have the Population [] collection. In Model Overview - Entity Types, display Population and double click on Population [] this opens both its diagram and the inspector.



In the Population diagram [] move the Aggregate icon from the icon bar to the drawing area. The variable Aggregate 1 has been created, right-click on that variable and choose Edit Aggregate.

Display the Aggregation Function options and choose Sum, in the Target Variable options choose rabbits. Close

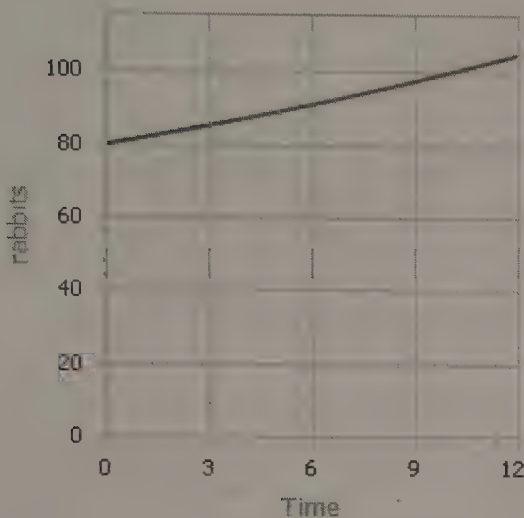
The name (Name) automatically changes to: Sum rabbits.



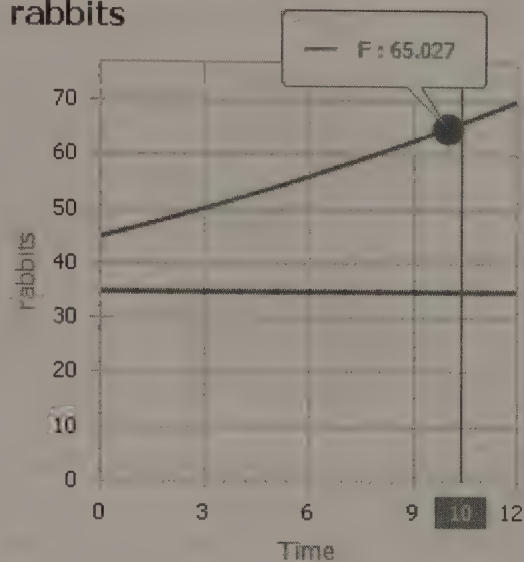
Press the green arrow to simulate the model.

Right click on the variable Sum rabbits and choose Plot Variable. The graph that appears shows the evolution of the total number of rabbits. Go to the Population tab and repeat the process with the rabbits stock, the graph shows the evolution of the number of rabbits of each entity (F or M).

Sum rabbits



rabbits



With a life expectancy of 10 months and a birth rate of 0.1, the population of males remains stable, while that of the female population increases since it has a longer life expectancy, 16 months. Save the model as Population 2

Model: Population of Rabbits 3

In fact, it is more realistic to consider that births do not depend on the total population of rabbits but on the number of females, for this we will modify the birth rate from 0.1 (of the total of rabbits) to 0.2 (of the number of females) and we will place a dependence of births on the attribute sex.

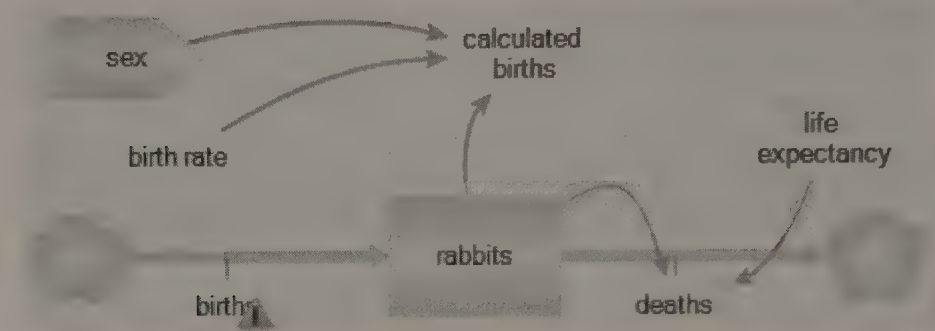
Make Edit of the variable birth rate and write 0.2 instead of 0.1. The next steps are:

1. Calculate how many rabbits are born (number of females * birth rate).
2. The result will be attributed to the group of females (as if all that were born were females). We will calculate how many rabbits are born in total (the value will be the same as that calculated for females, since the birth rate of males is 0), the total will be the sum of both origins: males plus females.
3. Assuming that the proportion of rabbits that are born is 50% male and 50% female, we must calculate the flow of births.

Step 1. Calculate the number of rabbits that are born. To do this create a new auxiliary variable (Auxiliary) with the name "calculated births", add arrows from birth rate, sex and rabbits. Write the equation:

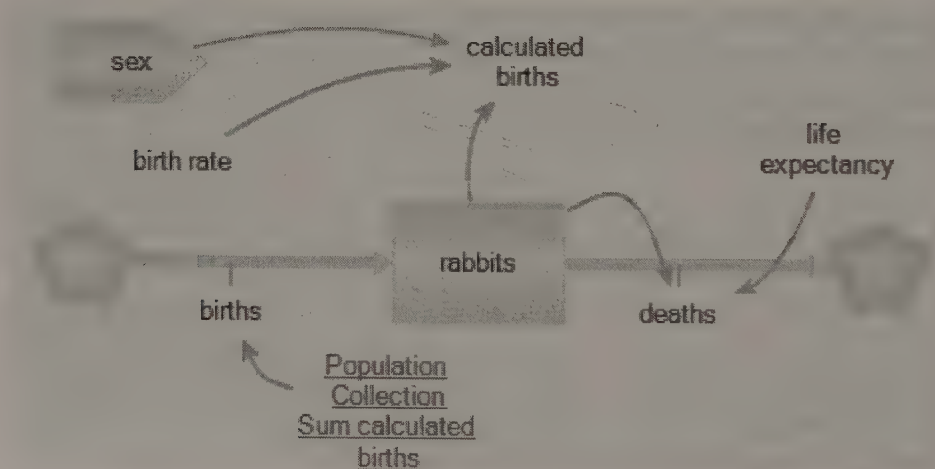
calculated births = if (sex = "female") {birth rate * rabbits} else {0}

In this equation if the sex is female, the births are obtained as the birth rate times the number of rabbits. If this condition is not met, the result is 0. Delete the arrows from birth rate and rabbits to births.



Step 2. In the Model Overview - Entity Types display Population and double click on Population []. This opens the diagram and also the inspector of this collection that adds the entities. In the Population tab [] move the Aggregate icon to the diagram, and as before to obtain the "Sum rabbits" create the aggregate "Sum calculated births". In the variables of the Population inspector [] located on the right, the "Sum calculated births" has appeared.

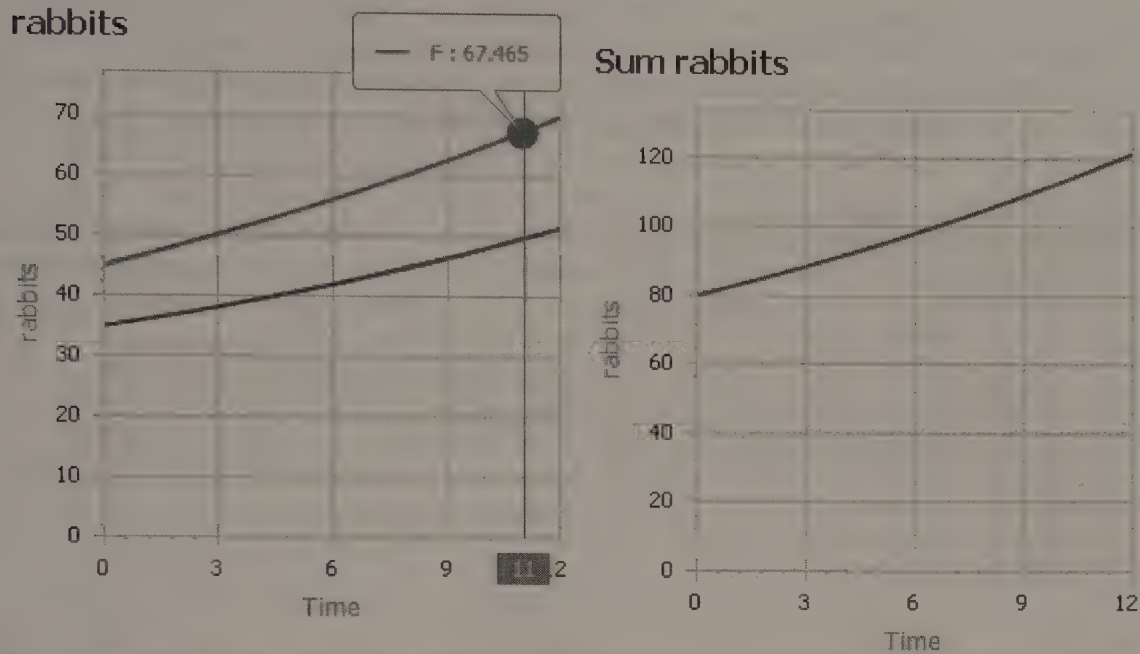
Step 3. Go to the upper Population tab to access the diagram, leaving the Population inspector [] open on the right. Move "Sum calculated births" to the diagram and add an arrow from it to births.



Make Edit Flow of births and write the equation that divides them in half so that each sex is assigned half of the total births.

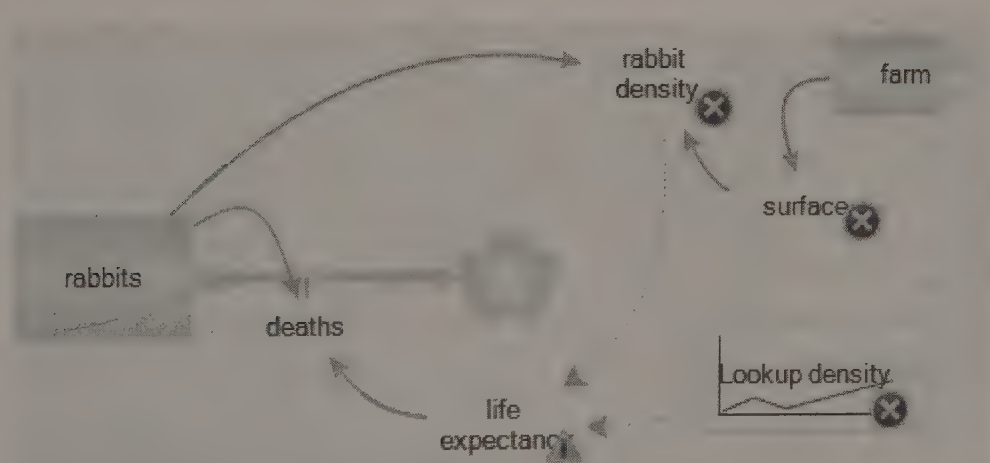
$$\text{births} = \text{Population Collection.Sum calculated births} / 2$$

Simulate the model. Right click on the rabbits stock, choose Plot Variable, go to the Population inspector [] select "Sum rabbits" and choose Plot Variable. Save the model.



Model: Population of Rabbits 4

The following model simulates that we are going to distribute the rabbits equally in two farms, North and South, with 500 and 1000 m² respectively, and we want to calculate the profit per farm taking into account that the life expectancy depends on the density of rabbits per m². The modifications that we are going to make to the model do not affect births but rather deaths. Create the following structure in the diagram.



Step 1. Farm attribute. Move the Attribute icon from the grid to the diagram, write the name farm.

In the diagram we use the attribute "farm" to identify if the rabbits are in the North or South farm. Based on this attribute we can to know the density in each farm, knowing the surface area (data that will be entered in the Initialization Data) and the number of rabbits. Then, based on the density we can calculate the life expectancy (using a table), and as in the previous version of the model, we calculate the deaths.

Auxiliary - Population.surface

Name:

Owner:

Units:

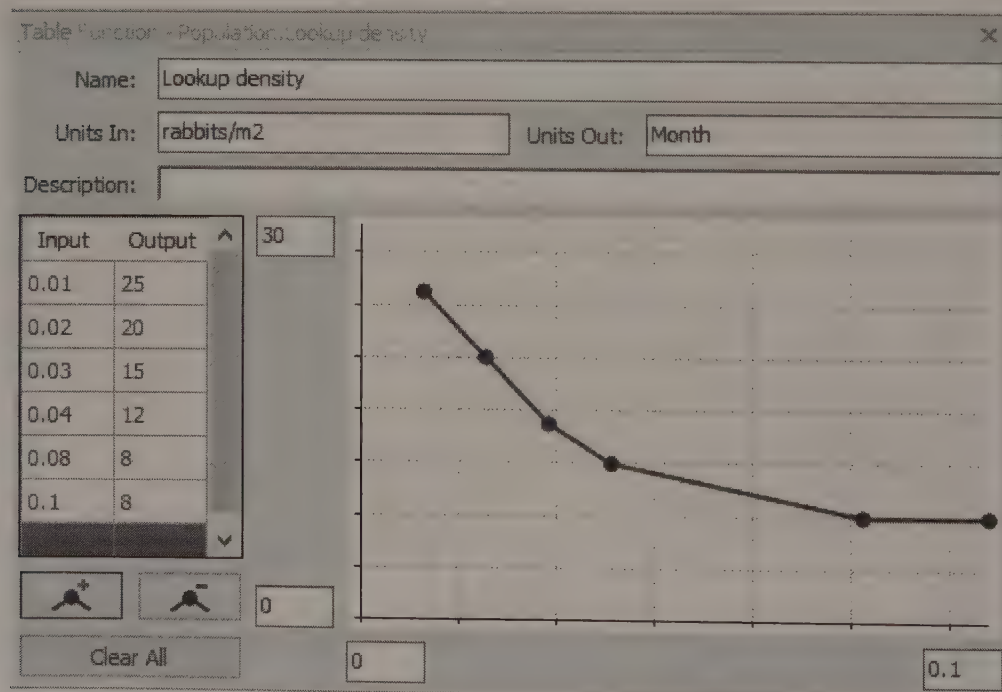
☐ Constant

```
if (farm = "North")
{500} else {1000}
```

Step 2. Use the Auxiliary icon to create the rabbit density and surface variables and move the life expectancy variable. Add the arrows from the diagram. The equations are:
 rabbit density = rabbits / surface
 units: rabbits / m2

surface= if (farm="North") {500} else {1000}
 units: m2

Step 3. Move with the table icon on the top bar to the diagram, right-click and choose Edit Table Function. Enter the values and units of the graph. With this we indicate that when the density is 0.01 the life expectancy will be 10 months, at higher density values there will be more life expectancy.



Modify the life expectancy equation (remove the Constant signal), it will be:

$$\text{life expectancy} = \text{Lookup density}(\text{rabbit density})$$

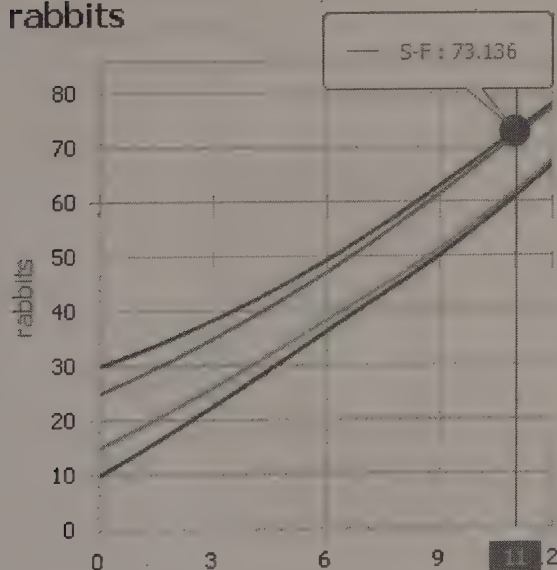
Tables are used when it is not possible to write an equation $Y = f(X)$ that easily relates two variables. In these cases, we go to draw some known or hypothetical points that show the values that the variable Y takes for certain values of the X .

Step 4. In Model Overview - Entity Initialization Data - Data Sources open Entities 1. In the Population tab, place the data that indicates the identifier of each entity (North-Male NM, etc.), the sex, the farm where they are located and the initial number of rabbits in each farm. Leave the rest of the columns blank.

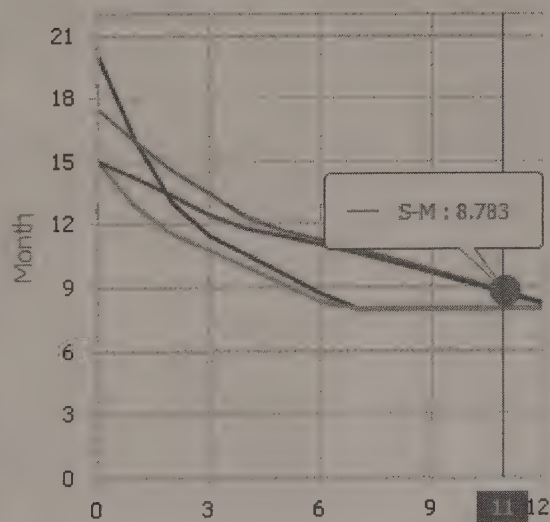
Entities 1									
Number of rows to add:									All None Invert ☐ Sho
Population									
	☒	Time	PopulationID	farm	sex	rabbits	life expectancy	birth rate	
1	☒		N-M	North	male	10			
2	☒		N-F	North	female	15			
3	☒		S-M	South	male	25			
4	☒		S-F	South	female	30			

Simulate, you can see the results in various graphs and formats. The growth of the rabbit population in the farms causes the life expectancy to be lower each time, since the surface is constant. Save the model.

rabbits



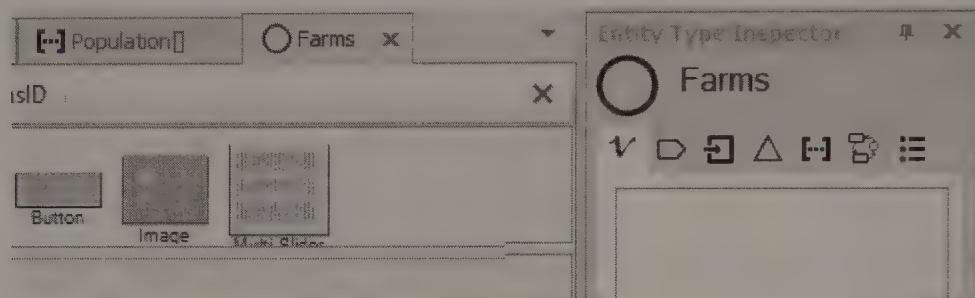
life expectancy



Model: Population of Rabbits 5

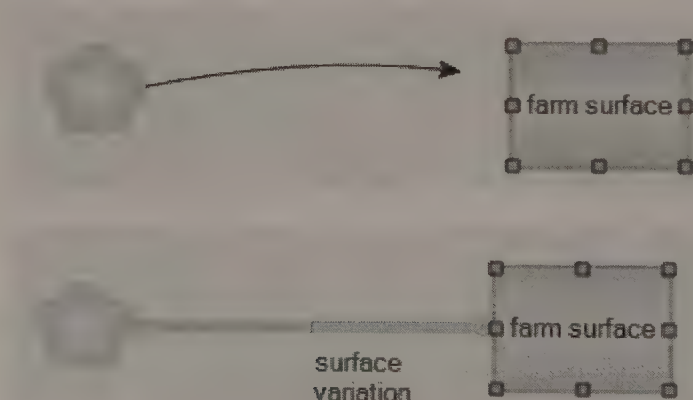
In order to increase life expectancy, it is proposed to act by increasing the surface area of farms. To analyze this proposal, a new type of entity called Farms will be created, and variables that link life expectancy with increased surface area will be included there. It is desired that the life expectancy is not below 10 months, and for this, if the life expectancy reaches that value, the surface area will be progressively increased.

Step 1. Create the new entity type. In Model Overview, right-click on Entity Types and select New Entity Type and write the name Farms. A tab with that name and an inspector appear.

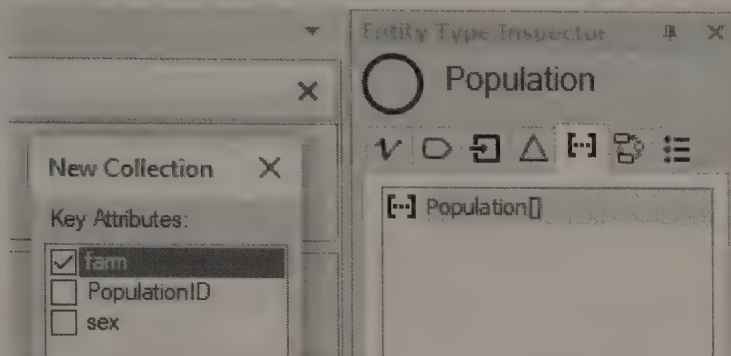


Step 2. Create a new variable for each farm surface and link it to life expectancy. Although there are many options to model the surface change, we will choose to create a stock with the name "farm surface".

On the Farms screen, move the Stock icon from the grid to the drawing area, write the name "farm surface", and an initial value of 1000 m2. Add a cloud to its left, moving the icon from the grid, and draw an arrow from the cloud to the stock, writing the name "surface variation", units: m2 / Month and value 0.



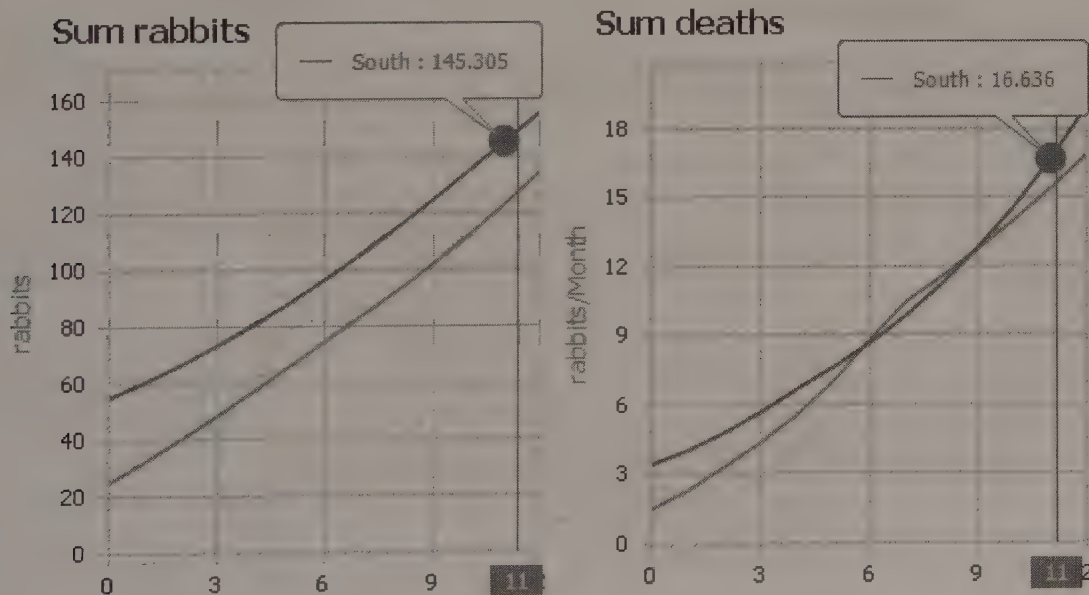
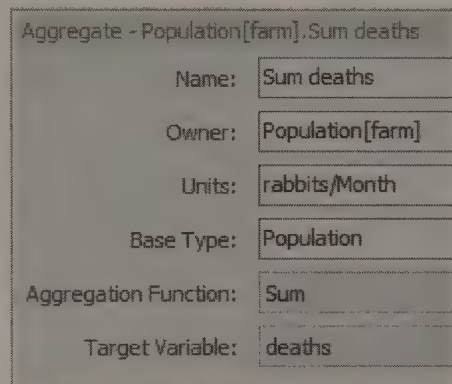
Step 3. Calculate the life expectancy on each farm. In each farm there are two groups of rabbits, males and females, the area will increase if the mean life expectancy of both falls below the minimum desired value (10 months). Total monthly rabbits and total deaths per farm need to be calculated to calculate life expectancy. We will create a collection (aggregate) of the rabbits per farm.

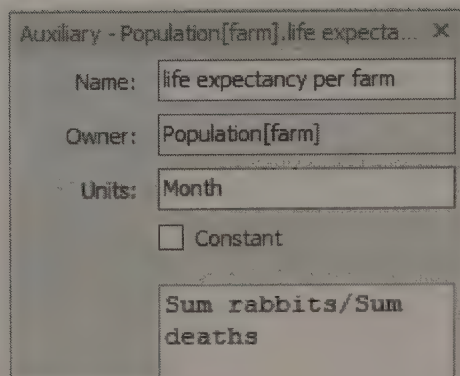


Open the Population inspector, to do this, double-click on Model Overview on Population. Go to the Collections tab and click New Collection (on the icon at the bottom). Select farm from the menu. The Population [farm] collection is created

In the Population [farm] diagram, move the Aggregate icon twice to the screen, create Aggregate 1 and 2. Make Edit in 1 and choose Aggregation Function = Sum and Target Variable = rabbits. Close. Press Edit on 2 and Target Variable = deaths. Close.

Simulate the model to verify that the created variables show the evolution of rabbits and deaths in the North and South farms. The created variables are shown in the Population [farm] inspector.



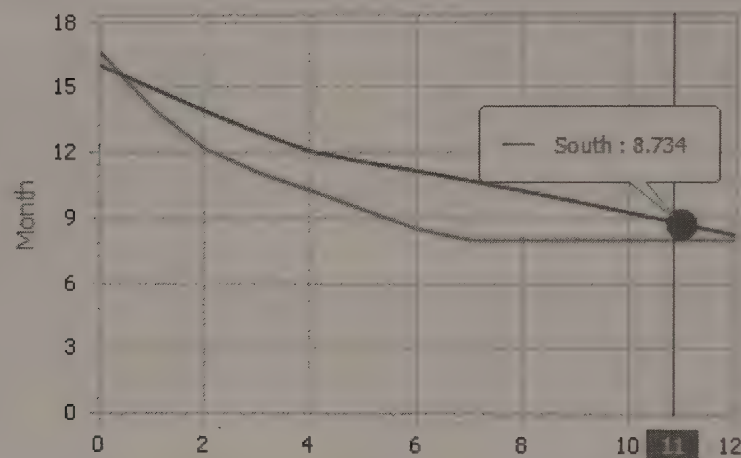


In the diagram Population [farm] create a new auxiliary variable (Auxiliary icon) that is the quotient between the sum of rabbits and the sum of deaths, which is the real-life expectancy per farm. Units = Month.

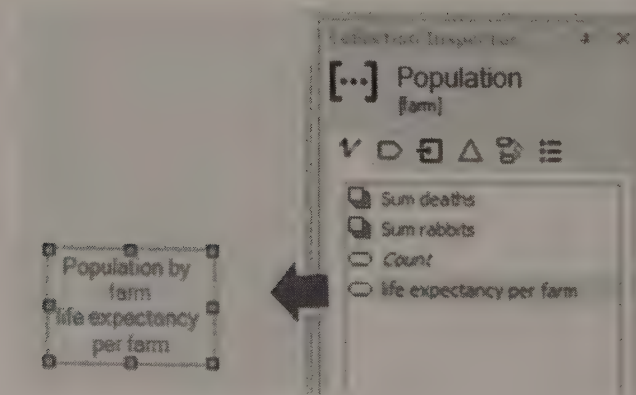
Simulate the model and make Plot Variable of the new variable.

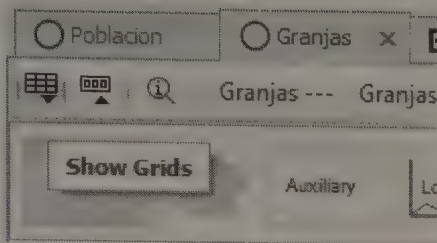
Both farms reach the established minimum limit (10) before the 12 months of study, therefore the action of increasing the surface will have an impact on both farms.

life expectancy per farm



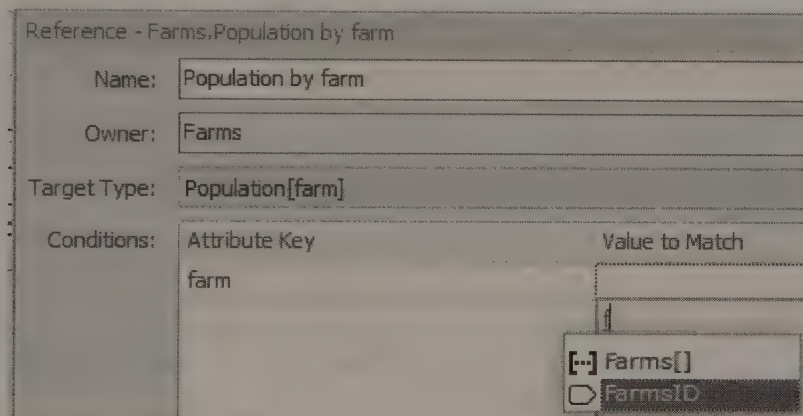
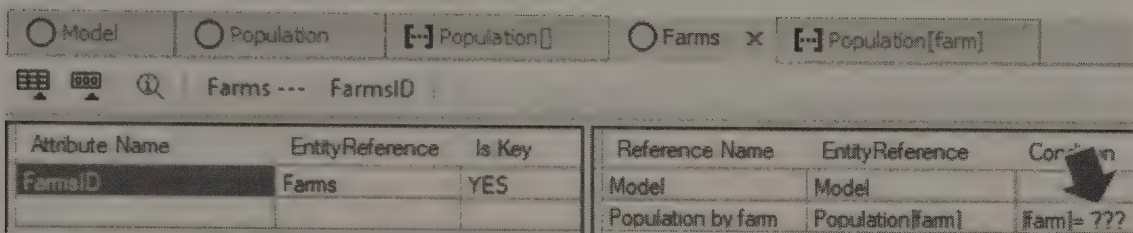
Step 4. From the Population inspector [farm] move the variable life expectancy per farm to the Farm diagram, since we will change the surface of the farms when necessary.





When a variable of one type is placed in another type, the reference grid must be opened. In this case the Farm reference grid is opened, which is where the variable has been placed, and on the right side ??? indicates what relationship is yet to be defined.

Click on ??? and in Value to Match select the FarmsID attribute, to link the farms attribute of the Population entities and the FarmsID attribute of the Farms entities.

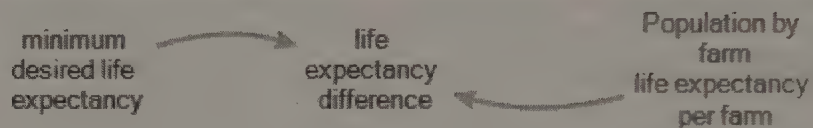


Step 5. We will create the variable "minimum desired life expectancy" in the Farm diagram, then we will calculate the difference with the real life expectancy, and this difference will be used to calculate the surface variation. We start by creating the auxiliary variable "minimum desired life expectancy" in the Farms diagram by moving the Auxiliary icon to the diagram, doing Edit and placing the value 10 in units Months. Mark Constant.

By marking Constant this variable is available in the Entity Initialization Data (Entities 1) and thus a value can be defined for each farm (which can be the same for all), but otherwise the value of that variable could not be applied to any farm. Click twice on Entities 1 to open it, if the Farms tab is not shown, check Show All Types, and click on the Farms tab. Enter the indicated values and close.

Entities 1						
Number of rows to add:						
Population		Farms		Model		
	☒	Time	FarmsID	farm surface	minimum desired life expectancy	Ca
1	☒		North	500	10	
2	☒		South	1000	10	

In the Farms diagram create a variable that measures the (relative) difference between the actual life expectancy and the minimum desired life expectancy. We are only interested in negative values, that is, when the life expectancy is less than the desired number, since if it is greater it will not reduce the surface. The MIN (0, ..) function suppresses the positive values.



Auxiliary - Farms.life expectancy difference

Name:

Owner:

Units:

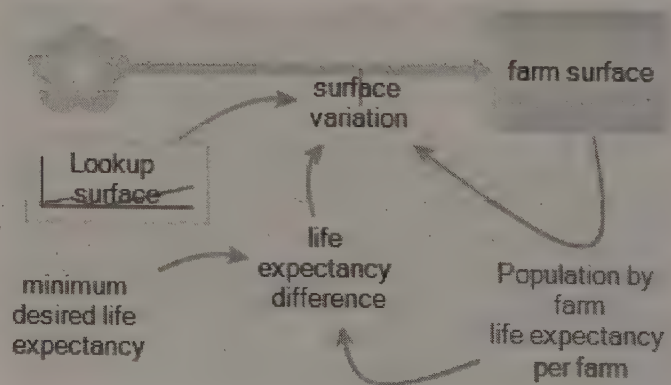
☐ Constant

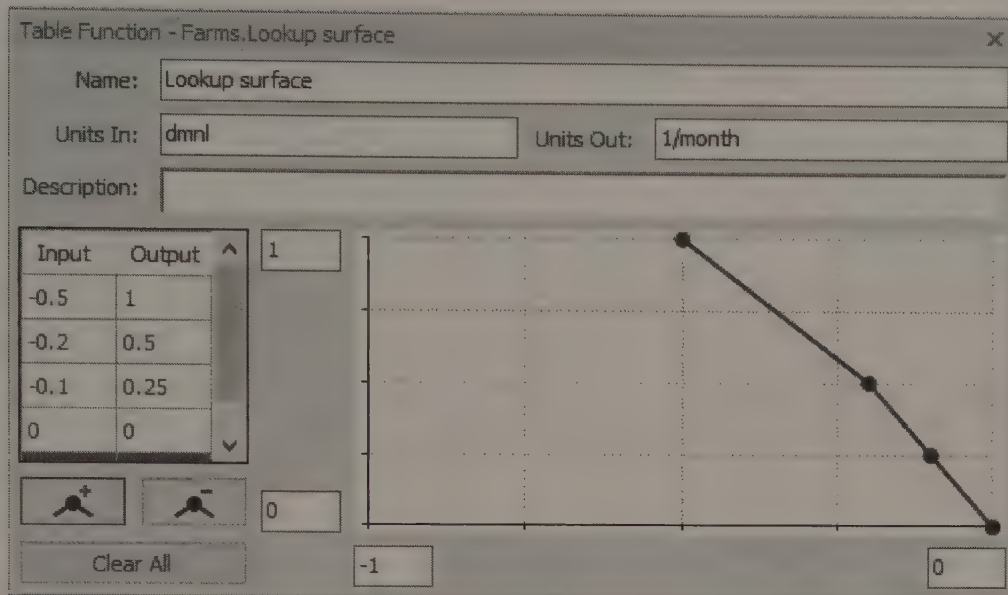
$$\min(0, (\text{Population by farm.life expectancy per farm} - \text{minimum desired life expectancy}) / \text{minimum desired life expectancy})$$

The "life expectancy difference" is ratio - that is why the units are dmnl (dimensionless) - instead of absolute values in order to later establish a simpler relationship: an x% difference in life expectancy generates a y% change in farm surface.

Step 6. Add the variables and relationships indicated in the following diagram.

Define the values of the "Lookup surface", for this, right-click on it and choose Edit Table Function. Enter the values and units.



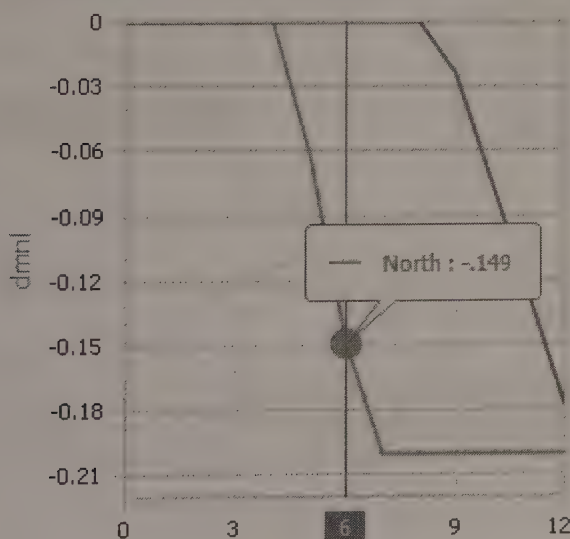


If there is no difference in life expectancy, or it is positive, the surface variation is 0, if the difference is -10%, because the life expectancy is 10% less than the desired minimum, the variation in surface area is increased by 25%. farm area.

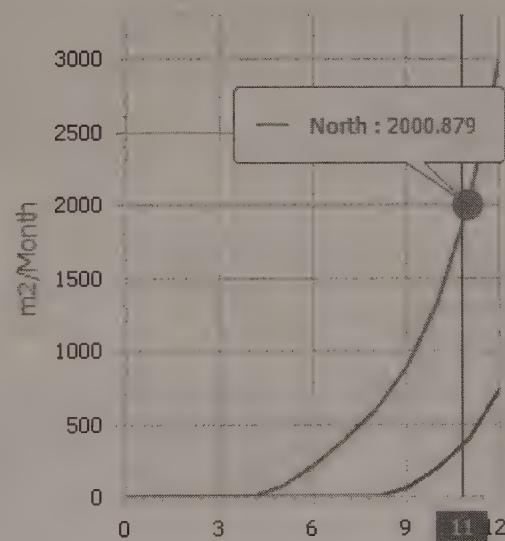
Enter the equation of: surface variation = surface table (life expectancy difference) * farm surface area, with this the difference in life expectancy is taken as input of the table - which is a % - and another % is obtained than is the increase in the monthly area of the farm. This % is multiplied by the surface of the farm to obtain the m2 of the extension.

Simulate the model. The graphs shown that the North farm and then the South have negative values of the difference in life expectancy, this increases the variable "surface of the farm".

life expectancy difference

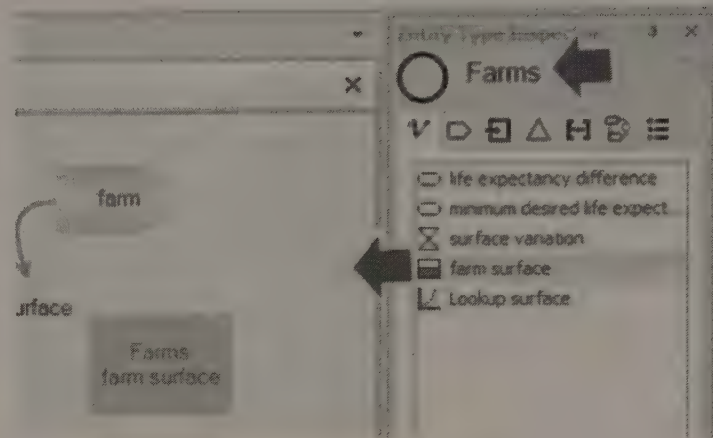


surface variation



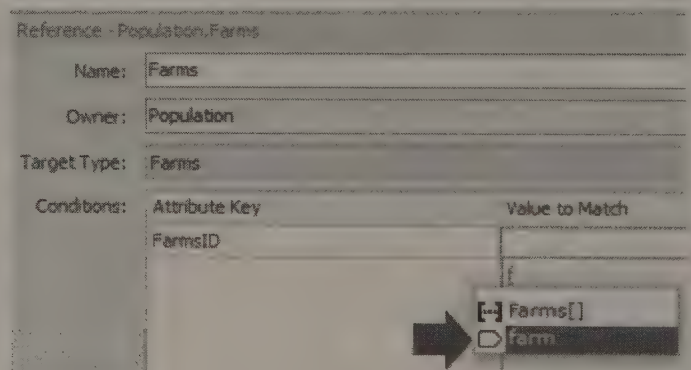
Step 7. Despite the stock "farm surface" does not act on the density of rabbits in the entity "Population", which continues to depend on the auxiliary variable "surface", the next step is to replace the dependency in the entity Population of that constant by the value of the stock.

From the Variables tab of the Farm inspector move the farm surface stock to the Population diagram. You have placed a variable from an entity (Farms) in another entity (Population), you must open the Population references grid and complete the Condition field.

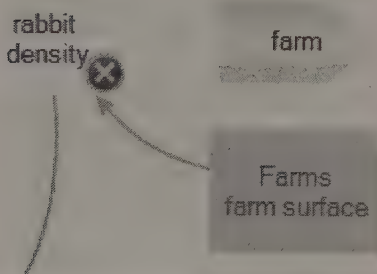


Reference Name	EntityReference	Condition
Farms	Farms	{FarmsID}= ???
Model	Model	
Population Collection	Population[]	

In the Value to Match field for Farms, select that FarmsID corresponds to the farm attribute. Close. Hide the grid of references.

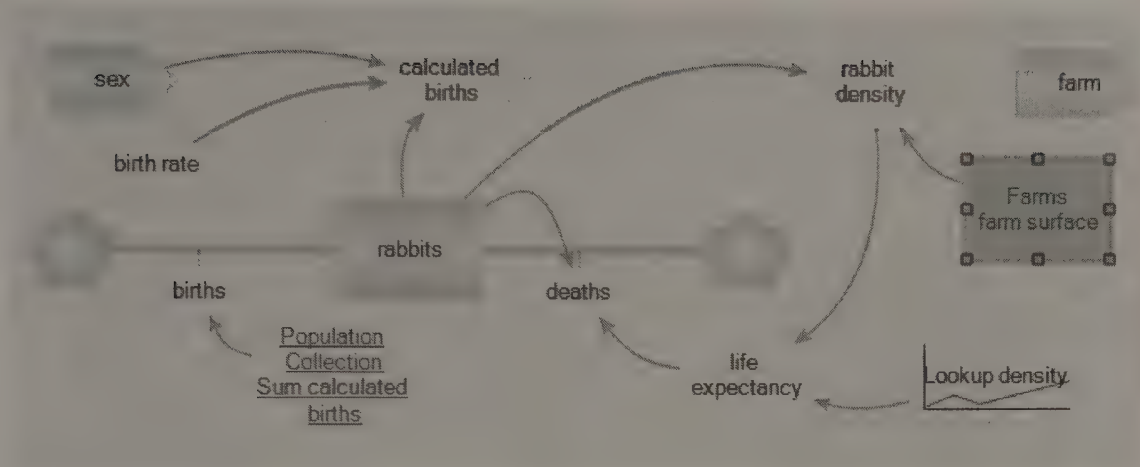


In the Population diagram delete the link from surface area to rabbit density and create a new one starting from "Farms farm surface".



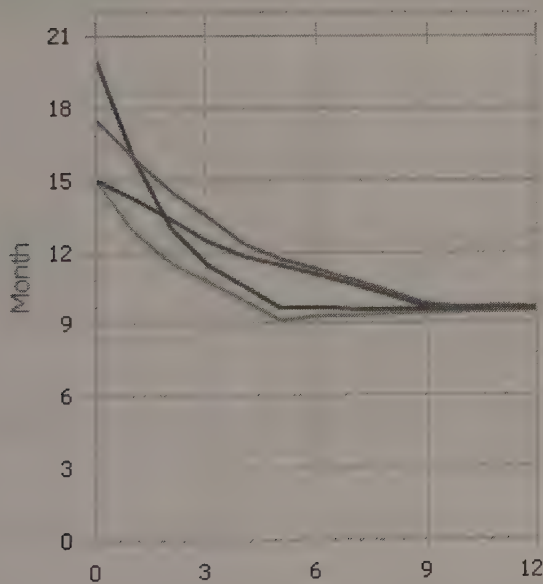
Edit the variable density of rabbits and replace the old surface with the new one.

In the Population inspector delete (option "Delete from Model") the surface variable.

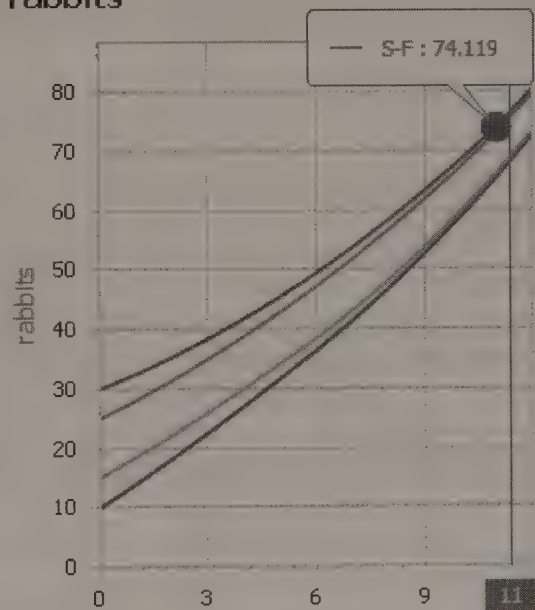


Simulate the model, go to Population tab and see the graph of the life expectancy and the stock of rabbits.

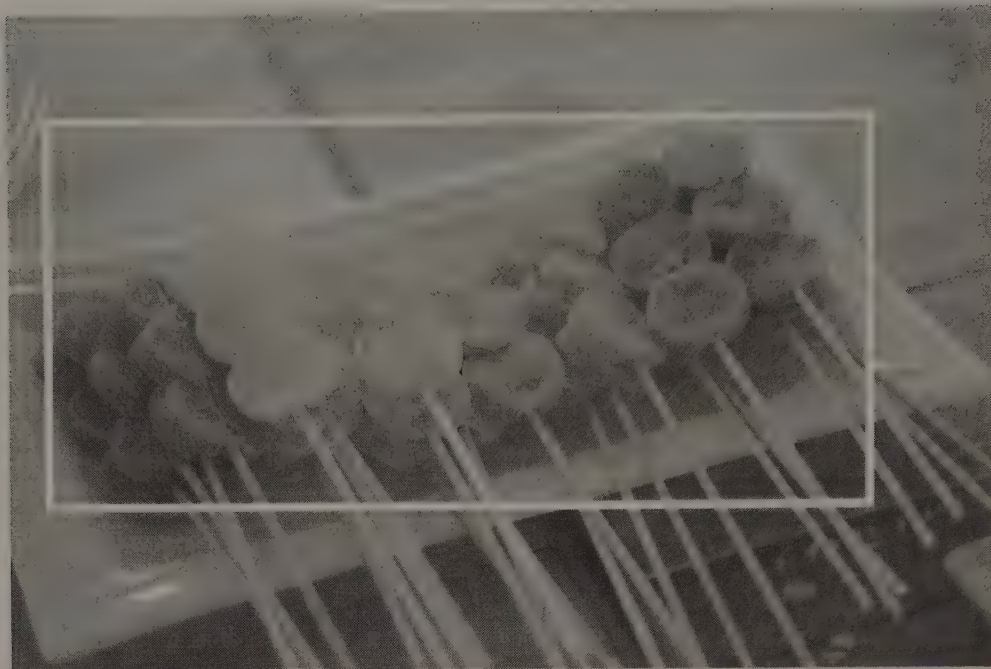
life expectancy



rabbits



The graph shows that life expectancy corrects and decreases to 10 months for all groups. The following graph shows how the number of rabbits increases (since a greater life expectancy reduces deaths).



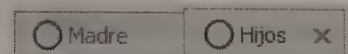
Model: Sweet Candies

A mother gives a candy to her son and another to her daughter when they come home from school, and she gives her daughter another candy on days when she is in a good mood. In this case we define the entities "Mother" and "Children".

To start, make File -New Model and create it with the name Candies. In Model Overview an entity called Candies is created. Click on it with the right mouse button, choose Rename and write Mother. Next, click on Entity Types again with the right button and choose New Entity Type, give it the name Children.

The Model Overview shows the two entities created, plus the Model entity that is created by default with the temporary parameters.

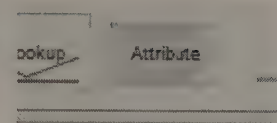
At the top of the screen there are two tabs, one for drawing the Mother diagram and one for the Children.



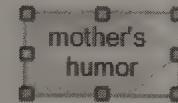
Initial Time	Final Time	Initial Calendar Time	Final Calendar Time	Time Step
0	30	09/28/2020	01/28/2021	1 (31 tot...)

Go to Model Overview - Run Control and type Final Time = 30 and Units for Time = Day, and close, to analyze what happens during a month.

It is necessary to include a variable that shows what mood the mother is in, which will not be numeric 0/1 but good/bad text, therefore we will not use a variable (Auxiliary) but an attribute (Attribute).



In the Mother entity window move the Attribute icon to the diagram and write the name "mother's humor". This way you can define in the initialization what the mother's mood is for each day of the month.



In the Children window, it is necessary to create an attribute called "mother", press Edit and in "Refers to" select Mother, to be able to tell which mother each child is from.

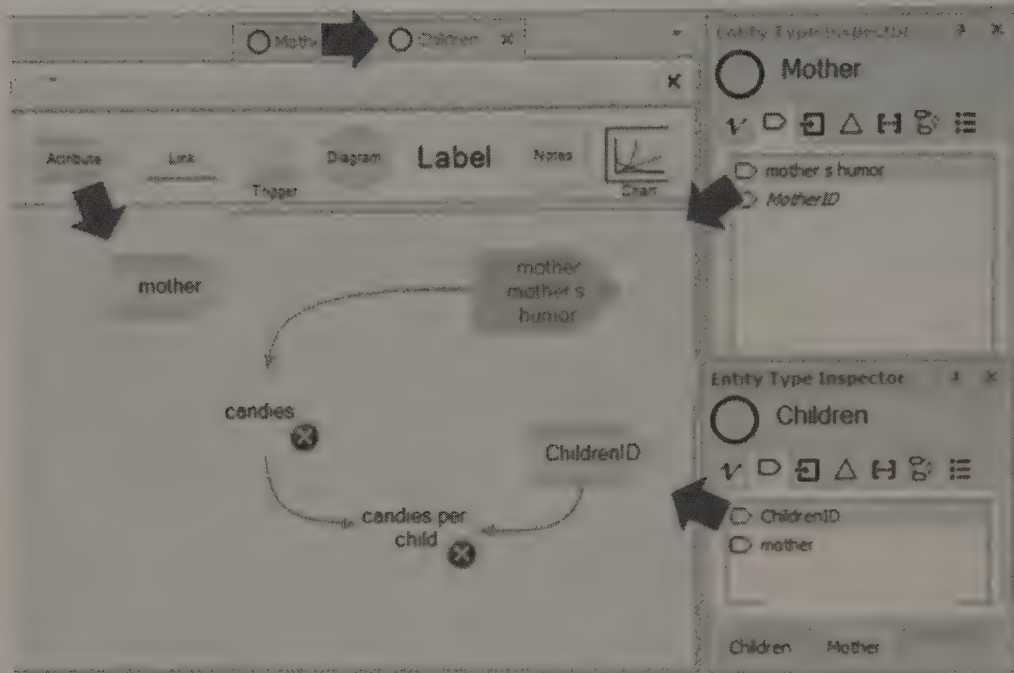
Go to Entity Initialization Data - Data Sources and choose Add Builtin Data, to create Entities 1. Double click on Entities 1 to open it. If any tab is missing, be it Mother or Children, check Show All Types. Go to the Children tab, and in Children ID enter the name of each one. In the mother column enter the mother's name.

Children				
	☒	Time	ChildrenID	mother
1	☒		Julio	Anna
2	☒		Rosalia	Anna

In the Mother tab fill in the MotherID field and then type the initial mood state (Time = 0) is good, it changes to "bad" on day 6, and it becomes good again on day 12. The way to do it is writing the sequence of changes.

Entities 1				
Number of rows to add:		+ - X Copy Paste Up Down		
Children		Model		Mother
	☒	Time	MotherID	mother s humor
1	☒		0 Anna	good
2	☒		6 Anna	bad
3	☒		12 Anna	good

In the tab of attributes of the inspector of Children take ChildrenID, and of the inspector of Mothers take "mother humor", moving both to the area of the diagram of Children. Finally, create two variables, one called candies and the other candies per child.



The variable "candies" is used to know if the mother is in a good or bad mood on the day of the week, and transform that text into values. In this case, if the mood is "good" the value 1 is generated, which indicates that there is 1 additional candy for someone, if it is "bad" there is no additional candy.

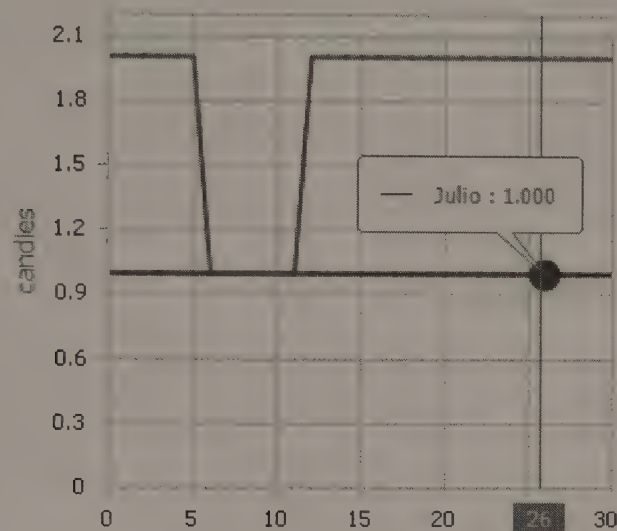
The variable candies per child results in that her son Julio always has 1 candy and that her sister Rosario has 1 plus another that depends on the mood of her mother.

Auxiliary - Children.candies	
Name:	candies
Owner:	Children
Units:	candies
☐ Constant	
<pre>if(mother.mother s humor="good") {1} else {0}</pre>	

Auxiliary - Children.candies per child	
Name:	candies per child
Owner:	Children
Units:	candies
☐ Constant	
<pre>if(ChildrenID="Julio") {1} else {candies+1}</pre>	

Click the Simulate icon. Then click with the right mouse button on the variable candies per child, in the menu choose Plot Variable. The result as expected is that Rosario receives two candies except between days 6 and 12.

candies per child



These results can be exported in numeric format. Go to Model Overview, click twice on Results and in the menu that opens click on the XLSX icon:



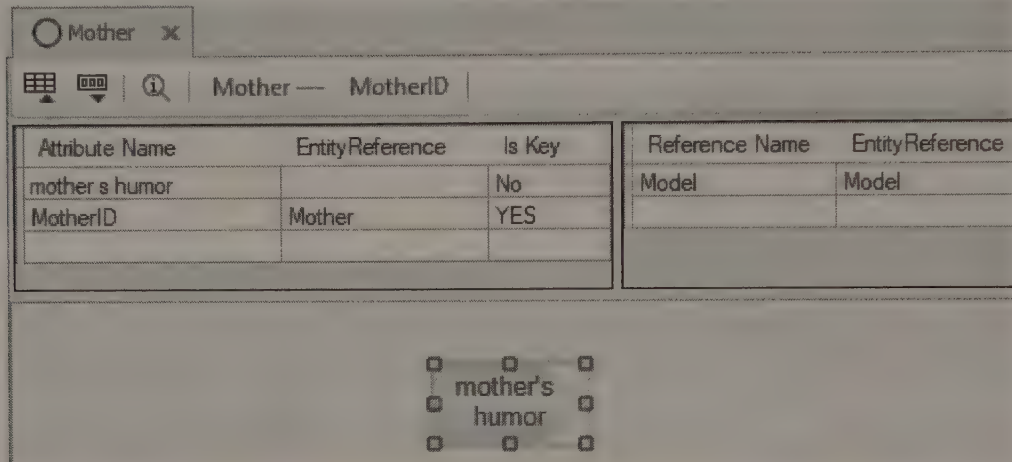
A window opens to choose the name of the file to save and its location. The Excel file contains all the simulation values on several sheets.

	A	B	C	D	E
1	DateTime	ChildrenID	mother	candies	candies per child
2	0	Julio	Anna	1	1
3	1	Julio	Anna	1	1
4	2	Julio	Anna	1	1
5	3	Julio	Anna	1	1
6	4	Julio	Anna	1	1
7	5	Julio	Anna	1	1
8	6	Julio	Anna	0	1

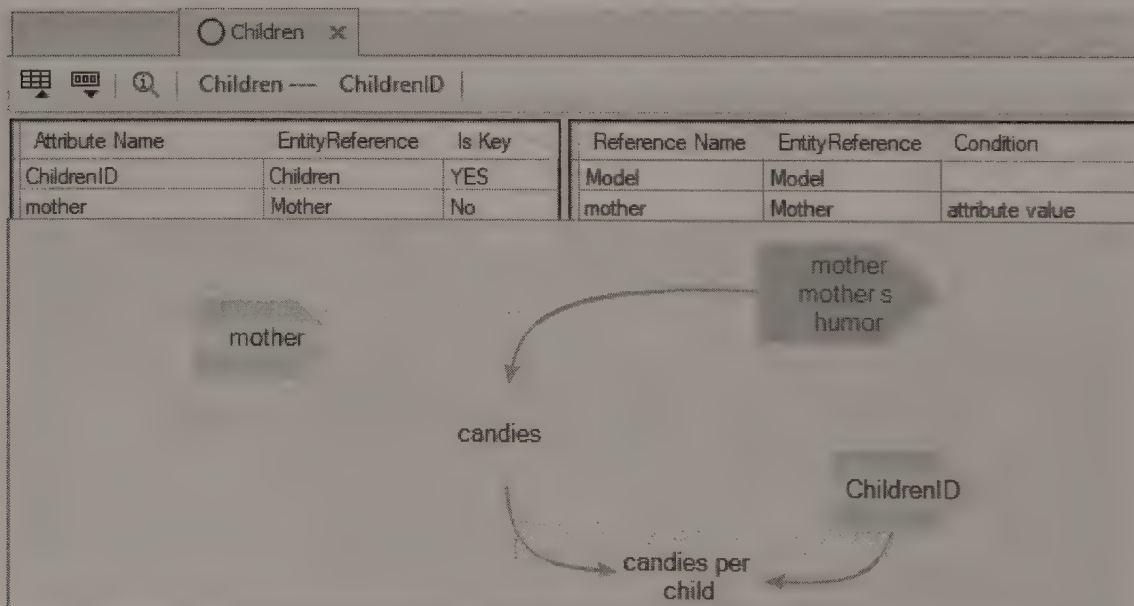
Children Mother Children() Mother() Model

Review of concepts: Attributes, Collections and References

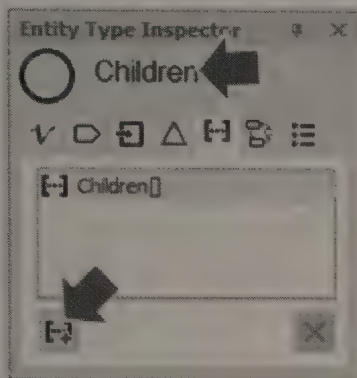
Go to the Mother diagram, click the icon to hide the drawing icons and click the icon to show the attributes and references. In the attributes column there are two, the one for MotherID which is by default, and other for mother humor that has been created since its content is alphanumeric (if it were numeric, an auxiliary variable would have been placed). In the reference column there is only the Model column, by default.



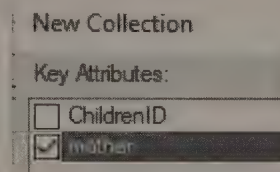
In the Children diagram, the attribute Mother appears in the Attribute column, which has been placed to define in the Initialization Data which mother each child is. This attribute belongs to the Mother entity, therefore in the Reference column it appears that the Children entity has a reference to the Mother entity.



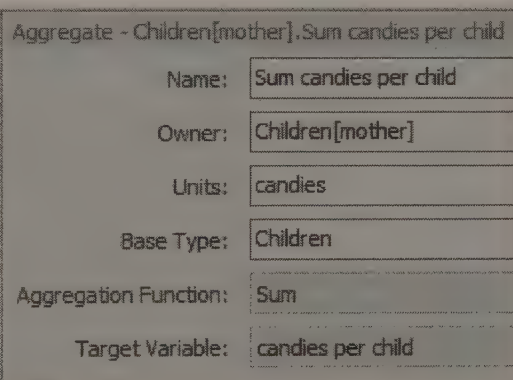
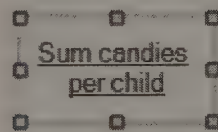
The next step is to calculate how many candies the mother has given each day, for this we must create a collection of Children who have the same mother (there is only one in this case).



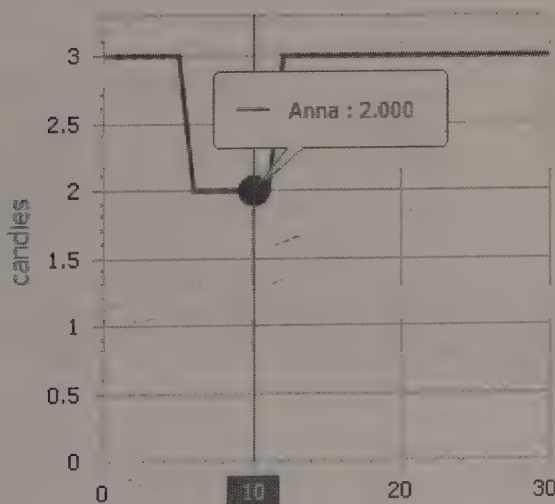
Go to the Children inspector (by clicking twice on the word Children in the Model Overview to open that inspector), and in the Collections tab, click the lower icon Add Collection. Click mother and OK.



Once the collection Children [mother] is created, move the Aggregate icon to the diagram, the variable Aggregate 1 is created, click on it twice and a menu opens, choose Sum and candies per child and close. The name of the variable is automatically written. This variable shows the total number of sweets per child delivered by each mother.

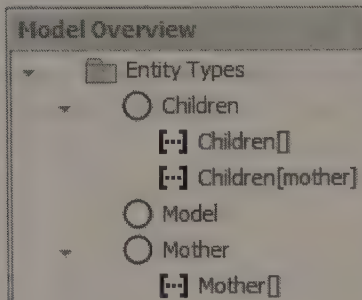


Sum candies per child



Simulate the model (with the green arrow), select the variable "Sum candies by child", press the right button and select Plot Variable. The graph shows the candies given by Ana.

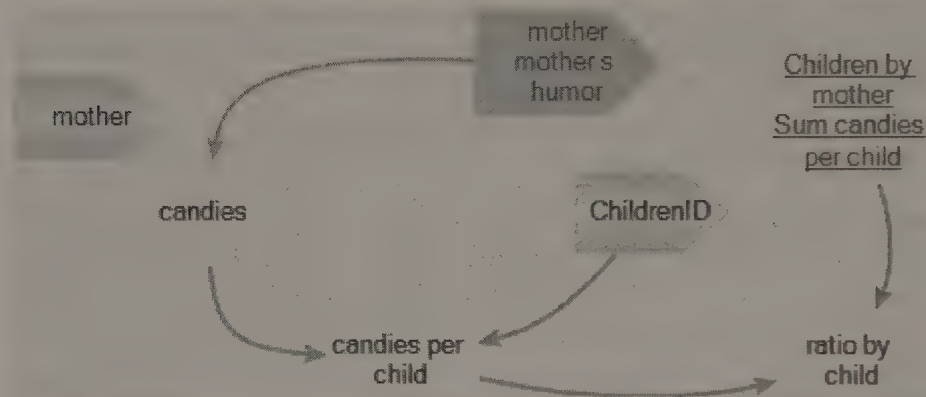
Now we are going to calculate the proportion of total delivered candies each day, which each child receives. To calculate the quotient of candies per child relative to the total, we will place the variable "Sum candies per child" in the Children diagram.



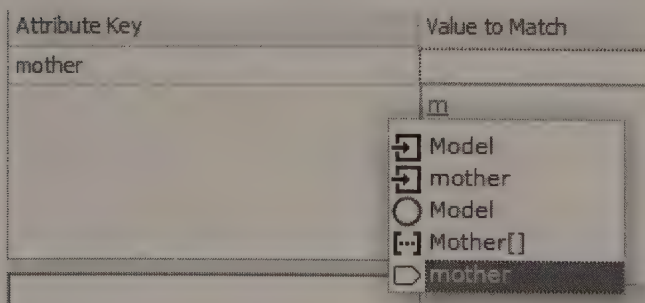
Expand the Children [mother] inspector by going to the Model Overview and double clicking on Children [mother]. The inspector opens on the right side of the screen.

Open the Children diagram and move the variable "Sum candies per child" from the Inspector of Children [mother] to the Children diagram.

Then move the Auxiliary icon to the diagram and create the variable "ratio by child", add arrows to that is dependent on Sum candy per child and candy per child.

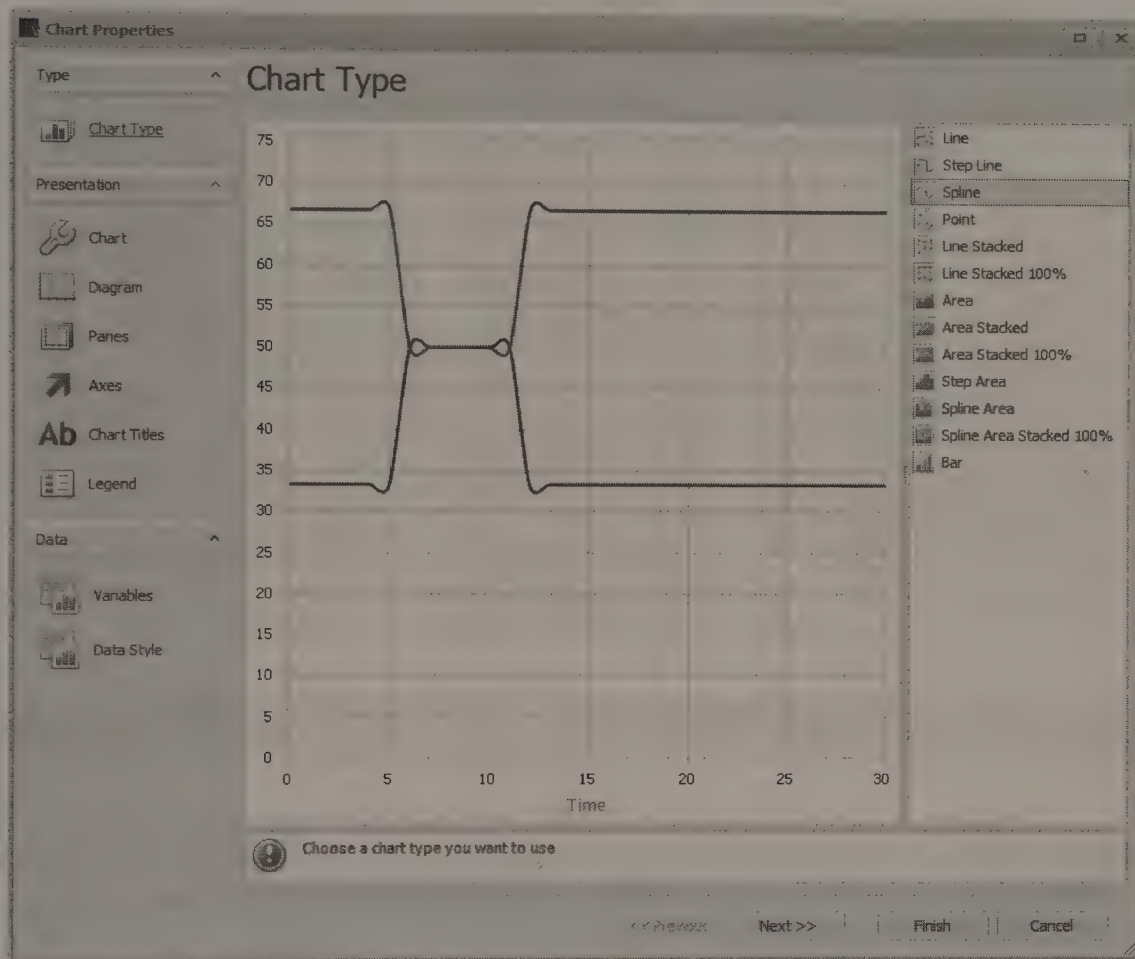


Press twice on "ratio per child", in the window that opens write the equation: $\text{candies per child} * 100 / \text{Children by mother Sum candies per child}$. In units write Dmnl (dimensionless) since it is a ratio and has no units.



When placing the Children [mother] aggregate on the Children diagram, a Reference to Children [mother] appears in the Condition column of the reference grid, which displays???. Click on [mother] = ??? In the window that opens, select the mother attribute in Value to Match.

Simulate, and see the evolution of the ratio by child with Plot Value, the graph shows that Borja always has 33% except between days 6 and 12 which has 50%. Then press the Edit Chart Properties button and try several options.



Save the model (File - Save).

Add entities

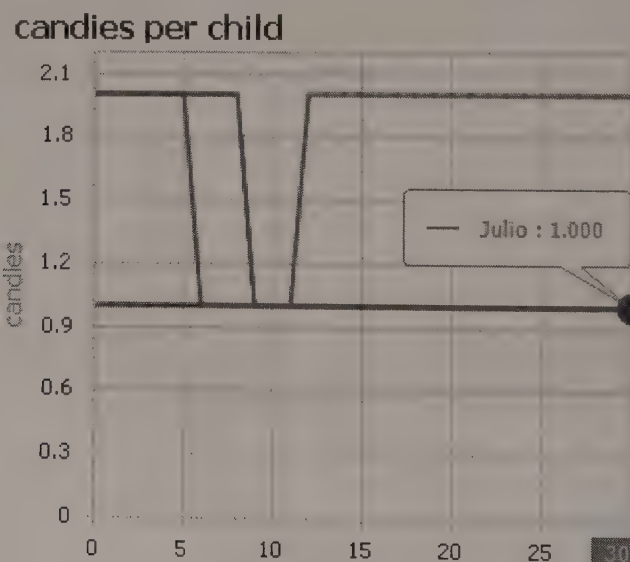
We are going to add a mother entity, and two child entities of this new mother. To do this in Entities 1 enter the following data:

Entities 1					
Number of rows to add:					
Children			Mother		
	☒	Time	CalendarTime	MotherID	mother s humor
1	☒		0	Anna	good
2	☒		6	Anna	bad
3	☒		12	Anna	good
4	☒		0	Eva	good
5	☒		9	Eva	bad

Entities 1					
Number of rows to add:					
Children			Mother		
	☒	Time	CalendarTime	ChildrenID	mother
1	☒			Julio	Anna
2	☒			Rosalia	Anna
3	☒			Mateo	Eva
4	☒			David	Eva

It is necessary to define the attribute "mother's mood from period 0, since the simulation require her to be informed. Save and simulate the model.

We can see that the model variables have included these new entities. The mother "Eva" delivers 4 candies on days when she is in a good mood and 2 on days when she is not, since none of her children has Borja's restriction.





Model: Cheese Shop

We are going to simulate the evolution of the stock of a store that sells cheeses, and its relationship with the distributor, with the aim of seeing the use of various delay functions, random functions, step and ramp.

Select File - New Model and write Cheese as the name of the model. Then go to Run Control and type Final Time = 60 and Units for Time = Days. In View - Units Manager uncheck the Check Units option.

Functions reminder

RandomNormal (mean, stdev)

Creates a random value with normal distribution, with its mean and standard deviation.

Step (height, time)

It has the value 0 until the period time, then the value defined in height.

Ramp (slope, start, stop)

It has the value 0 until the start period, increases with the slope until the stop period, and is a constant value thereafter.

DelayInformation(variable,timedelay,initial value)

Returns the value of the input variable with a delay based on the timedelay variable. DelayInformation can only be used to calculate values based on past data, and only positive integers are accepted in the timedelay variable.

SMOOTH (input, delay time)

SMOOTHI (input, delay time, initial value)

DELAY1 (input, delay time)

DELAY1I (input, delay time, initial value)

DELAY3 (input, delay time)

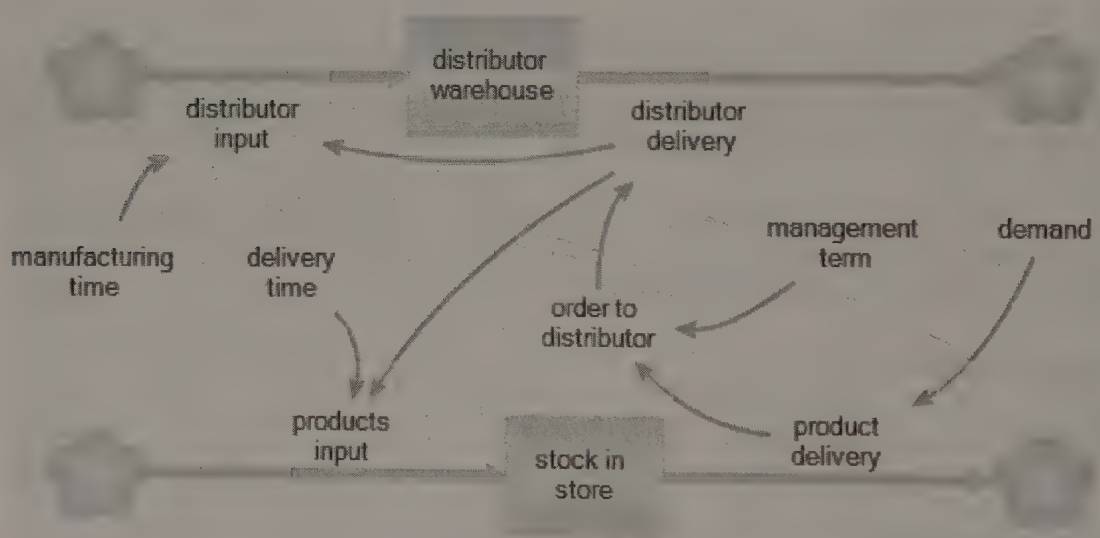
DELAY3I (input, delay time, initial value)

Returns a first or third order exponential smoothing of the input variable during the delay time with / without initial value.

Create the Diagram

The diagram shows the inputs and outputs of the dealer and store warehouses, and the variables that influence those flows. The only independent variable in this scheme is consumer demand.

In this model there is only the entity Cheeses. Draw the following diagram starting with the Stocks, add the clouds and the Flows, complete the diagram with the Auxiliary Variables and the arrows that relate these elements.



Once you have drawn the diagram, it is necessary to write the equations. For this, right-click on each variable, choose the Edit option and enter the equation.

Equations

distributor warehouse = 10000

They are the units that are in the distributor's warehouse in the initial period.

stock in store = 100

They are the units that are in the store's warehouse in the initial period.

demand = 10

In the first version of the model we will simulate a CONSTANT demand of 10.

order to distributor = DELAY1(product delivery, management term)

DELAY1 function provides a first-order response faster than other types of delay functions, so we simulate that this process is quite fast.

management term = 2

It is the number of days late for the order

distributor delivery = order to distributor

Orders are assumed to be delivered the same day they are received

distributor input = DELAY3(distributor delivery, manufacturing time)

It simulates that there is a delay of order 3, i.e. little response flexibility in relation to the value of the supplier's producer output.

manufacturing time = 30

It is the number of days of the manufacturing delay.

product delivery = demand

The products are delivered on the same day that the buyer demands them.

delivery time = 5

It is the number of days of delay in arrival since the distributor makes the shipment.

products input = delayInformation(distributor delivery, delivery time, 10)

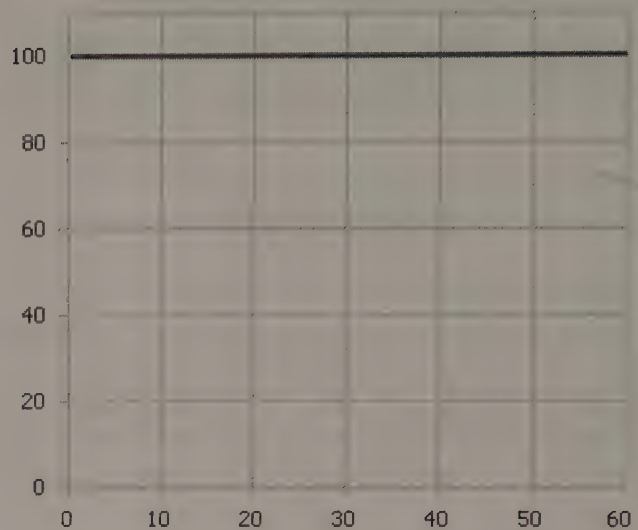
It is simulated that there is a fixed delay based on the delivery period. An initial value of 10 is used for this flow, which assumes that before the initial period there is already activity. Manually write the function since it does not work if you click on the option offered by the software.

Simulate the model.

Results

With the hypothesis that we have introduced of a constant demand, we can observe after simulating the model that all the variables remain constant at their same initial values, for example the stocks of the stores in 100 units.

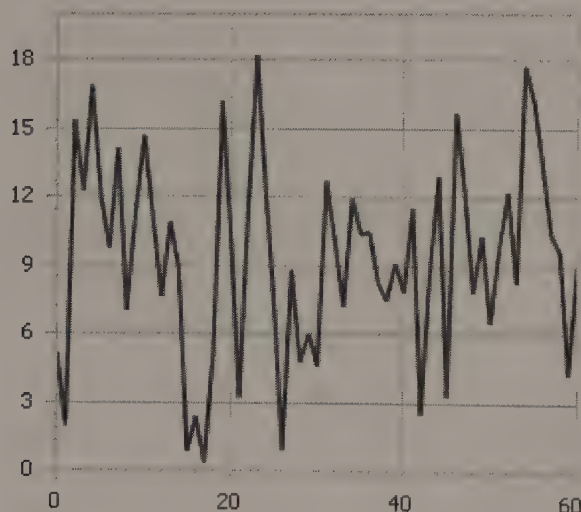
stock in store



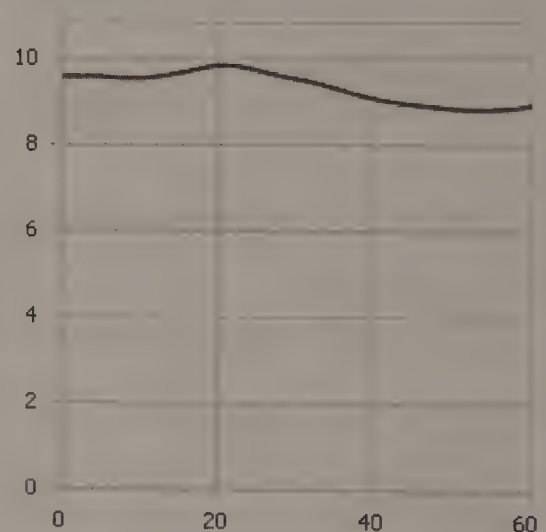
Simulation with random demand

Modify the demand equation = $\text{RandomNormal}(10, 4)$. Go to Random Control, in Random Seed write 6789 (so you get the same results as the book), close. Simulate and see the result.

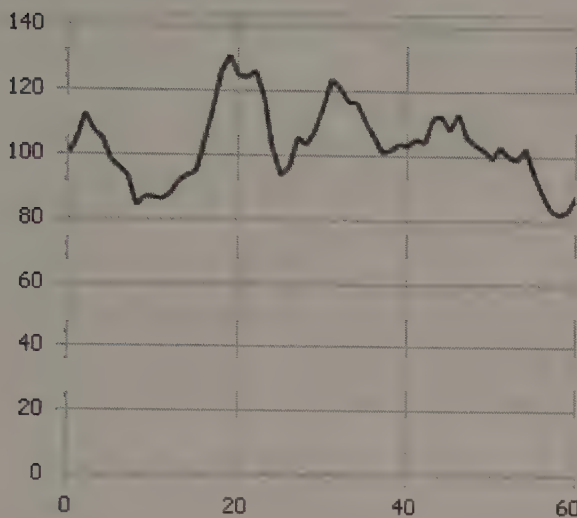
demand



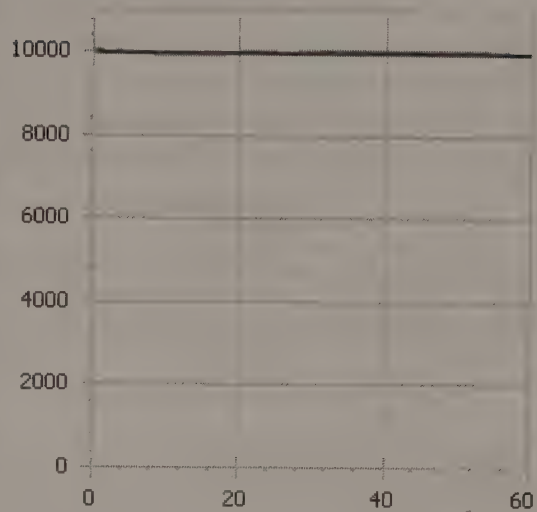
distributor input



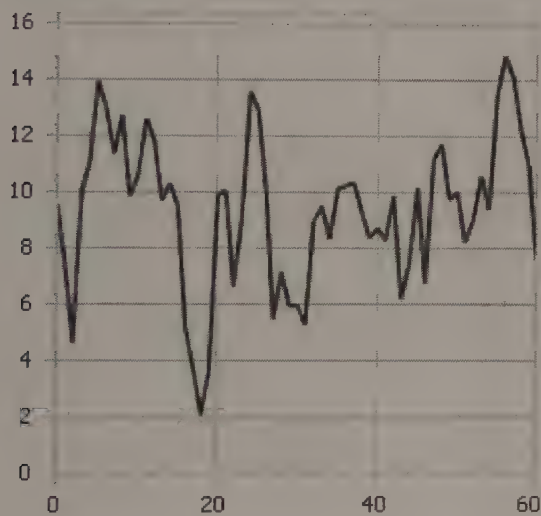
stock in store



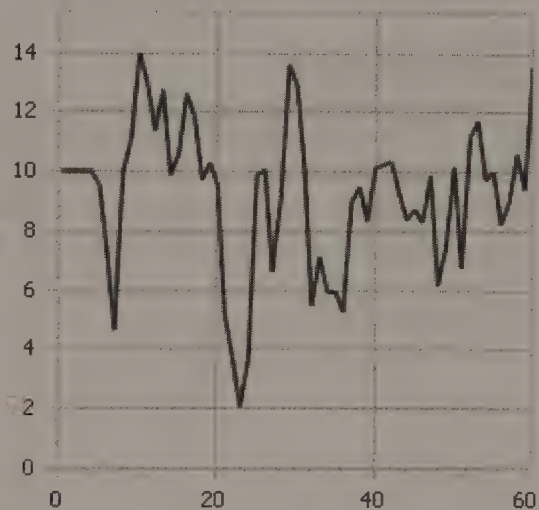
distributor warehouse



distributor delivery



products input



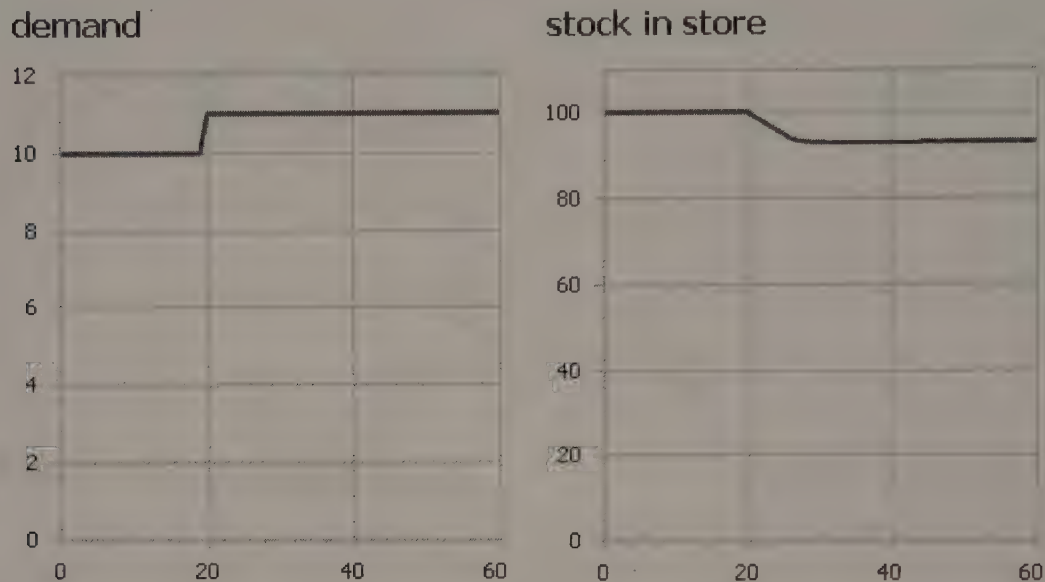
The values obtained depend on the defined value of the Random Seed. The results are not a forecast but show the relative impact of a random factor.

The previous graphs show the degree of impact that introducing a random factor in the demand variable has on the system variables, based on the hypotheses that have been taken.

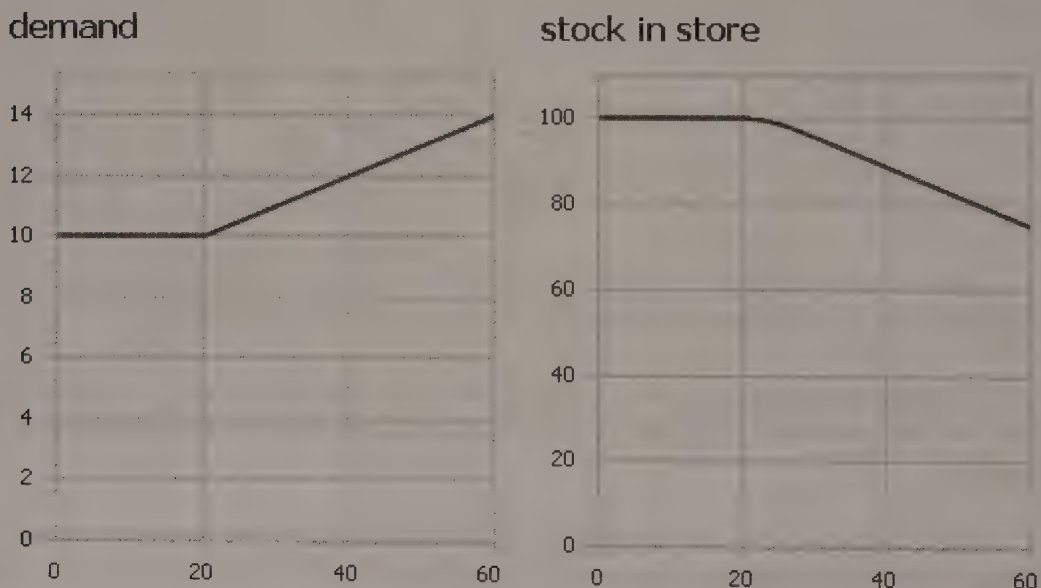
Now we will return to the model with a stable demand to introduce changes in the form of STEP (increasing demand by 10% in period 20), and in the form of RAMP (an increase of 1% from period 20).

Demand simulation

Simulate that demand is constant (value 10) until period 20, when it increases by 10%. Modify the equation: $\text{demand} = 10 + \text{Step}(1, 20)$



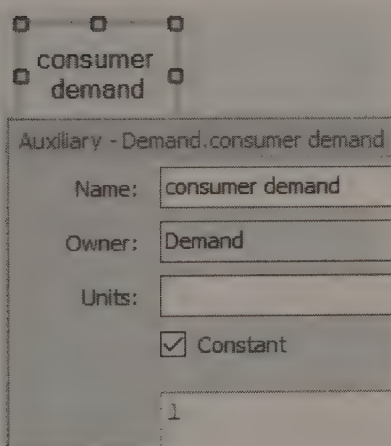
The decision-making system reflected in the model shows that an increase in demand translates into a decrease in the quantity of products in the store's warehouse, since the input of products has not equaled the output for some time. Write the new equation $\text{demand} = 10 + \text{Ramp}(0.1, 20, 99)$ and simulate.



ABM models allow one to simulate many entities and we will simulate that we have 10 consumers. Right click on Entity Types, choose New Entity Type and write Demand. A new type of entity is created with its diagram in the center and the inspector on the right.

If we open the Attribute tab of the Demand inspector, we will see that a DemandID attribute has been created that identifies each of the customers. Move the Auxiliary icon from the bar to the diagram and put the name "consumer demand", press Edit in it, write the value 1 and mark Constant.

Next, we will define the 10 customers of the store. In the Model Overview go to Entity Initialization Data, display the options, with the right button click on Data Sources and choose Add Builtin Data. In the window that opens, click on the Demand tab. Enter the following data in which we put a "name" to each client (A, B, C, ...) until 10, although it is not essential, and in consumer demand place the value 1 for each.



Auxiliary - Demand.consumer demand

Name: consumer demand

Owner: Demand

Units:

☒ Constant

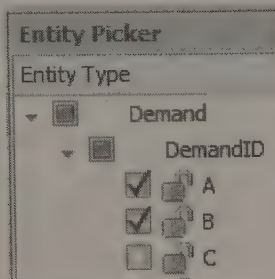
1

Demand			Model	
	☒	Time	DemandID	consumer demand
1	☒		A	1
2	☒		B	1
3	☒		C	1
4	☒		D	1
5	☒		E	1
6	☒		F	1
7	☒		G	1
8	☒		H	1
9	☒		I	1
10	☒		J	1

We are assuming that each customer makes one purchase per day.

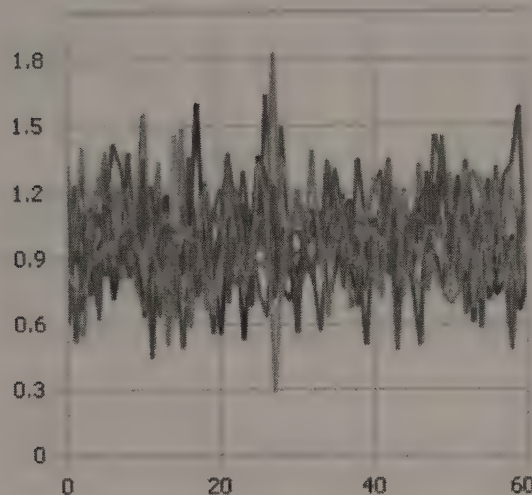


Let's add random consumer behavior. Create a new auxiliary variable called random demand, which depends on the customer's demand. Write the equation: $\text{random demand} = \text{RandomNormal}(\text{consumer demand}, 0.2)$

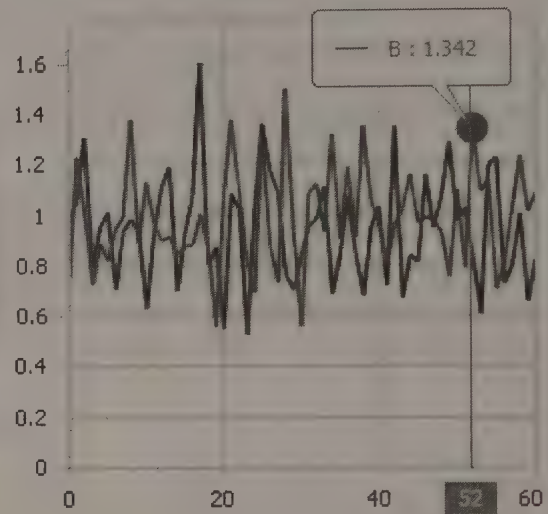


Simulate the model. The result shows the random demand graphs of the 10 clients, with values between 1.8 and 0.3. Since it is difficult to analyze we are going to select a couple of clients, for example "A" and "B". Go to Entity Picker (bottom left window), display Demand and Demand ID, then deselect all of them and mark four and five.

random demand



random demand



The graph now shows only those two entities A and B.

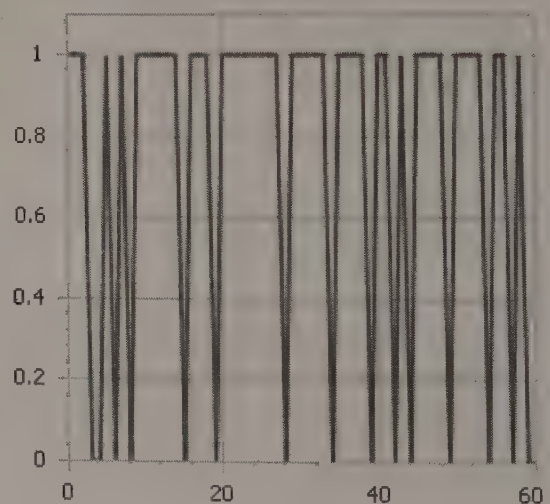
Since the results are slightly unrealistic fractions, we are going to modify them to be binary (buy or not buy). To do this modify the equation to:

```
random demand = if (RandomNormal
(consumer demand, 0.2) > 0.8 &&
RandomNormal (consumer demand, 0.2)
< 1.2) {1} else {0}
```

In this equation we add the "if" function so that if a condition is met (that the variable is between 0.8 and 1.2) the result is 1 and if it is not, it is 0. The && signs are the equivalent of: AND: from Excel. In the Entity Picker we leave only the client "C" active, select Plot Variable.

The graph shows on which days the customer "C" will make purchases.

random demand

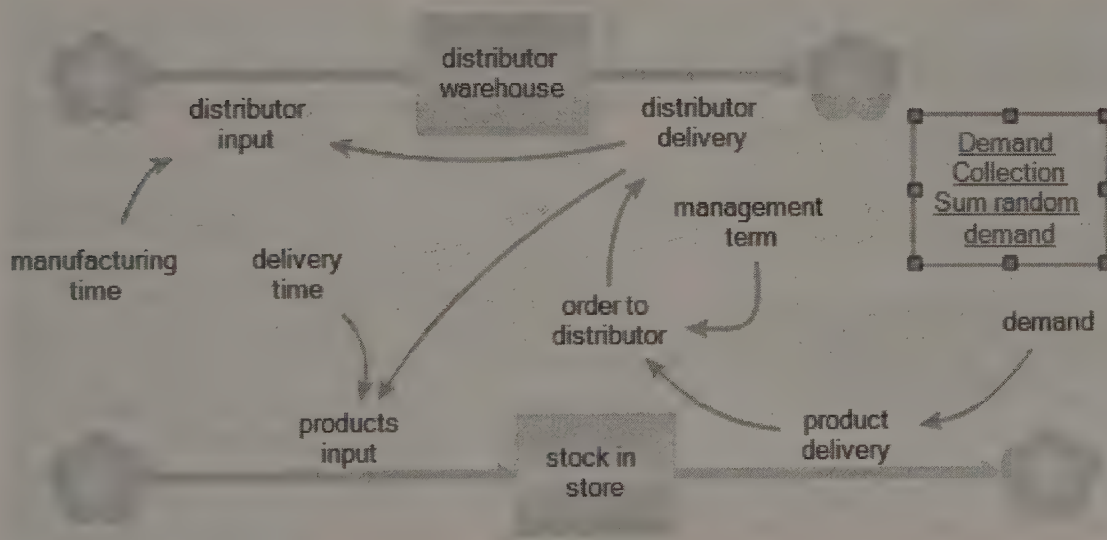


Now we are not interested in knowing the individual demand of each client, but the aggregate demand of ALL of them. It is not necessary to create a collection of all entities, since it is created by default in Demand []

In the Model Overview display Demand, double click on Demand [], this opens the screen. In that window move the Aggregate icon to the diagram, right-click on it and choose Edit Aggregate. In Aggregation Function choose Sum (since we want the total demand) and in Target Variable choose "random demand". Close.

Aggregate - Demand[], Sum random demand	
Name:	Sum random demand
Owner:	Demand[]
Units:	
Base Type:	Demand
Aggregation Function:	Sum
Target Variable:	random demand

The next step is to substitute the "demand" variable in the Cheese diagram for the total "random demand". Keeping the Demand [] inspector opened, open the Cheese diagram (by clicking on its tab at the top of the screen). Move the variable "Sum random demand" from the inspector to the diagram and the variable "Demand Collection Sum random demand" is created. It is not necessary to define any reference because ALL the entities of the demand are taken.



With the right button click on the variable "demand" and choose "Delete from Model". Add an arrow from the "Collection Demand" to "products delivery", and edit the latter, modify the equation, which is now equal to the "Collection Demand ...".

Go to the Entity Picker - Entity Type and activate the total of the Demand to see the global results.

Go to Entities 1, click on the Cheese tab (check Show All Types if it is not visible), check the box on the left to create a default entity and enter the data shown in the image. Close.

Entities 1

Number of rows to add:



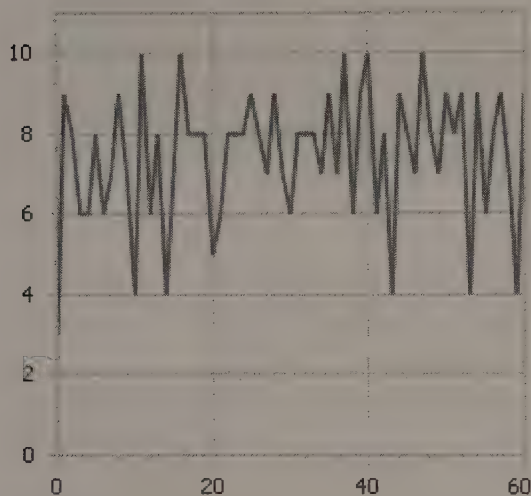
AllNone

Chesse

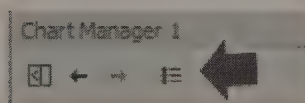
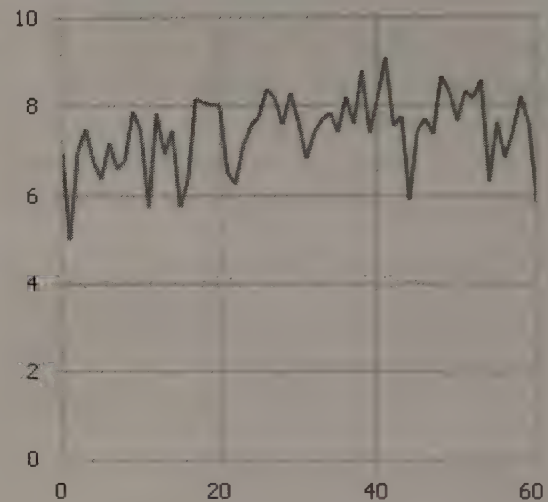
	☒	Time	ChesseID	distributor warehouse	stock in store
1	☒		Seville	10000	100

Simulate.

product delivery

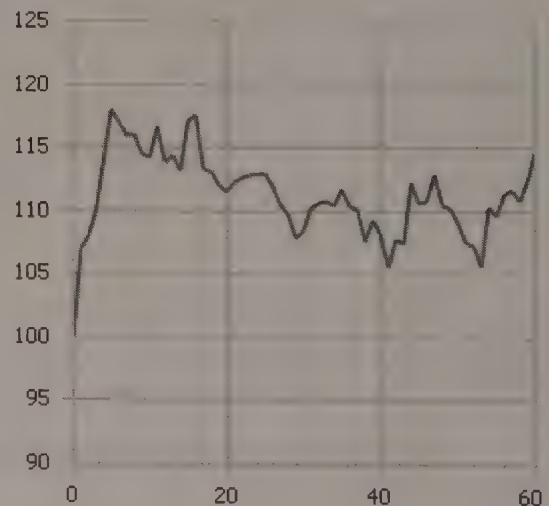


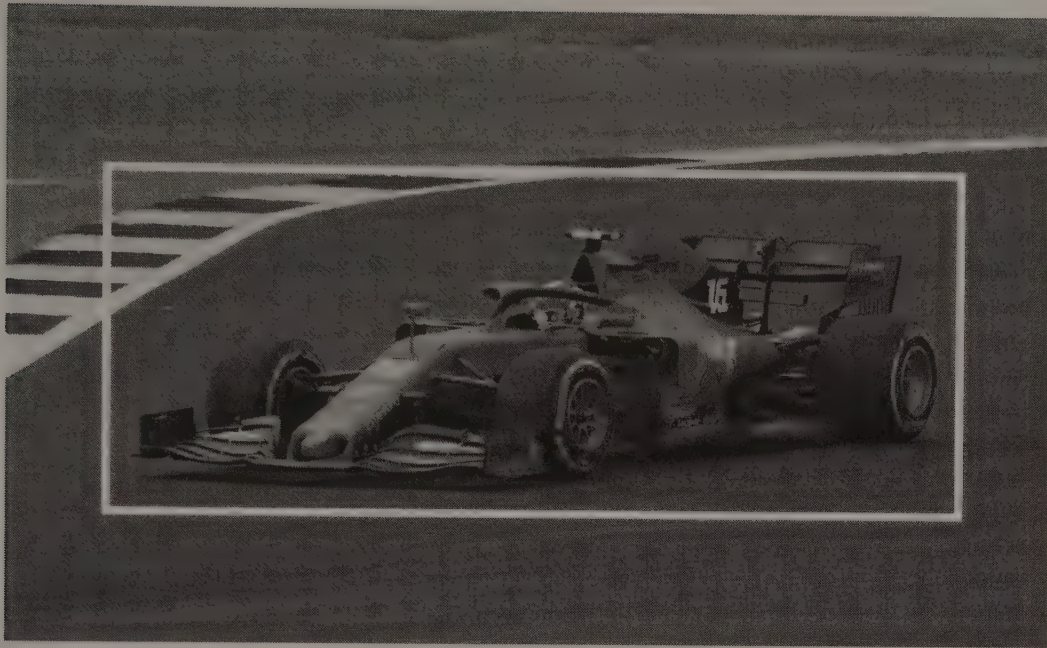
distributor delivery



The "store warehouse" graph is very off-center, click on the Edit Chart Properties, and then on the Axes option, at the top choose Axis Y. Go to the Whole Range tab, uncheck the automatic options and enter the values Min = 95 Max = 120 and Side Margins = 5. Finish.

stock in store



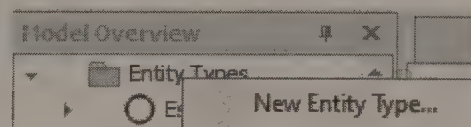
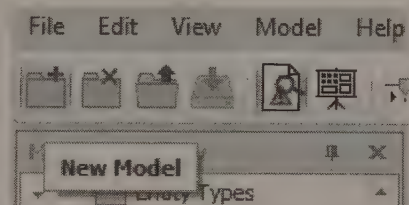


Model: Pilots of Formula 1

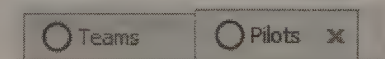
This model allows simulating the profits of 3 Formula 1 teams for a year, taking into account their advertising income and the salaries of the pilots.

Create the entities

Press the New Model icon and write the name Formula 1. In Model Overview an entity with the name Formula 1 appears. Press the right mouse button, choose Rename and write Teams. In Entity Types, right-click and choose New Entity Type, write Pilots.

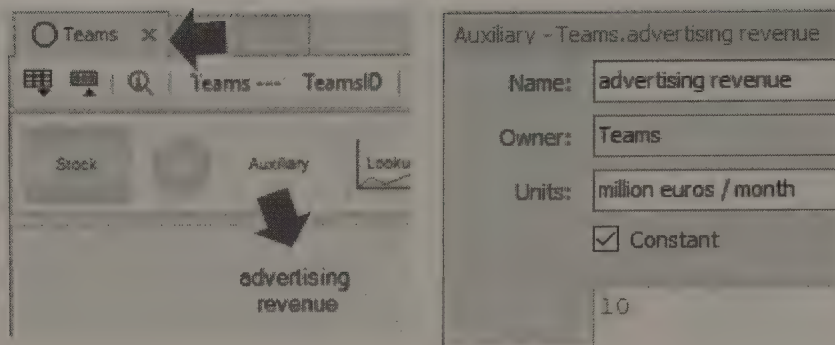


At the top there are two tabs to access the entity diagrams.



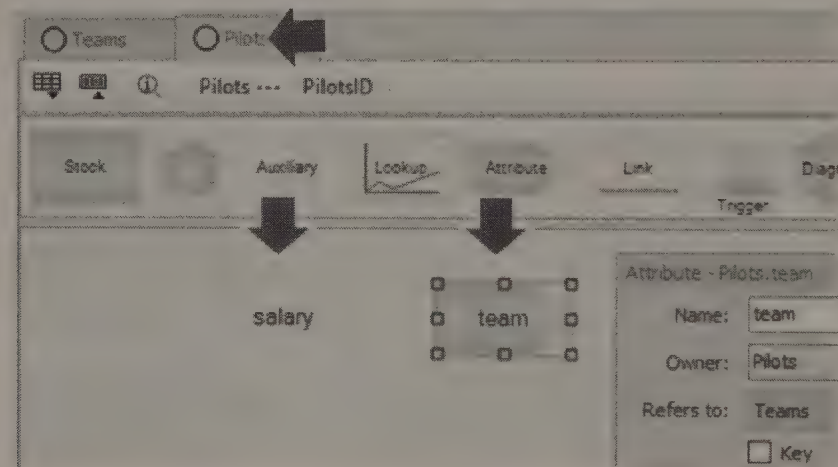
Define entities and relationships

Click the tab of Teams, move the Auxiliary icon to the diagram area, write the name "advertising revenue" and Enter. Right-click on the variable, choose Edit Auxiliary, and define a value (10) and the units (million euros / month), ticking Constant (so that different initial values can be defined for each team).



Go to the Pilots tab, move the Auxiliary icon to the screen, Enter, right-click on it, choose Edit Auxiliary, and create a variable with the name salary, amount 0.1 (zero-point-one), units million euros / month and click Constant.

Since the data of the pilots grouped by team is necessary to calculate the profit of this, create an attribute on the Pilots page that links them to each team. To do this, move the Attribute icon to the diagram, write the name "team", then edit it and in Refers to indicate Teams. Close the window.

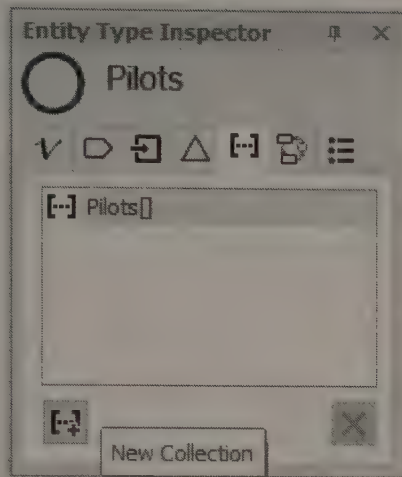


The team of a pilot is an alphabetical variable, so we cannot create an auxiliary variable (which can only have numbers) and we create an attribute of the drivers.

Inspector

On the right of the screen are shown the details of each entity, they can be opened by going to the Model Overview and double-clicking on the name of the entity. Open the Pilots inspector.

Collection

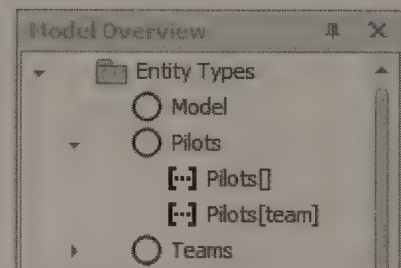


The groups of entities of the same type are called collections. We will create a collection with the pilots, to do this in the Pilots inspector, go to the Collections tab.

A Pilots [] collection appears that counts the number of pilots there are. We will create a collection that groups them by team. At the bottom of the Collections tab, click on "New Collection", in the menu that opens, select "team" to link the driver with the team. The collection of drivers grouped by team has been created: Pilots [team]

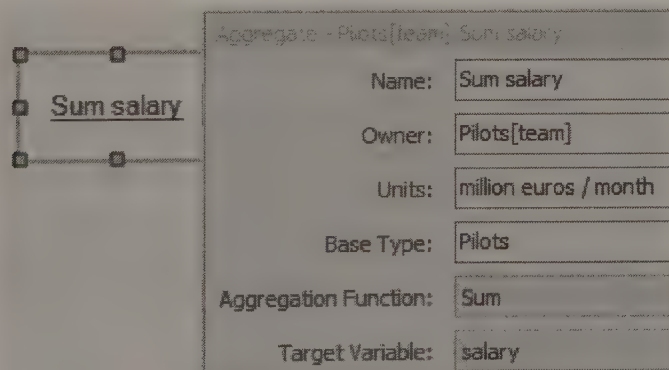
Using a collection

In the Model Overview, display the Drivers entity, a new collection appears, which is Drivers [team], double-click on it so that it is displayed in the inspector.

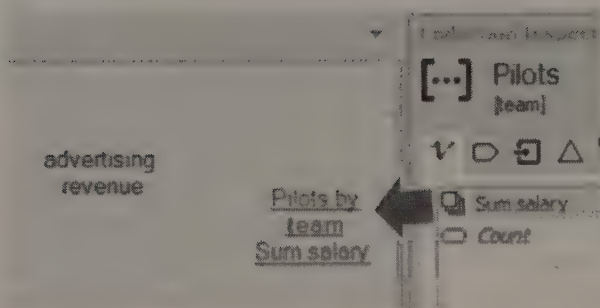


Aggregates

In Model Overview display the Pilots options. Double click on Pilots[team] to open the diagram and the inspector. In the Pilots[team] diagram we are going to calculate the total salaries of the drivers of each team. To do this, move the Aggregate icon to the diagram, press Edit Aggregate, and choose Sum and salary. Close.



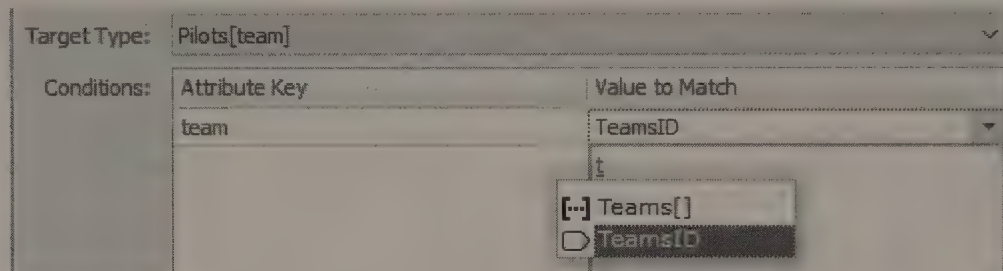
Link entities



Click on the Team tab to open the diagram.

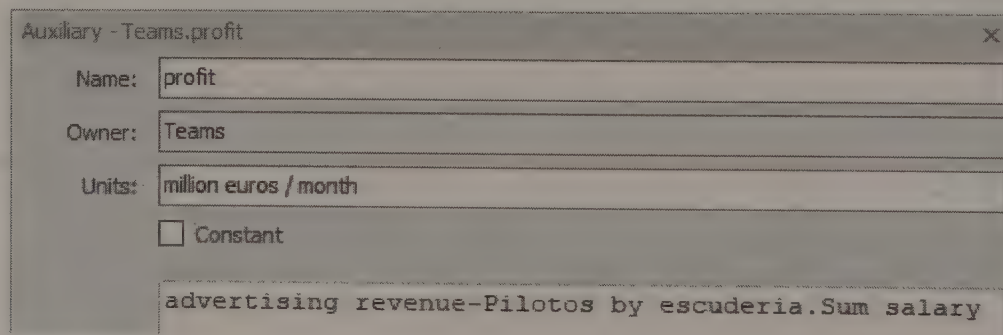
In the Pilots [team] inspector press the Variables tab, there is "Sum salary" created before. Move it to the Teams diagram.

Placing the addition of Pilots [team] in the Teams diagram creates a new relationship in it. Open the grid of references of Teams, the expression [team] = ??? in Conditions appears, click twice on ???, in the window that opens in Value to Match, write teams, select the attribute TeamsID and close.



Create the auxiliary variable "profit". Add arrows from "advertising revenue" and "Pilots by team Sum salary" to Profit. To do this, move the mouse over the arrow's origin variable and when a hand appears, click on the variable, move the mouse to the destination variable and click again.

Edit "profit" and enter the equation, income minus salaries by clicking on the equation area and selecting the variables.

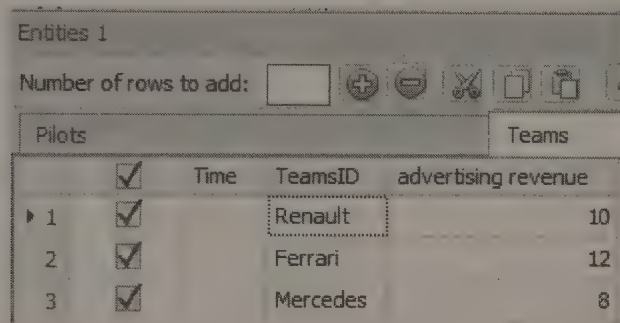


Temporary data

In Model Overview, right-click on Run Control and choose Edit. As Final Time type 12 and in Units for Time = Month. Close.

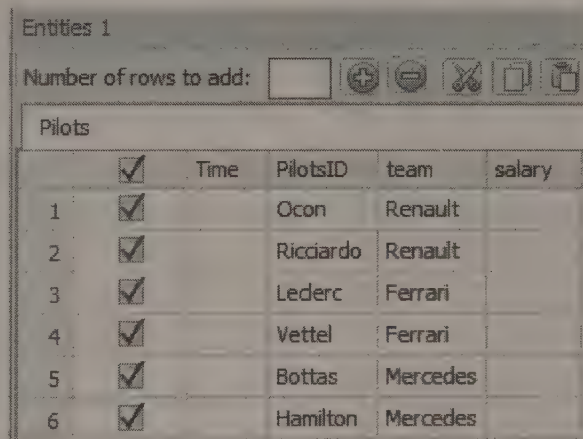
Initial data

In Model Overview - Entity Initialization Data, right-click Data Sources and choose Add Builtin Data. Check Show All Types if it is not there. In the Teams tab, enter the names and monthly income.



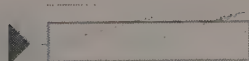
Pilots	Time	TeamsID	advertising revenue
1	☒	Renault	10
2	☒	Ferrari	12
3	☒	Mercedes	8

In the drivers tab, enter the drivers' names and those of their teams. We will leave the salary field blank so that the predefined 0.1 million euros / month is used.



Pilots	Time	PilotsID	team	salary
1	☒	Ocon	Renault	
2	☒	Ricciardo	Renault	
3	☒	Lederc	Ferrari	
4	☒	Vettel	Ferrari	
5	☒	Bottas	Mercedes	
6	☒	Hamilton	Mercedes	

Simulate

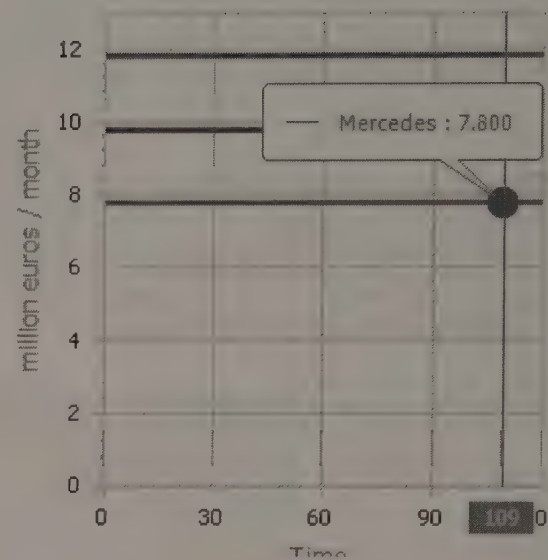


Press the green date icon. It is not necessary to write a name in the simulation.

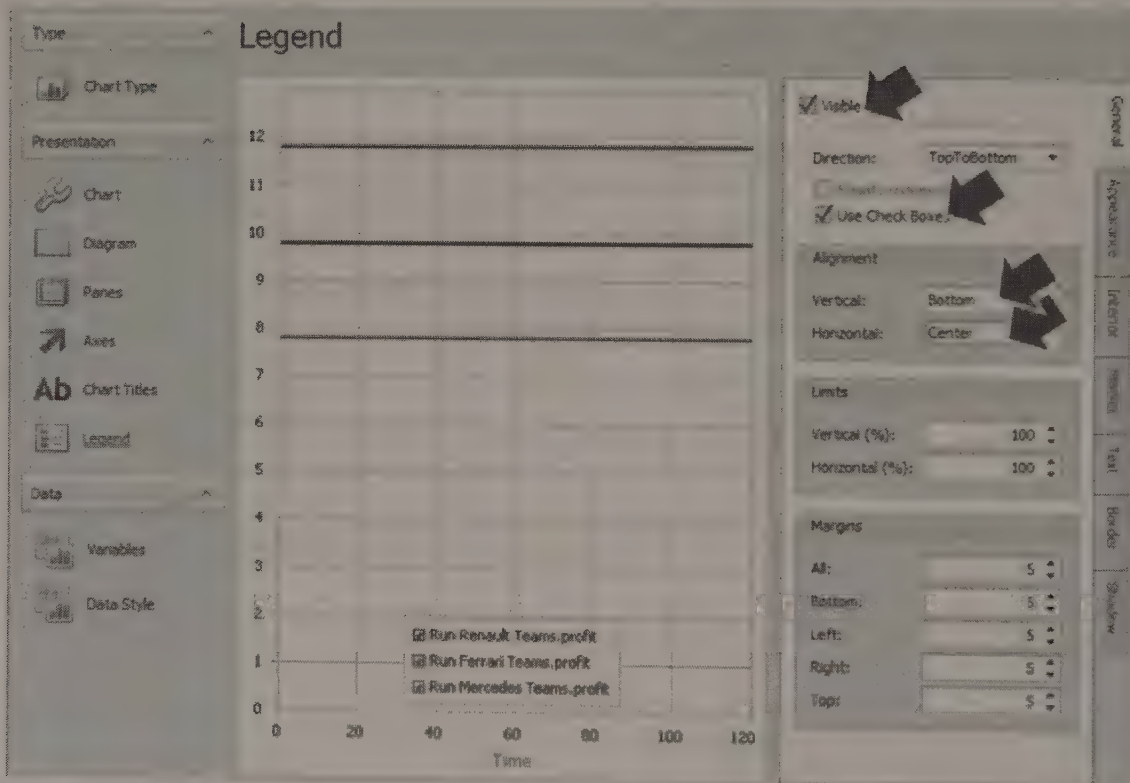
Results

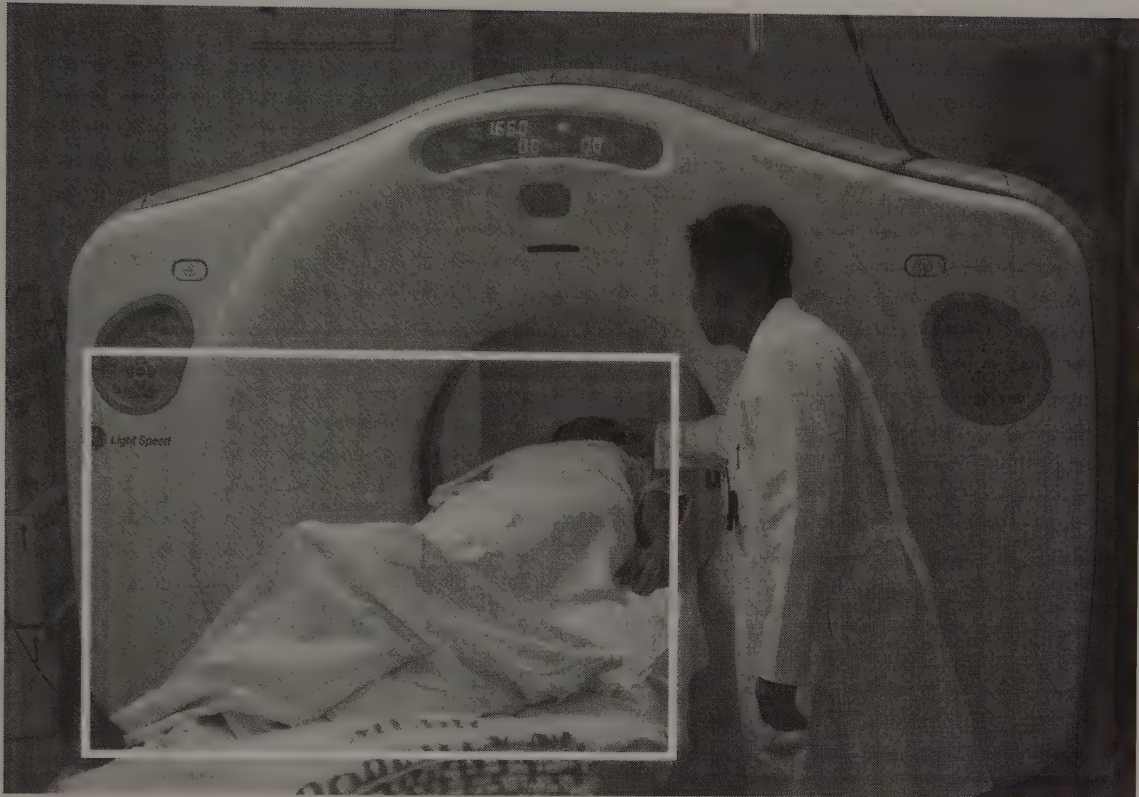
Go to the Teams page to see the evolution of the Profit variable. Click on the Profit variable, press the right button and choose the Variable Plot option. The graph shows the monthly profit of the teams. Place the cursor over the lines to see which team each line corresponds to.

profit



Click Edit Chart Properties. Check the options to display the names of the teams. Then click Finish





Model: Patients and Hospitals

We have 3 patients who are admitted to 2 hospitals and we know that the net cost per day depends on their ailment. They pay this cost less the amount that the insurance covers them for, which depends on the category of the hospital. We want to know what the daily net cost is for each patient. The data of patients, ailments, hospitals, and amounts are provided during the exercise as they are necessary.

Remember that from the Model Overview - Entity Types you can open the screen of an entity type by clicking twice on its name and you can also open the inspector, right-clicking on its name and choosing Inspect.

Attributes

In the following pages we will see in detail an example related to the use of attributes, collections, aggregates and references.

Click File - New Model and enter the name "patients". The patients entity type is created. Go to Run Control and change the Final Time to 8.

Go to the patient inspector, click the Attribute tab, the patientsID attribute has been created. Double-click on it and on the screen that opens, check that the Owner and Refers to are the type of patients entity, and the Key indicates that it is the key attribute, which defines each entity. Close the window.

Simulate the model with the (Run Model) icon at the top. We have not entered any data in the model. With the right mouse button click on patientsID, and then on View Data.

The window that opens shows a single entity called patients_0001, which is created by default. The simulation has the name Run since no name was specified when simulating.

Results: Run - Component: patients - Element: patientsID

DateTime	Run	patientsID
	0 Run	patients_0001
	1 Run	patients_0001
	2 Run	patients_0001
	3 Run	patients_0001
	4 Run	patients_0001
	5 Run	patients_0001
	6 Run	patients_0001

We are going to name each of the patients, go to Entity Initialization Data - Data Sources - Add Builtin Data, Entities 1 is created. Go to the patients tab.

Enter the following data, to indicate their names and the period in which they begin to be entities (when they enter the hospital). Close.

Results: Run - Component: patients - Element: patientsID

DateTime	Run	patientsID
	0 Run	Adrian
	1 Run	Adrian
	2 Run	Adrian
	3 Run	Adrian
	4 Run	Adrian
	5 Run	Adrian
	6 Run	Adrian
	7 Run	Adrian
	8 Run	Adrian
	2 Run	Barbara
	3 Run	Barbara
	4 Run	Barbara
	5 Run	Barbara

Entities 1

Number of rows to add:



hospitals		patients		
	☒	Time	CalendarTime	patientsID
1	☒	0		Adrian
2	☒	2		Barbara
3	☒	4		Alex

Simulate the model and show PatientID View Data. We can see that now the indicator takes the assigned name, and that each one starts at the indicated time (0, 2 and 4 respectively).

Go to Model Overview, display "patients" and right click on patients []. Choose Inspect to open the inspector on the right side of the screen.

Right click on Count and choose View Data. We can see the number of patients in each period.

Results: Run - Component: patients[] - Element: Count

DateTime	Run	Count
	0 Run	1
	1 Run	1
	2 Run	2
	3 Run	2
	4 Run	3
	5 Run	3
	6 Run	3
	7 Run	3
▶	8 Run	3

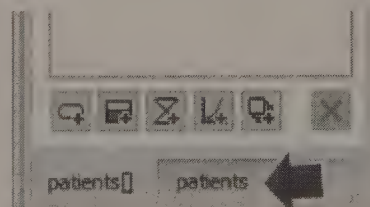
Move the Attribute icon from the icon bar to the screen and name the new attribute "disease". Right-click on the attribute and on Refers to choose patients, which is simple because there is only that type of entity. Do not mark Key, since several patients may have the same disease.

Go to Builtin Data, click twice on Entities 1 to open it and in the patients tab write the disease of each patient. Enter the additional data.

hospitals			patients	
	☒	Time	patientsID	disease
1	☒	0	Adrian	trauma
2	☒	2	Barbara	trauma
▶ 3	☒	4	Alex	diabetes

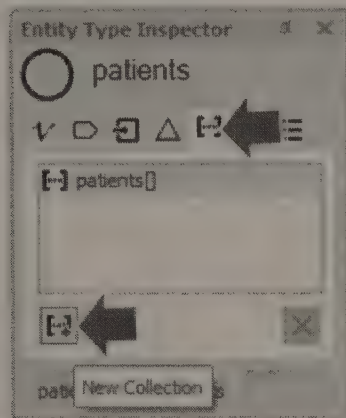
DateTime	Run	disease	patientsID
	0 Run	trauma	Adrian
	1 Run	trauma	Adrian
	2 Run	trauma	Adrian
	3 Run	trauma	Adrian
	4 Run	trauma	Adrian
	5 Run	trauma	Adrian
	6 Run	trauma	Adrian
	7 Run	trauma	Adrian
	8 Run	trauma	Adrian
	2 Run	trauma	Barbara
	3 Run	trauma	Barbara
	4 Run	trauma	Barbara

Simulate the model, right-click on the disease attribute and then View Data. A list is shown in which each entity (that is, each person) has their disease assigned in each period of time, since we have assumed that when they begin to be a patient they have that disease and does not recover before the end of the simulation.

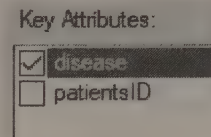


At the bottom of the Inspector, click on "patients" to open that inspector. We are going to create a collection of patients grouped by their disease.

Collections and aggregates

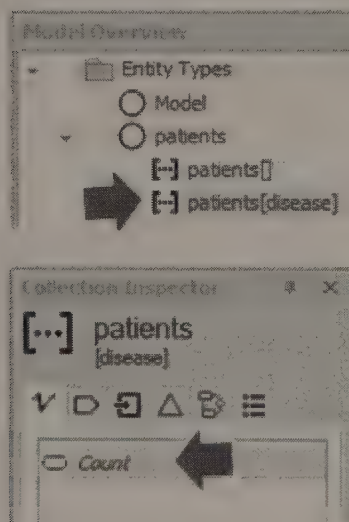


Open the Collections tab and at the bottom click New Collection. In the window that opens, check the disease box so that they are grouped by that concept.



The Model Overview shows patients [disease] and also their inspector.

Simulate the model, right-click on Count and choose View Data.



DateTime	Run	disease	Count
0	Run	trauma	1
1	Run	trauma	1
2	Run	trauma	2
3	Run	trauma	2
4	Run	trauma	2
5	Run	trauma	2
6	Run	trauma	2
7	Run	trauma	2
8	Run	trauma	2
4	Run	diabetes	1
5	Run	diabetes	1

The table shows for each period how many patients there are of each type. Those with trauma go from 1 to 2 and those with diabetes remain constant in 1 patient from period 4.

We can add qualitative variables that depend on an attribute, for example, the daily cost of each patient, according to the type of disease. Go to the patients' diagram and create the daily cost variable, adding an arrow from disease.



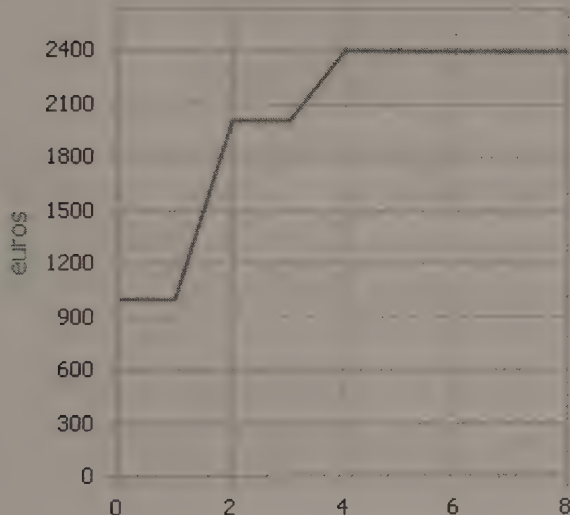
Make Edit in daily cost, and write the equation that indicates that it is 1000 euros if the patient is in trauma and 400 if it is not (it would be diabetes). Units = euros. Close.

daily cost = if (disease = "trauma") {1000} else {400}

Simulate the model. Click on "daily cost" with the right button and choose View Data, the day, name and daily cost of each patient are shown.

DateTime	Run	patient...	daily cost	
	4	Run	Barbara	1.000
	5	Run	Barbara	1.000
	6	Run	Barbara	1.000
	7	Run	Barbara	1.000
	8	Run	Barbara	1.000
	4	Run	Alex	400
	5	Run	Alex	400
	6	Run	Alex	400
	7	Run	Alex	400

Sum daily cost

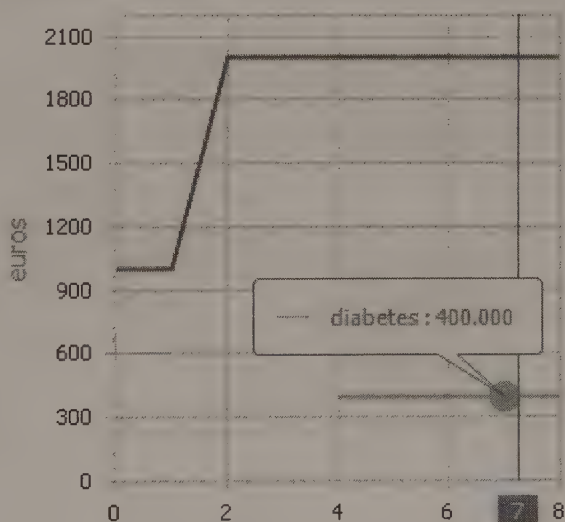


Go to the patient diagram [] and move the Aggregate icon to the center of the screen. Right-click on Aggregate1, press Edit Aggregate and choose "Sum" and "daily cost" to create "Sum daily cost". Close the screen. Simulate. Click on Plot Variable.

The graph shows the total daily cost, since patients [] includes all patients.

Sum daily cost

Go to the diagram patients [disease] and create an aggregate of the cost as you did in patients [], the variable "Sum daily cost" appears, but although it has the same name as this variable in patients [] the calculation is made by type of disease. Simulate the model and make Plot Variable. The graph now shows two lines, one for trauma and one for diabetes. In the aggregate calculations, information on each entity (such as its name) disappears and the results are total or subtotal.

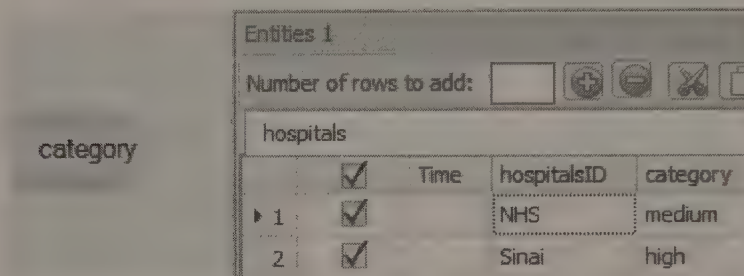


References

We are going to do an analysis from the point of view of the hospitals where these patients are treated. Create an entity type with the name hospitals. (Model Overview- Entity Types - New Entity Type)

In the hospital inspector you can see that the attribute (Key) hospitalsID exists. In the diagram create the attribute "category", and in Refers to select hospitals. Do not select it as Key because several hospitals can have the same category.

Now you must define what hospitals exist and what category each has. Go to Entity Initialization Data and click twice on Entities 1, check Show All Types so that the hospitals tab appears. Write the following data:

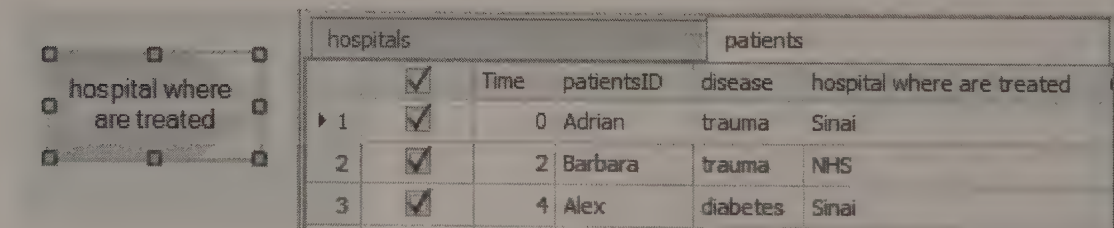


	Time	hospitalsID	category
▶ 1		NHS	medium
2		Sinai	high

There are therefore two hospital entities. It is not necessary to place values in the column Time because it is assumed that both exist in period 0 and during the days of the simulation.

Until now there is no established reference to which hospital each patient is in. In the patients' diagram add an attribute with the name "hospital where are treated", then go to Entities 1, open the patients tab and complete the data for each one.

There is still nothing that indicates that the hospital "Sinai" where the patient is the same as the hospital "Sinai" from the list of hospitals that we have entered in the type of entity hospitals. You have to create a referral between patients and hospitals.



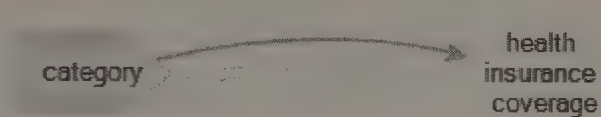
	Time	patientsID	disease	hospital where are treated
▶ 1	0	Adrian	trauma	Sinai
2	2	Barbara	trauma	NHS
3	4	Alex	diabetes	Sinai

It is not the same to create a reference to know in which hospital a patient is (Luis is in Cruces), compared to which patients that a hospital has (Cruces has Luís and Andrés).

If we create the reference patient [hospitals] we will obtain information about the hospital where each patient is (patient view). On the other hand if we create a reference hospitals [patient] we will obtain how many patients each hospital has (view hospital).

We want to know the daily cost for each patient, taking into account what the medical insurance covers, which depends on the category of the hospital. For this, in the hospitals diagram add the auxiliary variable "health insurance coverage", which depends on the category with the following equation, with units of euros:

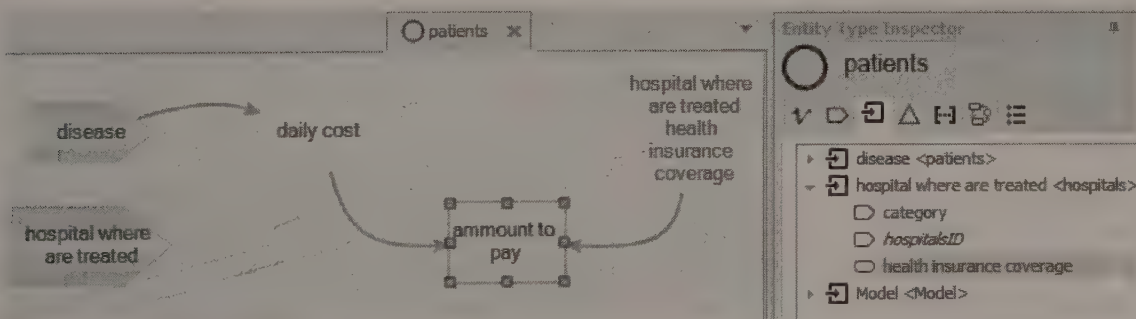
health insurance coverage = if (category = "medium") {200} else {300}



Go to the patient inspector. To do this in the Model Overview - Entity Type, right-click on patients and choose Inspect. Then click the tab References and open "hospital where are treated".

The variable "insurance coverage" appears, move it to the patient diagram, which is open in the center of the screen, or select it at the top. The shadow variable "hospital where medical insurance coverage is treated" appears.

Add the variable "amount to pay" which is the difference between the daily cost and the medical insurance coverage, euro units.

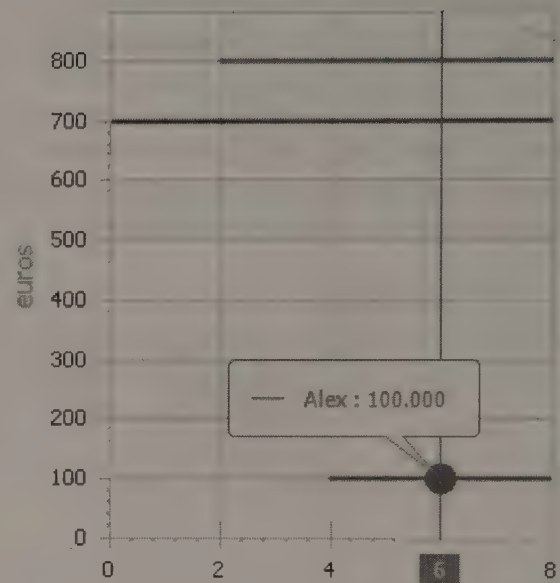


Simulate the model

Click on Plot Variable on the amount to pay, the graph is displayed for each patient, from the moment they enter your hospital.

DateTi...	Run	patientsID	ammount to pay
0	Run	Adrian	700
1	Run	Adrian	700
2	Run	Adrian	700
3	Run	Adrian	700
4	Run	Adrian	700
5	Run	Adrian	700
6	Run	Adrian	700
7	Run	Adrian	700
8	Run	Adrian	700
2	Run	Barbara	800
3	Run	Barbara	800
4	Run	Barbara	800
5	Run	Barbara	800
6	Run	Barbara	800
7	Run	Barbara	800
8	Run	Barbara	800
4	Run	Alex	100
5	Run	Alex	100
6	Run	Alex	100
7	Run	Alex	100
8	Run	Alex	100

ammount to pay



Click on View Data to obtain the values of each entity. For example, Alex has diabetes, which is a cost of 400 euros every day, he is in the Sinai hospital, which is of a high category, so the insurance covers him 300 euros, the difference is the amount to pay, which is 100 euros, as the results of the model show us.



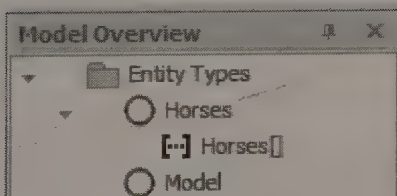
Model: Horse Breeding

We are going to create a model to simulate the behavior of a group of 10 horses distributed in 5 pastures that have different types of grass. The objective is none other than to learn the basic characteristics of the software: create the diagram, define the initial values and equations, simulate and see the result.

Default values:

- density of grass: 280 kg / (Ha * cm)
- area: 40 Ha
- grass growth: 0.01 cm / day

Create the model

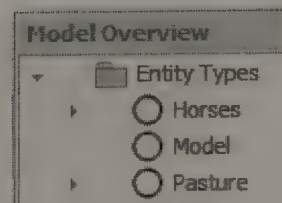


Select "File - New Model" and write the name "Horses". Two entities appear in Entity Types: Model (where are the Time parameters of the model) and another entity that has the same name as the model, Horses.

In the Model Overview open Run Control, in Units of Time select Day. Close.

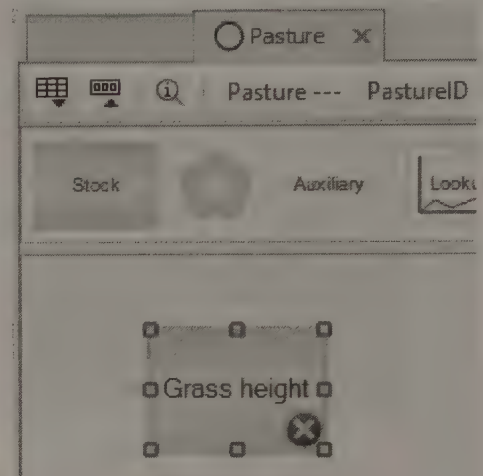
Create a new entity type

In the Model Overview, right-click on Entity Types, choose New Entity Type and write "Pastures", which will be a type of model entities.

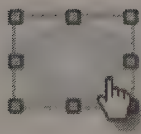


Create the diagram of an entity

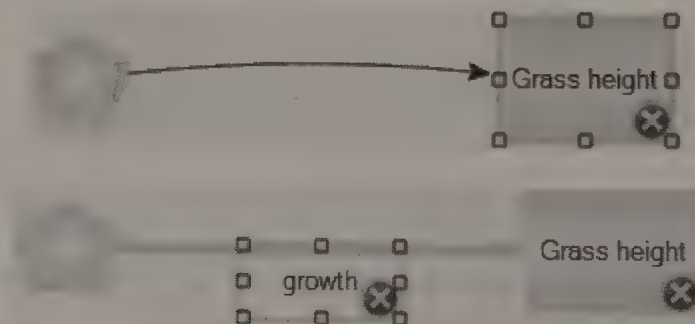
We are going to define in the Pastures entity type what is the height of the grass that exists (as Stock), the speed at which it grows (a Flow) and the mass or quantity of the grass (an Auxiliary Variable). Go to the Pastures diagram (by clicking on its tab at the top of the screen). Move the Stock icon from the palette to the diagram. When you release it, it will be automatically named "Stock1". Replace "Stock1" with the name "Grass height". Press Enter:



The symbol  reminds us that the equation is yet to be defined. Next add a cloud to the left of the Stock to the diagram, dragging that symbol from the panel.



Move the cursor over the edges of the cloud until a hand appears. At that moment click on the cloud and move the cursor to the right, drop it on the Stock and an arrow and a box appear to indicate the name of the Flow. Write "growth" and Intro.



Stock - Pasture.Grass height

Name: Grass height

Owner: Pasture

Units: cm

Default Initial Value: 2

The upper tab of the screen shows that you are in the Parcels entity. Once the diagram is complete, the equations can be added.

Right click on "Grass height" and select "Edit Stock". Write the units of the height of the grass (cm) and the initial value 2. Close this window.

Right click on "growth" and select "Edit Flow". Write 0.01 and cm / Day. Remember: In case of error go to Edit - Undo.

We have already defined the Pastures entity, which is characterized by the height of the grass, which starts at the value of 2 cm and grows 0.01 cm / day. The diagram shows the Stock and the Flow.

Flow - Pasture.growth

Name: growth

Owner: Pasture

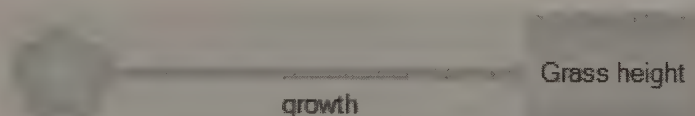
Units: cm/día

From Stock:

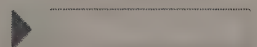
To Stock: Grass height

☐ Constant

0.01



Simulate



By default, it is assumed that there is only one entity of each type. Therefore, our simulation will include a plot, with an initial grass height of 2 cm and a grass growth rate of 0.01 cm / day. Press the green Run Model icon

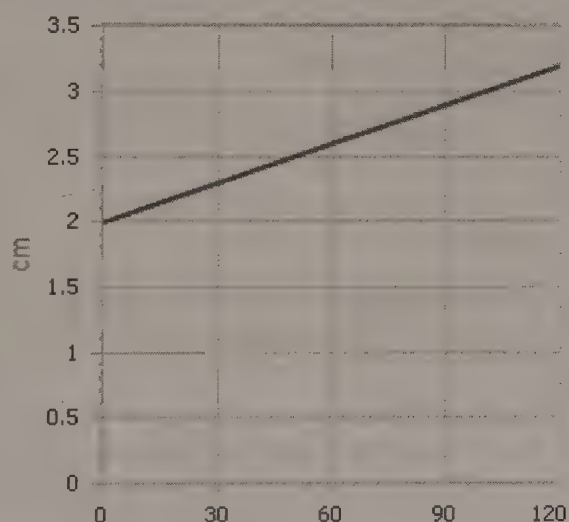
Results

Click with the right button of the mouse on the Stock, and in the menu that appears choose Plot Variable.

The graph shows the evolution of the height of the grass, we can see the values of each point by placing the cursor on the graph.

Save the model.

Grass height

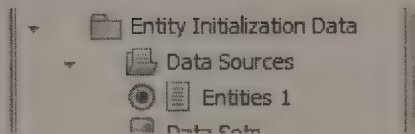
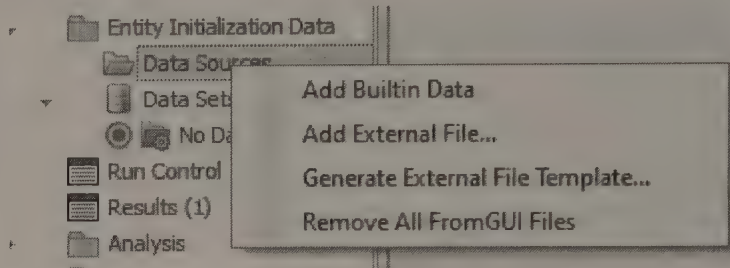


Model: Horse Breeding 2

We have simulated by writing the values of the parameters in the equations (2 cm in height, 0.01 cm / day of growth), which are the values that will be used by default. But we can define specific values for each of the entities of each type, when there is more than one, which is usual.

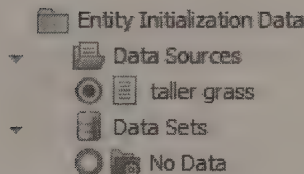
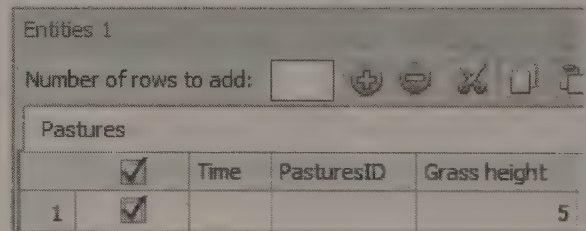
Our purpose now is to change the starting height of the grass since, unless we change it, the simulation will use the default value (2 cm).

In Entity Initialization Data, display the Data Sources menu, right-click and choose Add Builtin Data.



A data file has been created, which is in the Data Sources tab with the name Entities 1. Double click on Entities 1 to open it.

Select the Pastures tab. Check "Show All Types" if this tab is not shown. Enter 5 in Grass height, the validation box is activated. Close.

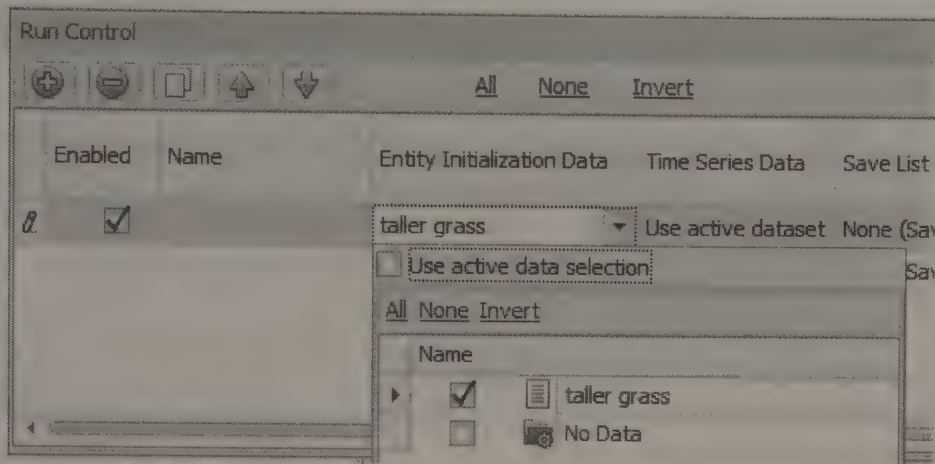


Click on Entities 1 with the right button, choose Rename. Rename Entities 1 to "taller grass"

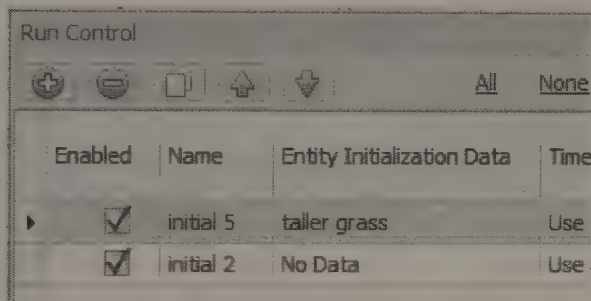
Simulate

In Model Overview double click on Run Control. When it opens, click on the Add Run (+) button in the upper left to add a row.

In the new window go to the first row of Entity Initialization Data, click on the box to open the menu, uncheck the option "Use active data selection", and in the menu that will appear, check the option "taller grass". Press on the screen to close the menu.

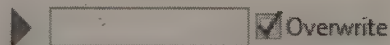


Repeat the process for the second row, and check the No Data option (which will use the default values for execution). In the Name column write a descriptive name for each simulation. Close this window.



We have defined that we want to simulate the same model with two different hypotheses, that the initial grass is 2 cm and that the initial grass is 5 cm. We will see the results in the same graph in a comparative way.

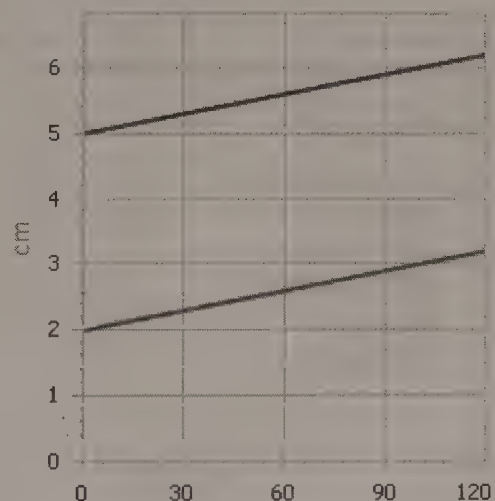
Simulate the model.



In the diagram, right click on "Grass height" and choose the Variable Plot option. The graph shows us the evolution of the grass height with the two defined initial values (2 and 5 cm). You can explore other options.

Save the model with a new name, for example Horses2, so we create different versions of the model and in case of problems we can go back to a previous version.

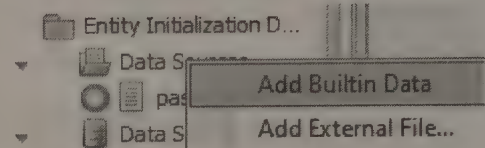
Grass height



Model: Horse Breeding 3

When defining the initial conditions of the system (initialization), how many entities there are of each type is defined, and their characteristics. Thus, an entity type can be "Cars", and its entities the individual license plates of each car. We will create an input data file in which each row will show what is the grass of each pasture.

Go to Entity Initialization Data - Data Sources - Add Builtin Data, a new Entities 1 folder appears.

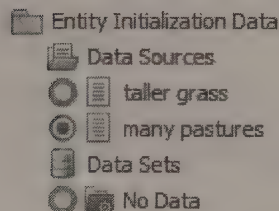


Note: Sometimes the software behaves unstably, if so, save the model, press Open Model and reopen the same model.

The screenshot shows a window titled 'many pastures'. It has a toolbar with icons for adding, deleting, and other actions. Below the toolbar is a table with two tabs: 'Horses' and 'Pastures'. The 'Pastures' tab is active. The table has columns: 'Time', 'PasturesID', 'Grass height', and 'CalendarTime'. There are 5 rows of data, each with a checkbox in the 'Time' column.

	☒	Time	PasturesID	Grass height	CalendarTime
▶ 1	☒		Big	12	
2	☒		River	10	
3	☒		Deep	8	
4	☒		Sunny	11	
5	☒		Dark	9	

Go to the Pastures tab (click Show All Types if necessary). In the PastureID column enter the code of the pasture and in the column "Grass height" the initial cm of the pasture. Enter the data for the first row, press Enter, open a second row and repeat. Close the window when finished.

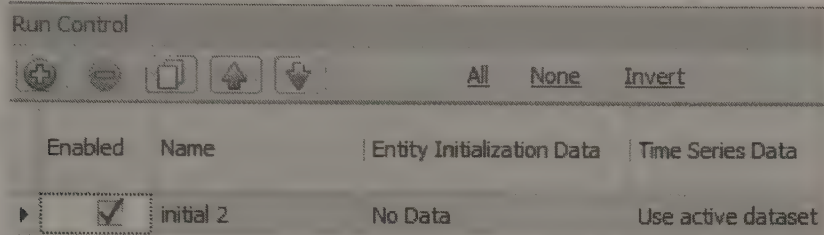


Go to the Model Overview - Data Sources press Rename from Entities 1 to "many pastures".

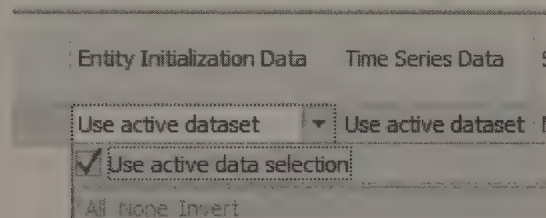
Activate the option "many pastures" by clicking on the green circle, so that it changes from empty to full.

Simulate

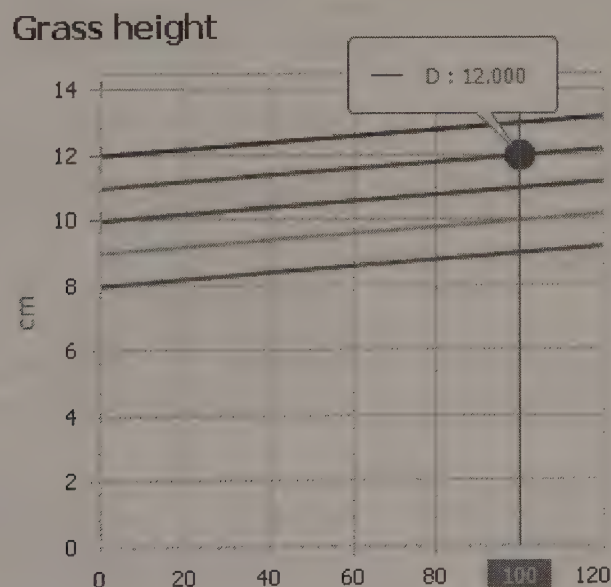
First of all, it is necessary to delete the results of previous simulations. To do this go to Run Control and press Edit. Press the icon (-) at the top, the simulation "initial 5" disappears.



In the Entity Initialization Data column open the side drop-down and check "Use active data selection" so the model uses the initial data by default. Close.



Press the green icon (Run Model) to Simulate. Go to variable "Grass height", right click on it, and choose Plot Variable. The graph of the height of the pasture in each plot is obtained, with the initial value defined for each plot and a growth of 0.01 cm / day. Go to File - Save As and save the model as Horses 3.

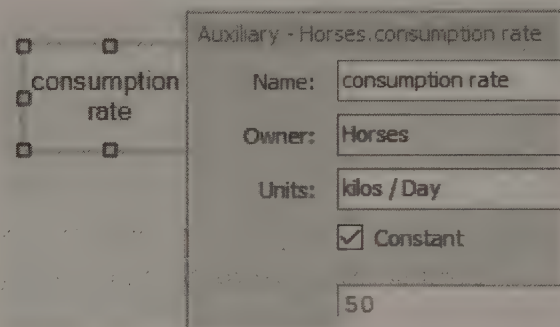


In the model we have created two types of entities: Pastures and Horses. We are now going to define each of the entities of the type Horses.



Select the Horses tab at the top and the diagram shows empty. We are going to add to the model how much grass each horse eats and in which pasture it is located.

Create an auxiliary variable called "consumption rate" (drag the Auxiliary icon to the drawing area of the diagram), and press Enter. Right click to edit the variable. Mark the Constant box (because we are going to define an individual value for each horse), and write the value 50 kilos / Day. Close the window.

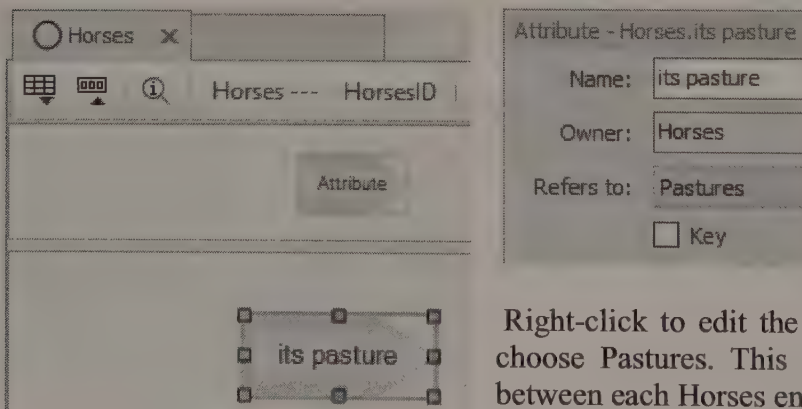


Attribute

Attribute

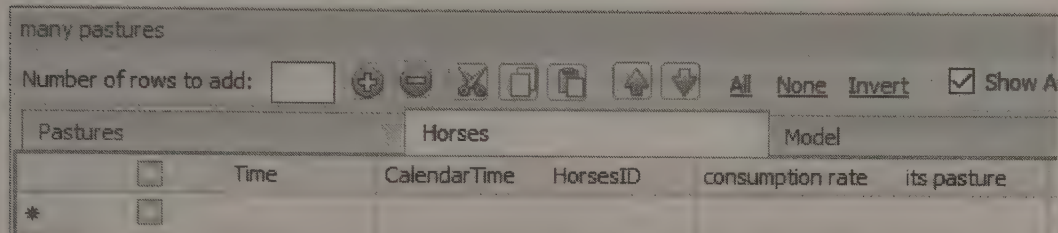
To specify the grass that each horse consumes, we will introduce an "attribute", which is a property or characteristic of an entity. In this case, the attribute will indicate in which pasture each horse is located

Move the Attribute icon to the diagram and write the name "its pasture".



Right-click to edit the attribute. In "Refers to" choose Pastures. This attribute shows the link between each Horses entity and a Pasture entity.

When the attribute is created, in the initial data of a simulation the tab of the entity type Horses allows linking each entity (HorsesID) with each one of the entities type Pastures. To check it go to Entity Initialization Data - many pastures and right click to edit it. At the top check the Show All Types box if the Horses tab does not appear. Click on the Horses tab, the "its pasture" attribute appears. Each HorseID will be assigned a pasture.



Later on, we will define that the "its parcel" attribute corresponds to a specific entity of the Pastures type, identified by its PasturesIID field.

Key attributes

An "entities type" is made up of entities be they groups, such as sports cars, or individuals such as their license plate. Each entity has multiple variables, e.g., price, age, owner's name, number of seats, etc., and the one that identifies an entity is designated as a key attribute (Key), for example "sports cars" or the "license plate".

In this model, the key attributes have the name (by default) HorsesID for the Horses entities, and PasturesID for the Pastures entities, and each one of the pastures has its Key PasturesID attribute with a unique value (Dark, Large, ..) and each horse will have its own Key HorseID attribute.

References

In the next simulation we will use the attribute "its pasture" as a label to organize the horses. We will group the horses according to their pasture and calculate the total grass consumption for each pasture. The attribute "its pasture" is now a property of the horses: each horse is in a pasture. Later we will use information from each horse's pasture to make calculations about the horse.

Each horse entity will need to know the available pasture per horse for the pasture entity it is on. Getting information from another entity or set of entities is called a "reference."

Click on the grids button to see the Horse references.

Attribute Name	EntityReference	Is Key	Reference Name	EntityReference	Condition
HorsesID	Horses	YES	its pasture	Pastures	attribute value
its pasture	Pastures	No	Model	Model	

In the left part (second row) the entity type Horses has a field to identify each entity, which is called HorsesID, this is the key (Key) that is used to link with other entities.

In Horses there is an attribute (Attribute Name) called "its pasture" that is linked (EntityReference) with the Pastures entity.

The references grid of the Pastures tab shows that its key is called PasturesID, and it identifies each pasture, and it will be used to link with the attribute "its pasture" of the Horses type entity.

Attribute Name	EntityReference	Is Key
PasturesID	Pastures	YES

Initial values

Click twice on "many parcels", click Show All Types, it will show all type of entities, go to the Horses tab, each row corresponds to a horse.

Time	CalendarTime	HorsesID	consumption rate	its pasture
I				

The HorseID column gives each horse a unique name. If you leave it blank, the horses will be automatically named horse_1, horse_2, horse_3, or you can type their names, e.g., Black, Champion, Tornado, as long as two horses do not have the same name. In the "consumption rate" column you could enter a value for each horse, instead of the default value of 50. The "its pasture" column contains the name of the pasture where each horse grazes. In this example, we have assigned 14 horses to the already

defined pastures. We have also assigned horses to a parcel ("Communal") that does not exist in the previous Parcel simulation.

many pastures

Number of rows to add:













All

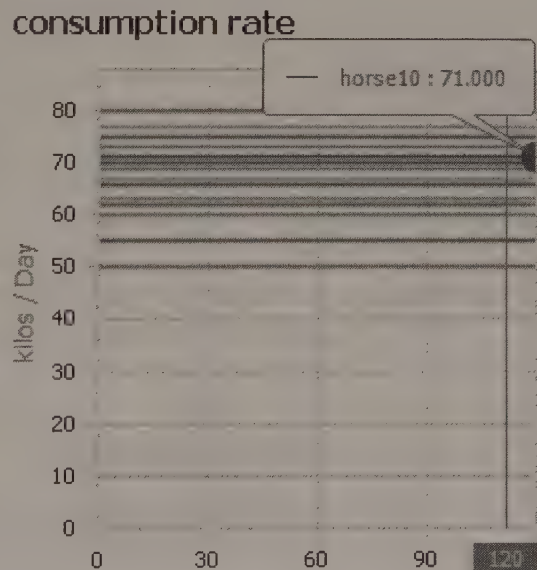
Pastures		Horses		M	
	☒	Time	HorsesID	consumption rate	its pasture
1	☒		horse1	50	Big
2	☒		horse2	55	Big
3	☒		horse3	60	River
4	☒		horse4	62	Dark
5	☒		horse5	63	Sunny
6	☒		horse6	66	Deep
7	☒		horse7	67	Sunny
8	☒		horse8	69	River
9	☒		horse9	70	River
10	☒		horse10	71	River
11	☒		horse11	73	Deep
12	☒		horse12	75	Big
13	☒		horse13	77	Communal
14	☒		horse14	80	Communal

Enter the values shown in the previous figure and close that window. Simulate the model by pressing the green arrow icon.

On the Horses diagram, select the variable "consumption rate", right-click on it and in the menu choose the Plot Variable option. The graph shows the daily grass consumption of each horse.

The Console displays a message that the "Communal" plot could not be calculated, since there are horses in that pasture but it has not been defined as a pasture. Go to View - Console if it is not shown at the bottom.

Save the model as Horses 4



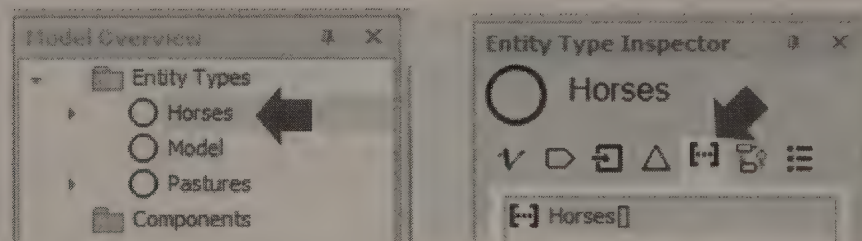
We are going to calculate the consumption of grass in each pasture. Then we will change the properties of the pastures so that the grass loses height as it is eaten.

Collections

A Collection (Collection) is a grouping of entities (several horses) of an entity type (Horses), using some of its attributes. In this example, to know how much grass is being eaten in each pasture, we will add the consumption of all the horses in a pasture using the attribute "its pasture" of each horse.

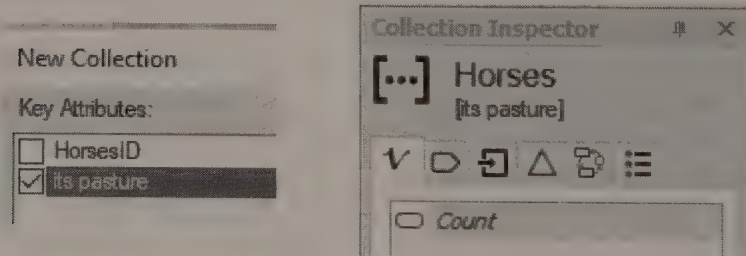
The first step is to create a "collection" of horses per pasture, which is done from the "Entity Type Inspector" of the Horses entity. In the Model Overview double click on "Horses" to open its inspector.

Press the "Collections" icon. The Horses [] groups ALL the horses is not useful because we want information only on the horses that are in a pasture and the consumption of grass they make.



Click on the "New Collection" icon at the bottom. A list of Horse attributes appears and we can divide the horse population by these attributes. Check the box to divide our horse population by "its pasture" and OK

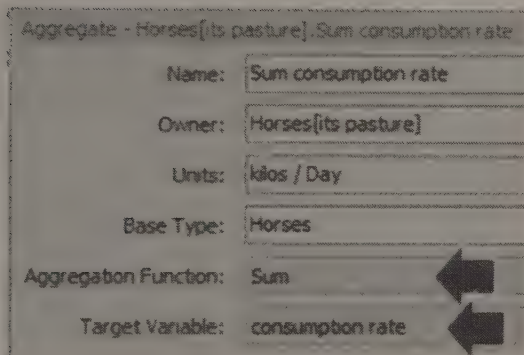
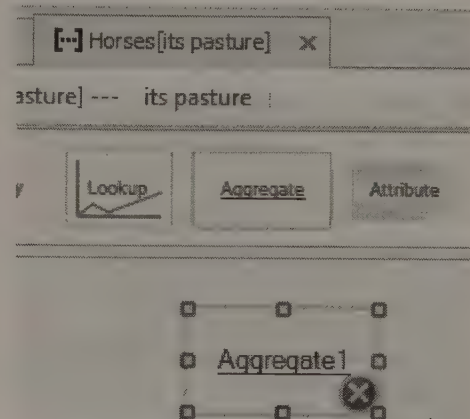
In the Inspector, the "collections" tab for Horses there is a new collection Horses [its pasture] that includes all the horses grouped according to the pasture they are in.



Aggregates

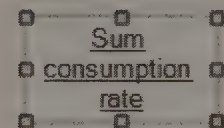
A collection can aggregate variables (sums, averages, products, minimums, maximums and standard deviations) and we will use one now to calculate the consumption of grass in each pasture.

We are going to create an aggregate (sum) of a variable of the entity Horses grouped by the parcel where they are. In the Horses [its pasture] diagram move the Aggregate icon to the center. The new aggregate will be named Aggregate1.



Right click on Aggregate1 and then Edit Aggregate. In the new window display the Aggregation Function menu, choose Sum, and in Target Variables select the consumption rate. The software changes the Name and Units. Close.

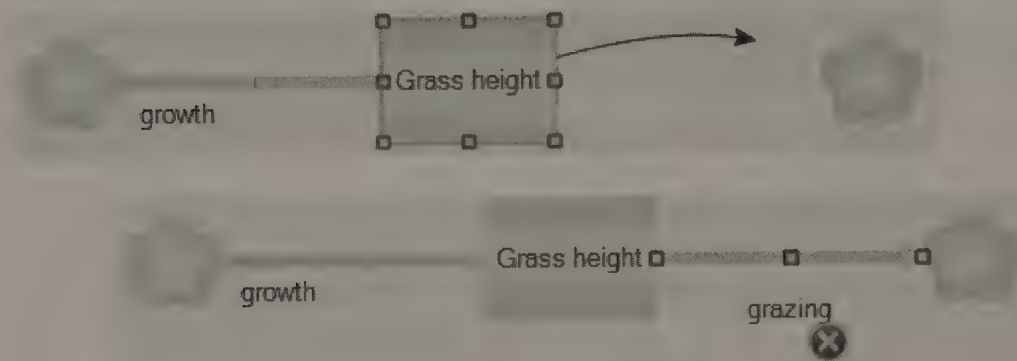
The appearance of the variable in the diagram is updated. Remember that if you want to open a new diagram or inspector, you can go to the Model Overview, expand the entities, and double-click the type of entity you want to display its diagram and inspector.



You can run the model and see the graph (Plot Variable) that shows the variable "Sum consumption rate" which are the kilos of grass consumed each day in each pasture, since the Key attribute of this collection is the attribute "its pasture". That is, this collection of data for each horse (consumption) added by [its pasture].

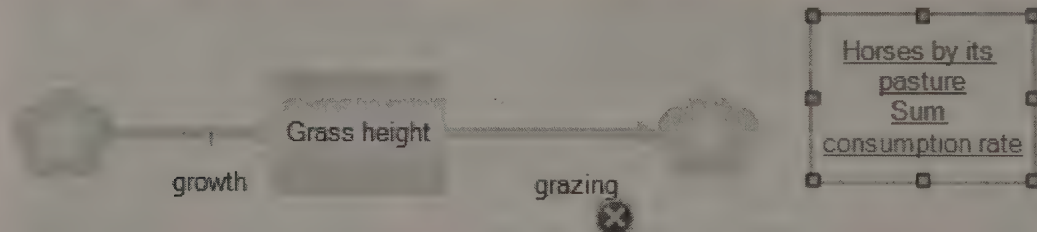
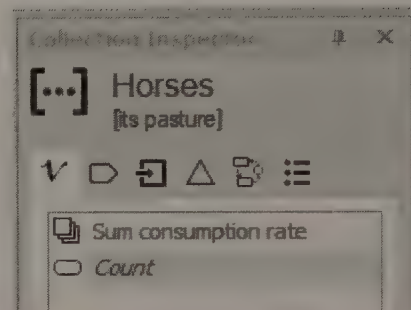
Note that although there is no parcel named "Communal" in the model, a total consumption value is calculated for that plot. This is because the calculation is made on the basis of the horses, not on the basis of the pastures, and there are horses in the model that are assigned the value parcel = "Communal". We use the consumption of each horse to calculate the height of the grass in each pasture.

Go back to the Pastures diagram, move a cloud to the right of the Grass height stock, draw an arrow from the Stock1 to the cloud and write the name "grazing".



Open the Horse [its pasture] inspector by clicking on it with the right mouse button in the Model Overview, click Inspect.

The Variables tab has the "Sum consumption rate", move it to the Pastures diagram. The variable is created (duplicated, in green color) "Horses by their pasture Sum consumption rate"



You will see that it appears in a different color than other variables and is labeled "Horses by its pasture Sum consumption rate". This points out that it is a reference to information of the collection of Horses [its pasture]. References are displayed in green on diagrams when shadow variables (duplicates).

By placing this duplicate variable (Sum consumption rate) that belongs to Horses [its pasture] in the Pastures entity, a link has been created between the two entities that we must define.

Open the Pastures references grid, in the Conditions column it is shown [your parcel] = ??? because this correspondence remains to be defined. Press twice on ???. In the window that opens, complete the Value to Match field, type pasture and choose the

PasturesID option, close. Thus, it has been defined that the "its pasture" attribute of the Horses entity corresponds to the pastures (PasturesID) of the Pastures entity.

Reference - Pastures.Horses by its pasture

Name:	Horses by its pasture	
Owner:	Pastures	
Target Type:	Horses[its pasture]	
Conditions:	Attribute Key	Value to Match
	its pasture	PasturesID

Thus, whenever a Pastures entity needs information about the horses, it will look for the group associated with the parcel name. We use this to calculate the reduction in grass height due to grazing.

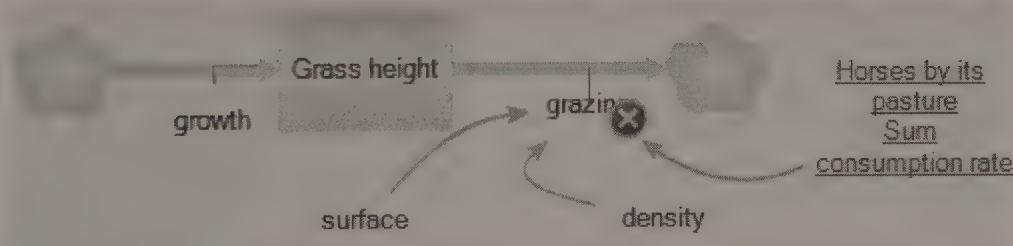
If you open the grid of references, you will see on the right that the entity "pasture" is already associated with the PasturesID.

The height of the grass is in centimeters, but the total consumption of horses in each pasture is measured in kilos. We need to do a conversion. Go to the Pastures diagram. Add the auxiliary variables "surface" (with a value of 40 Ha) and "density" with a value of 280 kilos/(Ha*cm). To do this, move the Auxiliary icon to the diagram, write the name of the variable and enter the value and the units. Mark both with the Constant option.

Auxiliary - Pastures.density

Name:	density
Owner:	Pastures
Units:	kilos/(Ha*cm)
☒ Constant	
280	

Draw arrows from each variable to "grazing" (by floating the cursor near each variable until the cursor turns to hand, then dragging an arrow to "grazing").



Select grazing by clicking with the right mouse button, choose Edit Flow and enter the equation and the units.

To write the equations select variables by double-clicking on their name which appears when clicking in the area of the equation below. The result should be:

Flow - Pastures.grazing

Name:

Owner:

Units:

From Stock:

To Stock:

☐ Constant

`Horses by its pasture.Sum consumption rate/surface/
density`

You can now calculate the total consumption of the horses in each pasture and its effect on the height of the grass. We will use the same horses and pastures from the previous simulation.

Click on "many pastures" to see the data previously entered. In the "Pastures" tab you will see surface and density columns. Use the default values for all except "River", which will be (10 Ha) with little vegetation (density of 100 kilos for each cm of grass in each Ha).

many pastures

Number of rows to add:       All None Invert ☐

Horses				Pastures			
	☒	Time	PasturesID	Grass height	surface	density	CalendarTime
1	☒		Big	12			
2	☒		River	10	10	100	
3	☒		Deep	8			
4	☒		Sunny	11			
5	☒		Dark	9			

Let's reconsider the growth of the grass, modify it from 0.01 to 0.1. Click on the variable "growth", press Edit Flow, change the value and close.

Simulate

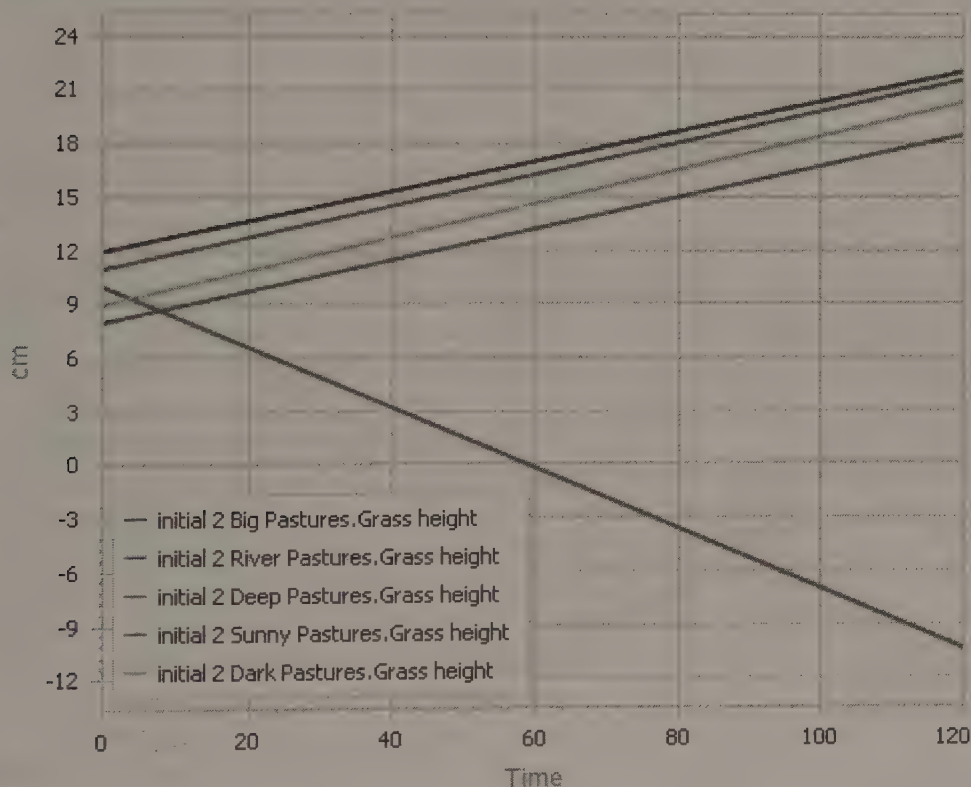
Simulate the model, pressing the green arrow, then select the variable "Grass height", with the right mouse button open the menu and select Plot Variable to see the evolution of each pasture.

Even with horses, grass grows a lot in all pastures except in the "River" pasture where horses balance what they can grow.

In summary, we have created a collection of horses divided into pastures and calculated the total consumption of grass in each pasture. Then in the definition of the pasture entity, we have created a reference to that collection, so the total consumption rate is used to calculate the net growth of grass in each pasture.

To display the graph, select the variable Grass height, click on Edit Chart Properties - Legend - Visible - Alignment bottom - Horizontal left, and Finish.

Grass height

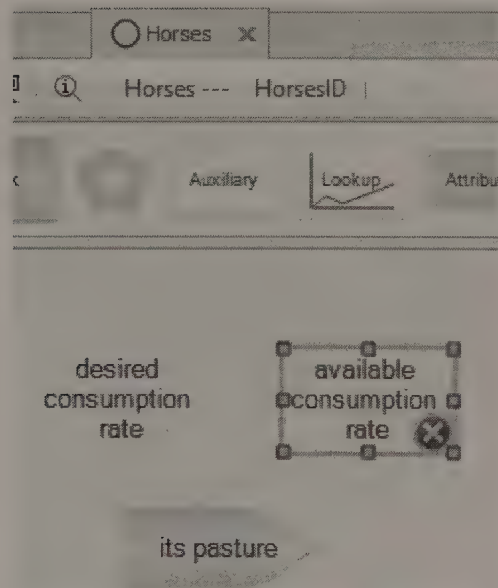
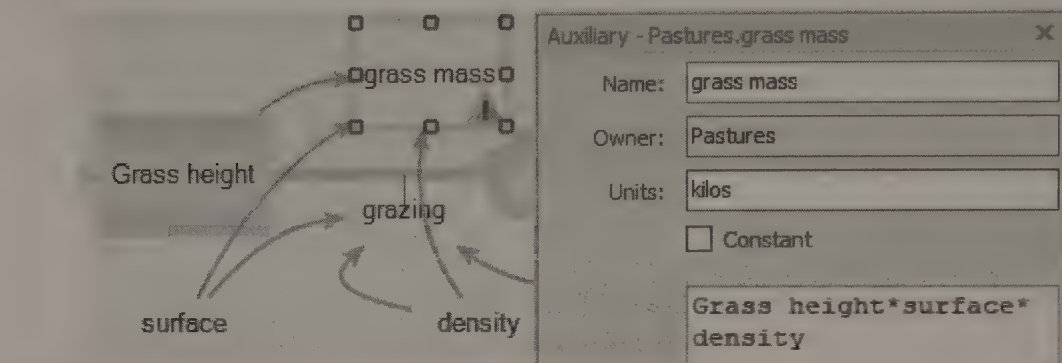


Save the model as Horses 5

Model: Horse Breeding 6

We want to simulate that grazing is limited to the amount of grass in each pasture. First, we must address a problem of units: horses consume grass in kilos per day, while grass is measured in centimeters per day, so it is necessary to calculate the kilos of grass available in each pasture.

In the Pastures diagram, add an auxiliary variable "grass mass", equal to the product of the surface, density, and grass height, with units of kilos.



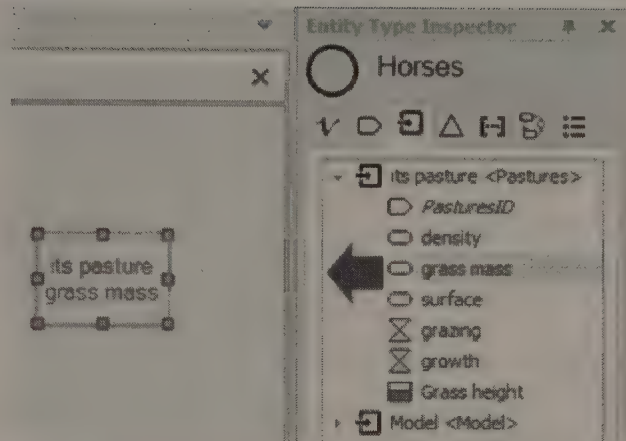
We will limit the consumption of grass to the amount available. In the Horses diagram change the name from "consumption rate" to "desired consumption rate" (right-click and Rename). Also create the auxiliary variable "available consumption rate" (to do so, move the Auxiliary icon to the Horses screen).

Display (Show Grids) the attributes and references table. In the entity Horses there is the attribute "its pasture" (Attribute Name, to the left of the next box). That attribute is also a reference to the Pastures entity (Reference Name, on the right). The reference means that the properties of the parcel are available to use in calculating the consumption of each horse.

Horses --- HorsesID			Horses --- HorsesID		
Attribute Name	EntityReference	Is Key	Reference Name	EntityReference	Condition
HorsesID	Horses	YES	its pasture	Pastures	attribute value
its pasture	Pastures	No	Model	Model	

Open the Horses inspector (click Horses in the Model Overview and choose Inspect) and go to the references tab. 

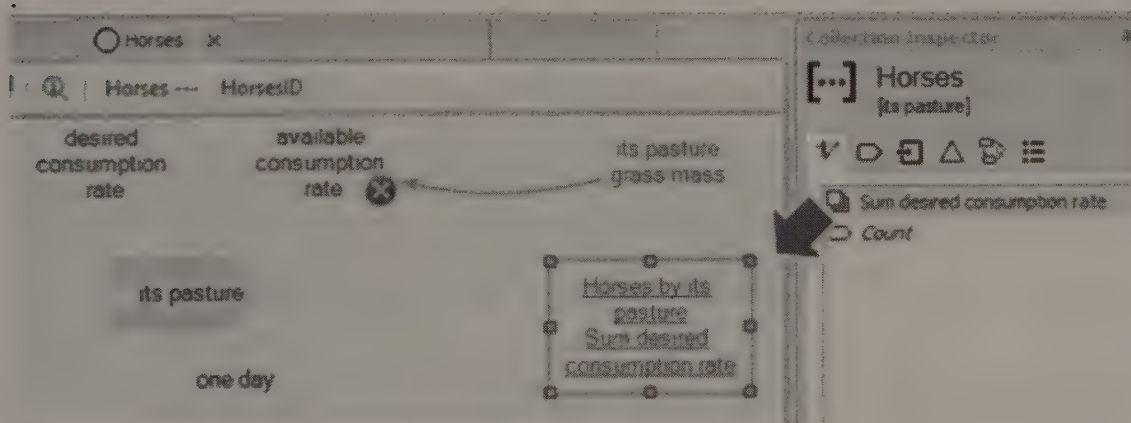
From "its pasture" <Pastures> menu, move the "mass grass" variable to the Horses diagram.



Then link the grass mass with the available consumption rate.

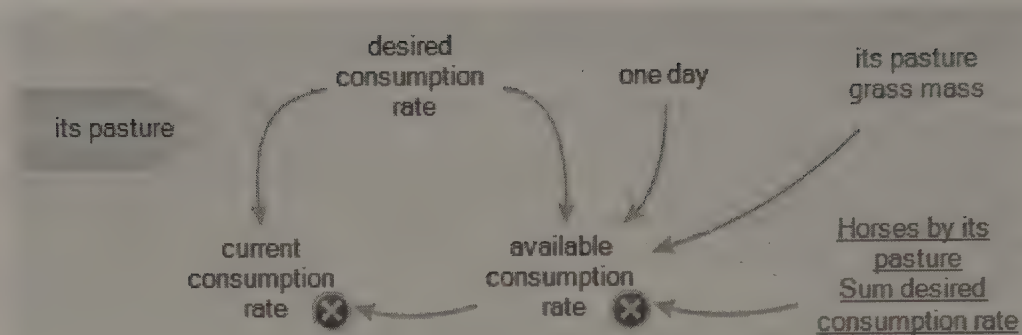


The maximum that a horse can eat is all the grass in a day. We will create another auxiliary variable called "one day", with value 1 and units "Day". Of course, every horse doesn't have all the grass to himself. The amount available to eat depends on the grass eaten by the other horses in the pasture. Therefore, we need to know the total of the desired consumption rates in each pasture.



Open the inspector of Horses [its pasture], drag the "Sum desired consumption rate" to the Horse diagram.

Create an Auxiliary variable of "current consumption rate" so that each horse obtains an amount proportional to its desired consumption. Reposition the variables in the diagram.



To write the equations, click on the area to the right of =, a menu will open with the available variables, click to select them.

Auxiliary - Horses.current consumption rate

Name:

Owner:

Units:

☐ Constant

`Min(desired consumption rate
, available consumption rate)`

Now, the current consumption rate will be the smaller value between the desired consumption and the available consumption, if there is no grass the horse cannot eat it.

On the other hand, the available pasture for each horse (grass mass of a pasture) is calculated as a ratio between what the horse wants to eat (desired consumption rate) and what the horses of the same pasture want in total (Sum desired consumption rate). Thus, each horse will eat a proportional part of the total grass in the pasture.

Auxiliary - Horses.available consumption rate

Name:

Owner:

Units:

☐ Constant

`( its pasture.grass mass/one day) * (desired consumption rate/Horses by its pasture.Sum desired consumption rate)`

To calculate the sum of the "available consumption rate" of each pasture we need to create a reference for each horse with the collection of horses in its pasture.

Horses x					
Horses --- HorsesID					
Attribute Name	EntityReference	Is Key	Reference Name	EntityReference	Condition
HorsesID	Horses	YES	Horses by its pasture	Horses[its past...	[its pasture]= ???
its pasture	Pastures	No	its pasture	Pastures	attribute value
			Model	Model	

Reference - Horses.Horses by its pasture x

Name:

Owner:

Target Type:

Conditions:

Attribute Key	Value to Match
its pasture	it

☒ its pasture
☐ its pasture

Open the grid of References of Horses entity, in Condition is it shown (???) the reference pending to be defined. Click twice on ???, open the options for Value to Match and select the attribute "its pasture", to link both attributes and entities.

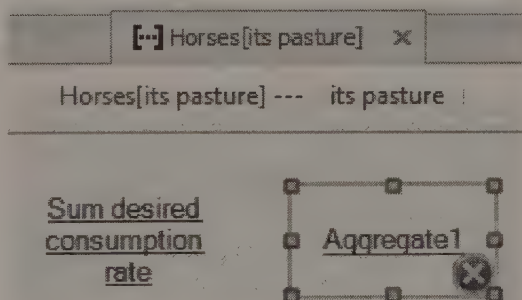
In summary, in the diagram of the Horses entity, the pasture attribute that has been placed links Horses with Pastures, and from the Pastures entity the grass mass of each parcel has been taken.

On the other hand, a reference has been created in the collection Horses [its pasture] with the variable of the desired consumption of each horse. From Horses [its pasture] the total desired consumption of the horses in a plot has been taken.

Thus, we can calculate the actual consumption of each horse based on its desired consumption and the total desired consumption of the horses that are in the pasture (Sum desired consumption rate), taking into account the existing pasture (grass mass).

The next step is to calculate the grazing of the horses in each pasture, as the sum of their actual consumption, not their desired consumption.

In the Horses collection [its pasture], create another aggregate variable to calculate total current consumption. To make the variable go to the diagram Horses [its pasture] and move the Aggregate icon to the diagram.



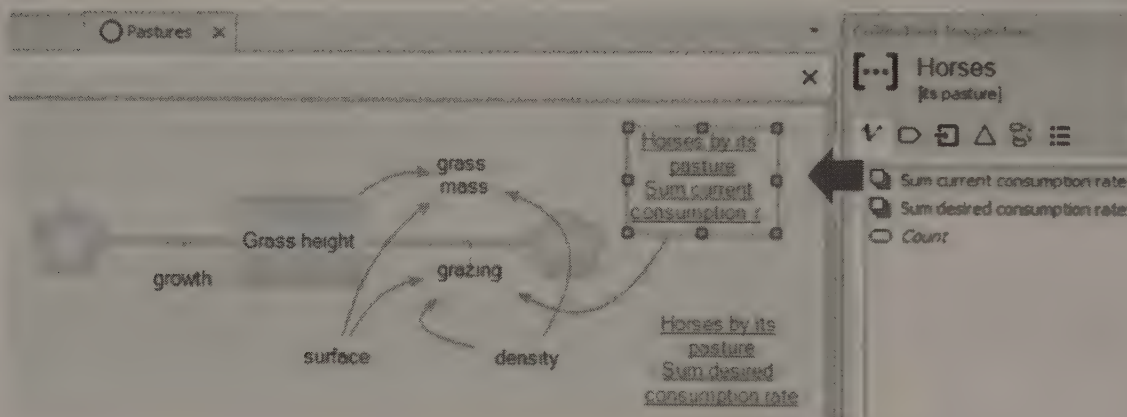
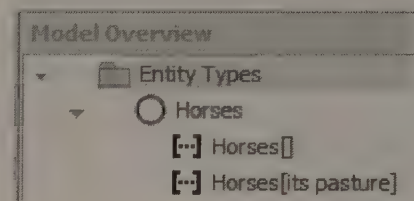
Aggregate - Horses[its pasture].Sum current consumption

Name:	Sum current consumption rate
Owner:	Horses[its pasture]
Units:	kilos/Day
Base Type:	Horses
Aggregation Function:	Sum
Target Variable:	current consumption rate

Right click and press Edit Aggregate. On the screen select Aggregation Function = Sum and Target Variable at the current consumption rate.

The Name and the other fields are filled in automatically. Close the window.

Open the inspector of Horse [its pasture] by double clicking on the detail of the Model Overview. Go to the Variables tab.



Flow - Pastures.grazing

Name:	grazing
Owner:	Pastures
Units:	cm/Day
From Stock:	Grass height
To Stock:	
☐ Constant	
$\text{Horses by its pasture. Sum current consumption rate} / \text{surface} / \text{density}$	

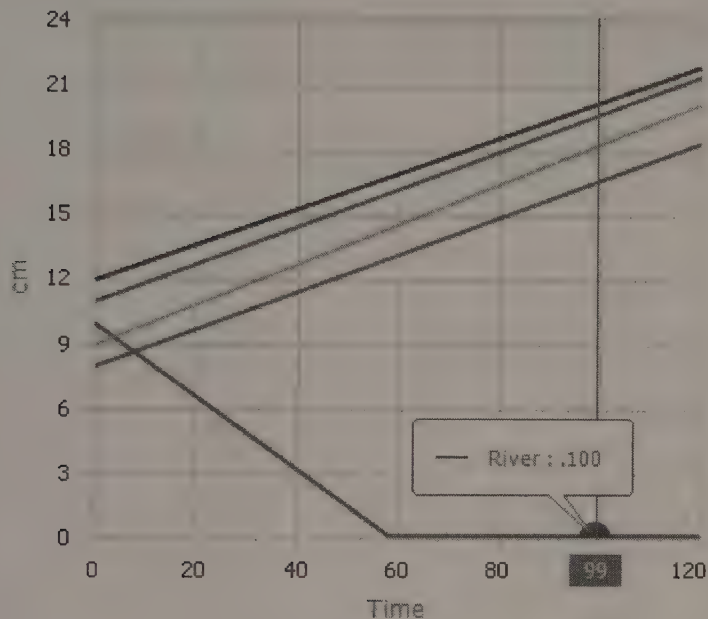
Use the "Sum current consumption rate" to calculate the amount of grazing by dragging it to the Pastures diagram and checking the grazing equation.

Delete the old link from "Horses by parcel. Sum desired consumption rate" to grazing (click on the link with the right button and then press delete) and modify the equation:

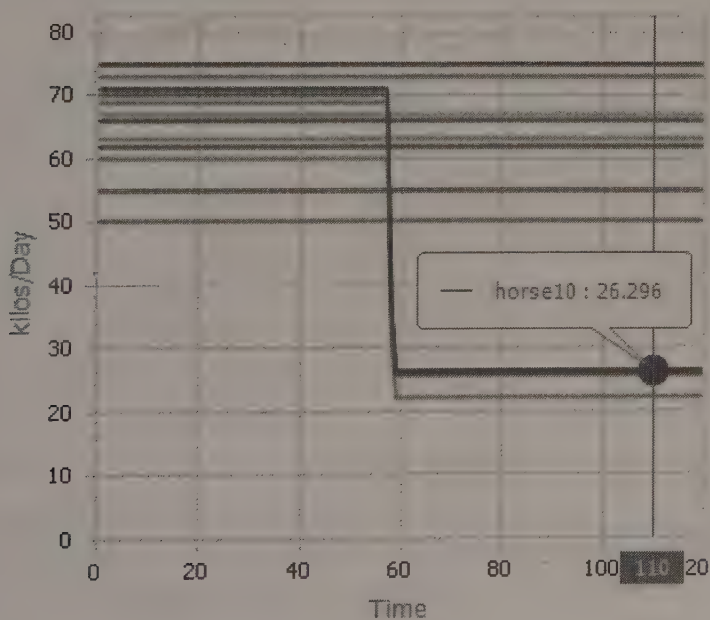
:

Simulate the model, take a look at the height of the pasture (in the Pastures tab). Now the growth lines are no longer parallel because some horses consume more than others depending on the availability of grass and see the available consumption rate (on the Horses tab).

Grass height



current consumption rate



In the Horses tab you can see the current consumption of each horse.

Save the model as Horses 6

Model: Horse Breeding 7

In this model we are going to introduce some horses in different periods of time. Go to Entity Initialization Data - Data Sources right click on "many parcels", choose Edit and Horses tab. Enter future dates at 15-day intervals for the last 5 horses. Close.

many pastures

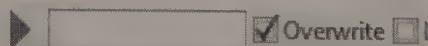
Number of rows to add:       All None Invert

Horses					
	☒	CalendarTime	HorsesID	desired consumption rate	its pasture
1	☒		horse1	50	Big
2	☒		horse2	55	Big
3	☒		horse3	60	River
4	☒		horse4	62	Dark
5	☒		horse5	63	Sunny
6	☒		horse6	66	Deep
7	☒		horse7	67	Sunny
8	☒		horse8	69	River
9	☒		horse9	70	River
10	☒	09/30/2025	horse10	71	River
11	☒	10/15/2025	horse11	73	Deep
12	☒	10/31/2025	horse12	75	Big
13	☒	11/15/2025	horse13	77	Communal
14	☒	11/30/2025	horse14	80	Communal

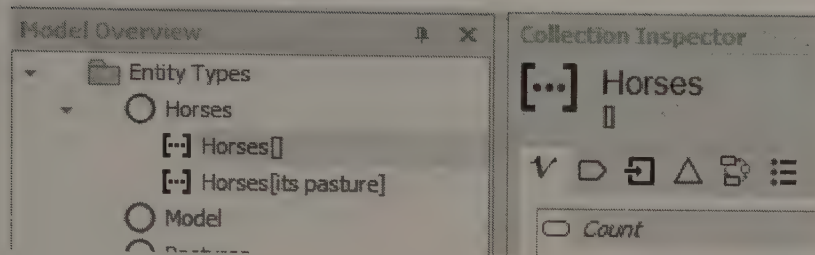
Go to Run Control, activate Using Calendar Time and enter Final Calendar Time 12/31/2025 and Initial Time = 8/15/2025. Units Of Time=Day.

Using Calendar Time	Initial Time	Final Time	Initial Calendar Time	Final Calendar Time	Time Step	Unit Of Time
☒	0	120	08/15/2025	12/31/2025	1 (139 t...	Day

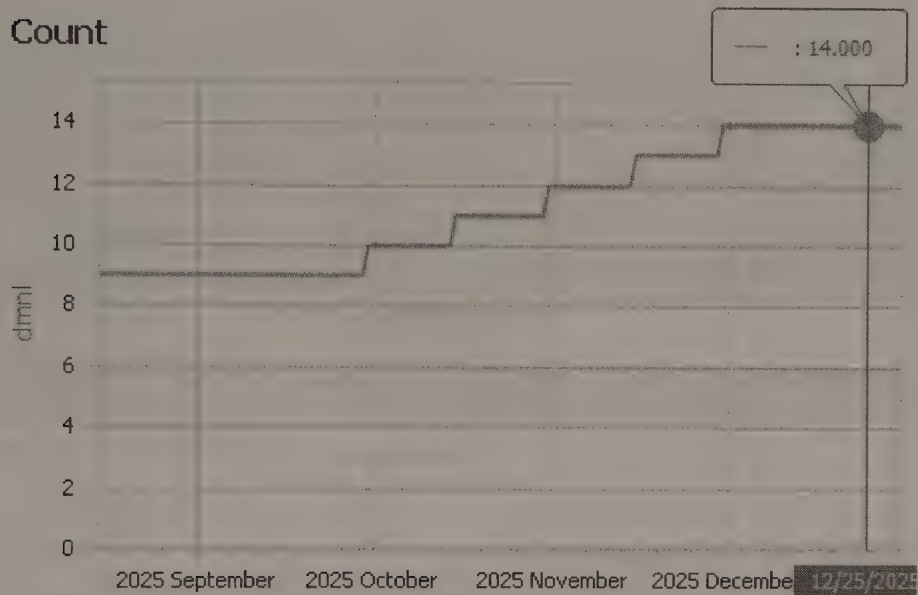
Simulate the model.



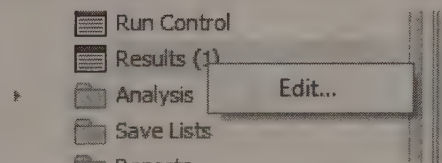
The first step is to check that the horses have been added as we have defined them. To do this in Model Overview double click on Horses [] to open the inspector. There we have a counter for the number of horses, click on it with the right button and choose Plot Variable.



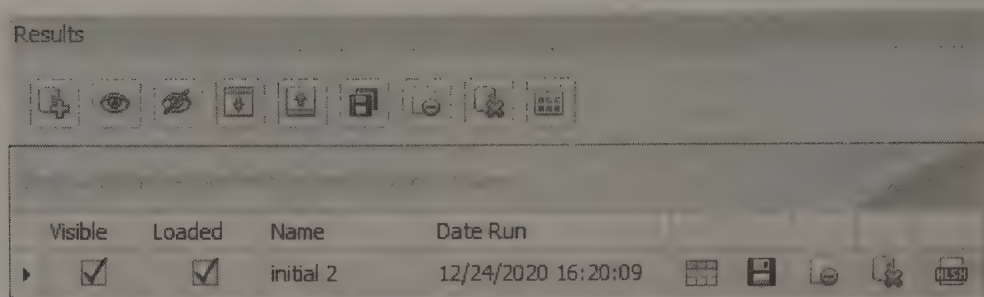
The graph allows one to verify that the model works well.



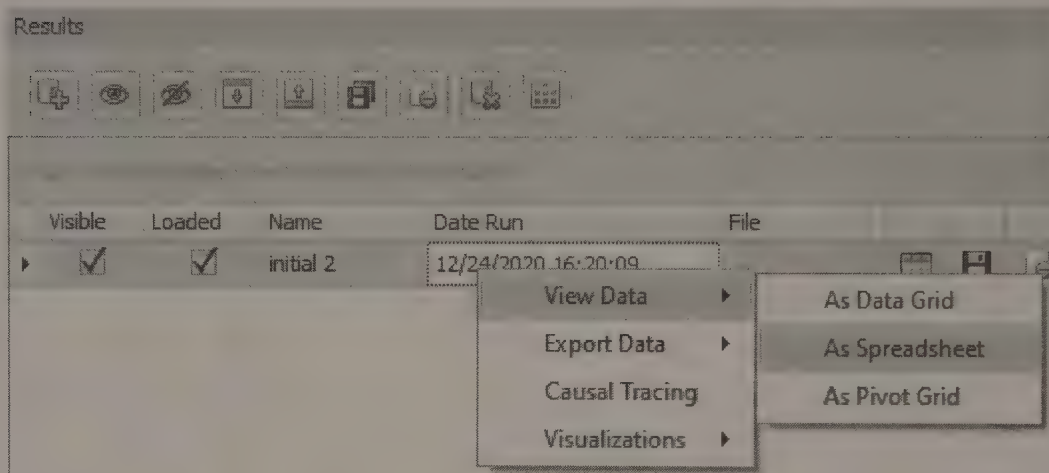
In addition to obtaining results in graphical form, we can obtain them in numerical format and export them. Go to Results, right-click and choose Edit.



In the window that opens, right-click on Run 1 (name assigned by default), display the options and select the icon As Spreadsheet.



Or right-click Date Run and select another option.

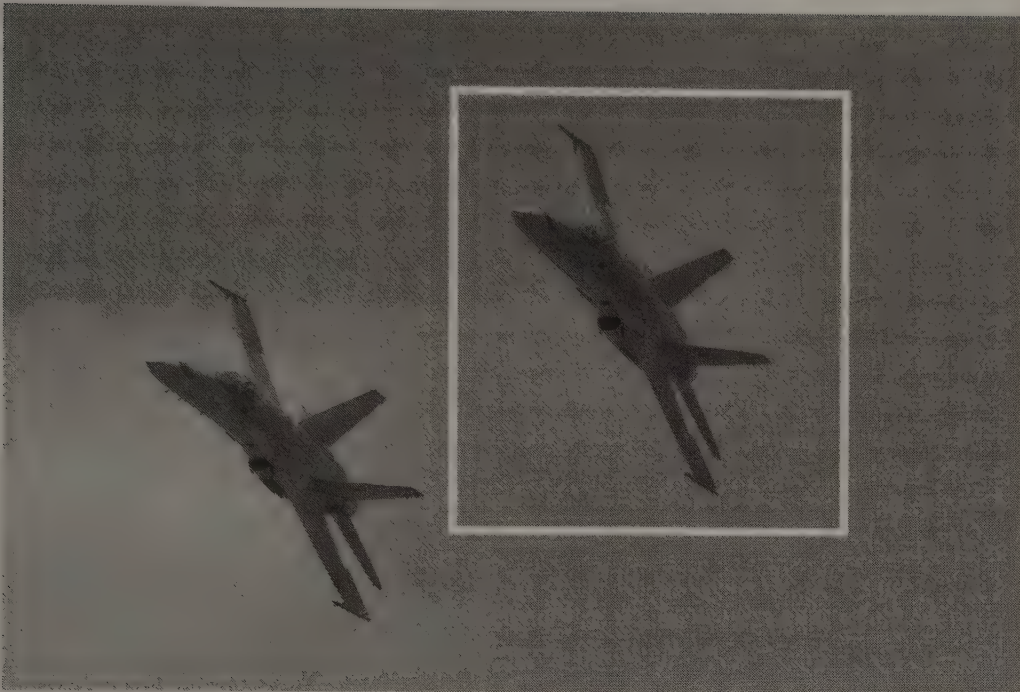


In a few seconds the information appears

	J8					
	A	B	C	D	E	F
1	DateTime	HorsesID	its pasture	available con:	current cons	desired cons
2	08/15/2025	horse1	Big	64.000	50	50
3	08/16/2025	horse1	Big	64.483	50	50
4	08/17/2025	horse1	Big	64.967	50	50
5	08/18/2025	horse1	Big	65.450	50	50
6	08/19/2025	horse1	Big	65.933	50	50

There are several options to export the results (Export Data), either directly to Excel or to CSV files without first viewing them.

Save the model as Horses 7.



Model: Aircraft Squadron

We will simulate the maintenance service of a fleet of aircraft. Aircraft are classified according to their operational status, if they are ready to take off on a mission, if they are in a scheduled maintenance service or if they are in a repair service due to any anomaly or fault detected. In these last two cases, the aircraft may be waiting to be attended to in service.

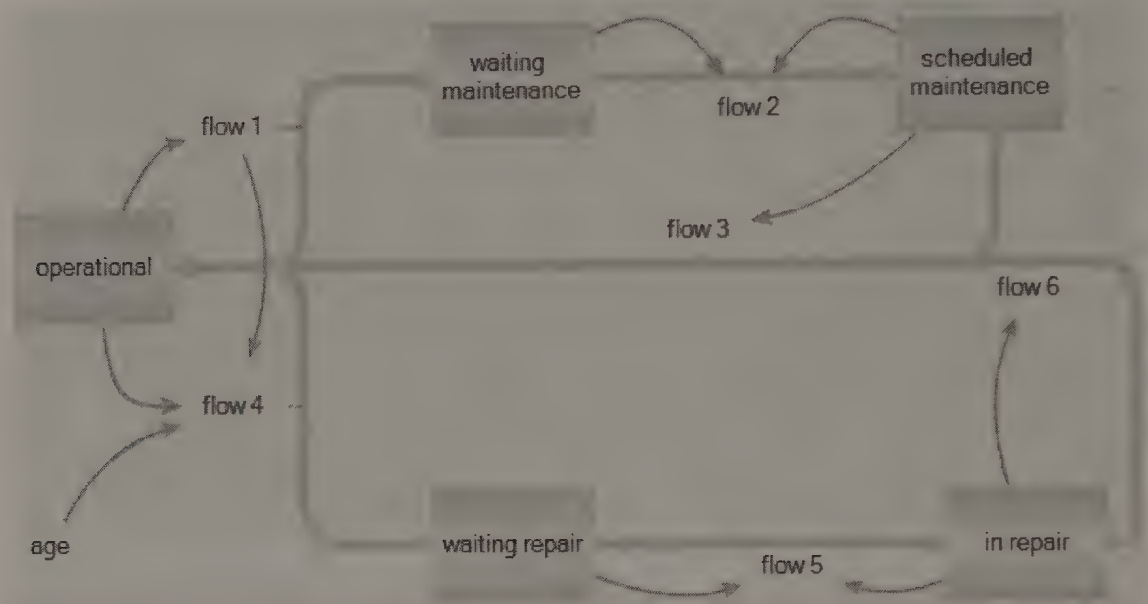
The objective of this model is to learn how to draw diagrams and equations with a certain complexity.

Press File - New Model and write Aircraft; a type of entity with the same name is created. Go to Run Control and modify the Final Time to 60 and in Units for Time type Day.

Initial Time	Final Time	Initial Calendar Time	Final Calendar Time	Time Step	Unit Of Time
0	60	11/11/2020	03/11/2021	1 (61 tot...	dia

Go to View - Units Manager and uncheck the Check Units, since in this model we are not going to use the units.

On the Aircraft screen, draw the following diagram in this order: first the Stocks, then the Flows, the Auxiliary variables and finally the arrows. Each Flow and each arrow have small green rectangles that can be used to modify their shape.



The diagram shows the changes of state of an aircraft through the 5 stocks that are shown. Thus, it can be operational and from there go to waiting maintenance or waiting repair. From these states it can be under maintenance or repair and from there it will be operational again. There is an auxiliary variable that is the age of the aircraft that determines how often the plane must be repaired.

We must define some entity for the model to work. Go to Entity Initialization Data, click with the right button on Data Sources, choose Add Builtin Data and Entities 1 is created. Enter the following data, to indicate that there is 1 plane and that it is operational and close the window.

Aircrafts						
	☒	Time	AircraftsID	operational	in repair	scheduled maintenance
► 1	☒		one	1	0	0

Equations

Stocks

operational, initial value 1

waiting maintenance, initial value 0

scheduled maintenance, initial value 0
waiting repair, initial value 0
in repair, initial value 0

Auxiliaries

age = 0 (check Constant)

Set a range from 0 to 1, with 0 if the aircraft is new and 1 if it is very old. This helps calculations and avoids working with real dates (current year minus year of purchase)

Flows

flow 1 = PulseTrain (5, 1, 15, 99) * operational

We simulate that the plane has a revision every 15 days and that the last one was made 10 days ago, so that the most immediate is in period 5. If the plane is operational, a pulse is programmed (and the flow value is 1) that begins in period 5, lasts for 1 period, repeats every 15 periods, and ends in period 99 (after Final Time, which is 60). If the aircraft is not operational (operational = 0), this flow is 0 and the aircraft is no longer waiting for a review.

flow 2 = IfThenElse(scheduled maintenance=0&&waiting maintenance=1, 1, 0)

If the value of "scheduled maintenance" is 0 (not in revision) and the value of "waiting maintenance" is 1 (it is waiting), the flow value is 1 and the aircraft leaves this state. Here a delay could be set if the total number of aircraft under review equals the capacity of the maintenance service.

flow 3=DelayInformation(scheduled maintenance,2,0)*IfThenElse(scheduled maintenance =1, 1,0)

If the aircraft is under review, the aircraft becomes operational after a delay of 2 periods. It is necessary to write the DelayInformation function without taking the written option offered by the software.

flow 4 = IfThenElse(operational=1&&flow 1=0, if(RandomNormal(operational, 0.2)>(1-age)&&RandomNormal(operational, 0.2)<(1+age)) {1} else {0}, 0)

flow 5 = IfThenElse(in repair=0&&waiting repair=1, 1, 0)

Same logic as flow 2.

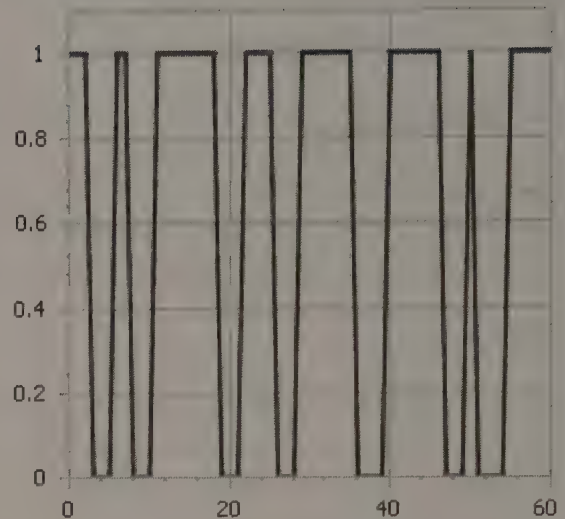
flow 6 = DelayInformation (in repair, 1.0) * IfThenElse (in repair = 1, 1.0)

Same logic as flow 3.

Simulate

Press the green arrow icon (Run Model). To see the results, click on any variable with the right button and choose the Plot Variable option. This shows the state of the aircraft during the time horizon. If it is a Stock, the value 1 indicates that it is at that state and the value 0 that it is not.

operational



Simulate several scenarios

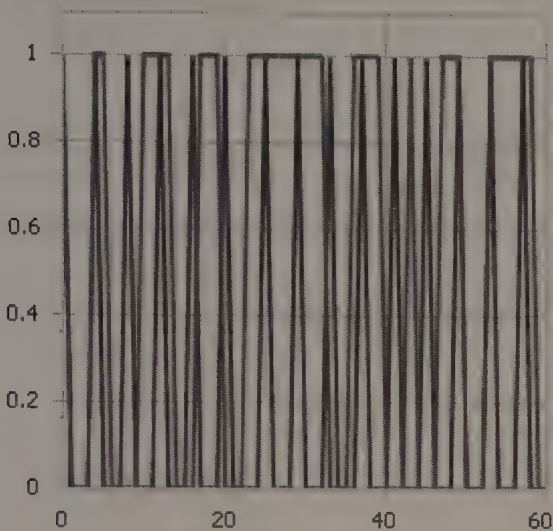
Go to Entity Initialization Data, right-click on Entities 1 and choose Rename, change to "age 0 years". Right click on Data Sources, choose Add Builtin Data, enter the following data. Then press Rename to "age 20 years", in this case we assume that the value of age is 1 (the maximum). Simulate.

Aircrafts					
	☒	Time	AircraftsID	age	operational
► 1	☒		one	1	1

Go to Run Control, uncheck "Use active data selection" and activate the data entity "20 years old" that has been created and "No Data" that uses the data defined in the diagram. Simulate the model again and see the behavior of the variables.

Entity Initialization Data		Time Series Data		Save Li	
age 20 years, No Data ▼		Use active dataset		None (	
☐ Use active data selection					
All None Invert					
Name					
►	☒	☐	age 20 years		
	☐	☐	age 0 years		
	☒	☒	No Data		

operational



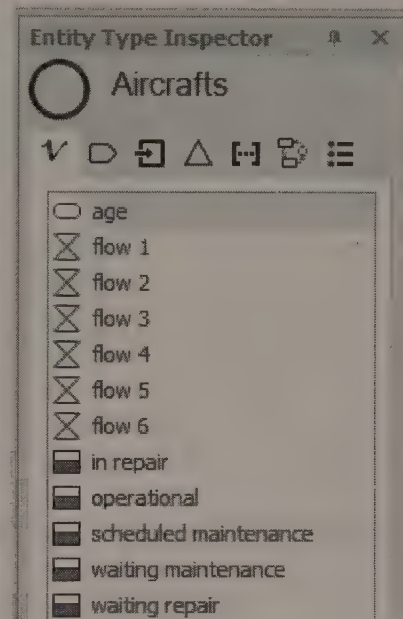
The result is very confusing, for that reason we will calculate the total of each scenario. For this it is necessary to create variables that serve as indicators of the monthly total accumulating the days that the aircraft has been operational.

Indicator

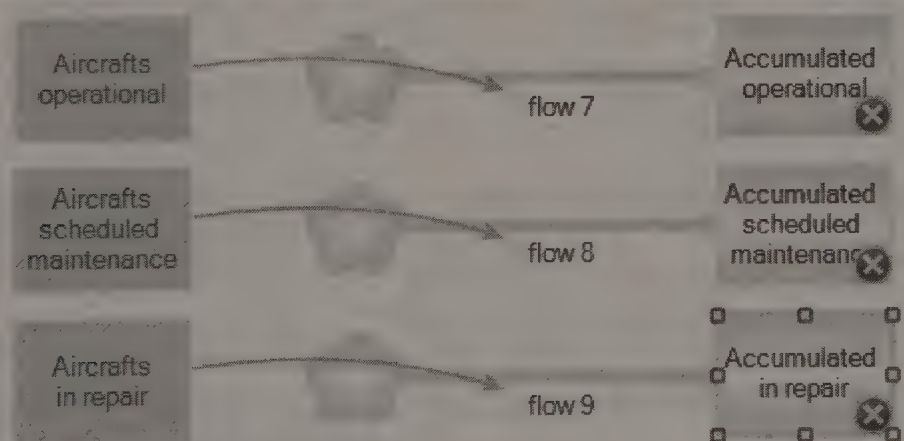
s

In Model Overview, right-click on Entity Type, choose New Entity Type and write the name Indicators. Press twice on Indicators so that it opens in the central area. Right-click on Aircraft and choose Inspect to open it in the right area of the screen.

The Aircraft inspector displays the Stocks, Flows and Auxiliaries. Move to the Indicators diagram: operational, in repair and scheduled maintenance.

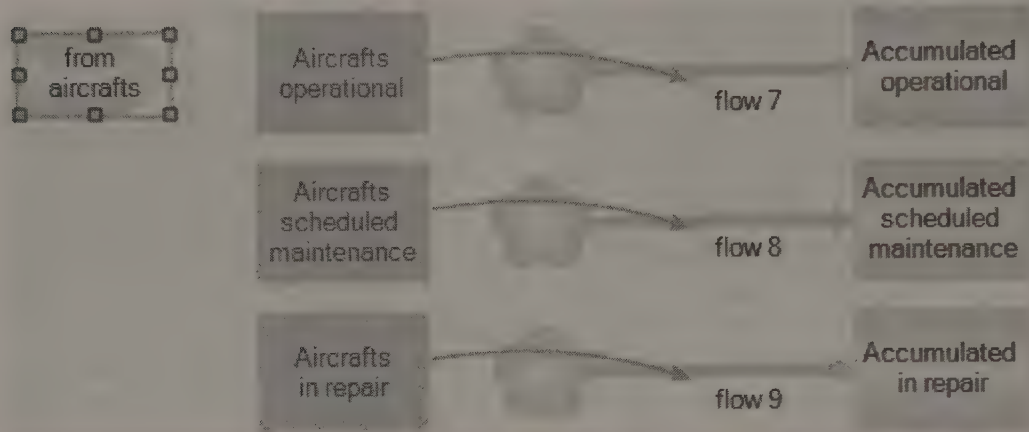


Add the 3 Stocks shown, place clouds, create flows and the arrows shown in the following diagram.



Edit the 3 flows and in the equation define that they are the same as the stock on which they depend. Edit the 3 new stocks and define that their initial value is 0 and assign the name "Accumulated".

It is necessary to add an attribute to be able to link each one of the aircraft type aircraft with the Indicator entities. Move the Attribute icon to the diagram and Edit. Write the name "from aircrafts" and in Refers to select that it belongs to Aircrafts.



Finally, you have to define the reference between Aircraft and Indicators and display the Indicators grid on the top bar. It shows that a relationship is pending definition. Click on ??? and in Value to Match write "from aircrafts", choose the attribute with that name from the list.

☐ Indicators
 ☐

-- IndicatorsID |

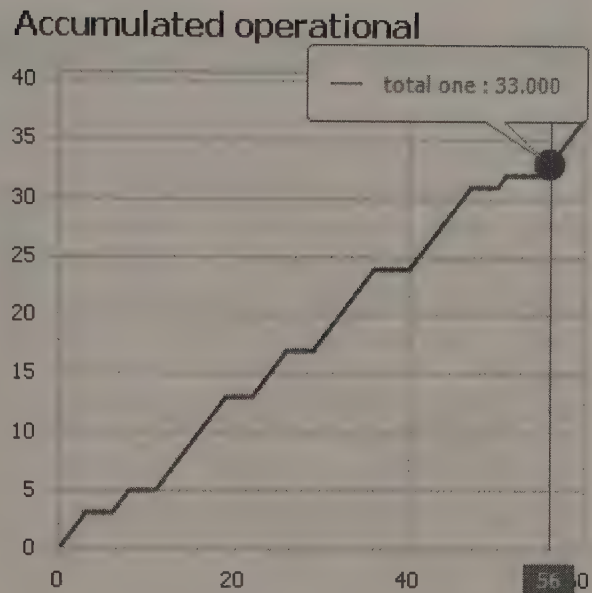
Reference	Is Key	Reference Name	EntityReference	Condition
aircrafts	No	Aircrafts	Aircrafts	[AircraftsID]= ???
indicators	YES	from aircrafts	Aircrafts	attribute value
		Model	Model	

Go to Entity Initialization Data, click twice on "age 0 years", check Show All Types if the Indicators tab is not shown, complete the following data:

age 0 years						
Number of rows to add:		+ - X Reset Undo Redo All None Invert ☐ Show All				
Aircrafts			Indicators			
	☒	Time	IndicatorsID	from aircrafts	Accumulated in repair	Accumulated scheduled
▶ 1	☒		total one	one		

Open the Run Control and Entity Initialization Data, check the "Use Active Dataset" box. Verify that it has been saved this option.

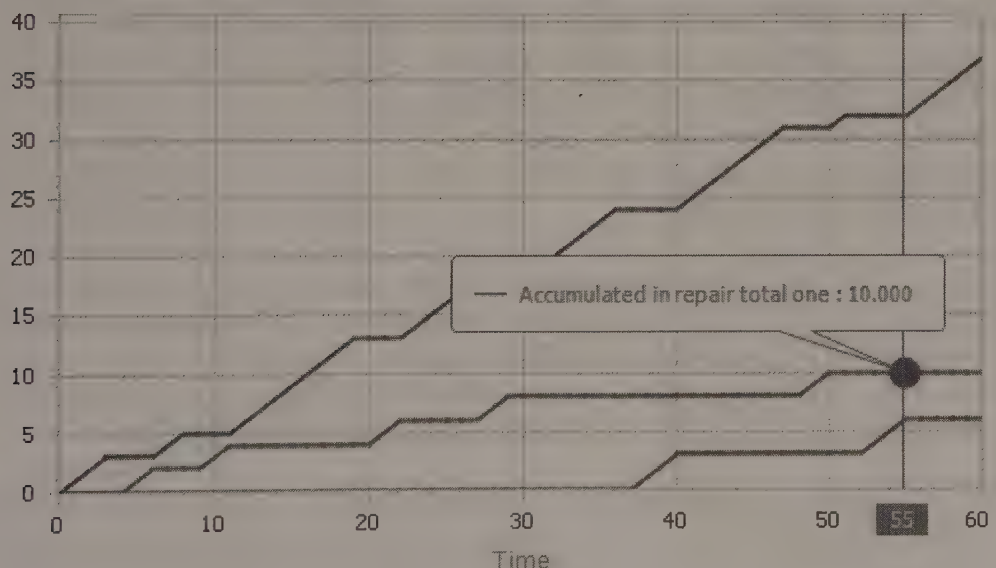
Simulate the model and with Variable Plot see the accumulated operating hours of aircraft "one".



Edit Chart Properties

In the upper left part of the screen, click on the "Edit Chart Properties" icon, in the menu that opens click on Variables, from the Indicators group select the 3 stocks: "Accumulated operational", "Accumulated scheduled maintenance" and "Accumulated in repair". Finish. The accumulated values of the 3 variables are shown.

Accumulated operational, Accumulated scheduled maintenance, Accumulated in repair



Validation: If we add the accumulated values of the 3 stocks shown plus the 2 waiting stocks, we obtain the value 61 (60 periods + period 0), which are the periods analyzed.

Model: Aircraft Squadron 2

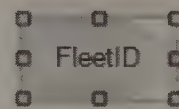
Once the model is made for a single entity, we are going to expand it to a fleet of 12 aircrafts with the information about each one that is in the Excel "Confidential.xlsx" (that is with the files of the book). Save it in the folder Data of this new model (Aircrafts 2).

The DateTime column could be used to indicate the start date of the entity. FleetID is a unique identifier for each entity (each aircraft). The AGE column is a parameter between 0 and 1 equivalent to a range of 0 to 20 years from its purchase. The "model" field is an attribute of the type of aircraft. The data is located in the sheet BaseMarbella.

	A	B	C	D
1	DateTime	FleetID	AGE	model
2		N751	0	F18
3		N752	0	F18
4		N753	0	F18
5		N754	0	F18
6		N755	0	F18
7		N756	0	F18
8		N757	0,5	F18
9		N758	0,5	F18
10		N759	0,7	F18
11		N760	0,7	F19
12		N761	0,7	F19
13		N762	1	F19

Step 1. Create an Entity Type

We will create the entity where the data imported from Excel will be. Click on Entity Types and then on New Entity Type, write the name BaseMarbella. The name of the new entity must be the same as the Excel sheet with the data.



Attribute - BaseMarbella.FleetID

Name: FleetID

Owner: BaseMarbella

Refers to: BaseMarbella

☒ Key

The BaseMarbella entity has a Key attribute with the name BaseMarbellaID created by default. The Key attribute identifies each of the entities. In Model Overview double click on BaseMarbella and the inspector opens on the right. Go to the Attribute tab, click on BaseMarbellaID and rename FleetID. The name of the Key attribute of the entity must be the same as the Excel column that identifies them.

Move the FleetID attribute from the inspector to the screen. On the same screen, move the Auxiliary icon to the diagram and create the AGE variable. Edit it with the value 0 and mark Constant. The name of the variables to import must be exactly the same as they have in the Excel column with the data.

Step 2. Define the entities

We are going to create a new entity with the actual aircraft data. Go to Entity Initialization Data, right-click on Data Sets and choose New Input Dataset. Input 1 is created. In this directory we will place all the data of the new simulation.

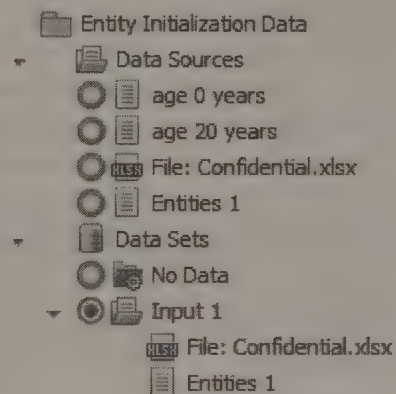
Right-click on Input 1 and choose Add Builtin Data. Entities 1 is created. In the Aircrafts tab define the values of AircraftsID and do the same in the Indicators tab with the IndicatorsID variable, entering the Excel data.

	☒	Time	AircraftsID	age
1	☒		N751	
2	☒		N752	
3	☒		N753	
4	☒		N754	
5	☒		N755	
6	☒		N756	
7	☒		N757	
8	☒		N758	
9	☒		N759	
10	☒		N760	
11	☒		N761	
12	☒		N762	

	☒	Time	IndicatorsID
1	☒		N751
2	☒		N752
3	☒		N753
4	☒		N754
5	☒		N755
6	☒		N756
7	☒		N757
8	☒		N758
9	☒		N759
10	☒		N760
11	☒		N761
12	☒		N762

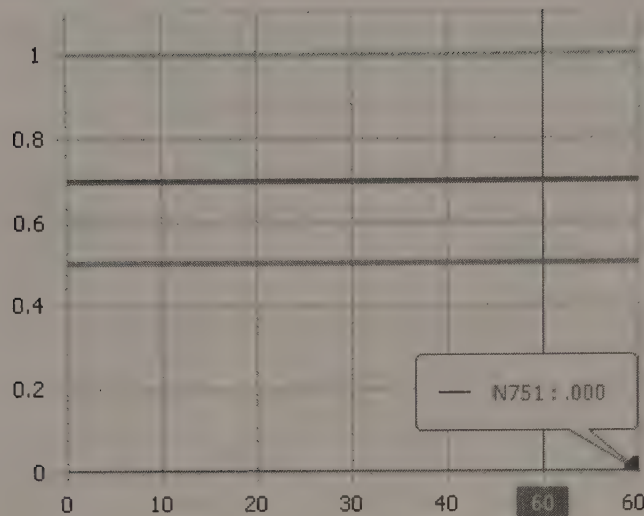
Step 3. Identify the Excel

Right click on Input 1 and choose Add External File. When you do this, the Data folder of the model automatically opens and the Excel Confidential.xlsx is displayed. Double-click on it. The name of the Excel is added in the Entity Initialization Data - Data Sets - Input 1 and that same Excel also appears in Data Sources.



If you make modifications in the Excel after you have linked it with the entity, right click on the Excel in Input 1 and choose Edit. Check the changes and then close. Then right-click on the Data Sources Excel and click Repoint.

AGE



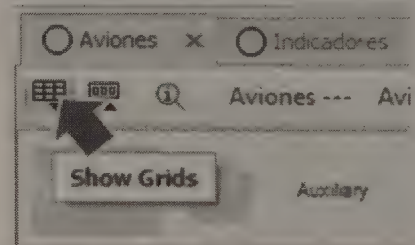
Simulate the model, and press Plot Variable in AGE.

The screen shows the age values of the planes: 0, 0.5, 0.7 and 1.

Step 4. Link Excel and entity

In the Model Overview, double-click on BaseMarbella to open its inspector on the right side. Then click on the Aircrafts tab at the top of the screen to display the diagram. Move the AGE variable from the BaseMarbella inspector to the Aircraft diagram. A variable is shown (duplicated with green color) with the name "BaseMarbella AGE". Place this variable next to the variable "age".

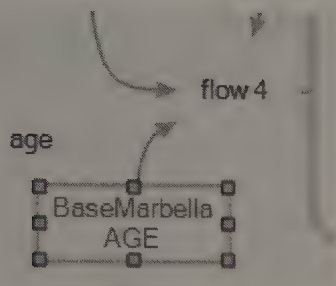
A variable (AGE) of an entity type (BaseMarbella) has been placed in another entity (Aircrafts). We need to create a reference that links them. Press the Show Grids icon.



The reference grid shows that the Aircraft entity type has an undefined BaseMarbella reference. Click on ??? and in Value to Match choose the AircraftsID attribute. Thus we indicate that AircraftsID = FleetID.

Aircrafts					
Aircrafts --- AircraftsID					
Attribute Name	EntityReference	Is Key	Reference Name	EntityReference	Condition
AircraftsID	Aircrafts	YES	BaseMarbella	BaseMarbella	[FleetID]= AircraftsID
			Model	Model	

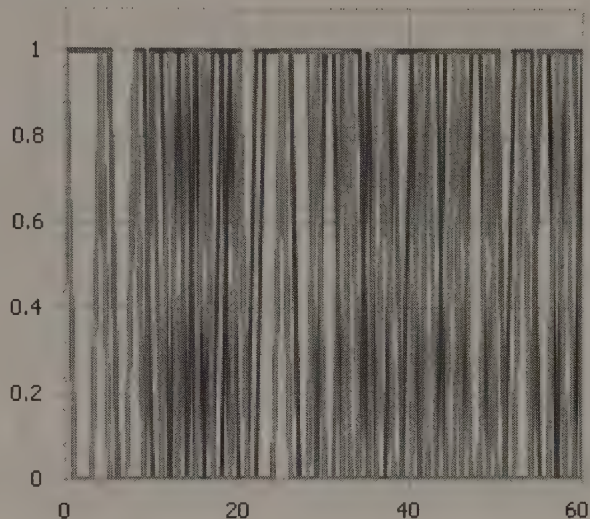
Step 5. Use of Excel data



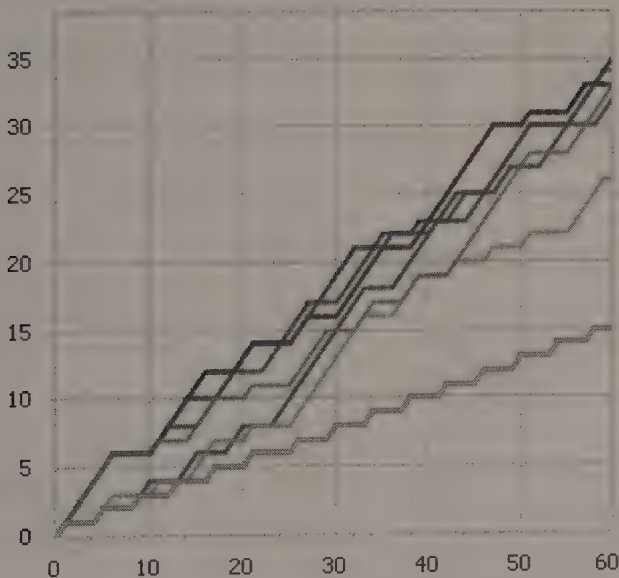
Delete the arrow that joins the variable "age" with "flow 4" and create a new variable from the duplicate variable "BaseMarbella AGE" to that flow. Edit flow 4 and in the equation replace the old variable with the new one.

Simulate the model and see the evolution of operational aircraft. The graph is confusing because it shows all the planes.

operational



Accumulated operational



Step 6. View accumulated data by entity

Go to the Indicators screen, click on the Show Grids icon to see the relationships and click on the equality AircraftID = "from aircrafts" of the previous model.

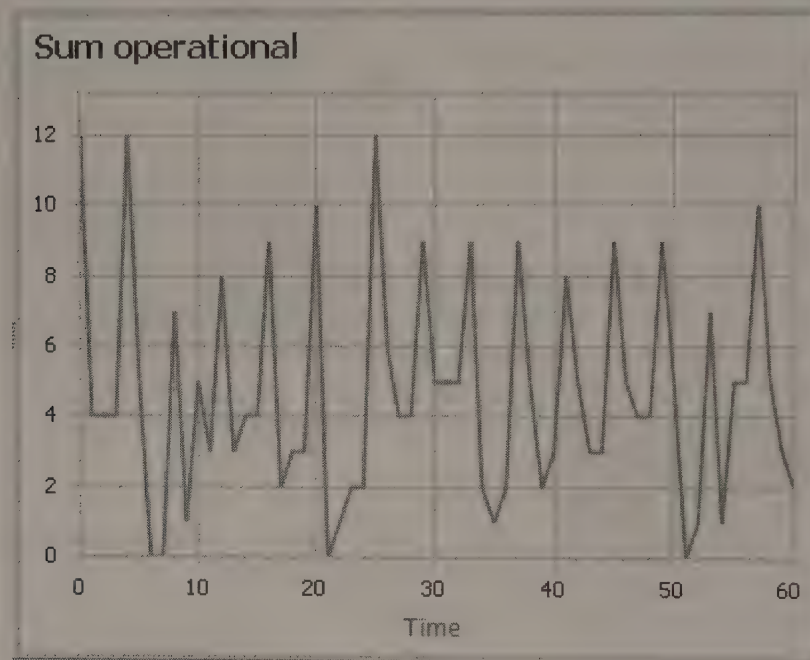
In the window that opens, go to the Value to Mach box and write IndicatorsID (which is where the identifier of each aircraft has been placed in Entity 1).

Close. Simulate and see the result.

Thus, we can already obtain in graphic or numerical format the evolution of the stocks and flows of each aircraft, both in daily and accumulated values. In the next step we will see the aggregated values of the 12 aircraft.

Step 7. Accumulated data of all entities

In Model Overview, display the options of Aircrafts and click on Aircrafts [], which is the collection of all aircraft. Move the Aggregate icon to the diagram. The Aggregate 1 is created. Right click on it and choose Edit Aggregate. In Aggregation Function choose Sum and in Target Variable the operational option. Close, simulate and select Plot Variable.



The graph shows the number of aircraft that are operational each day

Save the model.



Model: Fishing in Three Seas

A company has 10 boats spread over 3 fishing grounds. We have information on the catch capacity of each boat and the estimated value per ton of its catches. In this first model we will calculate the catch capacity in each fishing zone without further restrictions.

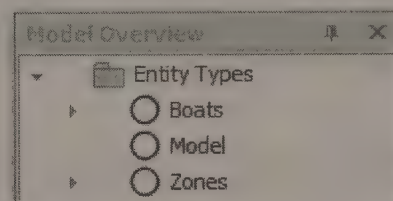
boat	zone	catch capacity Tm/Year	value euro/Tm
B-01	North Sea	10	70
B-02	North Sea	15	80
B-03	North Sea	20	90
B-04	North Sea	25	100
B-05	North Sea	30	110
B-06	South Sea	35	120
B-07	South Sea	40	110
B-08	Est Sea	45	100
B-09	Est Sea	50	90
B-10	Est Sea	55	80

Create a model to calculate with a horizon of 10 years:

1. The value of the catches by fishing zone, without any restriction.
2. If the maximum authorized catches per fishing zone are 100 Tm per year, calculate the value of the catches if they are distributed among the boats according to their capacity.
3. Recalculate the previous points if the catching capacity of the boats operating in the North Sea is increased by 10% annually.
4. Recalculate the previous points if the authorized catches fall by 5 Tm per year in all fishing zones.
5. Recalculate points 1 to 4 if boats 3, 4 and 5 start in year 6.

Create the model and its entities

Go to File-New Model and write Fishing as the name of the model. Two entities appear in the Model Overview: Model and Fishing. Rename Fishing to Boats (Rename) and add a new entity called Zones (press Entity Types - New Entity Type).

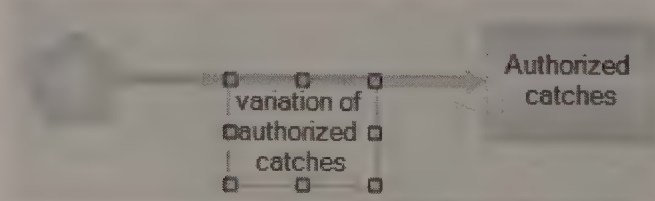


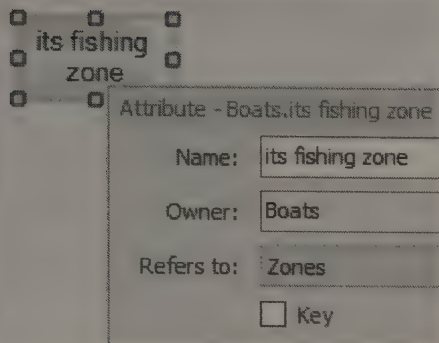
Press twice on Run Control to open it, in Final Time write 10, in Units of Time write "Year". Close.

Define the variables

We will start by creating the auxiliary variables to be used. In the Model Overview, click twice on Zones to open the central diagram of that entity.

In the Zones diagram create the Stock "Authorized catches", since one of the questions asks us to simulate the impact of its annual variation. We create that Stock, and the Flow will be that annual variation. Edit the "Authorized catches" (Edit Stock) and write 100 as initial value and Tm/Year in units. Create the flow "variation of authorized catches", with value 0 and units Tm/(Year*Year).



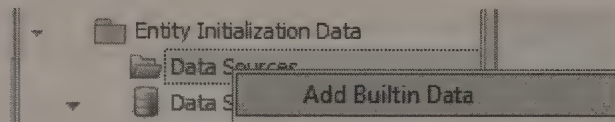


Go to the entity Boats (by clicking on the tab with that name) and create the attribute "its fishing zone" to identify which fishing ground (name) each boat is in. Edit the attribute and assign (Refers to) to the Zones entity.

In Boats create the variables "fishing value" and "catch capacity" with the values 100 euro/Tm and 40 Tm/Year respectively as default values. In both cases mark them as Constant, so that they appear

in the Entity Initialization Data. (*) Units: euros = 1000 euros.

We are going to introduce into the model the initial data of the boats and the zones. For this we do Add Builtin Data. Entities 1 is created.



Enter the data shown in the table (copy/paste available). Close.

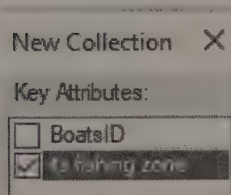
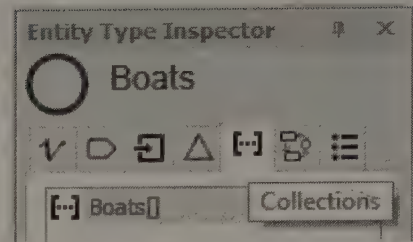
Boats						
	☒	Time	BoatsID	its fishing zone	catch capacity	fishing value
1	☒		B-01	North Sea	10	70
2	☒		B-02	North Sea	15	80
3	☒		B-03	North Sea	20	90
4	☒		B-04	North Sea	25	100
5	☒		B-05	North Sea	30	110
6	☒		B-06	South Sea	35	120
7	☒		B-07	South Sea	40	110
8	☒		B-08	Est Sea	45	100
9	☒		B-09	Est Sea	50	90
10	☒		B-10	Est Sea	55	80

Boats		Model		Zones
	☒	Time	ZonesID	Authorized catches
1	☒		North Sea	100
2	☒		South Sea	100
3	☒		Est Sea	100

Since we are looking for joint information on ships and zones, it is necessary to link the entity ships and that of zones. For this we will create a collection with the ships and their zones.

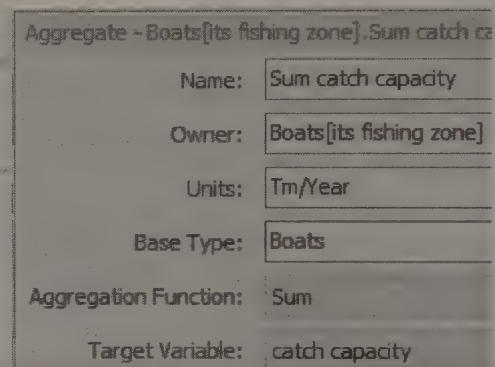
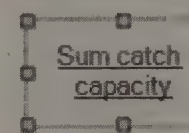
Collection

In the Boats inspector go to the Collection tab and at the bottom click New Collection. We want to have grouped information of the boats by zone. We already have the zone attribute in the Boats entity, now we are going to create an aggregate of its catch and sales capacity.

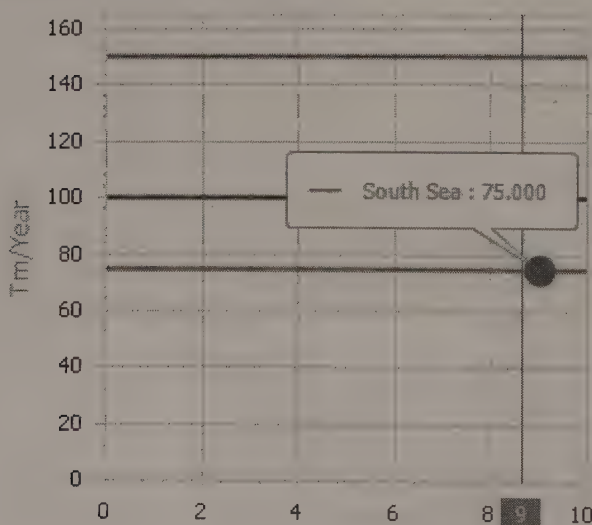


Mark the zone option, to group the boats by "its fishing zone", OK.

In the Boats [its fishing zone] diagram, move the Aggregate icon to the center of the screen, right-click on it and then Edit Aggregate. Fill in the "Aggregation Function" and Target Variable fields. Create the variable that is the sum of the catch capacity.



Sum catch capacity



Simulate the model by pressing the green icon and then, with the right button on "Sum catch capacity" to select it, choose Plot Variable.

The graph shows the catch capacity of the boats in each zone. For example, in the South Sea it is 75 Tm/Year, sum of boats 6 and 7, which was the answer to point 1.

We are going to solve the second point: "If the maximum authorized catches per zone are 100 Tm/Year, calculate the value of the catches if these are distributed among the boats according to their capacity." We know the capacity of each boat and in which zone it is. With this information we can calculate the total capacity of the boats in the zone. If that total exceeds the limit it is necessary to calculate the authorized catches per boat.

Click on the upper tab of Zones to open the screen for that entity. Open the Boat [its fishing zone] inspector (in the lower left part of the screen there is a tab that allows you to choose the inspector, or in the Model Overview display the Boats collections, right-click on Boats [its fishing zone] and select Inspect).

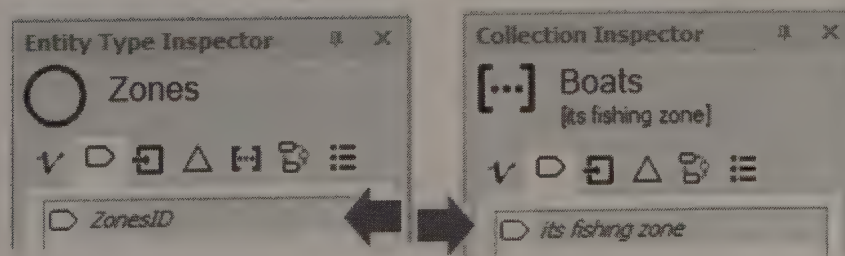
Move "Sum catch capacity" from the inspector of Boats [its fishing zone] to the Zones diagram to compare the capacity of the boats in each zone with the total authorized and calculate the ratio of authorized capacity by zone.

Press the Show Grids icon, the expression [its fishing zone] = ??? points out that this reference needs to be defined. Click twice on ???, in the window that opens go to Values to Match and write ZonesID. Select the attribute with that name. Close.

Reference - Zones.Boats by its fishing zone

Name:	Boats by its fishing zone	
Owner:	Zones	
Target Type:	Boats[its fishing zone]	
Conditions:	Attribute Key	Value to Match
	its fishing zone	

[-] Zones[]
[-] ZonesID



Now the values of the attribute "its fishing zone" of Boats [its fishing zone] are linked to the attribute ZonesID

In Zones create the variable "authorized capacity ratio", add arrows to it from the "Authorized catches" stock and from "Boats by its fishing zone Sum catch capacity". Edit and write the equation of the image. Units: Dmnl (dimensionless) means which is a ratio without units, write if, select the function, then write the initial of each variable and a menu is displayed where you can choose.

Auxiliary - Zones.authorized capacity ratio

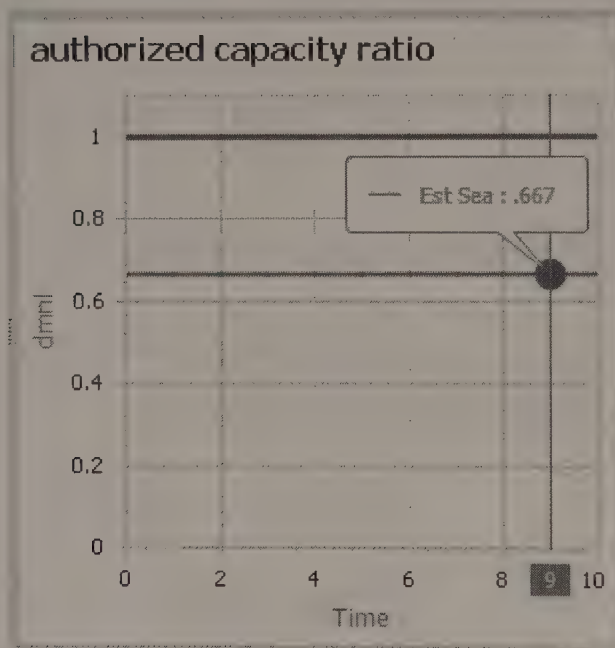
Name: authorized capacity ratio

Owner: Zones

Units: dmn1

☐ Constant

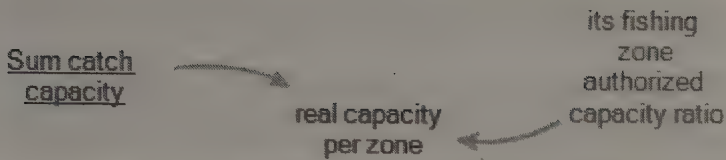
```
if(Boats by its fishing zone.Sum catch capacity>=
Authorized catches) {Authorized catches/Boats by
its fishing zone.Sum catch capacity} else {1}
```



Simulate, then right-clicking on the variable "authorized capacity ratio", and selecting the Plot Variable option, shows us that the East Sea fishing zone exceeds the authorized limit and its boats can only operate at a capacity of 66.7%. To see the numerical results, click on View Data in that menu.

Now we are going to calculate the authorized catches of each boat. To do this go to the diagram Boats [its fishing zone] and on the other hand open the Zones inspector. Move the variable "authorized capacity ratio" from the inspector to the diagram.

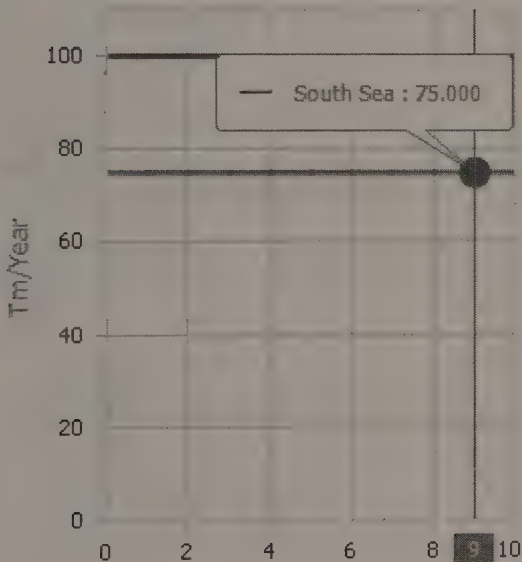
Add the variable "real capacity per zone", which will be the product of the capacity of the vessels in a fishing ground and the authorized ratio per fishing ground. Units Tm / year.



Auxiliary - Boats[its fishing zone].real capacity per zone

Name:	real capacity per zone
Owner:	Boats[its fishing zone]
Units:	Tm/Year
	☐ Constant
Sum catch capacity*its fishing zone.authorized capacity ratio	

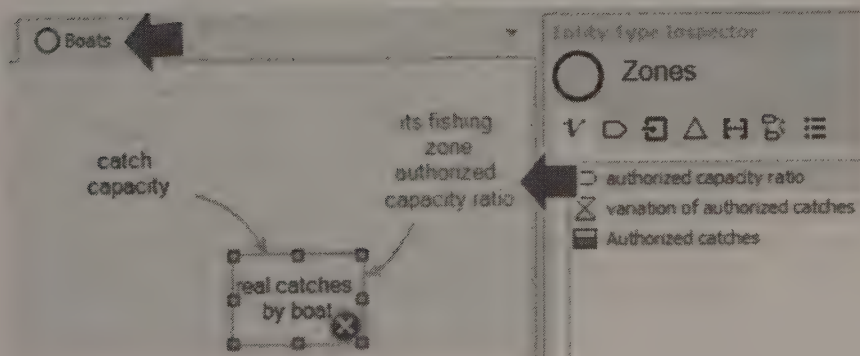
real capacity per zone



Simulate. The East Sea and North Sea zones have 100 Tm/Year and the South Sea 75 Tm / year. The only one affected by the limitation is that of East Sea, which goes from 150 to 100. The objective of this step was to validate that the variables are working correctly.

Go to the diagram of the entity Boats and move from the Zones inspector the variable "authorized capacity ratio". See next image.

Create the auxiliary variable "real catches by boat", which will be the product of the "fishing ground ratio authorized capacity" and "catch capacity" of each ship. Units Tm/Year. Simulate the model.



Auxiliary - Boats.real catches by boat

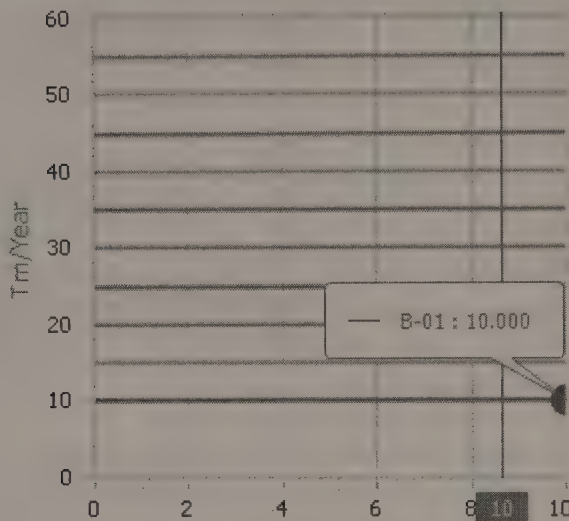
Name:

Owner:

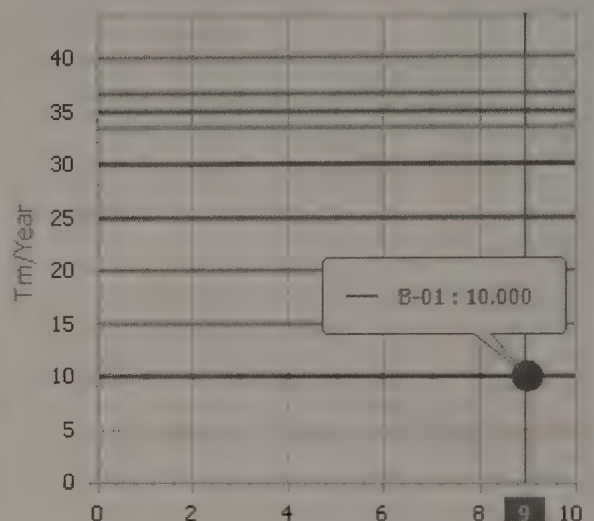
Units:

☐ Constant

catch capacity

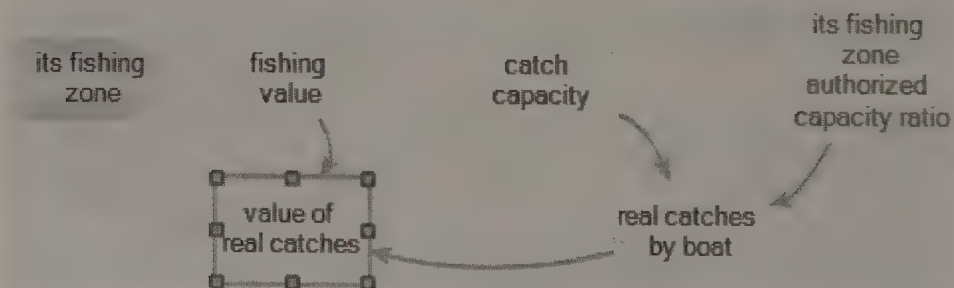


real catches by boat

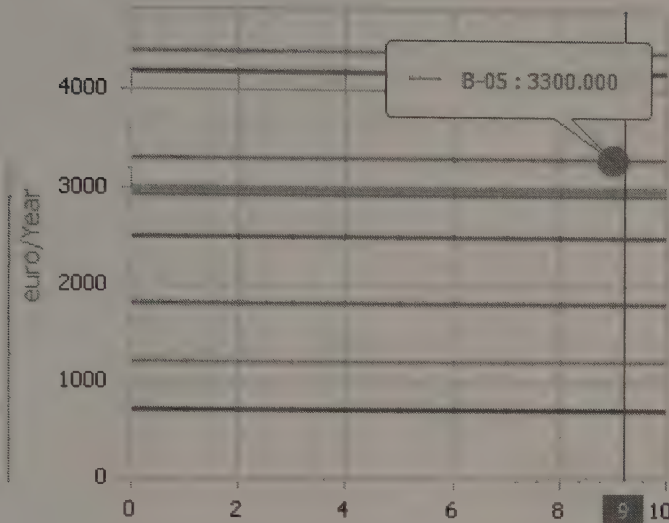


Boat B-09 has a catch capacity of 50 Tm/Year, but when applying the limit reduces to 33.3 Tm/Year. The next step is to calculate the value of each boat's catch, and then calculate the total value of the catch.

In the Boats diagram, add the variable "value of real catches", which is the product of "fishing value" by "real catches by boat". Units: euro/Year. Simulate.

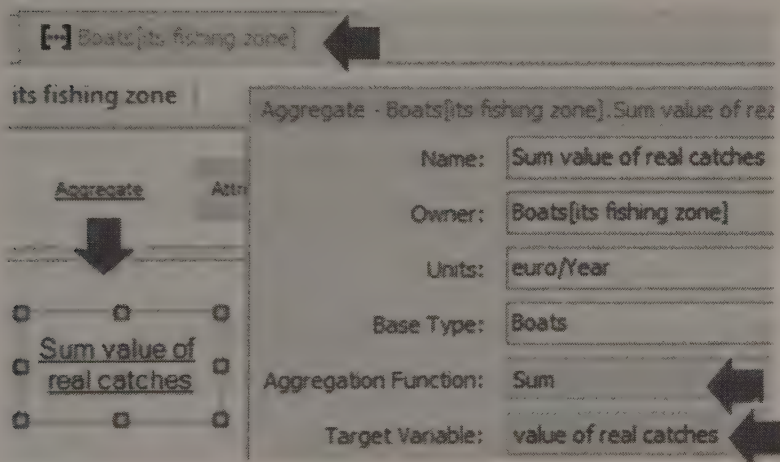


value of real catches



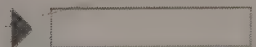
We can check one of the ships, for example the B-05, with these data: 30 Tm / year and 110 euros / Tm. It has not been affected by the limitations, so its income is 3,300 euros (actually 3,300,000 euros because it has been indicated that we will use euros for 1000 euros).

To calculate the total value of the actual catches by zone, go to the diagram Boats [its fishing zone] and create an aggregate that is the sum of the value of the actual catches.



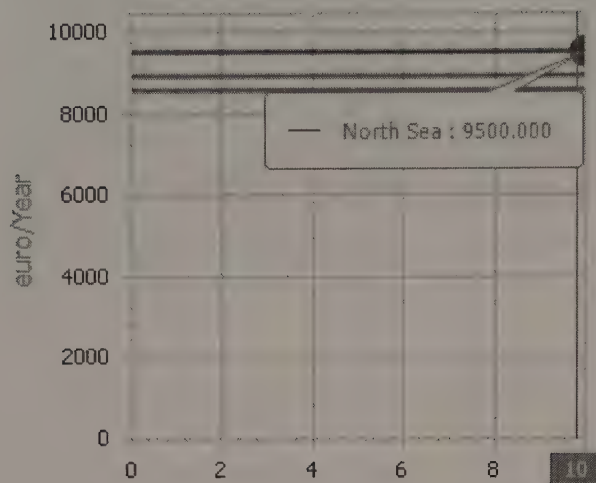
Select the "Aggregate Function" and the "Target Variable". The other fields included in the name are placed automatically.

Simulate.

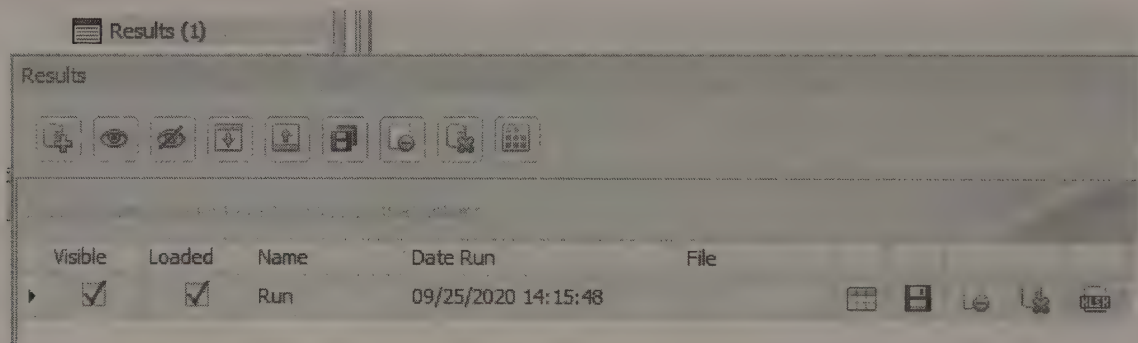


View the results by right-clicking on the name of the variable "Sum actual catch value" in graphic (Plot Variables) and numeric (View Data) format.

Sum value of real catches



Export the results, to do this, click on Results, and then on the XLSX icon, a screen opens in which we can indicate the name of the file and the desired location.



The file has different sheets, with the detail of each view or Entity.

24	0 B-03	North Sea	20	90	20
25	1 B-03	North Sea	20	90	20

Boats
Boats{its fishing zone}
Zones
Boats{}
Zones{}
Model

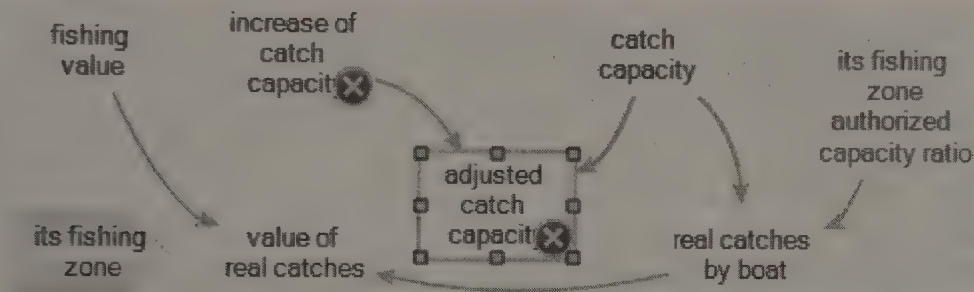
In Boats (its fishing zone) add the annual value of the actual catch, which is € 27 million.

Save the model.

Model: Fishing in Three Seas 3

The objective now is to: "Recalculate the previous points if the catch capacity of the boats operating in the North Sea is increased by 10% annually."

It is necessary to introduce some changes in the model to allow one to simulate the increase of the fishing capacity of the boats. In the Boats tab, create the auxiliary variables "increase of catch capacity" and "adjusted catch capacity".



Auxiliary - Boats.increase of catch capacity

Name:

Owner:

Units:

☐ Constant

Equations:

The RAMP function allows us to simulate an increase (simple, not compound) of 10% per year, it takes the values: 1, 1.1, 1.2, 1.3, 1.4, ...

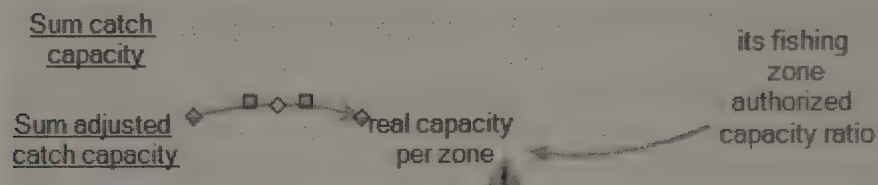
The adjusted catch capacity is the product of the catch capacity and the increase in catch capacity.

Units: Tm/Year

In the Boats [its fishing zone] diagram create an aggregate the sum of the "adjusted catch capacity" to find out the new catch capacity each year.

Aggregate - Boats[its fishing zone].Sum adjusted catch c	
Name:	Sum adjusted catch capacity
Owner:	Boats[its fishing zone]
Units:	Tm/Year
Base Type:	Boats
Aggregation Function:	Sum
Target Variable:	adjusted catch capacity

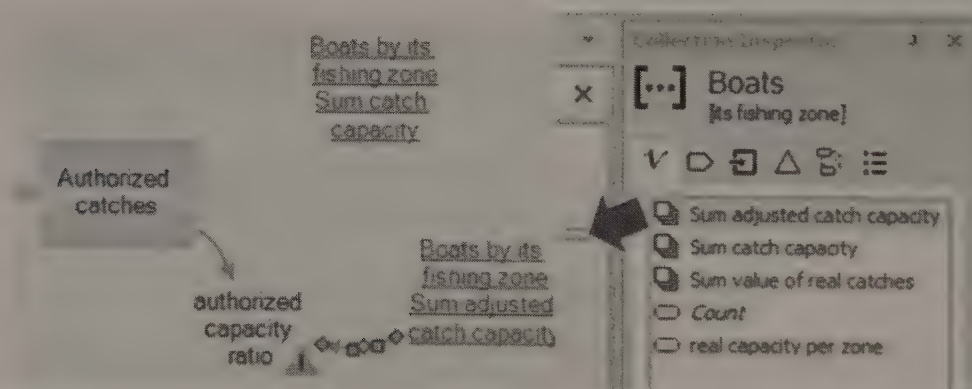
In the Ships [its fishing zone] screen it is necessary to change the diagram, since now the real capacity depends on the adjusted capacity, and modify the equation of the "real capacity by fishing ground"



Auxiliary - Boats[its fishing zone].real capacity per zone

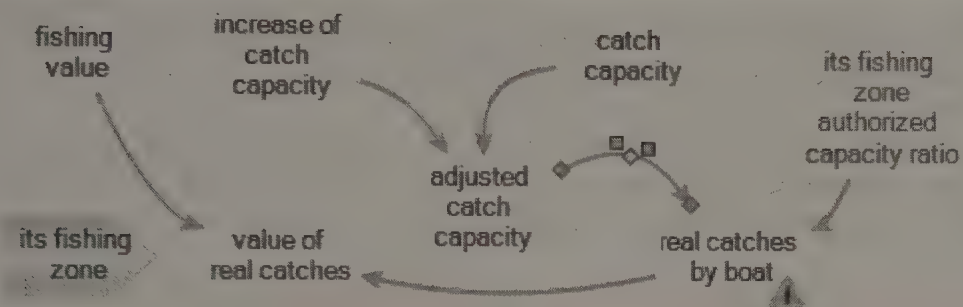
Name:	real capacity per zone
Owner:	Boats[its fishing zone]
Units:	Tm/Year
☐ Constant	
Sum adjusted catch capacity*its fishing zone. authorized capacity ratio	

Lastly, it is necessary to modify the diagram of the Zones entity to add the "Sum adjusted catch capacity" from the inspector Boats [its fishing zone]. This variable replace the old catch capacity in the equation of the "authorized capacity ratio".



Name:	authorized capacity ratio
Owner:	Zones
Units:	dmnl
☐ Constant	
if(Boats by its fishing zone.Sum adjusted catch capacity>= Authorized catches) {Authorized catches/Boats by its fishing zone.Sum adjusted catch capacity} else {1}	

Return to the Boats diagram to modify the equation for the actual catches, which becomes dependent on the "adjusted catch capacity" instead of the "catch capacity"



Auxiliary - Boats,real catches by boat

Name:

Owner:

Units:

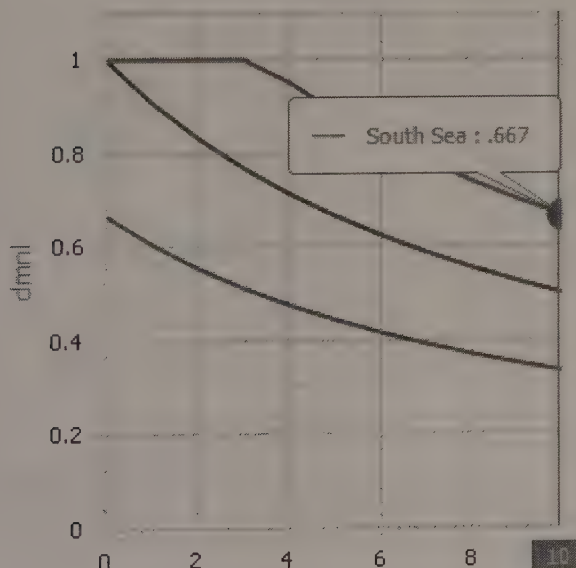
☐ Constant

$$\text{adjusted catch capacity} * \text{its fishing zone. authorized capacity ratio}$$

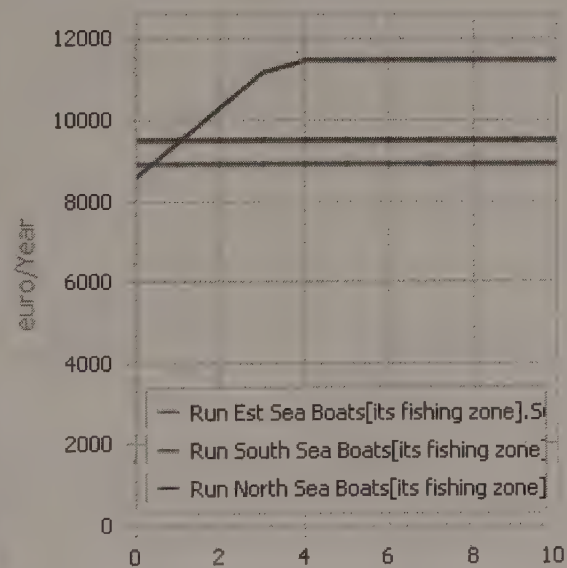
Simulate the model. The results show that the limit of 100 Tm/Year per zone, together with the growth in the capacity of the boats, affects the boats in different zone unevenly. Thus the South Sea boats -which operated at 75% of their capacity- it takes 3 years to reach the catch limit. In the other zones, an increase in capacity has no impact because they were already operating limited by the restriction of

catches.

authorized capacity ratio



Sum value of real catches

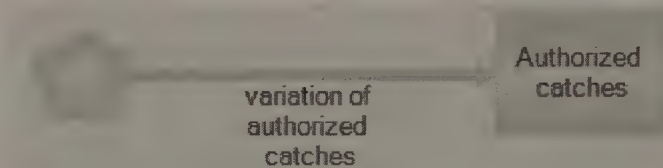


Thus, only in the South Sea fishing zone there is an increase in income.

Model: Fishing in Three Seas 4

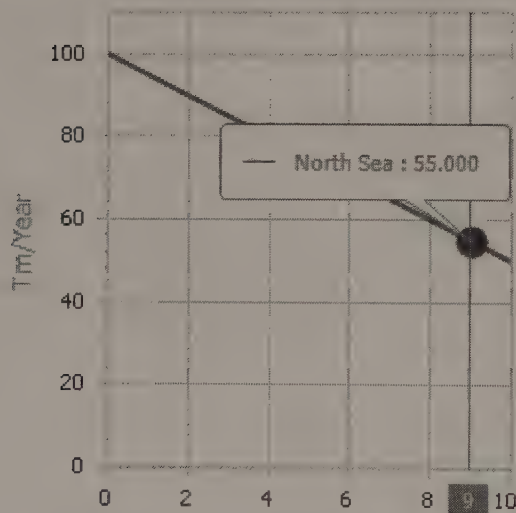
The objective of this exercise is: "Recalculate the previous points if the authorized catches fall 5 tons per year in all fishing zones."

In the Zones entity, a Stock of "Authorized Catches" was created together with a Flow of its variation. Modifying the value of this flow from 0 to -5 allows simulates the impact of reducing the authorized catches by 5 tons per year.

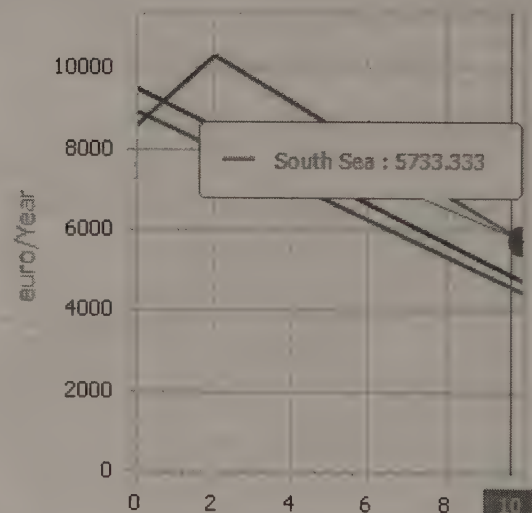


Right-click on "variation of authorized catches", choose Edit Flow and write -5. In fact, the flow drawing and the sign in the Stock equation are intended for increases, but it is easier to modify only the (negative) value of the flow. Close and Simulate. The authorized catches decrease from 100 to 50 tons per year in all the fishing grounds.

Authorized catches

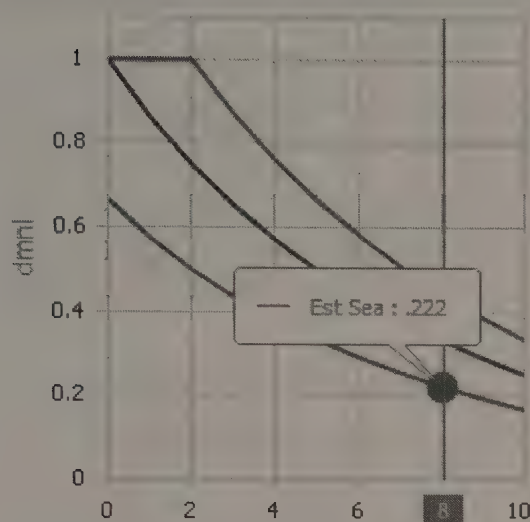


Sum value of real catches



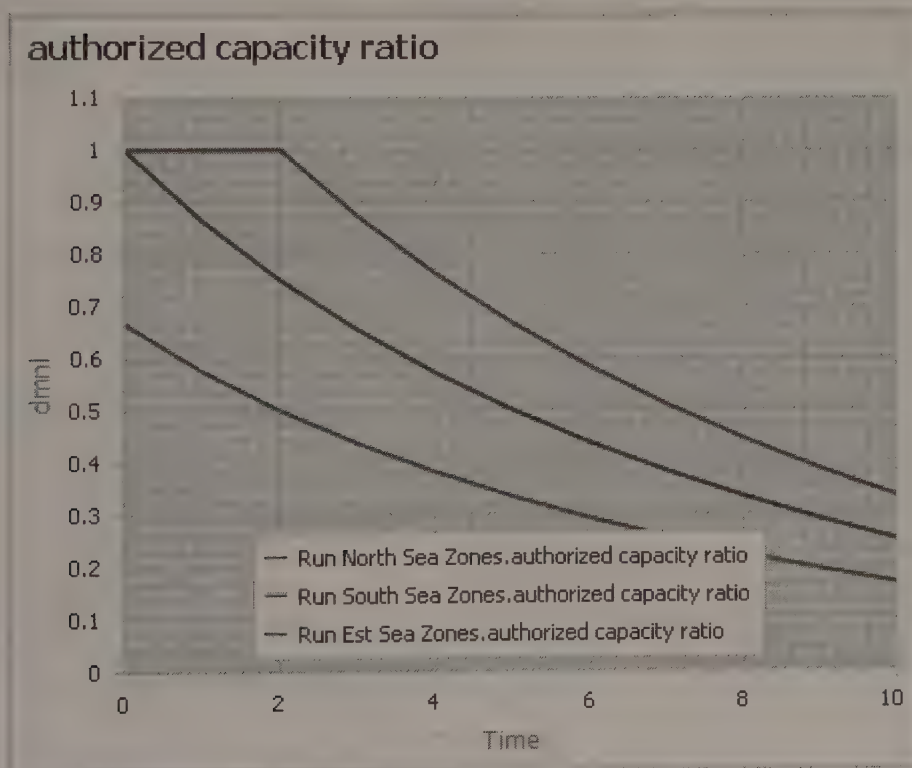
The value of real catches decreases each year in proportion to the decrease in authorized catches, except in the South Sea, which starts below the authorized limit and increases as its capacity increases in the previous year.

authorized capacity ratio



The East Sea fishing zone, which is the one with the greatest capacity for catches above the limit, is the one that most shows the reduction in authorized catches.

To edit some properties of the charts, click on Edit Chart Properties, and select
 Legend - mark Visible and select Vertical (Bottom) - Horizontal (Center).
 Panes - Background color (Green) Fill Style (Fill Mode Hatch - Max)
 Finish



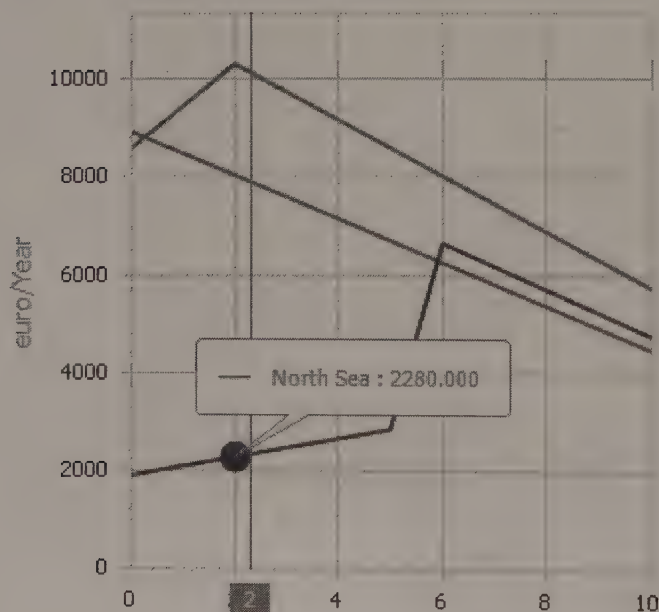
Model: Fishing in Three Seas 5

The objective of this exercise is "Recalculate points from 1 to 4 if boats B-03, B-04 and B-05 enter within 6 years"

In Entity Initialization Data - Entities 1, place the value 6 in the Time column for boats B-03, B-04 and B-05, leaving the rest of the boats unchanged, since by default the value 0 is assumed and the entity is active in the year initial. Thus boats B-03, B-04 and B-05 will start operating in year 6.

Boats						
	☒	Time	BoatsID	its fishing zone	catch capacity	fishing value
1	☒		B-01	North Sea	10	70
2	☒		B-02	North Sea	15	80
3	☒	6	B-03	North Sea	20	90
4	☒	6	B-04	North Sea	25	100
5	☒	6	B-05	North Sea	30	110
6	☒		B-06	South Sea	35	120
7	☒		B-07	South Sea	40	110
8	☒		B-08	Est Sea	45	100
9	☒		B-09	Est Sea	50	90
10	☒		B-10	Est Sea	55	80

Sum value of real catches



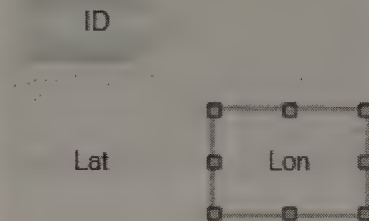
Simulate the model.

This change affects only the North Sea fishing zone since those boats are assigned that area. In this period there is an increase in catches, which then decrease due to the reduction of the simulated limit in the previous point. Boats 1 and 2 located in the North Sea from the beginning increase their catches annually because they do not reach the catch limit.

We want to know if any of the boats are in a restricted area. We have information on the position of the boats, and we want to include it in the model for future use. With the Fishing 5 model open, make Save As, the folder and the Fishing 6 model are created. Copy the file Position.xlsx (available with this book, see annexes) in the Data folder of this new model. This Excel has the ID code of each boat, and the latitude and longitude coordinates.

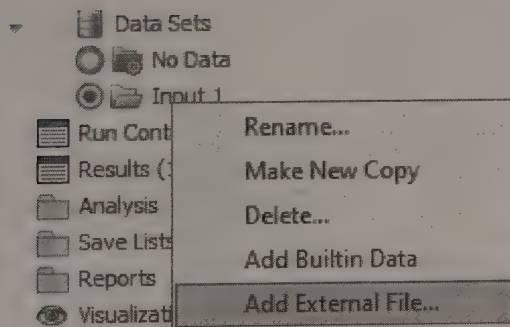
In Model Overview - Entity Types, right click, select New Entity Types, create one with the name "Coordinates". The name of the entity must be the same name as the Excel sheet where the data is.

On the right side of the screen, open the "Coordinates" inspector and click on the Attribute tab. Move the "CoordinatesID" attribute to the diagram area. Press Rename of this attribute and change from "CoordinatesID" to "ID", which is the name used in the Excel in the column that identifies the boats.



Move the Auxiliary icon to the diagram area, create the variable "Lat", edit and write the value 0 and units degrees. Mark Constant. Repeat and create the variable "Lon". These are the names used in the Excel data. Go to Entity Initialization Data, open Entities 1, click Show All Types and verify that there is a new tab, with the name Coordinates, close.

Boats	Zones	Coordinates	Model
☐	Time	ID	Lat
☐			Lon

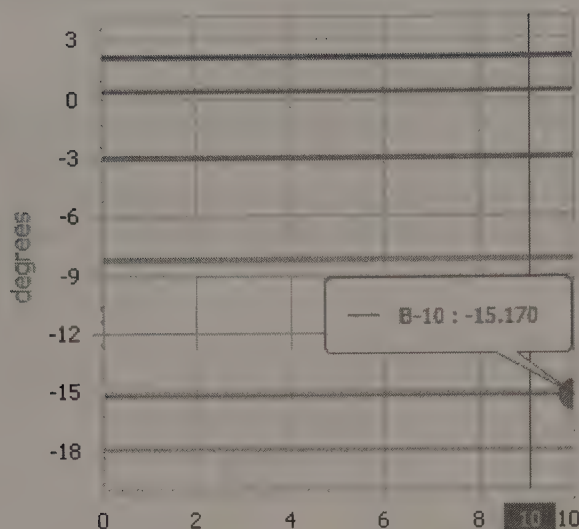


The next step is to include the xlsx file into the model. In Entity Initialization Data - Data Sets choose the option - New Input Data Set and the Input 1 folder is created.

Then, click on Input 1 and choose Add External File. A window opens that shows the Excel Position.xlsx, double-click on it, the window closes and is displayed within Input 1.

Simulate the model, and make Plot Variable of the variable Lon

Lon



The graph shows us the longitude coordinate of each of the boats.

Next, we will create the variable that tells us if a boat is in the restricted area. Keep in mind that: AND: is written "&&" in this software.

We enter the 4 vertices of the rectangle that defines the zone restricted to fishing.

ID

Lat

Lon

Auxiliary - Coordinates - restricted zone

Name:

Owner:

Units:

☐ Constant

```
if (Lat>24&&Lat<26&&
Lon<-15&&Lon>-16 )
{1} else {0}
```

Simulate the model. In this case, the graph is not very helpful since it indicates that ship B-10 is in that area but we do not know if there are any more. Click on the View Data option of the restricted zone variable to have the results in table format.

Results: Run - Component: Coordinates - Element

DateTime	Run	ID	restricted zone
8	Run	B-09	0
9	Run	B-09	0
10	Run	B-09	0
0	Run	B-10	1
1	Run	B-10	1
2	Run	B-10	1



Model: Gold Market

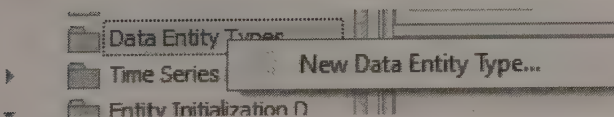
Importing time series

In this model we are going to import a time series from Excel. Create the model Gold, then open the Excel "gold prices.xlsx" (available with this book) and save it in the Data folder of the Gold model.

Column A contains the Key indicator, B the dates and C the data to be exported to the model.

Go to Run Control and define Final Time = 10 and Units Of Time = Year.

	A	B	C
1	goldID	DateTime	PRICE
2	oro	0	1200
3	oro	1	1400
4	oro	2	1300
5	oro	3	1200
6	oro	4	1600
7	oro	5	1800
8	oro	6	1200
9	oro	7	1300
10	oro	8	1400
11	oro	9	1500
12	oro	10	1200



Create a new Data Entity Type, enter the name "prices", which is where the data will be imported.

Create a Key attribute that identifies these data entities. To do this move the Attribute icon to the prices diagram, right-click on it, choose Edit and write the name goldID (the same name as the Excel). Refers to: prices, mark Key and close.

Data Variable - prices.PRICE

Name: PRICE

Owner: prices

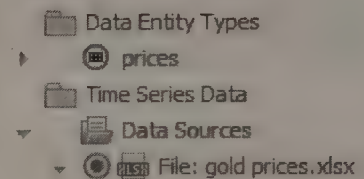
Units: euros

Input Name: PRICE

Interpolation: Straight

In the price diagram, move the Data icon to the screen, enter the Input Name PRICE and euros in Units. Close.

Go to Time Series Data - Data Sources, click right button and choose Add Data File. A window opens with the Data folder for this model. Double click on "gold prices.xlsx". The data series must be 0, 1, 2, ... because in Run Control Calendar has not been marked, as will be seen in the following exercise.



In the new window that opens click on Detect Source Entity Types, to locate the file (wait a few seconds) and then click on Data Entity Type. Choose the option price, to link data from Excel to the entity "prices" created (wait a few seconds again). Then click on Data Name boxes, display options and choose the name of the attribute or variable (in the diagram) corresponding to each Source Name (from Excel). Close.

Time Series Data Mapping - File: gold prices.xlsx

Mapping Name: gold pricesMapping

Time Series Data Description:

File: gold prices.xlsx

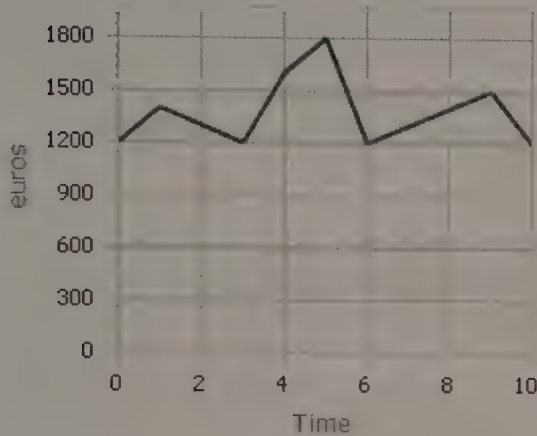
Detect Source Entity Types

Source Entity Type: page

Data Entity Type: prices

Source Name	Source Type	Interpolation Type	Aggregation Type	Source Units	Data Name	Data Type	Data Key	Data Units
goldID	Key				goldID	Attribute	Key	
PRICE	Variable	Straight			PRICE	Variable		euros
	Variable	Straight						

PRICE



Simulate the model. Select the PRICE variable, make Plot Variable to check that the behavior matches the content of the Excel. Now it is possible to use this variable like any other in the model.

Save the model.

Model: Gold Market 2

Importing a series with dates

	A	B	C
1	goldID	DateTime	PRICE
2	oro	31/12/2009	1200
3	oro	31/12/2010	1400
4	oro	31/12/2011	1300
5	oro	30/12/2012	1200
6	oro	30/12/2013	1600
7	oro	30/12/2014	1800
8	oro	30/12/2015	1200
9	oro	29/12/2016	1300
10	oro	29/12/2017	1400
11	oro	29/12/2018	1500
12	oro	29/12/2019	1200

In this case we are going to use the same data from the previous example except that the periods are going to be real dates instead of a sequence.

Go to Time Series - Data Sources - Add Data File and select the Excel "gold prices dates.xlsx" and follow the steps described in the previous example to link Excel and model. Select this Excel instead of the previous "gold prices.xlsx"

Open the Run Control, select Using Calendar Time.

Using Cale... Time	Initial Time	Final Time	Initial Calendar Time	Final Calendar Time	Time Step	Unit Of Time
☒	0	10	12/31/2009	12/31/2019	1 (11 tot...	Year

PRICE



Enter the dates in Initial Calendar Time and Final Calendar Time, keeping in mind that the notation MM / DD / YYYY is used. Write Year as Units Of Time

Move the cursor to the right to see additional fields, enter the Initial and Final Save Calendar Time dates. Close and simulate.

Dates now appear on the X axis instead of a sequence of numbers.

Save the model.

Model: Gold Market 3

Importing a historical series

In this example we will import a series of historical data, that can be used to calibrate the model or in the calculations of other variables. The first column of the Excel is the name of the key identifier, the second column is the date, and the third there is the historical price.

	A	B	C
1	goldID	DateTime	PRICE
2	gold	01/01/1950	35
3	gold	01/02/1950	35
4	gold	01/03/1950	35
5	gold	01/04/1950	35
6	gold	01/05/1950	35
7	gold	01/06/1950	35
8	gold	01/07/1950	35
9	gold	01/08/1950	35

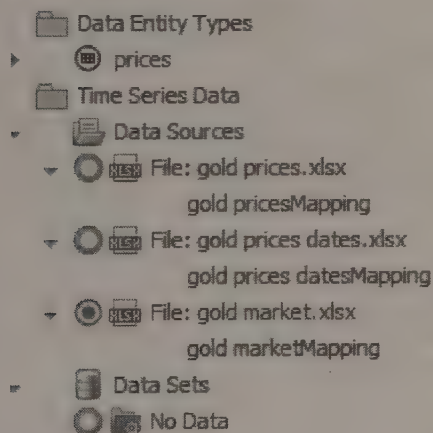
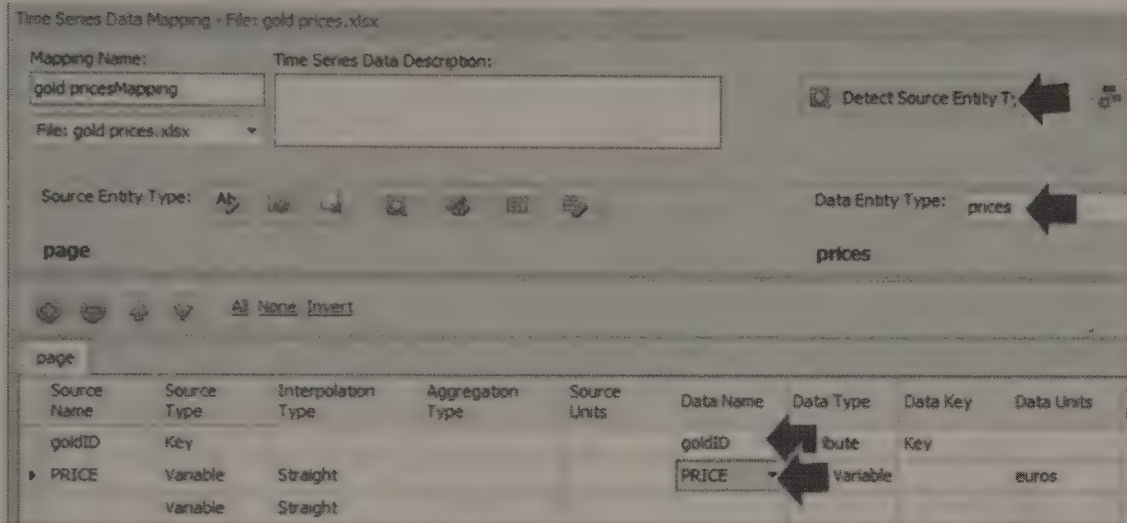
Copy the file "gold market.xlsx" in the Data folder.

In the Model - Overview open Run Control, activate the option Using Calendar Time. In the Initial Time enter the dates 01/01/1950, in the Final Time 07/01/2020, and the same in Initial and Final Save Calendar Time, this is in MM / DD / YYYY format. Close the window. Check that the Units Of Time are Year.

Go to Time Series Data - Data Sources - (right button) and choose Add Data File. A window opens with the Data folder for this model. Double click on "gold market.xlsx"

In the new window that opens, click on Detect Source Entity Types, so that it looks for the file (wait a few seconds) and as Data Entity Type select the only option that is price, to link the Excel data with the entity that was created (wait a few seconds).

Then click on the Data Name boxes, display the options and choose the name of the variable (from the diagram) that corresponds to each Source Name (from the Excel). Close.



Check the option "File: gold market.xlsx"

Simulate. On the screen, right-click on PRICE and choose Plot Variable. The graph is displayed.

PRICE



Although the Excel file has monthly data, the graph shows annual values because in Run Control Year in Units Of Time has been selected.

Results: Run - Component: prices - Element: PRICE

DateTime	Run	goldID	PRICE
01/01/1950	Run	gold	34,720
01/01/1951	Run	gold	34,660
01/01/1952	Run	gold	34,790
01/01/1953	Run	gold	34,850
01/01/1954	Run	gold	35,040
01/01/1955	Run	gold	34,970
01/01/1956	Run	gold	34,900

Click on View Data to verify that the results are annual.

Go to Run Control, change Year to Month, close and simulate the model. Check the results in graphic and table format.

PRICE



Results: Run - Component: prices - Element: PRICE

DateTime	Run	goldID	PRICE
01/01/1950	Run	gold	34,730
02/01/1950	Run	gold	34,730
03/01/1950	Run	gold	34,730
04/01/1950	Run	gold	34,730
05/01/1950	Run	gold	34,730
06/01/1950	Run	gold	34,730

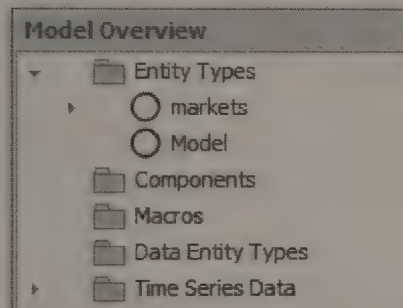
We can clearly see from the table that the results are now monthly.

Import of two historical series

In this case we are going to import two series of historical data from an Excel. Page1 has the prices for gold and Page2 for silver.

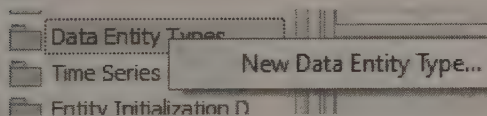
	A	B	C
1	goldID	DateTime	priceGold
2	gold	01/01/1950	34,73
3	gold	01/02/1950	34,73
4	gold	01/03/1950	34,73
5	gold	01/04/1950	34,73

	A	B	C
1	silverID	DateTime	priceSilver
2	silver	07/01/1991	4,47
3	silver	08/01/1991	4,08
4	silver	10/01/1991	4,14
5	silver	11/01/1991	4,11

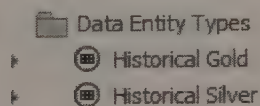


To start create a model with the name markets, and copy the file markets.xlsx in their Data folder.

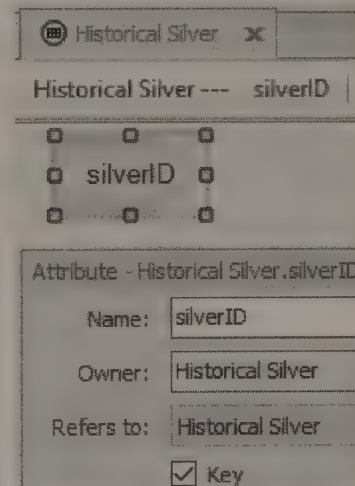
In the Model - Overview open Run Control, mark the option Using Calendar Time. In the Initial Time enter the dates 01/01/1950, in the Final Time 07/01/2020, and the same in Initial and Final Save Calendar Time, in MM / DD / YYYY format. Units Of Time=Month.



In Data Entity Types, choose New Data Entity Type. Write the name "Historical Gold" and repeat to create "Historical Silver"



Create an index to identify the variables of each entity. Move the Attribute icon to the Historical Gold diagram, right-click on it, choose Edit, write the name goldID (the same one used in Excel), Refers to "Historical Gold", mark Key and close. Create a similar attribute silverID in the Historical Silver entity.



Data Variable - Historical Silver.pri

Name: priceSilver

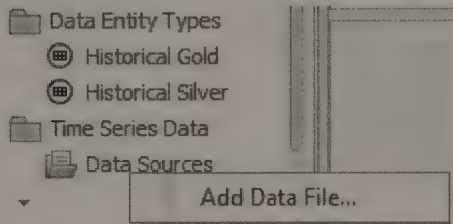
Owner: Historical Silver

Units: euros

Input Name: priceSilver

Interpolation: Straight

In the "Historical Gold" diagram, move the Data icon to the screen, press Edit, enter the Input Name priceGold and euros in Units. Close. Repeat in the diagram "Historical Silver" to create the variable priceSilver (those names do not need to be the same as used in the Excel file).



Go to Time Series Data - Data Sources - (right button) and choose Add Data File. A window opens with the Data folder for this model. Double click on "markets.xlsx"

In the new window that opens, click on Detect Source Entity Types to search for the file (wait a few seconds), and as Data Entity Type select Historical Gold to link the Excel data with the entity "markets" that has been created (wait a few seconds).

Next, on page1, click on the Data Name cells, display the options and choose for each column of the Excel (Source Name) the corresponding model variable (Data Name)

Time Series Data Mapping - File: markets.xlsx

Mapping Name: marketsMapping

File: markets.xlsx

Time Series Data Description:

Detect Source Entity Types

Source Entity Type: page2

Data Entity Type: Historical Silver

Historical Silver

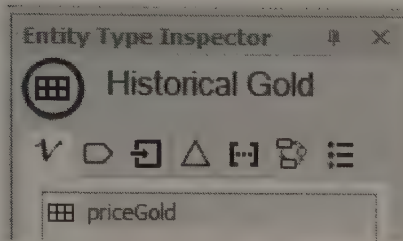
page1 page2

Source Name	Source Type	Interpolation Type	Aggregation Type	Data Name	Data Type	Data Key
silverID	Key			silverID	Attribute	Key
priceSilver	Variable	Straight		priceSilver	Data Variable	

Click on the page2 tab, go to the Data Entry Type and choose Historical Silver. Display the Data Name options and select the one that corresponds to each row. Close.

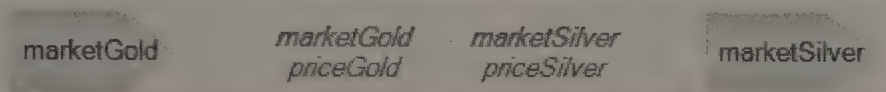
For future uses we want to have the prices of gold and silver in the entity markets.

In the market's entity create two attributes marketGold and marketSilver, press Edit Attribute by right clicking on them and choose Refers to Historical Gold and Historical Silver respectively. As Initial Value write "gold" and "silver" (with quotes) respectively.



Open the inspector of Historical Gold and the markets diagram. Move "priceGold" from the inspector to the diagram. Repeat the process from Historical Silver.

When a new window appears select Reference to marketGold or marketSilver respectively.



Close and simulate the model. Right-click on priceGold, choose Plot Variable and the graph is displayed. Repeat for priceSilver.

priceGold

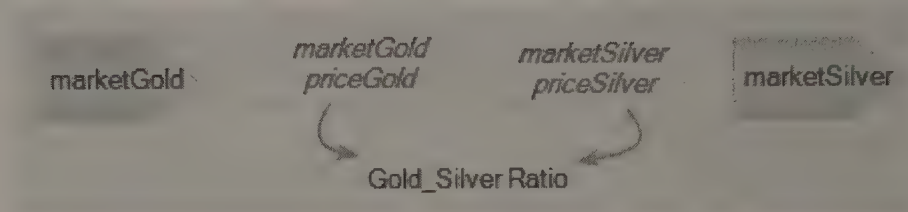


In the Run Setup 1/1/1950 has been defined as Initial Time and it is the prevailing date even though the silver data starts in 1991. Modify the Initial Time to 01/01/1992. Close. Simulate and see the result.

priceSilver

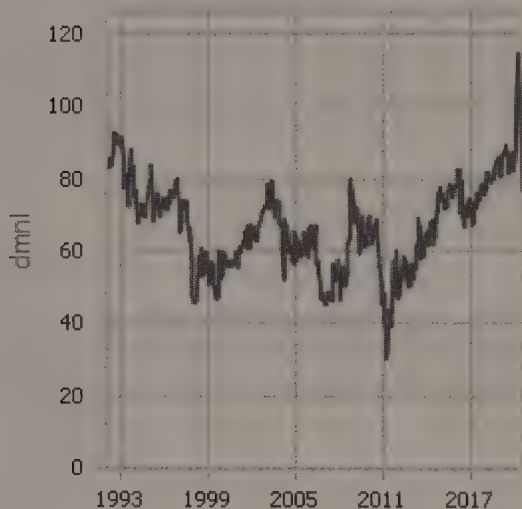


Finally, we can use the historical data to calculate a system variable, for example the ratio between the price of gold and silver. Create the auxiliary variable "Gold_Silver Ratio" with the equation: " $\text{marketGold.priceGold} / \text{marketSilver.priceSilver}$ " adding the arrows from both. Units dmn1.



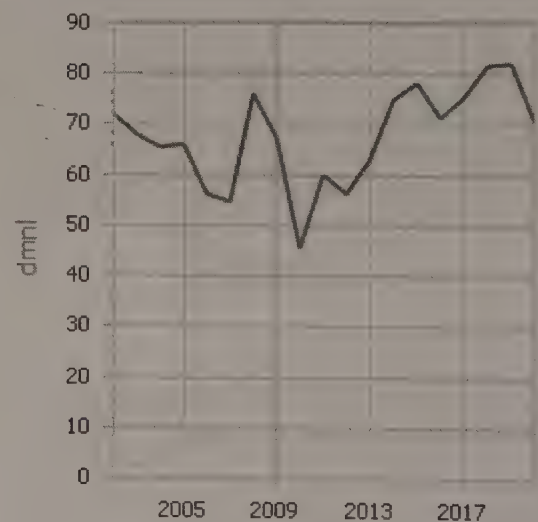
Simulate the model and see the graph of "Gold_Silver Ratio"

Gold Silver Ratio



Go to Run Control and change the Initial Time to 01/01/2002 and Units for Time=Year instead of Month.

Gold Silver Ratio





Model: Jewelry Store

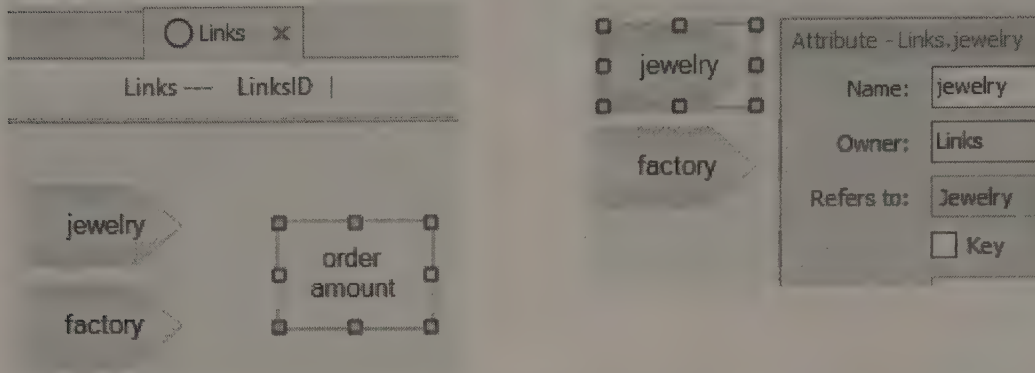
Some relationships are not exclusive. For example, a customer has many suppliers and a supplier has many customers. The goal of modeling many-to-many relationships allows either party, say a customer, to use information about their suppliers. For example, a model can allow each supplier to calculate the average size of their customers' orders. In this example the supplier role is played by a "Factory" entity, and the customer is a "Jewelry" entity. A third entity, called "Links", shows the relationship between the two entities.

Push File - New Model and write the name "Jewelry", which creates an entity with the same name, in addition to the Model. In Model Overview - Entity Types press the right button and choose "New Entity Type" and add the "Factory" and "Links" entities.

The entity "Links" includes:

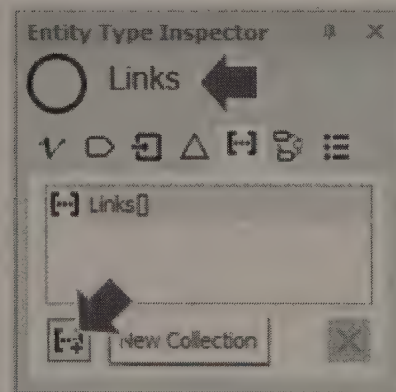
- An attribute "jewelry" with Refers to this entity.
- An attribute "factory" with Refers to this entity.
- The Auxiliary "order amount", mark Constant, Units=euro and value = 12000.

In the Links diagram define the two attributes that link factories and jewelry: jewelry for the Jewelry entity and factory for the Factory entity.

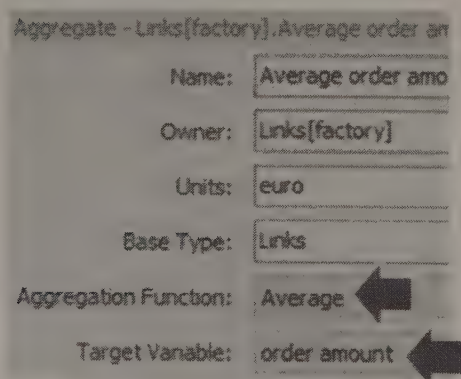
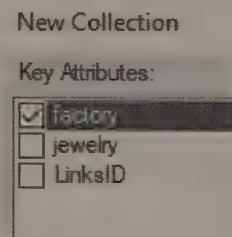


Therefore, each link is uniquely determined by the pair of attribute values. If we subsequently need to refer to a particular link, we can do so directly by supplying the specific factory and jewelry. This also means that we must be careful in initialization so that each link entity has a unique pair (factory, jewelry) specified.

We will define a sub-collection of Links grouped by factory. In this sub-collection we will include the desired aggregate variable. In this case the average order size.



Go to the Links inspector, Collections tab, click New Collection, and mark factory.



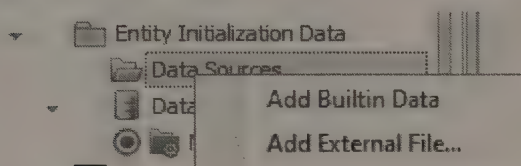
Then on the Links [factory] screen move the Aggregate icon to create a new variable.

Define the Average option as Aggregation Function, and in Target Variable choose the order amount.

Similarly, we will now define a sub-collection grouped by jewelry. In the Links inspector, Collections tab, click New Collection, select jewelry. Then on the Links [jewelry] screen move the Aggregate icon to create a new variable. Choose Average and order amount.

Both in the screen Links [factory] and Links [jewelry] we have two variables with the same name (Average order amount), but they mean different things. In one case it is the average of the amount of orders received by each factory and in the other is the average amount of orders placed by each jewelry.

In this example, we have 3 factories and 3 jewelry, each jewelry makes purchases from 2 factories and each factory supplies 2 jewelry.



Go to Entity Initialization Data - Data Sources and choose Add Builtin Data to define factory, jewelry and relationships.

Entities 1

Number of rows to add:

Factory		Jewelry	
	☒	Time	JewelryID
1	☒		J-01
2	☒		J-02
3	☒		J-03

Entities 1

Number of rows to add:

Factory		Jewelry	
	☒	Time	FactoryID
1	☒		F-01
2	☒		F-02
3	☒		F-03

Entities 1

Number of rows to add:

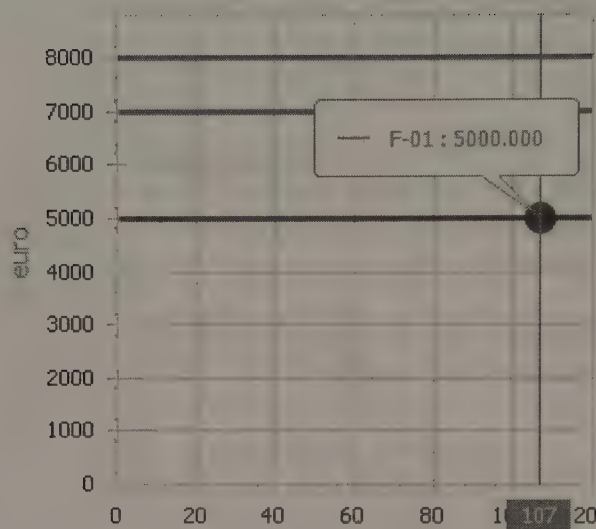


All

Non

Factory		Jewelry		Links			
	☒	Time	LinksID	factory	jewelry	order amount	Cal
1	☒			F-01	J-01	4000	
2	☒			F-01	J-02	6000	
3	☒			F-02	J-02	6000	
4	☒			F-02	J-03	10000	
5	☒			F-03	J-03	10000	
6	☒			F-03	J-01	4000	

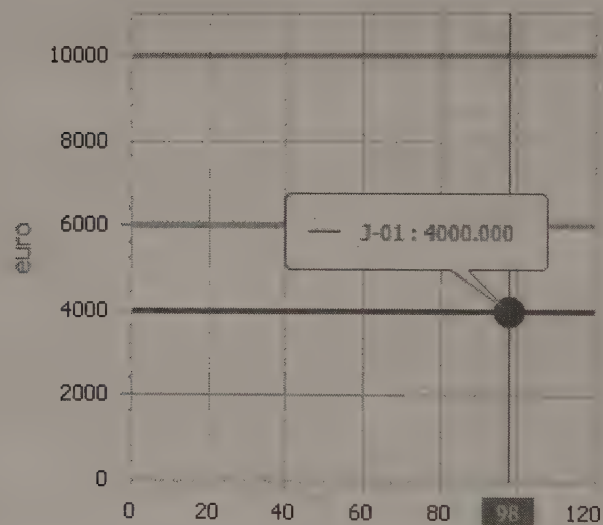
Average order amount



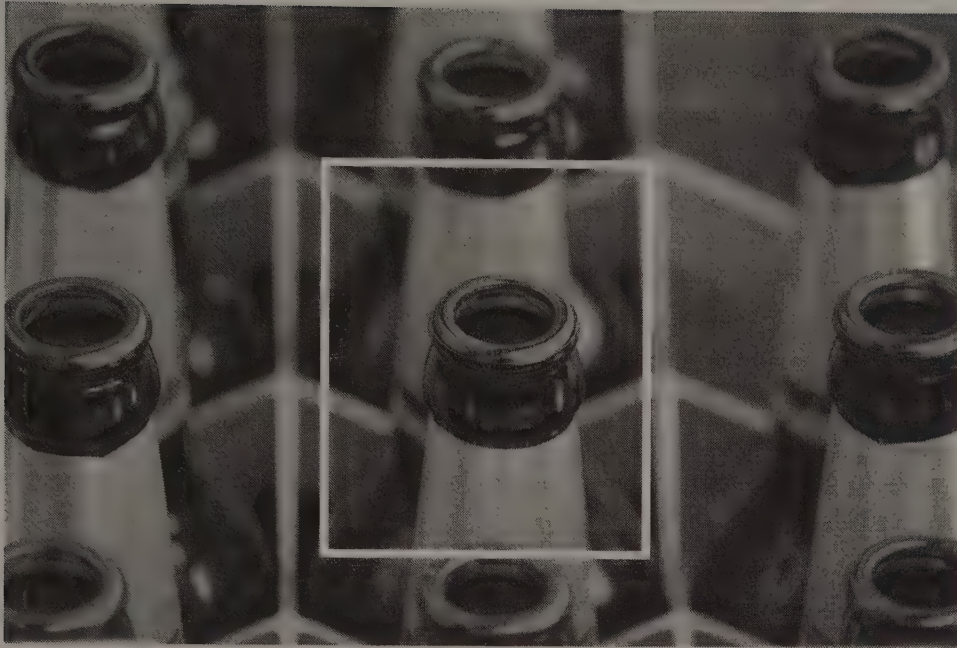
Simulate the model. Go to the Links [factory] screen, right click on Average order amount and choose Plot Variable. The average value of the order amount for each factory is obtained.

Then go to the Links [jewelry] screen, right click on Average order amount and choose Plot Variable. The average value of the order amount for each jewelry is obtained.

Average order amount



Save the model.



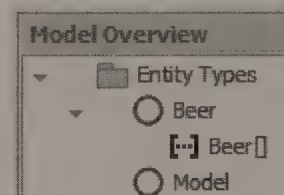
Model: Beer Game

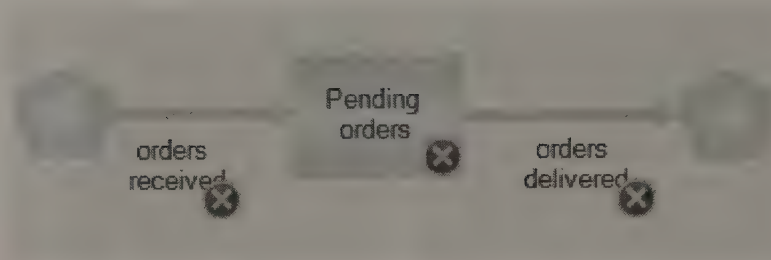
The Beer Game was created in 1960 by Jay Forrester. It allows one to see the usefulness and power of mathematical simulation based on the methodology called System Dynamics. It is used to show and analyze the typical coordination problems of a chain process supply and business decision making.

The game shows the importance of information exchange in supply chain management. Thus, due to lack of information, manufacturers, distributors, sellers and customers often have an incomplete understanding of what the impact of an order is. In the game the parts of the supply chain have no information about the other parts. Each part only has control over its own stage in the supply chain. Each part influences the entire supply chain, and by asking for too much or too little, it causes a bullwhip effect. https://en.wikipedia.org/wiki/Beer_distribution_game

In this model there is only one entity, Beer, which reproduces each part of the distribution process.

Make File - New Model and write the name Beer. The Model Overview shows an entity called Model, which has the temporary parameters of the model, and another named Beer.

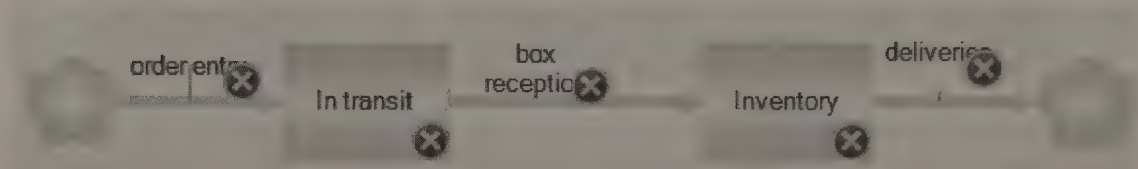




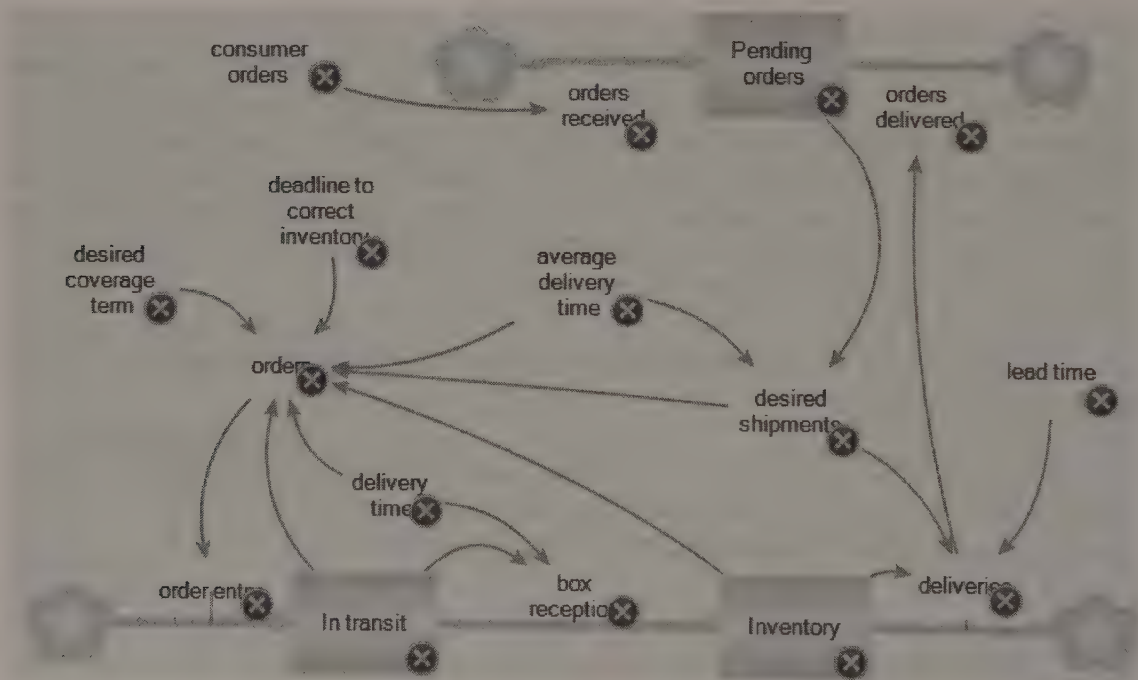
Draw the orders received, pending and delivered. To do this, move the Stock icon from the bar to the area of the diagram and write the name "Pending orders".

Move two clouds and place them as shown in the figure. Move the cursor over the left cloud until a hand appears, then click on the cloud and move the cursor over the Stock. Write the name "orders received". Repeat it to create the flow "orders delivered", this time starting from the Stock and ending in the cloud.

At the bottom of the screen add the Stocks and Flows shown in the following figure, with their respective names.

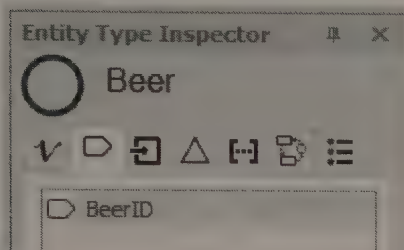


Add the auxiliary variables shown in the diagram below, moving the Auxiliary icon to the diagram area. Then add the arrows, to make it place the cursor on the source variable and move it until a hand appears, at that moment click on the variable, move the cursor to the destination variable.



If you get confused when drawing an arrow, select it and press delete on the keyboard. You can change the curvature of an arrow by moving the small yellow square in its center.

Attributes

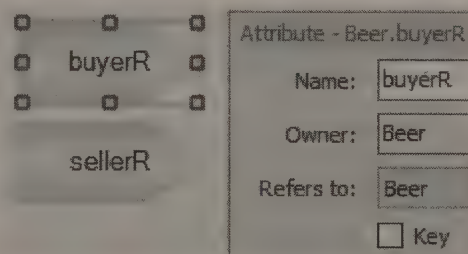


In the Overview Model - Entity Types right-click on Beer and choose the Inspect option, which opens the Beer inspector on the right side of the screen.

Click the Attributes tab, move the BeerID attribute three times to the left of the diagram.

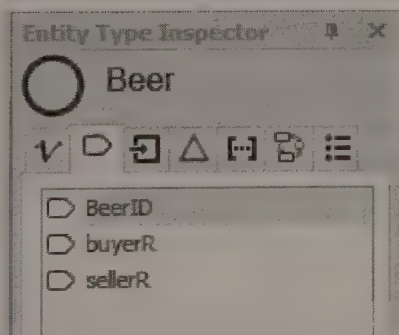
The attributes will allow us to identify the entity that is calculated, the origin entity (buyer) and the destination entity (seller). For example, when the store is calculated, it will have the consumer as the buyer and the distributor as the seller.

Create two new attributes, named buyerR and sellerR. Press the right button, press Edit and select Refers to Beer.

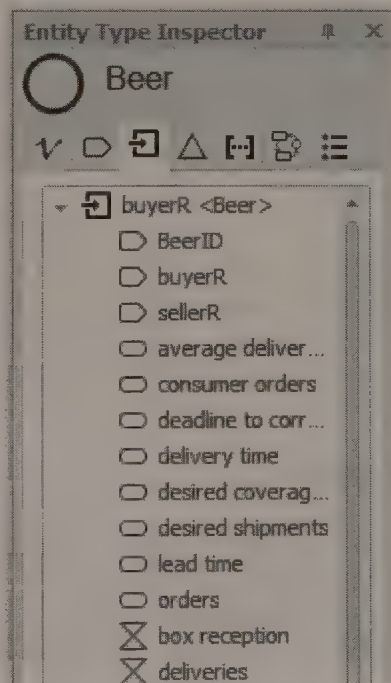


In the Beer inspector, attributes tab, the two newly attributes created ones appear.

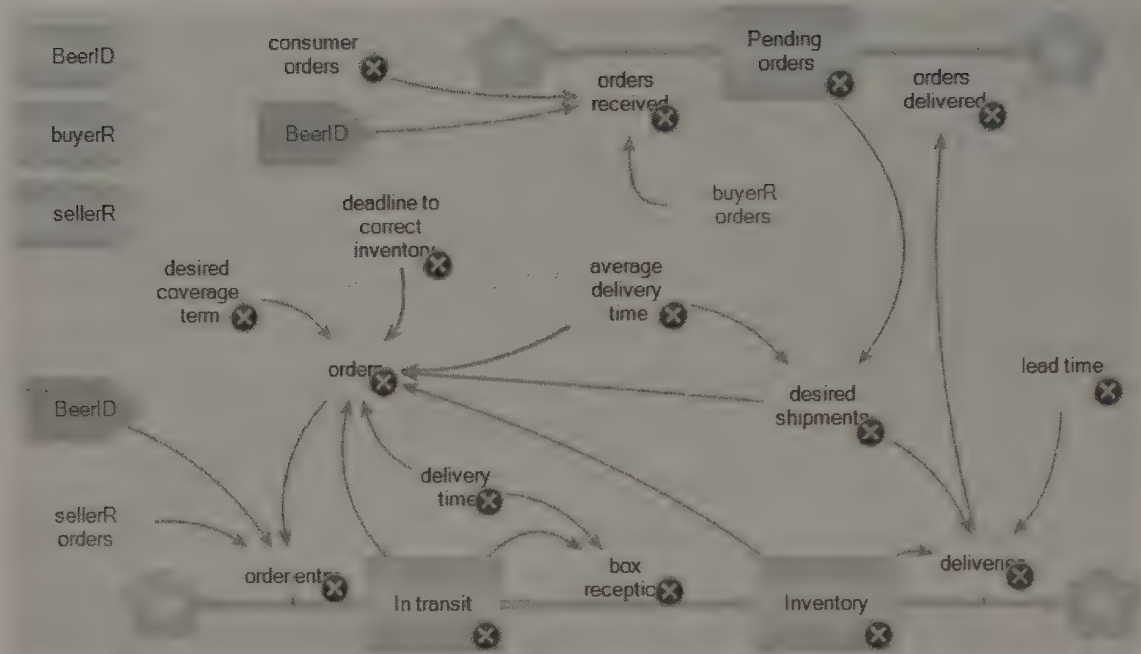
In addition, the references tab also shows buyersR and sellersR.



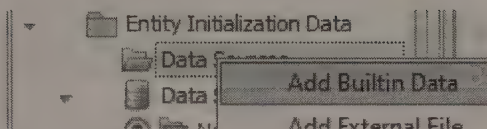
Go to the references tab, display buyersR and sellersR, and move the variable "orders" that are in both references to the diagram.



Add the arrows until you get the following diagram.



Initial parameters



In Entity Initialization Data - Data Sources click Add Builtin Data. Entities is created 1. Enter the following data

Entities 1					
Number of rows to add:					
Beer					
	☒	Time	BeerID	buyerR	sellerR
1	☒		consumer		seller
2	☒		seller	consumer	distributor
3	☒		distributor	seller	manufacturer
4	☒		manufacturer	distributor	

Equations

All the variables appear with the signal that the equation is pending to be written. Right-click on the variable, open the Edit Auxiliary menu, enter the units, and click in the equation area to display the variables related to it according to the drawing. Click on them twice to incorporate them into the equation.

In the Stocks you must enter the initial value. You will see that the software shows the input and output flows at the bottom.

The warning signs that appear when entering the equations indicate that this variable cannot yet be calculated because the equation of variables on which it depends is missing.

In the Stocks it is only necessary to write the Initial Value, since the input and output flows are shown in the lower part of the equation, according to the diagram.

Inventory

Initial value: 16, Units: boxes

In transit

Initial value: 8, Units: boxes

Pending orders

Initial value: 8, Units: boxes

All the Auxiliary have weeks as units, and the values:

desired coverage term = 8

lead time = 1

delivery time = 2

average delivery time = 2

deadline to correct inventory = 6

The Flows and other Auxiliary have as units boxes/week and the following equations.

consumer orders = if (CurrentTimeSlice ()> 6) {8} else {4}

It is indicated that if the period of time is greater than 6, that is from period 7, the value of consumer orders is 8, otherwise, that is until period 7, the value is 4 boxes per week constant. This allows us to validate that until period 7 the model shows a stable behavior since there are no changes in the initial input of the chain, which are consumer orders.

orders received = if (BeerID== "consumer") {consumer orders} else {buyerR.orders}

BeerID is the Key attribute that identifies each of the entities. In the Entity Initialization Data they have been defined: consumer, store, distributor and manufacturer. This equation points out that when the consumer entity is calculated, the orders received are the consumer orders, otherwise they are the orders from the buyer entity.

orders delivered = deliveries

The delivered orders, which come out of pending orders, are the deliveries.

order entry = if (BeerID=="manufacturer") { orders } else { sellerR.orders }

In this case, if the entity being calculated is the manufacturer, the input takes the value of the orders, otherwise it takes the orders of the sellerRef attribute.

box reception = In transit / delivery time

It is the quotient between the boxes in transit and the delivery period that has been defined.

deliveries = MIN(desired shipments, Inventory/lead time)

It is the lowest value between the desired shipments and the quotient between Stocks and delivery time. The delivery units are boxes/week, and in order to be consistent, Stocks must be divided by a temporary variable (weeks).

desired shipments = Pending orders / average delivery time

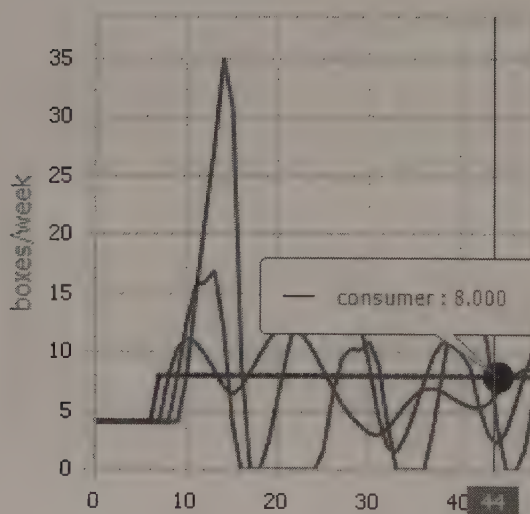
This variable is the quotient between pending orders and the average delivery time, in this way abrupt changes in specific pending orders are partially cushioned.

orders = max(0, (desired shipments * (desired coverage term + delivery time + average delivery time) - Inventory - In transit) / deadline to correct inventory)

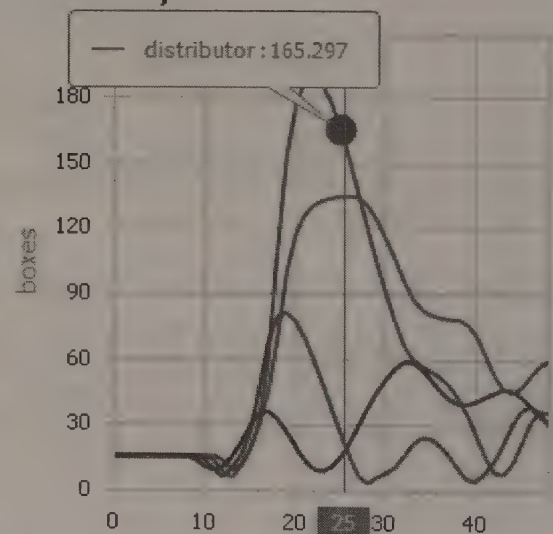
The "max" function that is used prevents orders from being negative at any time. First, the desired shipments are calculated multiplied by the sum of the deadlines that delay deliveries. That is, if the desired shipments are 10 boxes per week, but the sum of the deadlines that delay shipments is 4 weeks, 40 boxes would be requested. From this figure (40) the boxes that are in stock or in transit are subtracted. The resulting net figure is divided by the time that exists to correct the inventory to its desired value.

Simulate the model and view the results by selecting the variable and Plot Variable.

orders received

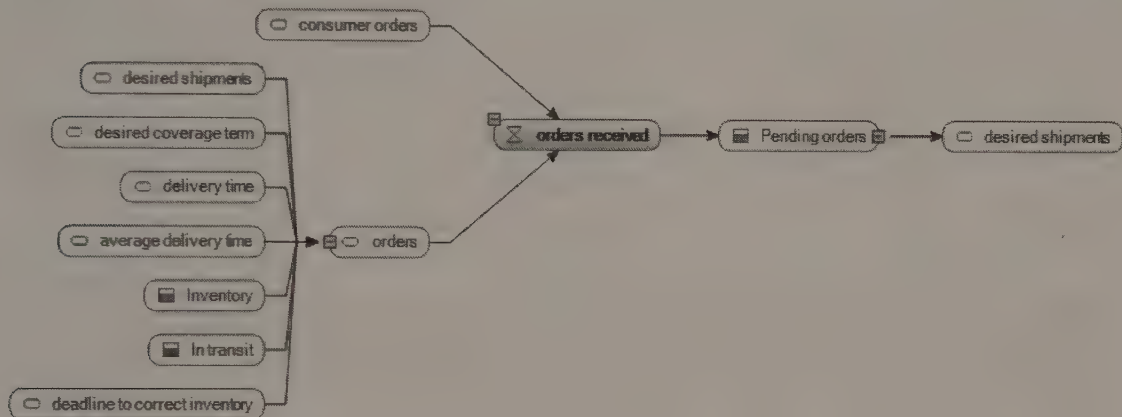


Inventory

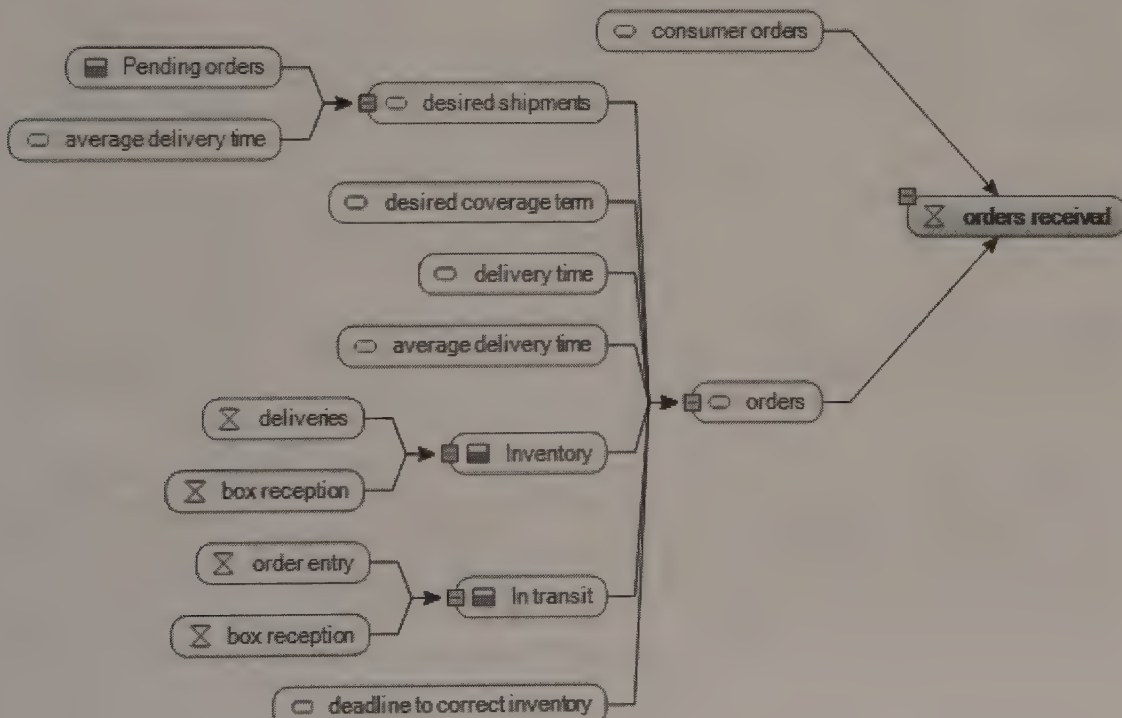


The system remains stable until period 7, at which time consumers go from buying 4 to 8 boxes/week and then the value of 8 boxes/week remains constant. This single change causes an increase in stocks and orders at all stages of the supply chain, reaching a peak of 65 boxes/week received by the manufacturer.

We can use the model to see some features of the software, such as the causal tree. Once the model has been simulated, we can right-click on any variable and in the menu that opens, click the Causes-Uses Tree option to see which variables the selected variable depends on and which variables it influences. We are able to choose the degree of detail of these dependencies in the menu on the left. In the case of "received orders" the equation is: if (BeerID == "consumer") {consumer orders} else {buyerRef.orders}



The Causes Tree and Uses Tree options provide a part of this diagram.



The Causes Tree Table option allows to see the values of the variables on which it depends for each period and entity and thus understand the calculations made by the model.

Causes Data Table - Results: (Run) - Variable: Beer.orders received

Run	Date Time	Entity ID	<< Beer.orders received >>	Beer ID	consumer orders	buyer R.orders
Run	6	consumer	4	consumer	4	4
Run	7	consumer	8	consumer	8	4
Run	8	consumer	8	consumer	8	8
Run	9	consumer	8	consumer	8	10.3333333333...
Run	10	consumer	8	consumer	8	11.1666666666...
Run	11	consumer	8	consumer	8	10.8055555555...
Run	12	consumer	8	consumer	8	9.73148148148...
Run	13	consumer	8	consumer	8	8.51157407407...

Causes Data Table - Results: (Run) - Variable: Beer.orders received

Run	Date Time	Entity ID	<< Beer.orders received >>	Beer ID	consumer orders	buyer R.orders
Run	6	seller	4	seller	4	4
Run	7	seller	4	seller	8	4
Run	8	seller	8	seller	8	4
Run	9	seller	10,333	seller	8	8
Run	10	seller	11,157	seller	8	12.6666666666...
Run	11	seller	10,806	seller	8	15.6944444444...
Run	12	seller	9,731	seller	8	15.9444444444...
Run	13	seller	8,512	seller	8	16.9398148148...

The variable "consumer orders" is an input that does not depend on other variables, and as the equation is defined, the orders made by the consumer entity (orders received) are always the "consumer orders".

The "seller" entity has as received orders the "buyer orders" from the "consumer", while its "buyer orders" are received by the distributor, as defined in the Entity Initialization Data.

We can review the calculation of the orders for the buyerRef attribute of the consumer when the change occurs from 4 to 8 boxes/week in period 7. The equation of

Week 6

consumer orders 4

BeerD

orders received 4

Pending orders 8

orders delivered 4

desired coverage term 8

deadline to correct inventory 6

average delivery time 2

buyerR orders

orders 4

desired shipments 4

lead time 1

BeetD

sellerR orders

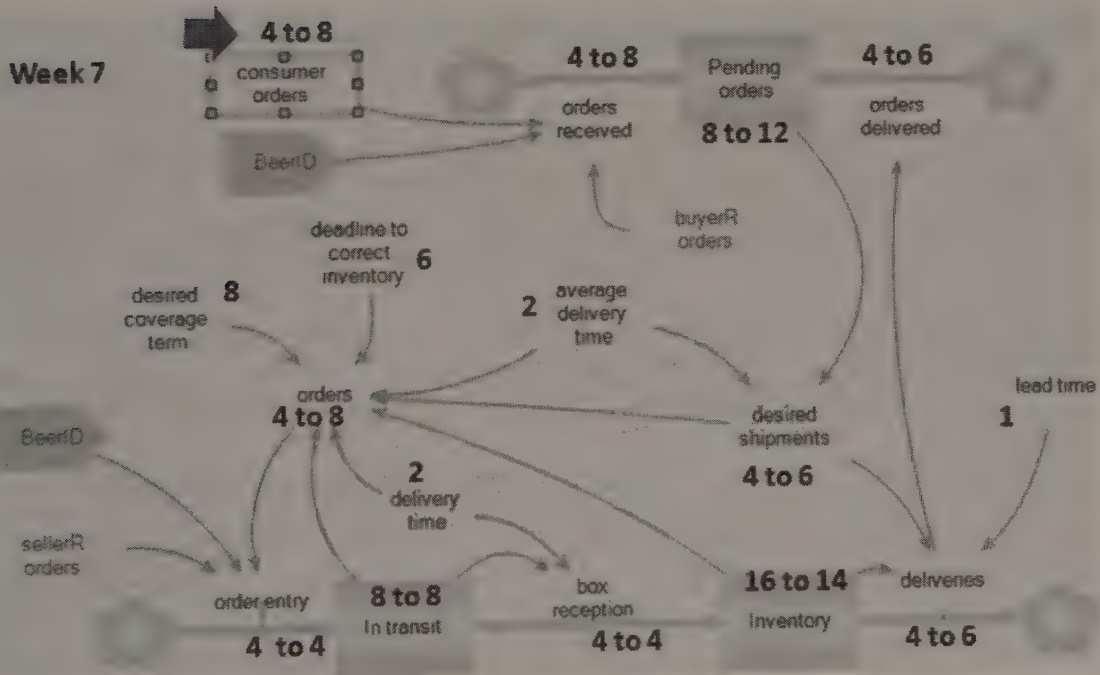
order entry 4

In transit 8

box reception 4

Inventory 16

deliveries 4



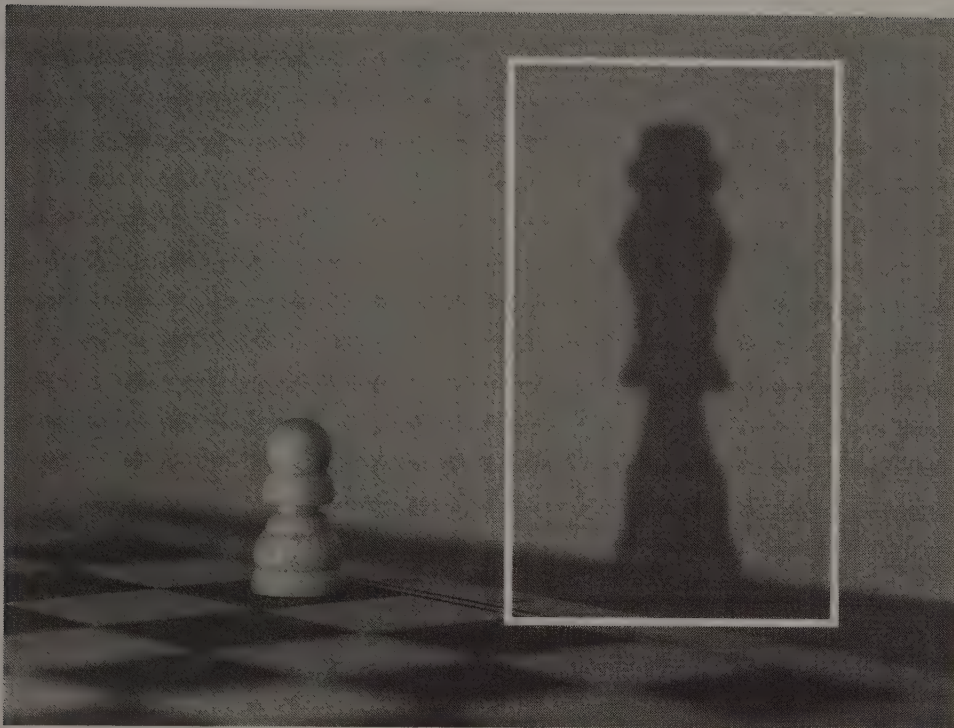
Results: Run - Component: Beer - Element: orders

DateTime	BeerID	orders delivered
	5 consumer	4
	6 consumer	4
	7 consumer	4
	8 consumer	6
	9 consumer	7
	10 consumer	7,500

In the previous image we can see the initial values of each variable when consumers always buy 4 boxes/week and in the following image the values when they go from 4 to 8 boxes/week in period 7

Thus, by making View Data of the variable "orders delivered" we can verify that in the model they go from 4 to 6 in the following period, as has been calculated manually on the diagram.

Although the calculation of orders is not complex, it is important to note that the "orders received" depends on the orders sent to the seller in the previous period, that are still 4. To calculate the status of a variable in the week 7 the values of the other variables in period 6 are used. In any case, this does not influence the result of the first weeks since the Inventory is greater than the "desired shipments".



Continuation

After reading this book, I hope that you have achieved some knowledge - not without a great effort that I recognize - that allows you to create basic ABM models.

Without a doubt your progress from zero to the point where you are is impressive. You can select entities, create a diagram, add variables and attributes, create collections and aggregates, define relationships, add equations with their functions, import data from Excel, etc. That is great progress.

This tool offers additional features like: define actions to add or delete entities, create cohorts of entities, sensitivity analysis, optimize and calibrate the model parameters. These tools are developed in the next book in this collection.

I hope to see you in the next book.

The author.

Agent-Based
Modeling and Simulation
II

Actions, Cohorts and Analysis of Results

Juan Martin Garcia



ISBN 9798599114710

Annex



Causal Loop Diagrams

What is a system?

A system is a set of interrelated elements, where any change in any element affects the set as a whole. Only elements directly or indirectly related to the problem form the system under study here. In order to study a system, we must know the elements that make it up, and the relationships between them.

When we analyse a system we usually focus merely on the characteristics of its constituent elements. However, in order to understand the functioning of a complex system, we must focus also on the relationships that exist between the elements which form the system.

It is impossible to understand the essence of a symphony orchestra by merely observing the musicians and their instruments. It is the coordination that exists between them that produces beautiful music. The human body, a forest, a country, or the ecosystem of a coral reef are all examples of systems that are far more than the sum of their parts.

An ancient Sufi saying can illustrate this: *You can think, because you understand 'one', and you can understand 'two', which is 'one' plus 'one'. However, you must also understand 'plus'.* For example, in a traffic problem, many related elements converge - the number of inhabitants, the number of cars, the price of petrol, parking spaces, alternative transport, etc., and it is often easier and more effective to attempt to solve the relationships between the elements (*'plus'*) than the elements themselves.

A good method to begin defining a system is to write the main problem down in the middle of a blank page, surrounding it with the directly related elements. The elements which affect the main problem indirectly go around the appropriate direct elements. This will be the system that we will study in order to consider possible solutions to a given problem.

Where does the system end?

We have all heard the theory that a butterfly fluttering its wings in China could cause a tornado in the Caribbean. In our study, however, we will include only elements with a reasonable influence on the behaviour of a system. We mustn't lose sight of the objective, that is, to propose practical action towards effectively solving the problem at hand.

The system must contain as few elements as possible while providing a simulation that will truly allow us to decide which of the possible courses of action studied are the most effective solution to the problem. The models are generally small to begin with, with few elements. They are then expanded and perfected. Later on, elements which don't play a decisive part in the problem are eliminated. During the construction of a model, there are several extension and simplification phases in which elements are added and subtracted.

We can't ignore the relationship between the consumption of petrol and lung health. When we analyse the carbon combustion process in an electric power plant, we can see that, apart from energy, the following is produced - ash, suspended particles, SO₂, CO₂, etc. We can also see that there is no barrier between the desired product (electricity) and the by-products. Sometimes, the so-called *side-effects* are as real and as important as the *main effects*. The beauty of a system in nature is that the waste produced by one process serves to feed the next. Perhaps this is the model to follow for industrial design in the future.

The final size of the model must be such that its main aspects can be explained in ten minutes. Any model larger than this will fail.

How do we represent the system?

A system is formed by the set of elements that bear relationship to the problem and that account for the observed behaviour, along with the relationships that exist between these constituent elements (which often involve feedback). The causal diagram represents the key elements of the system and the relationships between them.

It's important to draft versions that will bring us increasingly closer to the final complex model. The minimum set of elements and relationships that serves to reproduce the historical reference of a system is that which forms the basic structure of the system. Once the variables of a system and the hypothetical relationships between these variables are known, we can move on to produce a graphical representation. This diagram shows the relationships as arrows between the variables. These arrows are marked with a sign (+ or -) which indicates the kind of influence one variable exerts over the other. A '+' means a change in the influencing variable will produce a change of the same direction in the target variable. A '-' means the effect will be the opposite.

So, when an increase in A results in an increase in B, or a fall in A causes a fall in B, this is a positive relationship, as shown below:

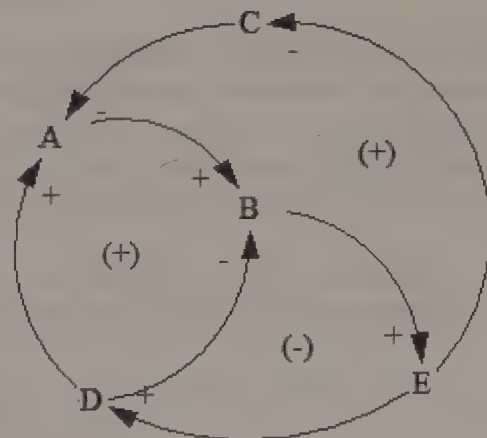


When an increase in A results in a fall in B, or a fall in A causes an increase in B, this is a negative relationship, which is expressed as follows:



What is a feedback loop?

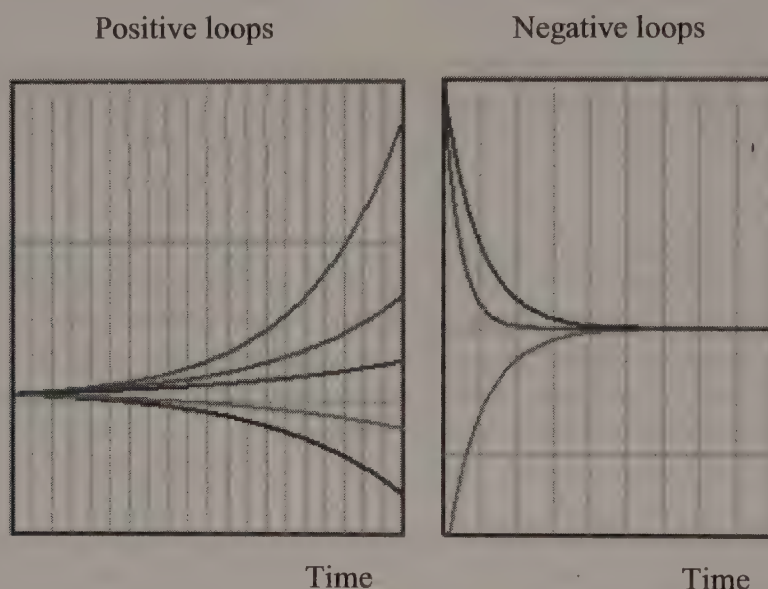
A closed chain of relationships is called a loop, or a feedback loop. When we turn on the tap to fill a glass with water, the amount of water in the glass increases. The amount of water in the glass, however, also has an effect on the speed at which it is filled. We fill it more slowly when it is fuller. Therefore, a loop exists.



The system formed by us, the tap and the glass is a negative loop, because it is designed to achieve a goal (fill the glass without spilling). Negative loops act as stabilising elements in systems designed to reach a given goal, like when a thermostat in a heating system guides the temperature towards the stock specified by the user.

When we construct a model, loops appear. For example, those formed by ABEDA, DBED and ABECA in the following causal diagram. Loops are defined as 'positive' when the number of negative relationships is even. If the number of negative relationships is odd, the loop is 'negative' (just as -3 multiplied by $+3$ gives -9).

Negative loops tend to stabilise the model, while positive loops tend to destabilise it, independently of the basic problem at hand, because in essence a positive loop is: $x_{n+1}=f(x_n * x_n)=f(x_n^2)$ and a negative one is $x_{n+1}=f(x_n * (1-x_n))$ that is, the state of a variable x in period $n + 1$ is a function of the state it had in period n , exponentially amplified if it is a positive loop, or not, if it is a negative loop.



Real-life systems contain both types of loops, and the ultimate behaviour will depend on the dominant type at any particular moment. When a country acquires more arms it makes its neighbours feel threatened and causes them to also acquire more arms. This is a positive loop, also called a vicious circle, and grows more and more as it feeds itself. Positive loops cause growth, evolution, and also the collapse of systems.

Naturally, socio-economic and ecological systems are made up of hundreds of interconnected positive and negative loops and its ultimate behaviour isn't obvious.

The concept of the loop is very useful because it enables us to start from the structure of the system that we are analysing and work towards its dynamic behaviour.

If a system fluctuates persistently, remains in equilibrium or drops off rapidly, we can identify the structural reasons and decide how to go about modifying the causal loops that are going to influence it. This procedure can be applied to anything from the control of an industrial process to the monitoring of diabetes or cancer, fluctuations in the price of raw materials, or economic growth.

Yet the most important use of this concept is in understanding how the structure of systems affects their behaviour. In the same market and in the same year, various firms that offer the same product present very different economic results. The less competent managers put this down to causes beyond their control – the cost of labour, competitors, customers' habits, and so on – when in fact they should study why the systems they control (their businesses) have a less competitive structure than those that show better results.

Country A perceives that the arms race was caused by country B and vice versa. But in reality we can also say that country A has caused its own rearmament by acquiring arms, as this causes the rearmament of country B. Similarly, the rise in oil prices is due both to the concentration of production in a few countries and to excessive consumption in developed countries of a product that's limited, inasmuch as it isn't renewable.

Identifying the cause of a problem as being something that is not external to the system doesn't tend to be very popular as it is easier to blame external factors beyond our control. The trouble is, if the exponents of the argument of the external cause really believe what they are saying, they will be unable to identify the true cause of the problem – inside the system – and obtain the desired results. If the system contains the elements that cause the problem, it also has those that can be used to solve it.

For example, a product's life curve can be said to be regulated initially by a positive loop that permits rapid exponential growth, followed by a steady state dominated by a negative loop involving the saturation of the market, and finally a usually sudden drop caused by the appearance of fast-growing substitute products.

Lastly, note that the causal diagram is very important for explaining the final model to the user if he or she isn't familiar with this technique, as is usually the case.



Download the models

All models of the book are available in:

http://atc-innova.com/mba_one.htm



Made in the USA
Middletown, DE
19 April 2022



64509942R00146



ISBN 9798599111306



9 798599 111306



90000

